前言

笔者是一名有6年嵌入式软件研发经验的工程师，由于工作需要和自己的兴趣，在最近3年的时间开始接触上层软件方面的技术，其中就包括学习和使用Python。越深入地学习，越让笔者觉得Python好用，于是陆续在博客上更新了很多Python的教程和资料，这本书的大多数素材均来自笔者的博客。

Python语言从出现到如今已经有很长时间了。市面上关于Python的入门读物更是数不胜数，所以想再写一本入门的好书需要极大的勇气和决心。真正触动笔者下决心写这本书的原因是笔者发现对于很多国内读者，尤其是很多计算机基础相对薄弱的国内读者来说，市面上的教程往往很难看懂。因为这些Python图书多数都是国外作者写的，面对的读者是具有较强的计算机基础的国外用户。不仅如此，这些书被翻译成中文，在国内销售，还忽略了中外读者的差异，导致很多国内读者即便是看入门级的Python读物，学习起来仍然觉得很吃力。国外的作者往往会省略一些背景知识，而国内读者又恰恰缺少这些相应的背景知识，自然很多内容无法完全理解。举个例子，在解释选择Python的IDE开发环境时，如果直接让国内初学者使用某个界面相对好看但是功能复杂的IDE，很多人会很难理解如何使用。初学者对于IDE和原始的命令行之间的关系本身就不是很清楚，在这种情况下，又怎么可能学会使用呢？笔者觉得应该换一种更好的方式去解释，即先分析透彻原始的开发环境和复杂的IDE之间的关系，然后解释为何要选择某个IDE，再进一步介绍IDE中每个部分的功能和之前原始的命令行方式的对应关系。只有如此解释，初学者才能明白选择Python开发环境的真正含义，才能理解为何选择和使用某个IDE。基于此，笔者才陆陆续续地编写了一系列针对Python初学者的帖子，并努力把每个知识点讲透。笔者觉得这样才能真正帮助读者学习和掌握Python。

全书共分为7个章节：

- 第1章主要阐述一些与Python相关的基本概念，并介绍其应用领域；
- 第2章介绍如何下载和安装Python；
- 第3章介绍如何在Windows、Linux、Mac等多种不同平台下开发Python应用，以及如何选择合适的开发环境；
- 第4章解释Python的基本知识；
- 第5章给出一些有趣的Python小实验；
- 第6章进一步给出Python在各种领域内的实际应用案例；
- 第7章介绍Python与开源硬件之间的关系，了解如何用Python搭配开源硬件以实现各种有趣的功能。

本书主要针对没有基础的初学者，包括但不限于具有硬件背景的工程师、非计算机专业读者、Python业余爱好者、学生等。鉴于轰轰烈烈的创客运动，笔者把这本书的定位描述成创客的Python入门书。其实在笔者看来，只要是想动手，实现自己心中所想的人都应该是创客。从这个角度出发，笔者觉得任何一个想要学习Python的初学者都是创客。这是一本真真正正的入门书，读者只需具备计算机编程语言的基本知识，即可阅读本书。本书会带领读者从了解Python的概念开始，到搭建Python的开发环境，再到真正使用Python语言实现一些应用。

本书中所用到的标识及其含义如下：

提示。提示信息，如用于解释一些名词的含义、阐述相关知识背景。

注意。用于强调一些注意事项、心得体会等内容。

重要。提示某些重要的信息。

警告。用于指出严重的错误，提醒读者一定要避免这类错误。

笔者首先感谢父母的教育，没有他们的悉心培养，笔者很难有今天的成就。笔者还要感谢朋友高静在此期间给予笔者的支持和无私的帮助，让笔者能够终成此书。由于笔者水平有限，书中错误在所难免，欢迎读者和同行们指出错误，提出您的宝贵意见或建议。

李茂

2014年11月

第1章　Python简介

1.1　Python是什么

Python作为一个英文单词，其本意是巨蟒、蟒蛇的意思。

Python这个词在计算机语言领域内指的是一种计算机语言叫作Python。

Python语言名字的由来

我们之所以把蟒蛇——Python用于命名计算机语言是有其历史典故的。

1989年在阿姆斯特丹，Python语言的创始人吉多·范罗苏姆（Guido van Rossum）为了打发圣诞节的无趣决心开发一个新的脚本解释程序，在给此新的计算机语言起名字时，由于其本人是巨蟒剧团[1]的忠实粉丝，所以就把此计算机语言的名字叫作了Python。这就是Python作为一种计算机语言的名字的由来。

Python语言被称为了蟒蛇，名字看似很凶猛，但并不代表Python语言本身很“凶猛”、“吓人”，反倒是谈论起Python语言功能的强大时，用“凶猛”来形容并不为过。

[1] 巨蟒剧团（Monty Python）是始于20世纪70年代而后风靡全球的一个英国六人喜剧团体。

第1章　Python简介

1.1　Python是什么

Python作为一个英文单词，其本意是巨蟒、蟒蛇的意思。

Python这个词在计算机语言领域内指的是一种计算机语言叫作Python。

Python语言名字的由来

我们之所以把蟒蛇——Python用于命名计算机语言是有其历史典故的。

1989年在阿姆斯特丹，Python语言的创始人吉多·范罗苏姆（Guido van Rossum）为了打发圣诞节的无趣决心开发一个新的脚本解释程序，在给此新的计算机语言起名字时，由于其本人是巨蟒剧团[1]的忠实粉丝，所以就把此计算机语言的名字叫作了Python。这就是Python作为一种计算机语言的名字的由来。

Python语言被称为了蟒蛇，名字看似很凶猛，但并不代表Python语言本身很“凶猛”、“吓人”，反倒是谈论起Python语言功能的强大时，用“凶猛”来形容并不为过。

[1] 巨蟒剧团（Monty Python）是始于20世纪70年代而后风靡全球的一个英国六人喜剧团体。

1.2 对Python的四种定义

对于某种计算机语言，根据其特点和语言本身侧重点的不同会有不同的分类和叫法。Python作为计算机语言的其中一种也不例外。接下来就来详细解释Python的各种不同的分类和叫法的详细含义。

1.2.1 一种脚本语言

脚本的英文是script。一般的读者看到script这个单词往往首先想到的是电影的剧本，我们大多数人都知道电影剧本其实就是由一段段的脚本所组成的，即电影剧本的脚本决定了电影中的人和物要做哪些事情以及具体怎么做。

与之类似，计算机中的脚本决定了计算机中的操作系统和各种软件工具要做哪些事情以及具体怎么做。

此外，根据笔者的理解，脚本这个词还会让人有另外一种感觉：随性。

所谓的随性就像在现实中写电影脚本，直接拿张纸和一支笔就可以写了，写完之后就可以拿去使用，即拍电影了。而对应的计算机中的脚本也是类似的过程，用户想要实现一个功能，在构思了如何做之后，就可以直接找个文本编辑器写上对应的脚本，也就是普通的文本，接着让计算机去运行，从而实现想要的功能。当然计算机脚本与电影脚本也有不同之处：计算机中的脚本所运行的环境中有对应的脚本解析器，可以解释并执行对应的脚本。

例如，若创建Linux中的shell脚本，可以先创建一个普通文本文件，然后在其中添加shell脚本代码，保存文件，接着就可以在Linux的带有shell解释功能的环境中运行了。

那些相对“随性”的脚本语言来说不那么“随性”的非脚本语言叫作编译型语言，比如C语言，它们需要再加上编译这个步骤之后才能生成可以运行的程序，然后才能执行程序。

编译型语言和脚本语言的对比

（1）编译型语言

简单地说，编译性语言就是需要用该语言的编译器将源代码编译为可执行程序，然后才能运行可执行程序的语言。此编译过程需要先将源代码编译为目标文件，然后把目标文件加上必要的库文件，最后再链接为最终的可执行文件。

（2）脚本语言

脚本语言是无需用编译器编译源代码，可直接运行该源码形式脚本文件的语言。而直接能够运行脚本文件的背后，是当前环境中存在着此脚本的解析器。解析器负责读入此脚本源码，以及后续解析并执行的动作。

编译型语言和脚本语言的执行示例

（1）编译型语言的执行

编译型语言的执行过程为

源代码 ⇒ （用编译器编译成）可执行程序运行程序（可执行文件）

以在安装了Visual Studio 2010的Windows 7中编译运行C语言为例，其过程为

hello.c ⇒ （用编译器cl编译为）hello.exe ⇒ 运行hello.exe

其中，hello.c是先被编译器cl编译为目标文件hello.obj，然后再调用link将目标文件hello.obj链接为可执行文件hello.exe的。

（2）脚本语言的执行

脚本语言的执行过程为：

源代码直接 ⇒ （在具有了解析器的环境中）运行程序（脚本源码）

以在安装了Python的Windows中运行Python脚本为例，其过程为：

demoPython.py ⇒ （直接在有了Python解析器python.exe的环境中）运行demoPython.py

其中，python.exe是安装了Python时就会被安装的Python解释器，负责解析并运行Python脚本文件。

1.2.2 一种解释型语言

我们有时候也会把Python称为一门解释型语言。这是因为脚本语言的天然特点之一就是解释性，而作为脚本语言的Python因此也可以被称为是一门解释型语言。

什么是解释型语言

脚本语言的特点是在具有解释器的环境中，以源代码的形式而无需编译就可以直接执行。脚本语言的内部执行过程是首先由解析器一行一行地读取脚本的源代码，同时解析每一行，然后给出执行后的结果。通俗地说就是：读一行，解释一行，执行一行。由此，脚本语言就天然地具有了解释性的特点，所以脚本语言也常被称为解释型语言。

1.2.3 一种高级语言

在解释Python为何是一种高级语言之前先来解释一下什么是高级语言。

什么是高级语言

事物的发展都是从低级走向高级的，计算机语言的发展也不例外。早期的计算机语言大多数都是针对硬件上的机器本身所开发的语言，一般叫作汇编语言。汇编语言之所以常被称为低级语言，是因为其语言本身直接和硬件打交道，而缺少对计算机细节的抽象，相对而言不是那么易于程序员理解和使用。

随着计算机语言的发展，出现了对计算机细节更宏观抽象的语言，这类语言更多地采用人类更容易理解的元素和概念，因此对于程序员来说，相对更容易理解、学习、掌握和使用这类语言，而这类语言被称为高级语言。常见的一些高级语言有Fortran、Pascal、Lisp、C、C++、Java、C#、Python、Ruby等。

高级语言和低级语言的对比

高级语言在本身的设计层面会考虑到对计算机细节的封装和抽象。

比如一个普通的计算机语言概念中的数组，在常见的高级语言中有具体的实现，程序员可以很方便地直接拿来使用。而作为大多是汇编语言的低级语言，它们对于数组的实现和使用则很复杂，需要手动去写很多复杂的汇编代码才能使用。因为汇编语言等低级语言和底层硬件有关系，比如不同的CPU会有不同的指令集和不同的寄存器等各种内部资源，所以要具体实现数组这样的功能时，会有千差万别，需要考虑到很多细节和不同的实现方式，需要每个程序员对当前所使用的CPU的所有架构、内部资源等细节都很清楚，同时也需要对当前特定CPU的指令的语法都很清楚。在此前提下，才可以写出对应的汇编代码，才可以实现对应的数组。

由此可见，汇编语言会涉及太多物理层面上和硬件上的实现细节，不利于人类以普通的逻辑去理解，更难学习和掌握。而相对来说，高级语言涉及的基本元素和概念和人类的思维很相似，所以高级语言对于程序员来说更加容易理解、学习和掌握。

然后我们再来具体地解释为何Python是其中一种高级语言。Python在语言设计的时候，和其他高级语言类似，也完全具有常用的各种基本元素，比如各种普通变量、列表、函数等内容，完全符合人类的逻辑，易于理解、学习和掌握，所以Python本身的确是一种高级语言。

1.2.4 一种面向对象的语言

在解释为何Python是一种面向对象的语言之前，先来简要介绍一下什么是面向对象的语言。

什么是面向对象的（高级）语言

首先要明确的是，面向对象的计算机语言这个概念是针对高级语言来说的。其次，在一堆高级语言里面，有些是在设计该语言本身时，对于语言本身的基本元素是以对象的方式设计的，而不同的对象之间的交互则成为整个程序运行的主要表现形式。

此处的对象往往和现实中的物体、概念、逻辑的整体等内容是一一对应的关系。由于程序员用基于对象的编程语言设计程序时往往更加直观和易于理解，更加容易简化问题的处理逻辑，所以相对而言，面向对象的语言简化了使用该语言解决实际问题的复杂度，提高了处理事情的效率，所以程序员更容易理解、学习、掌握和使用。

而Python语言本身的设计也是基于对象的，也是面向对象的各种概念和逻辑的，所以说Python也是一种面向对象的编程语言。

有了前面介绍的对于不同Python称谓的理解之后，我们再来总结一下作为计算机编程语言的Python的常见且基本的定义：Python是一种面向对象的、解释型的计算机高级语言。

至此可能很多读者才突然明白：哦，原来在各种资料或教程中所使用的Python的上述定义背后，还隐含着如此多方面的含义，原来Python的各种特点和称谓是这么回事。

如果读者会有此感悟，则才真正实现了笔者写本Python教程要实现的目的：希望学习技术的读者，不仅要知其然，也要知其所以然。将此逻辑应用到此处的含义就是：不仅要知道Python是如此定义的，更要知道为何会有这样的定义，即Python如此定义背后的真正完整的含义。

1.3 Python的特点

1.3.1 作为脚本语言的优缺点

上面已经介绍过，Python是一门脚本语言，也是一门解释型语言。下面就来简单解释一下作为解释型语言的Python有哪些特点。

1.作为脚本语言的Python的优点

· 快速开发：不需要编译即可运行

正如前面的解释，写完Python脚本后直接就可以运行而省去编译的步骤，使用起来相对省事和高效。

2.作为脚本语言的Python的缺点

· 性能相对不是特别强

Python的性能相对一些其他语言（比如C、C++等）来说不是特别强。对于性能要求比较苛刻的某些领域不太适合全部使用Python去实现所有的功能。现在已有的解决方法是：使用相对Python性能更好的其他语言去实现与性能相关的最核心部分的功能，然后再将此部分整合到Python内，这样即可高效地实现程序的功能，又可以很大程度上提高开发的效率。

为何不说是Python性能比较弱

此处之所以说是Python的性能不是特别强，那是相对于更加底层的语言（比如C、C++等）来说的，并不是说Python的性能本身很差。相对而言，C、C++等语言由于是编译型语言并且更加面向底层，所以在性能方面与Python等脚本语言相比，的确会更好一些。但是在现实中实现功能时，很多情况下并不那么关注代码本身的性能高低，而是更加关注完成任务的总体效率。

下面通过一个例子来解释上面这句话的含义。

假如想要写一段代码用于实现一个功能，而下面是所需要花费的时间。

（1）写代码实现功能所花费的时间

· 写C或C++代码用了2天时间实现了功能。

· 用Python写代码实现同样的功能只需半天时间。

（2）代码运行所花费的时间

· C或C++的性能比Python好20%，运行程序实现功能花费了10秒。

· Python代码效率稍差，比C或C++差20%，但运行程序实现功能也只是花费了12秒左右。

（3）完成总体任务所花费的时间

· 总体上来说使用C或C++完成总体任务花费2天多时间。

· 总体上来说使用Python完成总体任务只花费半天左右时间。

如果要实现同样的功能，很多用户其实更加关注整体效率。因此，在使用Python可以保持总体效率不太差但可以用更少的时间的前提下，用户肯定会选择Python。此时用户总体感觉就是：还是Python更好用。

除了上述的例子，现实中还有很多类似的情况：80%的用户只是为了实现某些功能，对于相对较小的性能差异并不是很关心，此时我们更倾向于采用Python实现程序的主体功能；而余下的对性能要求比较高的20%的情况则往往是采用C或C++等性能更好的语言去实现。

此外，Python语言本身支持将其他语言（包括C、C++等）通过相关的框架和手段集成到Python中，所以在这种情况下也是可以用Python作为主要语言去实现所需的总体功能的。

总结:

Python的性能只是相对弱一些但不差，而且多数情况下都能够满足需求，即多数情况下都可以选择用Python去实现自己需要的功能。

当Python性能不能满足要求时，也可以通过采用其他性能更好的如C、C++等语言去实现性能相关部分的功能，然后再集成到Python代码中，这样保持了Python仍然是实现功能的主要语言。

1.3 Python的特点

1.3.1 作为脚本语言的优缺点

上面已经介绍过，Python是一门脚本语言，也是一门解释型语言。下面就来简单解释一下作为解释型语言的Python有哪些特点。

1.作为脚本语言的Python的优点

· 快速开发：不需要编译即可运行

正如前面的解释，写完Python脚本后直接就可以运行而省去编译的步骤，使用起来相对省事和高效。

2.作为脚本语言的Python的缺点

· 性能相对不是特别强

Python的性能相对一些其他语言（比如C、C++等）来说不是特别强。对于性能要求比较苛刻的某些领域不太适合全部使用Python去实现所有的功能。现在已有的解决方法是：使用相对Python性能更好的其他语言去实现与性能相关的最核心部分的功能，然后再将此部分整合到Python内，这样即可高效地实现程序的功能，又可以很大程度上提高开发的效率。

为何不说是Python性能比较弱

此处之所以说是Python的性能不是特别强，那是相对于更加底层的语言（比如C、C++等）来说的，并不是说Python的性能本身很差。相对而言，C、C++等语言由于是编译型语言并且更加面向底层，所以在性能方面与Python等脚本语言相比，的确会更好一些。但是在现实中实现功能时，很多情况下并不那么关注代码本身的性能高低，而是更加关注完成任务的总体效率。

下面通过一个例子来解释上面这句话的含义。

假如想要写一段代码用于实现一个功能，而下面是所需要花费的时间。

（1）写代码实现功能所花费的时间

· 写C或C++代码用了2天时间实现了功能。

· 用Python写代码实现同样的功能只需半天时间。

（2）代码运行所花费的时间

· C或C++的性能比Python好20%，运行程序实现功能花费了10秒。

· Python代码效率稍差，比C或C++差20%，但运行程序实现功能也只是花费了12秒左右。

（3）完成总体任务所花费的时间

· 总体上来说使用C或C++完成总体任务花费2天多时间。

· 总体上来说使用Python完成总体任务只花费半天左右时间。

如果要实现同样的功能，很多用户其实更加关注整体效率。因此，在使用Python可以保持总体效率不太差但可以用更少的时间的前提下，用户肯定会选择Python。此时用户总体感觉就是：还是Python更好用。

除了上述的例子，现实中还有很多类似的情况：80%的用户只是为了实现某些功能，对于相对较小的性能差异并不是很关心，此时我们更倾向于采用Python实现程序的主体功能；而余下的对性能要求比较高的20%的情况则往往是采用C或C++等性能更好的语言去实现。

此外，Python语言本身支持将其他语言（包括C、C++等）通过相关的框架和手段集成到Python中，所以在这种情况下也是可以用Python作为主要语言去实现所需的总体功能的。

总结:

Python的性能只是相对弱一些但不差，而且多数情况下都能够满足需求，即多数情况下都可以选择用Python去实现自己需要的功能。

当Python性能不能满足要求时，也可以通过采用其他性能更好的如C、C++等语言去实现性能相关部分的功能，然后再集成到Python代码中，这样保持了Python仍然是实现功能的主要语言。

1.3.2 Python自身的特点

除了作为解释型语言的Python自带的优缺点之外，Python语言本身也有其自己的特点。此处讨论Python特点时，也是和评价其他事物一样去和其同类语言对比。

Python相对其他语言一个最大的特点是：代码缩进决定了代码的逻辑关系，而不仅仅是为了代码的美观性。

下面介绍更多的其他特点：

1.很多语言内置特性使得Python语言本身很方便编程者使用

Python中的字符串可以直接通过数组下标去获得对应子字符串。这类特性是其他很多语言本身所不具有的。Python的此类特点在第4章中会有更加详细的解释。

2.一种被广泛采用的胶水语言

Python相对其他语言有很强的粘合能力，其主要指的是Python本身支持很多框架和接口，它允许在Python和其他很多语言之间相互调用。如为了提高性能等，把与性能相关的部分用性能相对更好的其他语言实现，然后再整合到Python中，起到了作为不同语言的“粘合剂”的作用，以此达到提高Python程序整体性能的目的。

3.功能丰富的自带库以及种类和数量繁多且强大的第三方库

除了Python语言本身自带的标准库功能就很丰富、很强大之外，目前还有数量巨大的、各种领域的、功能强大的第三方库函数。

因为Python语言易于使用，所以有越来越多的用户愿意使用和发展Python，很多人把在Python开发期间积累的库函数贡献了出来，如此积累至今，就形成了数量众多且功能强大的第三方库函数。

如上特点对于Python语言的使用者来说有着显而易见的好处：当实现很多功能时，往往不需要重复“造轮子”，而可以找到现成的内置Python库或第三方的Python库去帮助我们实现想要的功能，从而节省我们大量的精力。换句话说，在实现一个复杂的功能时，往往变成了找到合适的库并使用即可，而无需从头到尾去实现一个很复杂的功能，从而节省了很多宝贵的时间和精力。而与之对应的是，当我们用其他语言实现同样的功能时，由于缺少好用的库而需要我们从无到有全新地去实现对应功能所需的代码。由此即可对比出Python更加好用与强大。

例1.1　Python库种类繁多和功能强大的好处

此处用制造汽车来类比解释Python的好处。

假如你需要制造一台汽车，如果使用Python去实现的话，你会发现，制造汽车所需要的核心部件，比如发动机、底盘、车架等，都已经有了，即可以直接从Python内置的相关功能和第三方库中找到实现对应功能的模块。并且实现同一种功能，比如发动机，还有多个模块可供选择，而你要做的只是稍微了解每个模块各自的优缺点，然后组合和校准之后很快就可以弄出一台汽车了。

与此对应的是，如果用其他语言去实现同样的功能，则很可能遇到的情况是：如发动机等核心部件没有现成的可用的模块，你要自己手动去打造，需要自己去实现对应的代码，或者即使有现成的但功能也不够强大，又或者选择余地不够多而没有找到让你满意的，最终还是需要你额外花很多精力去实现。而除了需要自己搞定发动机等核心功能外，还有很多其他功能也需要自己去实现。

这样，用其他语言从无到有去制造一台汽车的话，可能少则几周多则几个月。而用Python这种好用的语言的话，很可能是几天或者一两周就可以搞定了。

这就是我们所说的Python语言功能强大的自带库以及种类和数量繁多的第三方库，综合而言就是Python很强大很好用！

4.适合原型开发

原型开发指的是当有了一个概念或算法之类的东西，为了验证它的正确性或可行性而去找合适的语言快速实现出来。由于Python是脚本语言，且其本身功能丰富，所以特别适用于原型开发。

5.跨平台和可移植性很好

需要首先强调的是，所谓的可移植性其实也是相对的概念，比如Java也是个可移植性很好的语言。

简单地说，可移植性就是对于同样的代码可以不经过改动直接拿到其他平台使用，并可以实现同样的功能。与此相对的其他很多语言，比如C等，更换平台后则很可能还需要改动部分代码之后程序才可以正常运行，才可以实现同样的功能。有些语言更换平台后甚至都无法直接运行。比如C#的代码只能在Windows平台上运行，换个其他系统，比如Linux，基本上就没法运行没法使用了。

而Python的可移植性是因为不同平台中安装了Python环境，有对应的Python解释器，对于Python代码来说又无需编译，只要是个普通的文本文件就可以直接运行了。其内部实现过程是，Python解析器会把代码解析成可以在当前平台下运行的代码并运行。

正是由于Python语言具有如此多的优点，所以才有越来越多的人去认识和学习Python，实现Python更广泛的应用。

1.4 Python的应用

了解了Python是什么以及它的众多特点后，读者很自然地就会想到一个问题：Python能干什么?

而此处普通的这一问句“Python能干什么”的背后其实有着更深层的含义。严格地说至少包含如下三层含义。

1.Python能干什么

从语言本身的层面来说，Python这门编程语言可以实现哪些功能以及不能实现哪些功能。

2.Python更适合做些什么

作为其中一种编程语言，Python语言根据其语言的特点和优势更适合做哪些事情。

3.你能用Python干什么

即使Python可以做很多事情，但是由于每个人对该语言掌握的程度有差异，使得不同的人有不同的实现的能力，所以这句话还有这层含义：作为Python的学习者，尤其是初学者，你自己能用Python来实现什么。

下面就针对这三个层次的含义来分别解释一下Python能干什么。

1.4.1 Python能干什么

由于Python本身是一门高级语言且功能比较强大，所以，对于Python语言本身可以做什么，简单地说就是：可以做其他高级语言做的事情。这可能听起来没有什么实际意义，但是此句话本质上至少也包含如下两层含义。

1.可以像其他高级语言一样用来开发各种不同功能的软件

对于其他高级语言，笔者目前接触过的一些如C#、Java等都可以用来实现各种常见的功能，包括但不限于网络方面的应用，比如Web服务器、普通桌面软件等。事实上几乎任何一个常见且功能足够用的高级语言都可以实现，Python也不例外。唯一的区别在于不同语言的特点使得实现不同功能的复杂度和难易程度等不同。也就是说，Python作为一种高级语言也是可以用来实现各种常见的功能的。

2.和其他高级语言一样不能直接操作底层硬件

但是很明显，作为高级语言的Python不能像汇编语言那样直接去操作底层的硬件，尤其是那些汇编语言可以直接操作的寄存器级别的硬件。同时，作为更高层抽象的Python，和C#、Java等其他高级语言一样，在实际使用过程中也很难直接通过本语言去操作那些底层硬件资源。与此相对的是，C语言虽然也是高级语言，但因C语言的自身特性，它可以直接操作很多硬件层面的资源。也就是说，Python不能直接操作底层硬件资源。

1.4.2 Python更适合做些什么

介绍了Python语言本身可以做什么之后，接着就是对于普通的程序员、Python的使用者来说，在实际使用中更多的或者说更倾向于拿Python来做些什么呢？其主流应用领域和实例如表1-1所示。

表1-1 Python的主流应用领域及实例

Python 的应用领域	子应用领域	应用实例
网络和以太网开发	网络框架	Django、TurboGears、Pylons、异步网络编程框架 Twisted、flask、Webpy、Bottle
	内容管理系统	Plone、Mezzanine
	应用服务器	Zope
	Python 网站	❑ 社交分享网站：Reddit ❑ 文件分享服务：Dropbox ❑ 图书、唱片、电影等文化产品的资料数据库网站：豆瓣

（续）

Python 的应用领域	子应用领域	应用实例
网络和以太网开发	Python 网站	❑ MediaWiki 的机器人程序：Python Wikipedia Robot Framework ❑ Wiki 程序：MoinMoinWiki ❑ 图片社交分享网站：Pinterest ❑ 幻灯片存储、展示、分享的网站：SlideShare ❑ 其他各种网站：知乎、果壳网、美团、快盘、Wopus 问答、扇贝网、糗事百科等
	其他	❑ 编写 CGI 脚本 ❑ 支持 HTML 和 XML 的处理 ❑ 邮件处理，邮件列表：Mailman ❑ RSS 处理 ❑ 支持很多网络协议
数据库访问	自定义和 ODBC 数据库接口	MySQL、Oracle、MS SQL Server、PostgreSQL、SybODBC 等
	标准数据库接口 API	
	对象数据库	ZODB、Durus
图形桌面	图形桌面	❑ 内置 Tk 的 GUI 库 ❑ wxWidgets ❑ GTK+ ❑ 通过 pyqt 或 pyside 支持 QT ❑ 通过 Win32 扩展支持 MFC ❑ Delphi
科学计算	科学计算	❑ Bioinformatics ❑ Physics
教育教学	教育教学	❑ Education Special Interest Group ❑ pyBiblio ❑ Software Carpentry Course
软件开发	编译控制	SCons
	自动化连续编译和测试	Buildbot、Apache Gump
	缺陷追踪和项目管理	Roundup、Trac
游戏和 3D 图形处理	游戏框架	PyGame、PyKyra 等
	3D 渲染库	FreeCAD、Minilight
	3D 软件	以 C 与 Python 开发的开源 3D 绘图软件：Blender、Cinema 4D
	Python 游戏库	PySoy、Pyglet、Python-Ogre、Panda3d、Blender3D、GGZ Gaming Zone 等
	Python 游戏	EVE、Battle Field 2 and 2142 Battlefield Heroes、Mount&Blade、SolarWolf 等
	Python 在线游戏服务器	Evennia、GrailMUD

由表1-1可见，作为高级语言的Python可以用在很多领域去实现各种各样的功能。

1.4.3 你能用Python干什么

虽然理论上可以用Python实现前面所述的各种功能，但是实际上针对每个人来说结果并不太一样。不是每个人都真的能够用Python来实现上述如此多的应用，因为很多应用还是需要足够的软件背景知识，甚至需要足够高的智力才能搞定的。

比如，将Python应用于3D图形开发，事实上它的难度和工作量并不在于写Python代码，而在于要懂得相关的实现3D效果所涉及的算法，即如何实现对应3D图形背后所涉及的原理和机制。只有搞懂如何实现后才有可能用Python将代码写出来。另外，对于其他很多方面的应用，包括最常见的Python在网络方面的应用，也是类似的道理。

比如笔者之前所接触过的实现网络爬虫、从网页中抓取数据、模拟网页登录等内容，它们背后涉及的相关基础背景知识包括但不限于HTTP的GET和POST操作、Cookie、JSON、字符编码、HTML、JavaScript、Python中专门解析HTML的库BeautifulSoup等。只有搞清楚了这些背景知识和相关技术，分析了所要处理的网页的内部执行逻辑，然后才是用某种语言模拟实现这套逻辑。而实际上搞懂写代码之前所需要了解的逻辑，往往比知道逻辑后去用代码写出来更难。

可以这么说，如果是打算用Python去实现网络爬虫，那往往会是要花费80%的精力去搞懂网络爬虫背后的知识和逻辑，而剩下的20%的工作量才会涉及写Python代码，即利用相关的网络方面的库以及Python的第三方库去实现对应的功能。

换句话说，大多数情况下实现某种功能时，如果你不懂80%的背景知识和逻辑的话，只会写那20%工作量所对应的Python代码，也是无法实现你的目标的。

这就是所谓的“你要清楚你自己能用Python干什么”。对于实现某些方面的功能或应用需要哪些基本的背景知识，以及掌握了背景知识后如何写对应的Python代码，很明显的是不同领域所需要的背景知识是不同的。因此作为Python基础教程，本节不适合也没办法解释清楚所有领域的基础知识。

至此，你应该对于Python可以做的事、适合做的事以及自己能用Python做什么有个基本的了解了，接下来就是循序渐进地学习Python的基本知识了。

1.5 Python的必备常识

有一些和Python相关的常识性的知识，即使是Python初学者也需要先了解清楚。下面就来介绍这些内容。

1.5.1 Python文件的后缀

在计算机的世界里，多数文件的类型都是通过文件的后缀来区分的。Python文件的后缀是.py。换句话说，我们见到的Python代码文件的文件名就是类似于xxx.py的。

1.5.2 Python的缩写和简称

正是由于Python文件的后缀是.py，所以很多人也常用py来指代Python。比如后文中将会看到的，有些人把Python 2简写为

py2。

1.5.3 Python的官网

Python的官方网站是http://www.python.org/。

此处之所以介绍一下Python的官网，主要是提醒用心的读者：Python的官网中其实有很丰富的学习资料。

对于英文不是很好的中文用户，可能会遇到看英文吃力的问题。但是对于英文还不错的，建议多去官网上看看，多找找相关资料来参考和学习。上面一是会有最新的更新，二是内容权威，更适合学习和掌握Python。

1.5.4 Python的Logo

在学习Python期间你常会看到一个如图1-1所示的图标。

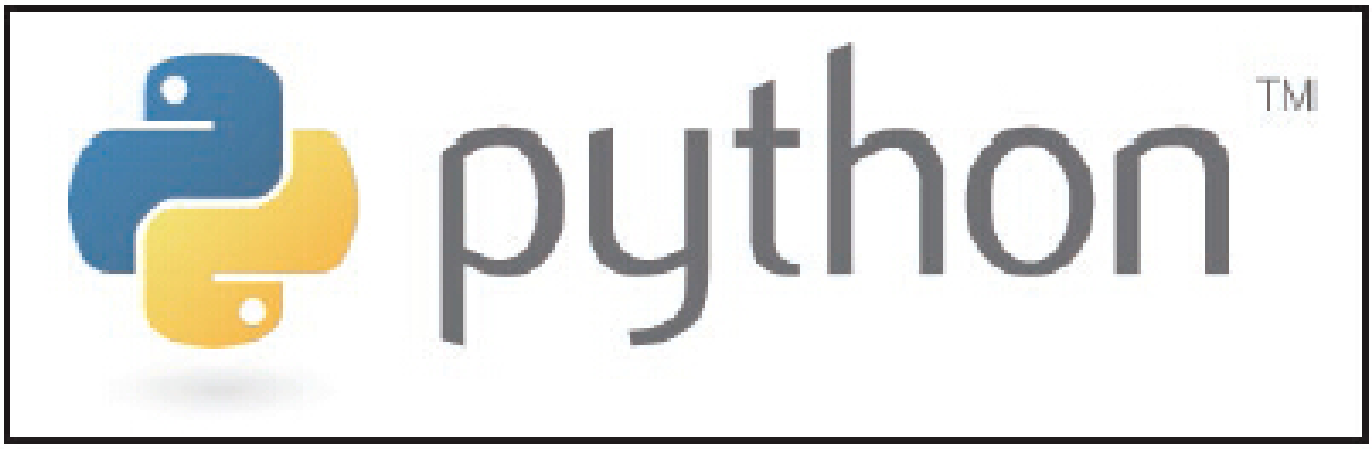


图1-1 Python的Logo

图1-1的图案就是经典的Python图标，可以看到是两种颜色（蓝黄）的蟒蛇交织在一起的效果。Python官网中有对此Logo的详细解释，感兴趣的读者可以深入了解一下，参考网址：http://www.python.org/community/logos/。

第2章 下载并安装Python

2.1 因Python版本不合适而导致的常见问题

对于下载Python这件事，如果Python没有多种版本，那么只需简单下载一个Python即可。但是实际上由于Python不仅仅有多个不同的版本，而且不同系统环境需要用不同版本的Python，这使得在实际中很多人对“下载一个合适版本的Python”感到无从下手。因此此处需要详细解释一下关于Python版本选择方面的背景知识，并给出对于初学者选择版本的建议，否则很可能会因

为最初Python版本选择的不合适而导致在后续学习和使用Python时遇到一些问题和错误。

下面总结一下由于版本选择不合适而导致的一些常见问题和错误。

1.语法错误

在安装了Python 3版本后却参考他人Python 2的代码从而导致出现语法错误。其中最常见的经典的例子是因print的语法不同而导致出现错误。

很多初学者在还没搞清楚Python的两大版本，即Python 2和Python 3的区别的背景下就安装了最新版本的Python 3，然后去参考，也是最常见的，别人提供的Python 2的代码，而遇到的第一行代码往往就是调用print这个函数去打印信息。但是由于Python 2和Python 3print的语法不同，所以导致了第一次学习和使用Python就出现print语法错误的问题。

相关的示例代码和错误详情请参见2.2.2节中例2.1中的详细解释。

2.无法使用

x64的系统中安装了x86的库而导致后续无法使用。笔者之前就曾遇到过一次这种问题：在x64的Windows 7中安装了x86的某个库而导致无法正常调用该库函数。所以才有必要解释清楚这方面的区别，以便于初学者选择适合自己的Python版本。

3.影响难易度

选择不同版本的Python会导致后面学习过程难易度不同。由于Python有两种不同的版本Python 2和Python 3，而对于Python版本的选择是2还是3，不仅仅是语法上的一些区别，对于初学者来说，更多的还可能影响到“可供用户参考的示例代码”和“可供用户利用的第三方库函数”等的数量和质量的不同。从学习和使用Python的角度来说，目前还是选择Python 2更合适。关于此部分的更详细的解释，详见2.3.1节。

当然，对于Python 2和Python 3的区别，除了此处提到的一些错误之外还可能导致一些其他问题，在此就不一一细说了。

因此，为了避免今后可能出现的因版本选择不合适而导致的问题，以及更方便地找到合适的参考资料和可使用的库，接下来应该搞清楚两个Python版本的区别，然后再去选择一个适合自己的Python版本。

第2章　下载并安装Python

2.1　因Python版本不合适而导致的常见问题

对于下载Python这件事，如果Python没有多种版本，那么只需简单下载一个Python即可。但是实际上由于Python不仅仅有多个不同的版本，而且不同系统环境需要用不同版本的Python，这使得在实际中很多人对“下载一个合适版本的Python”感到无从下手。因此此处需要详细解释一下关于Python版本选择方面的背景知识，并给出对于初学者选择版本的建议，否则很可能会因为最初Python版本选择的不合适而导致在后续学习和使用Python时遇到一些问题和错误。

下面总结一下由于版本选择不合适而导致的一些常见问题和错误。

1.语法错误

在安装了Python 3版本后却参考他人Python 2的代码从而导致出现语法错误。其中最常见的经典的例子是因print的语法不同而导致出现错误。

很多初学者在还没搞清楚Python的两大版本，即Python 2和Python 3的区别的背景下就安装了最新版本的Python 3，然后去参考，也是最常见的，别人提供的Python 2的代码，而遇到的第一行代码往往就是调用print这个函数去打印信息。但是由于Python 2和Python 3print的语法不同，所以导致了第一次学习和使用Python就出现print语法错误的问题。

相关的示例代码和错误详情请参见2.2.2节中例2.1中的详细解释。

2.无法使用

x64的系统中安装了x86的库而导致后续无法使用。笔者之前就曾遇到过一次这种问题：在x64的Windows 7中安装了x86的某个库而导致无法正常调用该库函数。所以才有必要解释清楚这方面的区别，以便于初学者选择适合自己的Python版本。

3.影响难易度

选择不同版本的Python会导致后面学习过程难易度不同。由于Python有两种不同的版本Python 2和Python 3，而对于Python版本的选择是2还是3，不仅仅是语法上的一些区别，对于初学者来说，更多的还可能影响到“可供用户参考的示例代码”和“可供用户利用的第三方库函数”等的数量和质量的不同。从学习和使用Python的角度来说，目前还是选择Python 2更合适。关于此部分的更详细的解释，详见2.3.1节。

当然，对于Python 2和Python 3的区别，除了此处提到的一些错误之外还可能导致一些其他问题，在此就不一一细说了。

因此，为了避免今后可能出现的因版本选择不合适而导致的问题，以及更方便地找到合适的参考资料和可使用的库，接下来应该搞清楚两个Python版本的区别，然后再去选择一个适合自己的Python版本。

2.2 Python的两大版本

目前Python主要有两个大的版本：

- Python 2
- Python 3

Python两大版本的不同写法

关于两种版本的对比，也常写成为

- Python 2.x VS Python 3.x
- Python 2VS Python 3
- py2VS py3

读者以后看到类似写法，明白其指的是Python 2和Python 3就可以了。

2.2.1 Python版本历史

为了更深入地理解Python 2和Python 3的区别，此处先简要介绍Python版本的发展历史。

Python从1989年诞生起到现在，按照主版本号的不同，主要有1、2和最新的3。下面介绍一下其中相对使用较广的几个版本。

1.Python 1.6

相对来说，最早的Python 1.x系列的版本中，使用得比较广泛的就是Python 1.6左右的版本了。

2.Python 2.7

截至笔者写稿的2014年，Python的2系列的版本中用得相对最广泛、最流行的版本要数Python 2.7.x系列的版本了。

很多操作系统和平台中，包括现在主流的Ubuntu，Mac OS等系统中预装的Python都是Python 2.7.x的版本。

3.Python 3.x

此处所说的Python 3.x主要指的是2014年前后，Python刚刚发展到的3系列的版本。从Python 2到Python 3是一个很大的变化。这些变化包括很多基本的元素、函数接口和定义、设计理念等方面。虽然，这些变化总的来说在向好的方向进化，但是由于Python 3的变化，导致和旧的Python 2不太兼容，而之前Python 2系列的版本使用得太广泛了，包括很多第三方的库函数都是Python 2版本的，从而导致对于第三方库函数来说，如果需要全部支持Python 3则需要很大的改动，涉及的代码工作量和测试工作很多，所以现在很多第三方库函数暂时没有支持Python 3，或者是对于Python 3的支持还不够稳定。

所以当前的结论是，虽然Python 3是以后的趋势，但是目前多数人还在继续使用Python 2。

2.2.2 Python 2和Python 3之间的区别

介绍了Python各版本的基本历史后，下面再来详细解释Python 2和Python 3的区别。

Python2和Python3在很多基本的、最常用的函数和接口方面都变了，所以也会导致一些问题。其中最典型的要数前面提到的print函数的语法错误了。

例2.1 Python 3中print出现语法错误

在安装了最新版本的Python 3后却参考并运行了他人Python 2的示例代码，其中有用到print函数，结果就会导致print函数的语法错误：“SyntaxError：invalid syntax”。

介绍一个真实的案例，之前遇到了某个Python初学者就出现了如图2-1所示的错误。

1.print函数的基本语法

为了更清楚地解释此问题，需要先解释一下print函数的基本语法：

· Python 2：print函数不带括号

· Python 3：print函数必须带括号

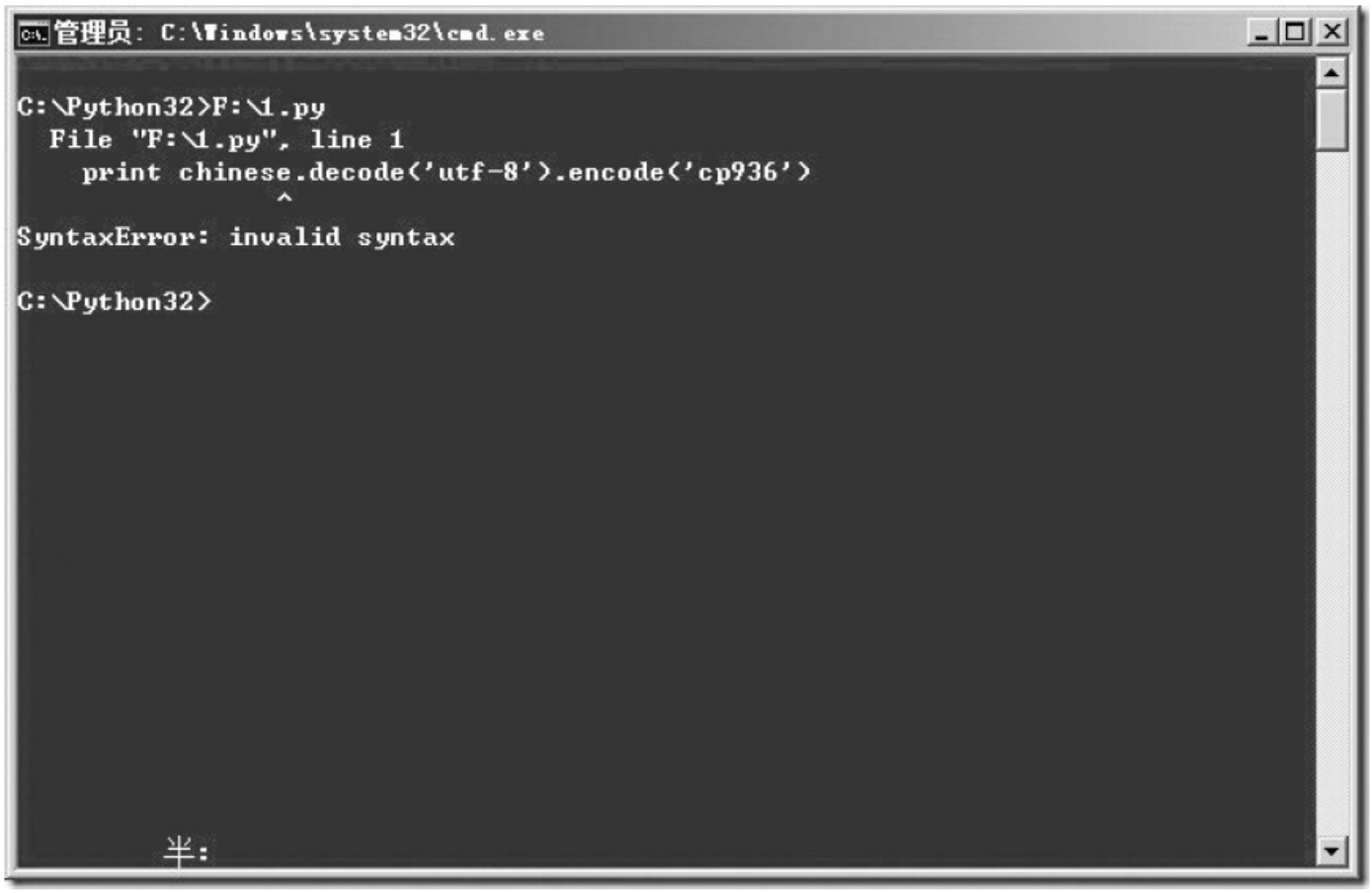


图2-1　Python 3中print出错

下面详细阐述：

（1）print中不带百分号格式化的

· python 2

```
print "Pyhon 2 can use print string without ()";
```

· python 3

```
print("Python3, print must use () to output string");
```

（2）print中带百分号格式化的

· python 2

```
print "old %s version is %d, print no ()"%("Python", 2);
```

· python 3

```
print("new %s version is %d, print must have ()"%("Python", 3));
```

2.官网中print的用法

关于更多的新旧版本中print的用法，此处再整理一些官网中的例子以供参考。

（1）末尾不换行

· python 2

```
print x,
```

· python 3

```
print(x, end=" ")
```

（2）输出空白行

· python 2

```
print
```

· python 3

```
print()
```

（3）重定向输出

· python 2

```
print >>sys.stderr, "fatal error"
```

· python 3

```
print("fatal error", file=sys.stderr)
```

（4）打印输出表达式

· python 2

```
print (x, y)
```

· python 3

```
print((x, y))
```

至此，读者对于Python 2和Python 3中的print的语法应该比较清楚了。

3.Python 3相比Python 2的其他变化

除了print之外，关于Python从2发展到3后其他的变化，虽然对于初学者来说暂时无需了解如此多的细节，不过为了知识的完

整性，以及为今后遇到的问题提供参考，所以还是有必要在此处一并整理出一些大的语法方面的变化。

（1）部分返回值用view和Iterator取代了list

· dict的有些方法不再返回list

以前dict的dict.keys()、dict.items()和dict.values()是返回list的，现在变为返回views。比如，以前用：

```
k = d.keys(); k.sort()
```

现在改用：

```
k = sorted(d)
```

· 去掉了dict的部分方法

去掉了dict的dict.iterkeys()、dict.iteritems()和dict.itervalues()。

· map()和filter()返回迭代器了

· range()变成以前的xrange()的效果了

所以去掉了xrange()。

· zip()返回迭代器了

（2）带排序的比较

· 简化了带排序的比较

· builtin.sorted()和list.sort()不再支持cmp作为参数，而改用key

· 取消了cmp()和__cmp__()，改用__lt__()，以及用带__hash__()的__eq__()实现更加复杂的比较

（3）整型变量

· long改名为int

· 类似于“1/2”这样的表达式返回float变量

· 去掉了sys.maxint

· repr()返回的值不再包含L后缀

· 八进制数值，不再是0720的形式，而变成0o720

（4）Unicode和8位字节的区别变为文本和数据的区别

· 从之前的Unicode字符串（unicode）和8位的字符串（str）变为文本（str）和二进制的数据（bytes）。

· 不能再用u"http://www.hzcourse.com/resource/readBook?path=/openresources/teach_ebook/uncompressed/15066/OEBPS/Text/..."，且对于二进制数据用b"http://www.hzcourse.com/resource/readBook?path=/openresources/teach_ebook/uncompressed/15066/OEBPS/Text/..."。

· 文本字符串和二进制数据之间不再支持混用，需要转换时，可以用对应的str.encode()和bytes.decode()，或者bytes（s，encoding=http://www.hzcourse.com/resource/readBook?path=/openresources/teach_ebook/uncompressed/15066/OEBPS/Text/...）和str（b，encoding=http://www.hzcourse.com/resource/readBook?path=/openresources/teach_ebook/uncompressed/15066/OEBPS/Text/...）。

· 反斜杠在原始字符串中就是字面意思，没有转义的含义。

举例说明：

1）Python 2.6中ur'\u20ac'表示单个的euro字符；

2）Python 3.0中r'\u20ac'表示对应的一个包含了6个字符的字符串。

· 去除了内置的抽象类型变量basestring而改用str。

· 打开文件后返回的是解码后的Unicode字符串，加上b前缀后返回的是字节数组，所以也无需再用之前的codecs模块了。

· sys.stdin、sys.stdout和sys.stderr的初始值从io.TextIOBase改为仅支持Unicode字符串，如果想要这些字符流的字节数据，可以使用io.TextIOBase.buffer。

· 文件名当做参数传递进入和返回时也都是Unicode字符串。

· 部分系统相关的API，比如os.environ和sys.argv，当系统用默认编码无法解析相关的字节时，可能会存在一些问题。

· 字符串的repr()不再返回转义后的非ASCII字符，但仍会转义那些控制字符和非打印字符。

· 默认字符串编码为UTF-8。

· 非ASCII字符不允许用于标识符。

· 去掉了StringIO和cStringIO模块，改为先导入io模块后再引用对应的io.StringIO或io.BytesIO。

更多细节读者可以参考官网进一步了解，地址为：http://docs.python.org/3/whatsnew/3.0.html#overview-of-syntax-changes。

2.3 如何选择合适的版本

选择合适的Python版本主要包含如下两方面含义：

· 选择Python 2还是Python 3

· 选择32位（x86）还是64位（x64）

下面来详细介绍它们的具体含义。

2.3.1 选择Python 2还是Python 3

通过前面的介绍，读者已经清楚Python 2和Python 3的区别了。但是作为普通Python使用者来说，到底是选择Python 2还是Python 3呢？在给出笔者的建议之前，此处先总结一下它们各自的优缺点，如表2-1所示。

表2-1 总结对比Python 2和Python 3各自的优劣

对比的内容	Python 2	Python 3	对比后的结论
版本比较	相对属于旧版本	版本更新	❑ Python 2 迟早要被淘汰 ❑ Python 3 是未来的趋势
使用程度	非常广泛	不那么广泛	❑ Python 2 是当今的主流 ❑ Python 3 暂时使用得不够广泛
现存教程和资料丰富程度	很丰富	不太丰富	❑ 学习 Python 2 期间可参考的教程和资料都很多 ❑ 学习 Python 3 期间万一遇到问题，可能会找不到合适的参考资料
现有第三方库的支持程度	绝大多数都支持	部分支持	❑ 使用 Python 2 期间可以“拿来主义”的库很多，并且稳定、好用 ❑ 使用 Python 3 期间想要借用第三方库可能出现其不支持 Python 3 的情况，或虽然支持却不够稳定的情况
学习上手难度	相对简单	相对简单	❑ 从 Python 2 开始和直接从 Python 3 开始，学习 Python 的难度相差不多。可以先学习 Python 2，今后必要时再转到 Python 3 ❑ 直接学习 Python 3，在学习难度差不多的前提下，可以免去从 Python 2 转到 Python 3 的过程

从表2-1的对比和结论中，很明显可以得到如下结论：

推荐Python初学者先用Python 2，待熟悉Python 2之后再换用Python 3。

因为先以Python 2上手开始学习Python，既可以在和Python 3的学习难度相差不大的基础上，同样快速学习和掌握Python；也可以在后续使用Python期间，在找寻学习参考资料和问题解决办法时，找到更多相关资料。而对于从Python 2转换到Python 3，则完全可以在对Python有了深入的了解之后，在了解了Python 2和Python 3的语法和概念上的区别后，就可以很顺畅地从Python 2转换到Python 3了。

2.3.2 选择Python是32位还是64位

在初学者选择了Python 2之后，还要搞清楚一个在体系架构方面的区别，即当前系统是x86还是x64，即选择32位的Python还是选择64位的Python。

什么是32位（x86）和64位（x64）以及如何才能发挥64位的优势

首先明确基本概念：

· 32位==x86

· 64位==x64

其次要知道对于x86和x64这个概念包含如下两层含义。

1.硬件上的x86和x64

此处的x86或是x64主要指的是硬件上的，尤其是CPU架构方面，是32位还是64位。它决定了当前的硬件是否可以支持64位。而64位较之于32位来说，性能会有很大的提升。现在主流的CPU多数都已经是支持64位的了。

2.软件上的x86和x64

想要发挥64位的更好的性能优势，除了硬件要支持64位之外，还需要对应的软件层面的支持。和64位相关的软件层面主要涉及如下两方面：

（1）操作系统是x86还是x64

我们所安装的操作系统，尤其是Windows系统中最常见的Windows 7，多数都是32位的。往往部分电脑的内存大于4G的用户才会安装64位的Windows 7，这对应着读者之前可能听到过的叫法：

· x64的Windows 7

· 64位的Windows 7

（2）应用程序是否支持x64

除了操作系统要满足是64位的之外，还需要在64位的操作系统上的软件应用程序也支持64位。而对于应用程序是否支持64位则取决于开发该程序的作者是否加入了64位的支持。另外还要确保你自己下载和安装的是否是对应的64位的版本，而不是下载和安装了32位的版本。

简言之，需要满足下述条件：

· 硬件上支持64位：目前多数CPU已支持。

· 软件上也支持64位：目前只有部分用户安装的Windows 7是64位的，且只有部分用户安装的软件是支持64位的。

最终才能发挥64位的性能。

解释完什么是32位和64位，以及如何才能发挥64位的优势之后，再来解释如何选择Python的版本。

此处要清楚我们的目标是知其然也要知其所以然，即不仅仅要知道应该选什么版本的Python，而且也要知道为何要选这个版本。

（1）如果你安装了32位的操作系统

假如你安装了32位的Windows 7，那么你的选择很简单，因为你无法在32位的Windows 7系统中去安装64位的Python，所以也只能安装对应的普通的32位Python。

（2）如果你安装了64位的操作系统

虽然你可以在64位的系统中去安装32位的Python，但是发挥不了64位的性能优势，所以为了更好的性能应该去安装对应的64位的Python。

但是，由于现实情况中很多第三方的Python库对于64位的支持不够好，所以从初学者的角度来说，为了减少可能存在的32位和64位之间版本兼容性问题，还是建议Python初学者去安装32位的Python。

所以最后的结论其实很简单：对于Python初学者，建议安装32位的Python。

2.4 常见软件的发布格式

至此，我们清楚了Python的版本选择：建议初学者选择32位的Python 2且是32位的Python。接下来，很自然所能想到的就是去合适的地方找到所需的版本再去下载即可。

不过在下载之前还要搞清楚一些关于软件发布格式方面的基本概念：在计算机领域内的软件方面，尤其是软件中的开源软件，在一款新的软件发布时往往有多种形式发布出来供用户下载。并且对于这些不同形式的可下载的东西，往往又提供了多种不同的下载源。只有对于这方面的内容有所了解，才能更好地在后续下载Python时明确和选择最合适的东西去下载。下面就来简要介绍一下这方面的基本概念。

2.4.1 源码格式

很多软件，尤其是开源软件，它们发布的策略中包含提供软件的源代码，这种做法有很多好处，包括但不限于：

· 用户可以提交已知的问题（bug），以便软件作者得知软件存在的相应问题，而后再去修复问题。甚至有些时候会出现提交bug的用户自己去解决该问题，并且公布和提交了自己的解决办法。

· 用户在自己写代码增加了新功能后，可以主动提交自己新增加的功能对应的代码。

很明显，上述的做法可以促进软件本身的发展，提高软件的质量，增强软件功能。

不过好处的另一方面也存在一些相对的缺点：对于软件普通的使用者，尤其那些专业知识水平相对不足，更加关注软件的使用而不关心软件开发的人来说，这些源码基本上是没帮助的，也是无需关心的。目前国内的这类用户比较多，所以此时只需要搞懂如何使用软件即可。虽然不需关心软件源码，要明白很多软件是有发布源码的，要知道有这回事。

下面简要介绍一下常见的源码的下载格式。

（1）.tar

把一个或多个文件的源码打包、集成到一起，成为一个单独的文件。

把tar解开为源码的方式有：

· Linux下用tar

常见的用法：

```
tar xvf xxx.tar
```

· Windows下用WinRAR或7-zip等压缩软件

用鼠标右击.tar文件，选择解压即可。

（2）.gz

常见的后缀是.tar.gz。表示在打包的基础上，又进一步去压缩而使得文件变得更小，更省空间。

将gz解开为源码的方式有：

· Linux下用tar

常见的用法：

```
tar xvzf xxx.tar
```

· Windows下用WinRar或7-zip等压缩软件

用鼠标右击.gz文件，选择解压即可。

（3）.zip

将源码压缩为单个文件。

将zip解开为源码的方式有：

· Linux下用unzip

常见的用法：

```
unzip xxx.zip
```

· Windows下用WinRar或7-zip等压缩软件

用鼠标右击.zip文件，选择解压即可。

（4）7z

由于7z的格式比zip格式压缩效果更好，所以很多源码也通过7z格式提供。

将7z解开为源码的方式有：

· Windows下用7-zip

用鼠标右击7z文件，选择解压即可。

（5）.tar.xz

.tar.xz为新出现的一种压缩格式，压缩效果比zip好。

2.4.2 二进制格式

与软件的源码相对应的是经过软件源码所编译后的那些可以直接拿来安装和运行的文件，即此处将要详细解释的二进制格式的软件。再具体点说就是，上述所提到的软件的源码，在某些平台和环境下，用对应的编译器去编译出来的，可以运行的二进制版本

的该软件。对于二进制版本的软件，其发布格式往往也有很多种。现在就来详细解释不同形式的二进制版本的含义。

1.二进制格式：免安装版和安装版

根据二进制文件是否需要安装又可以分为免安装版和安装版。

（1）免安装版

免安装即无需安装，这往往意味着是可移植、可移动的，所以免安装版本往往也被叫作：

可移植版本==可移动版本==portable version

它们一般都是以压缩包的格式提供，常见的压缩包的格式为：zip和7z。

将其下载后再去解压即可得到对应的文件夹，其中就可包含可执行文件，正常双击去运行程序即可而无需安装。

（2）安装版

安装版即所谓的安装包，就是去下载对应的安装版的程序。

· Windows中一般是exe或msi格式。下载对应的exe或msi文件，然后双击安装即可。

· Linux中不同发行版的格式不同。Debian类的Linux系统，比如Ubuntu等，支持.deb格式的文件；RedHat之类的系统支持.rpm格式的文件；其他Linux系统各自有自己所支持的不同格式的文件。

2.二进制格式：官网版和第三方版

根据二进制文件的发布者、制作者不同，又可分为官方版和第三方版。

（1）官方版

官方版==官网版，此处主要指的是某软件的官网制作和发布的版本。

（2）第三方版

此处主要指的是相对于软件官方来说的某个第三方，由于某些原因或为了特定目标专门在官网版本的基础上或者是独立从源码编译，同时可能额外添加很多模块、功能而发布的不同的版本，目的是为了满足不同人的不同需要。

2.5 下载合适的Python安装包

在了解了上述软件发布的背景知识后，再来看看Python提供了哪些供读者下载的形式。

2.5.1 Python提供了哪些形式

搞懂了上述软件发布的各种格式及含义，此处再来看看Python提供了哪些形式，以及我们需要选择哪种形式下载。

在Python官网（http://www.pythonorg/）中我们可以看到Python提供了很多不同的形式，同时来分析一下我们需要哪种形式。

1.源码形式

作为目前只关注Python软件使用的Python初学者，此处我们暂不关心Python的源代码。不过其中一点需要提示的是：Linux系统中是可以下载Python源码，然后自己编译和安装的。另外，实际上很多Linux的发行版系统中已经提供了更加方便的方式去安装所需要的软件，比如Ubuntu中的apt，它可以自己帮助用户下载和安装好对应的Python。所以，此处对于初学者来说，暂时可以无需关心如何在Linux中下载源码去安装。关于Ubuntu中如何用apt去安装Python，详见2.7.1节。

2.二进制形式

根据制作者和发布者的不同，有官网的Python和第三方版本的Python。

（1）官网的Python

截至目前，官网版还提供了如下两个版本：

· Python 3.3.3

· Python 2.7.6

根据我们之前的选择判断，此处选择Python 2系列的版本对应着的就是Python 2.7.6。

官网版还提供了针对不同平台（操作系统）的二进制安装包。

1）针对Windows操作系统的二进制安装包。

此处以相对比较常见的Windows平台为例来说明如何选择Python。

为了实现更好的版本兼容性，所以优先选择32位而不是64位的安装包。为了方便读者上手学习，所以选择示例和文档更丰富的Python 2而不是Python 3。为了更容易找到参考代码，所以选择目前最流行、使用最广的Python 2.7系列的版本，而目前最新的Python 2.7系列的版本是2.7.6。

结论：

对于Python初学者，选择对应的Python 2.7.6的Windows版本的那个32位的安装包，即Python 2.7.6Windows Installer，其下载地址是：http://www.python.org/ftp/python/2.7.6/python-2.7.6.msi。

如此才算真正搞清楚：为何选择以及如何选择你所需要的Python的版本。

2）针对Linux操作系统的二进制安装包。

Linux系统中已经自带一个（相对来说稍微有点老的）版本的Python了，对于Python初学者来说足够使用了，从而无需另外安装更新版本的Python。当然如果非要安装最新版本的Python的话，也是可以的。一般的做法是：去Linux中下载对应的Python源码包，然后在Linux中自己去编译并安装。

只不过如前所述，类似于Ubuntu等Linux发行版中已经带了方便的软件管理工具apt，它可以帮用户下载最新版，所以想要最新版的Python其实更方便的做法是通过apt等工具去自动下载和安装Python。

结论：

对于Python初学者来说，Linux系统中无需安装更新版本的Python，使用自带的Python就已够用。

3）针对Mac操作系统的二进制安装包。

Mac系统已经内置了Python，可以满足初学者的使用需求，所以对初学者来说无需再下载了。

而对于其他非要下载的读者来说可以参考上述的版本选择去下载对应的Python 2.7.6的Mac版本的安装包。

· Mac OS X 10.6

Python 2.7.6Mac OS X 64-bit/32-bit x86-64/i386Installer的下载地址是：http://www.python.org/ftp/python/2.7.6/python-2.7.6-macosx10.6.dmg。

· Mac OS X 10.3

Python 2.7.6Mac OS X 32-bit i386/PPC Installer的下载地址是：http://www.python.org/ftp/python/2.7.6/python-2.7.6-macosx10.3.dmg。

结论：

使用Mac系统的Python初学者也无需额外下载，而使用内置的Python即可。

（2）第三方版本的Python

此处对于第三方制作的Python，主要是为了某些特定领域的开发更加方便等目的而再次去打包发行的。

比如ActiveState的ActivePython就集成了很多用于科学计算方面的模块和功能。所以，如果你不是专门从事如天气预报、地震预测这类需要大量科学计算的对应领域的研究，那么就无需这类第三方版本的Python安装包了。

而对于Python初学者的学习重点是了解和使用Python的基本语法和知识，所以更无需这些第三方版本的Python。而且，即使以后需要用到这部分的功能，也完全可以在已经对Python有所了解甚至比较精通之后，将要开始用Python去用于特定领域内的开发时，再去安装和使用这些包含所需模块的第三方版本也不迟。

结论：

对于Python初学者而言没有必要选择第三方版本的Python。

Python初学者根据自己的系统平台和需求去选择上述相应的版本，就可以达到选择合适自己版本的Python并下载的目的了。

2.5.2 选择更稳定、更快速的国内下载源

知道了下载哪种版本的Python之后，按照上述的解释去Python官网中下载即可。

Python的官网：http://www.python.org/，以及其下的下载页面：http://www.python.org/ftp/python/和http://python.org/getit/releases/对于国内的网络来说不太稳定，导致很多国内开发者访问网站时，经常会遇到无法访问的情况。对此问题，很幸运的是有更好的解决办法：笔者后来找到一个Python官网的国内的镜像：http://mirrors.sohu.com/python/，此网站对于国内用户来说很稳定，并且速度也不错。所以当国内读者发现Python官网无法访问和下载时，就可以去上述镜像下载。

下载完Python后，接着就是安装Python了。接下来就详细解释对于主流的三大系统：Windows、Linux、Mac中，如何安装Python。

2.6 如何在Windows系统中安装Python

Windows系列操作系统有很多不同版本，目前主流的有Windows XP、Windows 7、Windows 8等。下面以使用最广的Windows 7为例来解释如何在Windows中安装Python。

2.6.1 在Windows 7中安装Python

此处假设已经下载了64位Windows 7系统的Python的安装程序：python-2.7.3.amd64.msi，下面详细介绍如何在64位的Windows 7下安装Python。

1.选择要安装的用户

双击python-2.7.3.amd64.msi，在弹出的对话框中选择默认的Install for all users（为所有用户安装此Python），如图2-2所示，然后单击Next按钮执行下一步。

2.选择目标安装路径

接着去选择对应的目标路径。此处的默认路径是“C：\Python27\”，如图2-3所示。

继续单击Next按钮执行下一步。有如下几点需要说明。

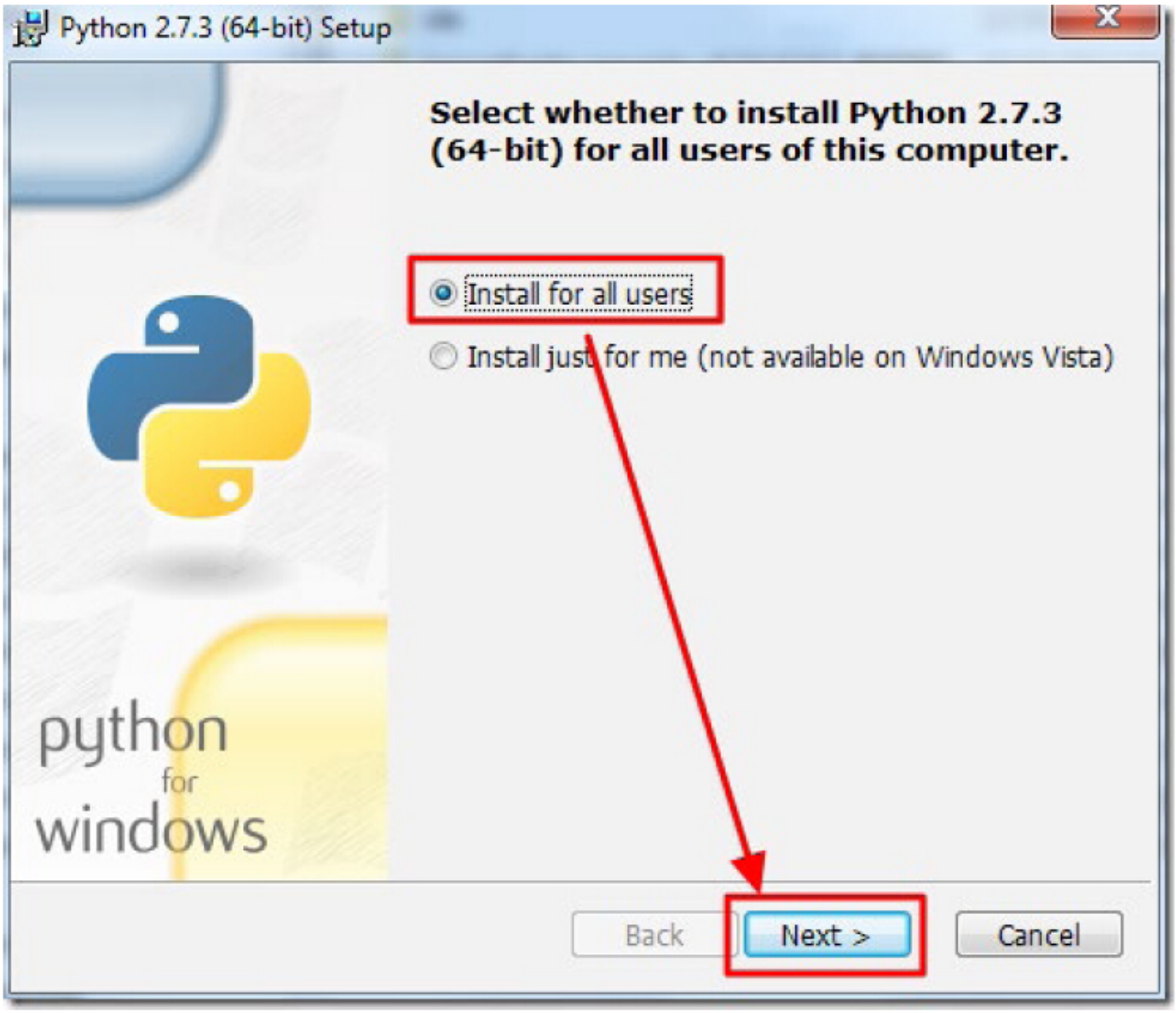


图2-2　选择为Python安装的用户

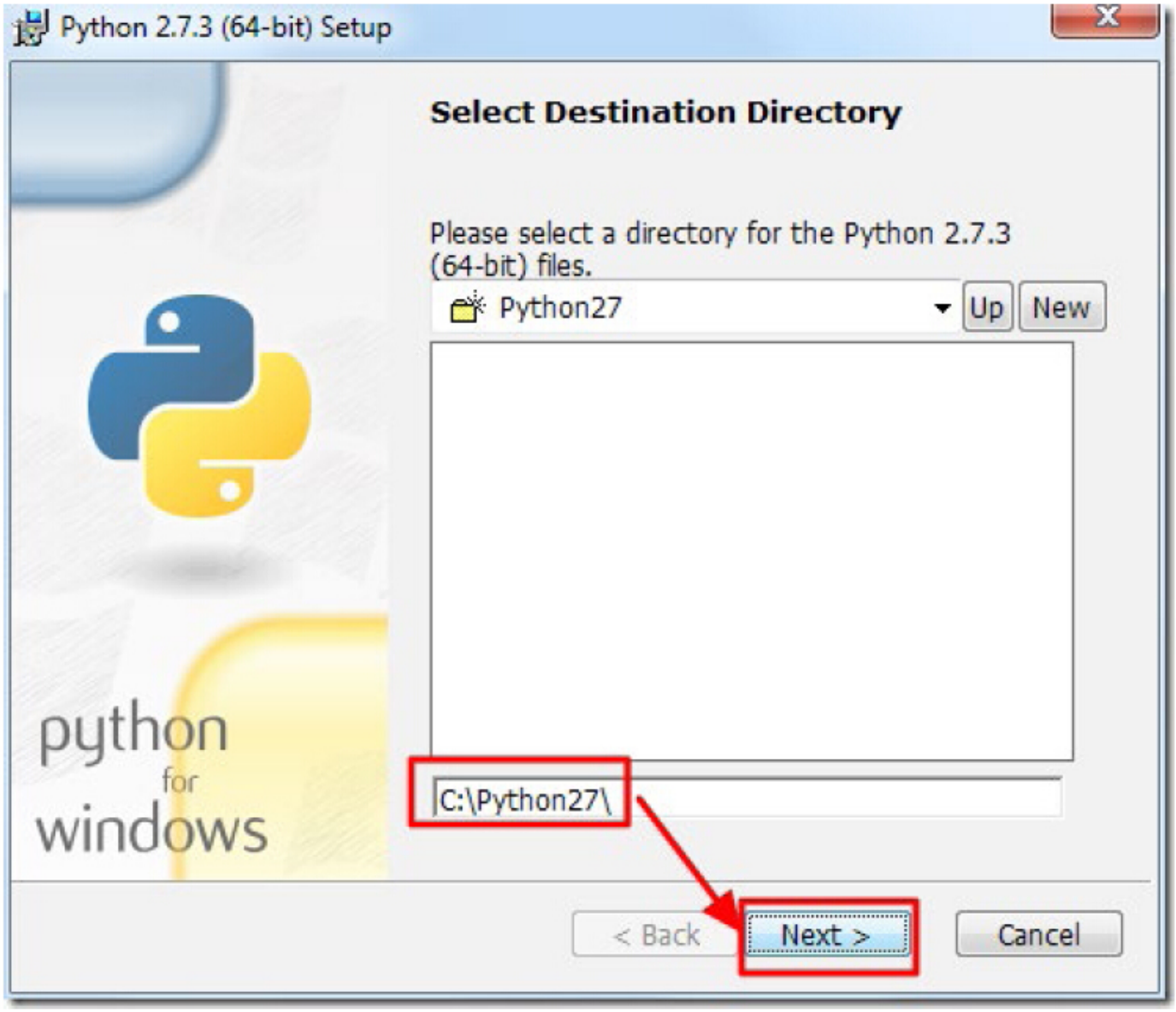


图2-3　默认的Python目标安装路径

很明显此处Python27的含义是：

Python27==Python+27==Python 2.7==Python 2.7.3中的主版本是2.7

依此类推：

对于Python 2.6.x的版本，其默认安装路径是C：\Python26\。

同理，Python 3.3.x版本的默认安装路径是C：\Python33\。

· 将Python安装的位置从默认为C盘改为E盘

此处笔者的一般选择是E盘，目的是符合自己统一的开发风格和习惯：对于和软件开发相关的软件、工具等内容都统一放到E盘。

所以笔者自己去安装Python时，往往将其安装到E盘对应的目录，比如E：\dev_install_root\Python27\或者E：\DevInstallRoot\Python27\。在安装其他开发工具时，也安装在对应路径下，如此以便于统一管理和开发相关的软件工具。

不过，由于笔者当前环境是在另外一台PC上，而对应的和开发相关的内容都放在tmp目录下，所以此处笔者的安装目录选择D：\tmp\dev_install_root\Python27_x64\，如图2-4所示。

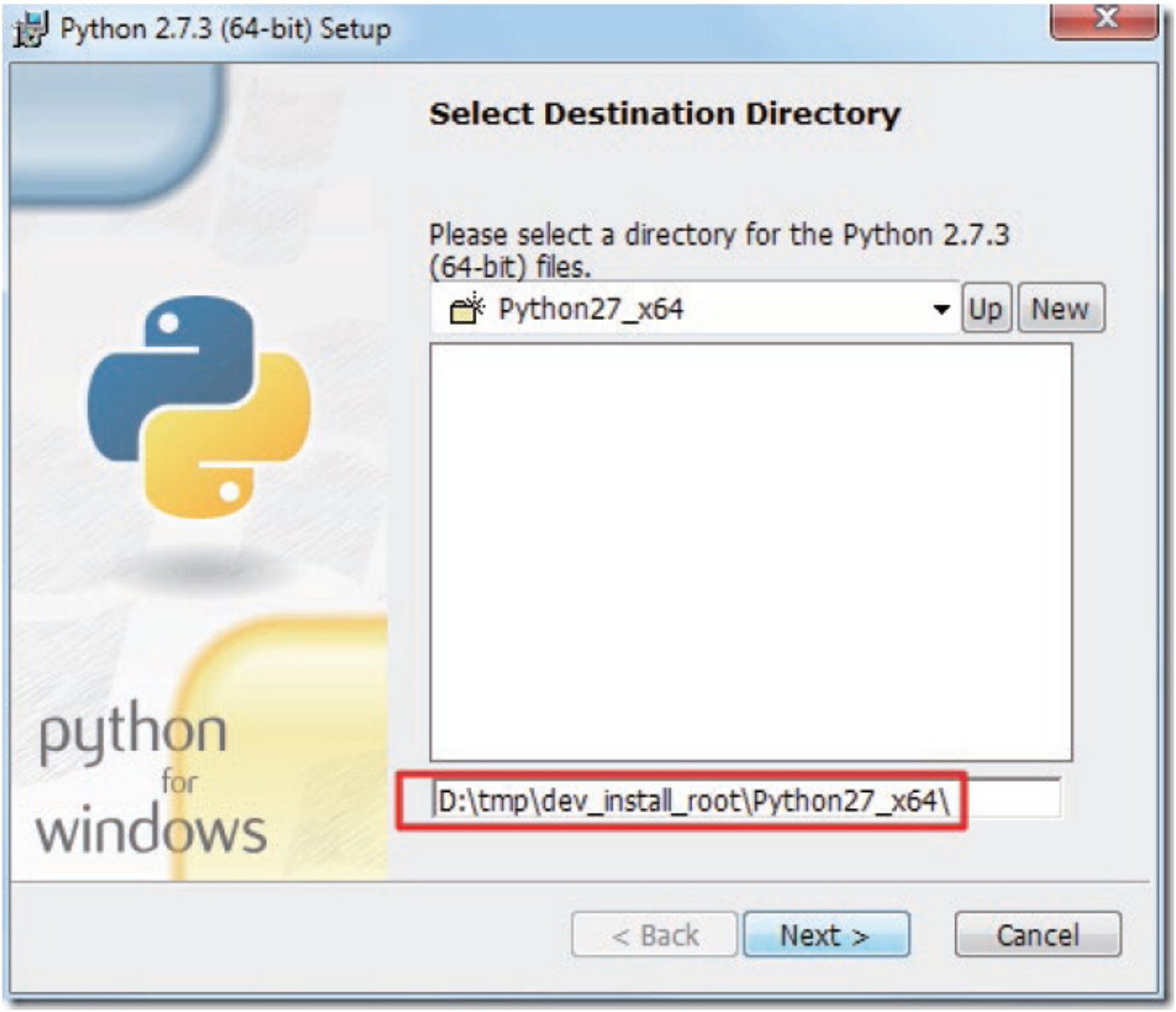


图2-4　选择安装目录

当然，笔者上述的做法只是供读者参考而已，并非强制性的，读者将Python安装到什么目录中，可以根据自己的情况而定。

Python安装路径不要包含特殊字符

对于上面提到的一些安装路径，可以看出全部是ASCII字符，没有特殊字符。而对于Python中安装路径需要提醒一点的是：在Python的安装路径选择中，最好不要包含一些特殊字符。

· 中文字符

对于国内用户来说，比较常见的现象就是在Python安装路径中包含了一些中文字符。多数情况下此做法对于Python的后续使用没什么负面影响。但是有时候路径中包含的非ASCII的中文字符可能会引起一些其他问题，比如某个Python内置或第三方的库对于此中文路径不识别，从而导致该模块使用时出现问题。所以，为了尽量避免后续出现这类问题，最好还是在Python安装路径选择时就确保不要包含中文字符。

· 空格

空格会导致将完整的路径分割成两个非法的路径。比如路径C:\Program Files在某些时候会被误认为是两个路径C:\Program和Files，而实际上这两个路径都是不存在的。所以，为了避免以后可能出现的路径问题，最好也不要在路径中包含空格。

· 其他特殊字符

同理，为了避免以后出现存在可能的路径问题，路径中也尽量避免其他特殊字符。

总之，Python的安装路径最好只包含普通的ASCII字符，即字母、数字、下划线，避免使用中文字符、空格等特殊字符。

3.选择要安装的部分

接下来选择需要安装的Python相关的部分，此处默认为全选，如图2-5所示。

图2-5中各选项的含义如下。

（1）Register Extensions

选择Register Extensions选项将使得此处安装的Python变成默认安装的Python。比如你之前也安装了其他版本，那么选择该项则使当前要安装的Python成为默认的。

（2）Tcl/Tk

Tcl/Tk包含了一些与图形环境相关的库、Tkinter和相关开发工具，如IDLE、pydoc。

（3）Documentation

Documentation是Python HTMLHelp文件，即安装之后自带的Python Mannual（Python手册）。

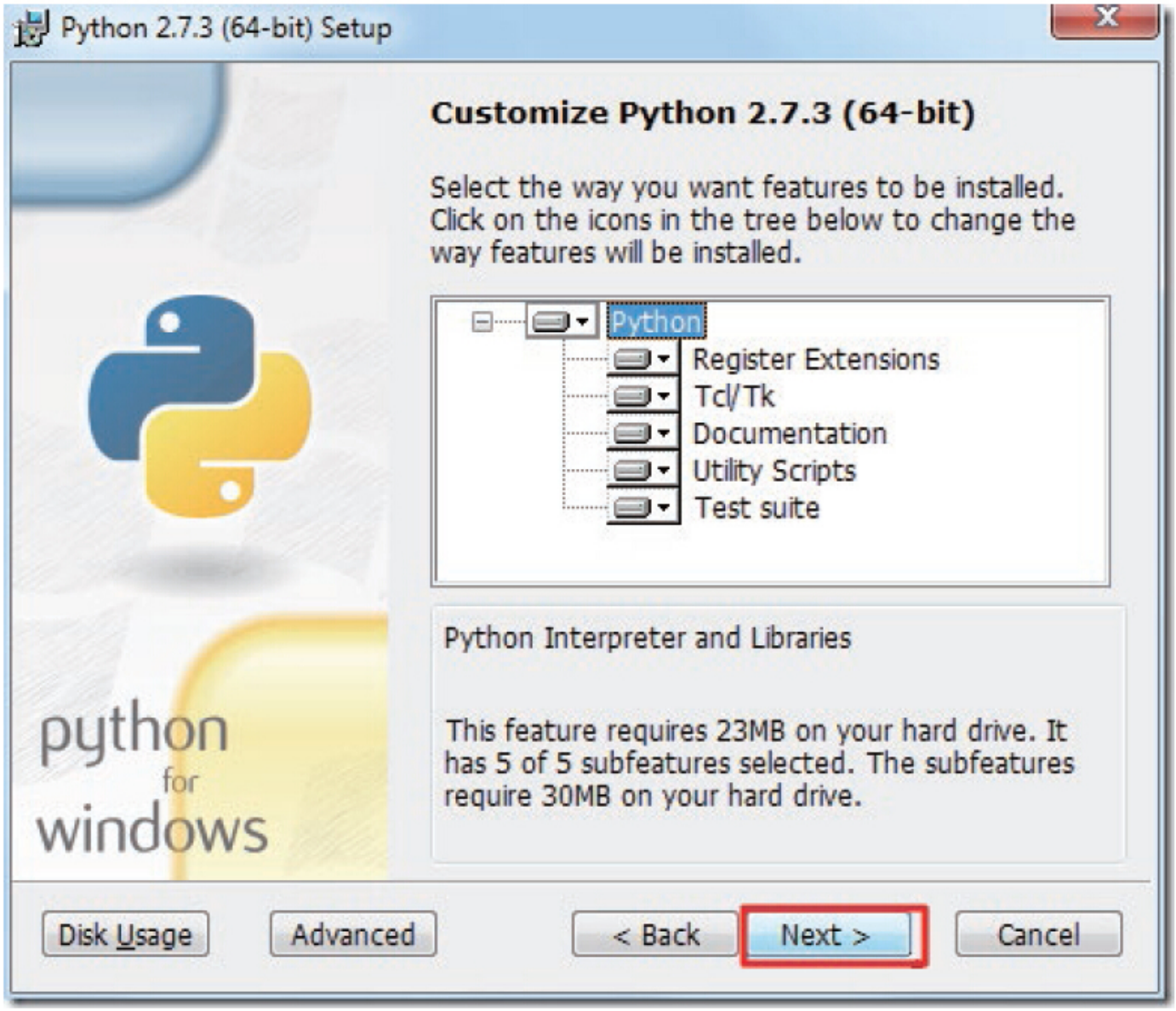


图2-5　定制选择所要安装的Python的部分

（4）Utility Scripts

Utility Scripts是一些用Python实现的有用的脚本工具。

（5）Test suite

Test suite为用于测试的Python工具。

继续单击Next按钮，执行下一步。

4.安装

接下来即正式开始安装Python了，如图2-6所示。

安装完毕后，单击Next按钮。

5.完成安装

至此，Python就安装完毕了，如图2-7所示。

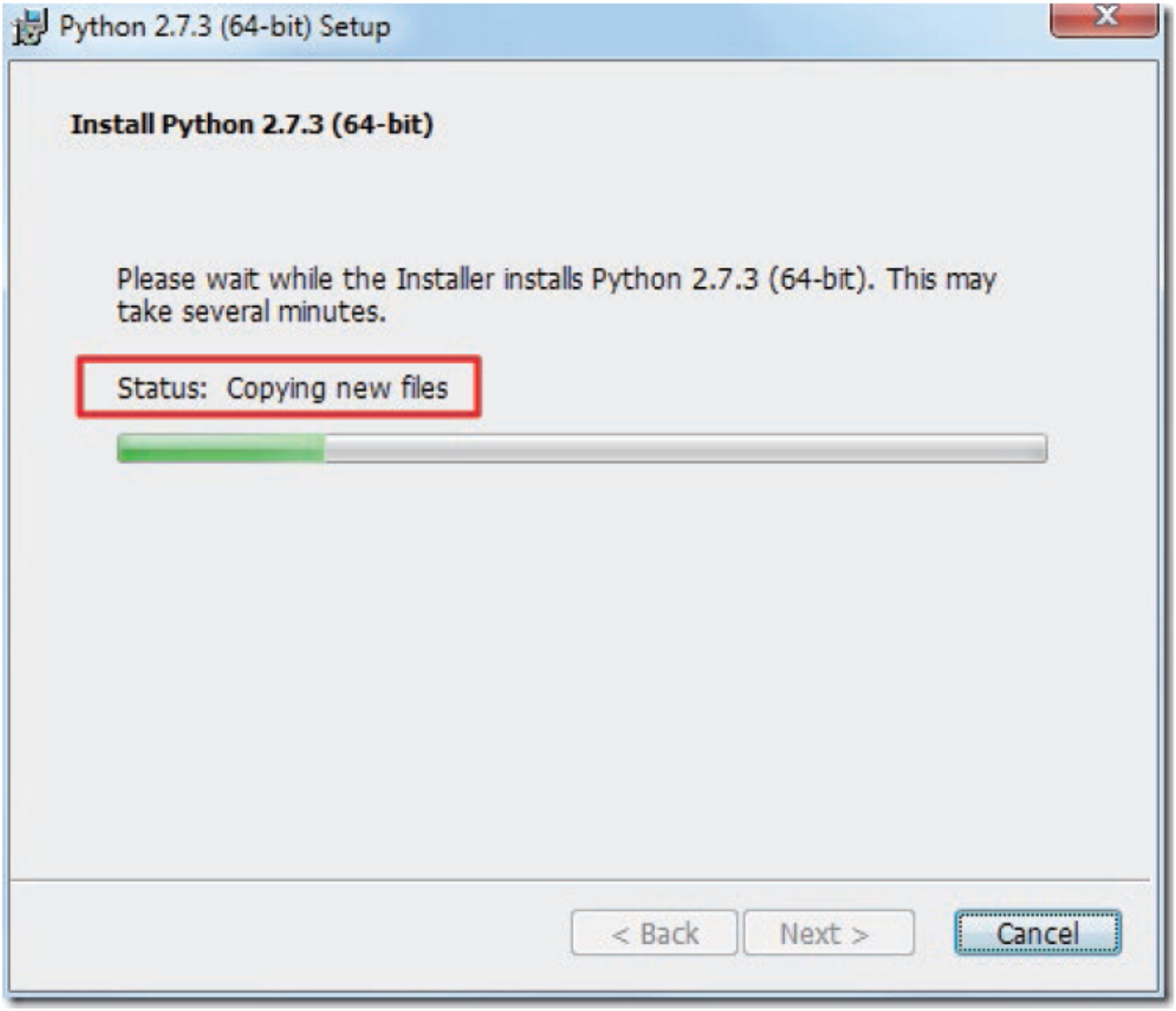


图2-6　正在安装Python

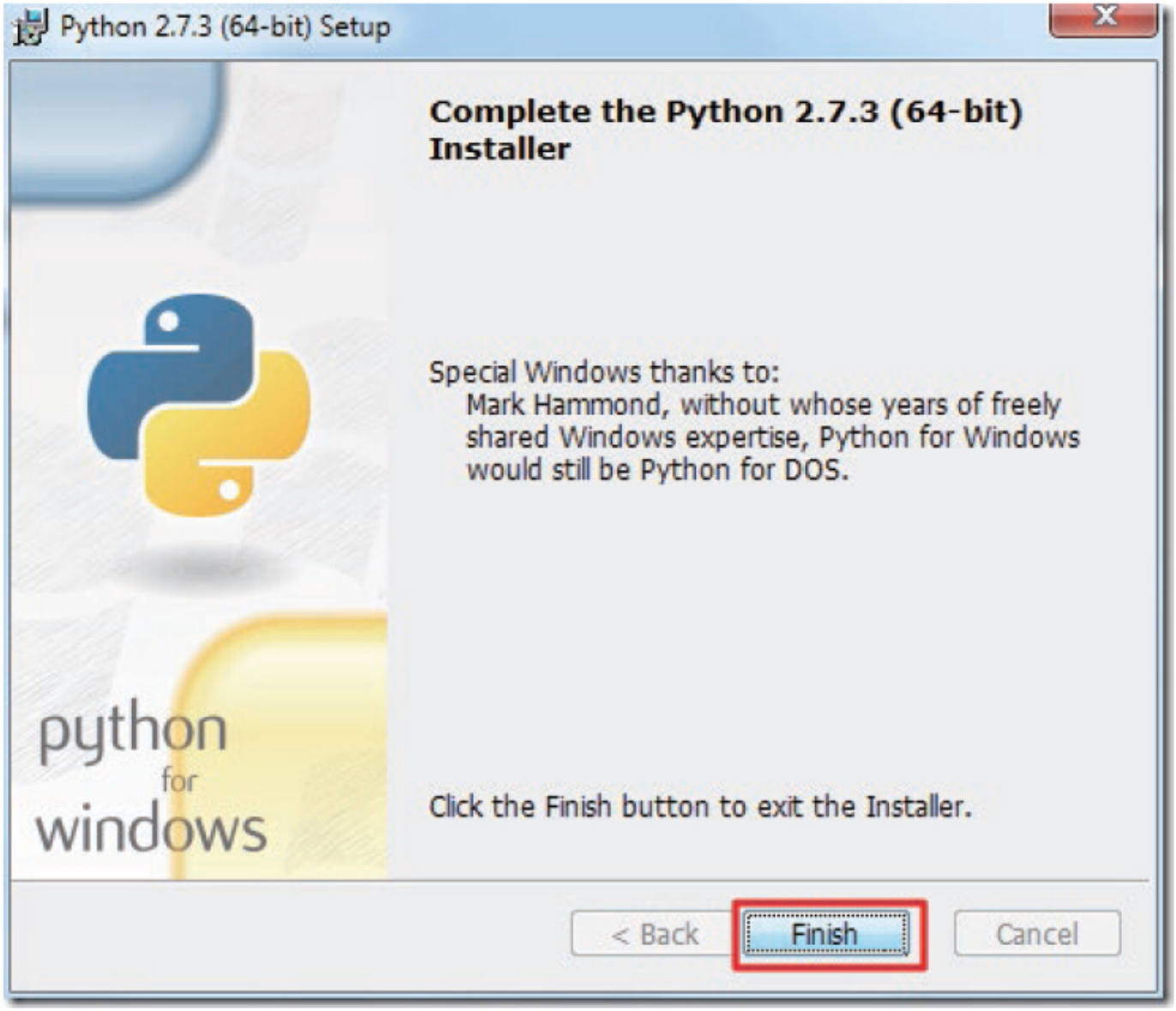


图2-7　Python安装完毕

2.6.2　在Windows中安装Python后的常见问题

1）如何确认Python已经安装成功？

确认的方法很简单，在Windows的开始菜单中可以找到Python即说明安装成功了，如图2-8所示。

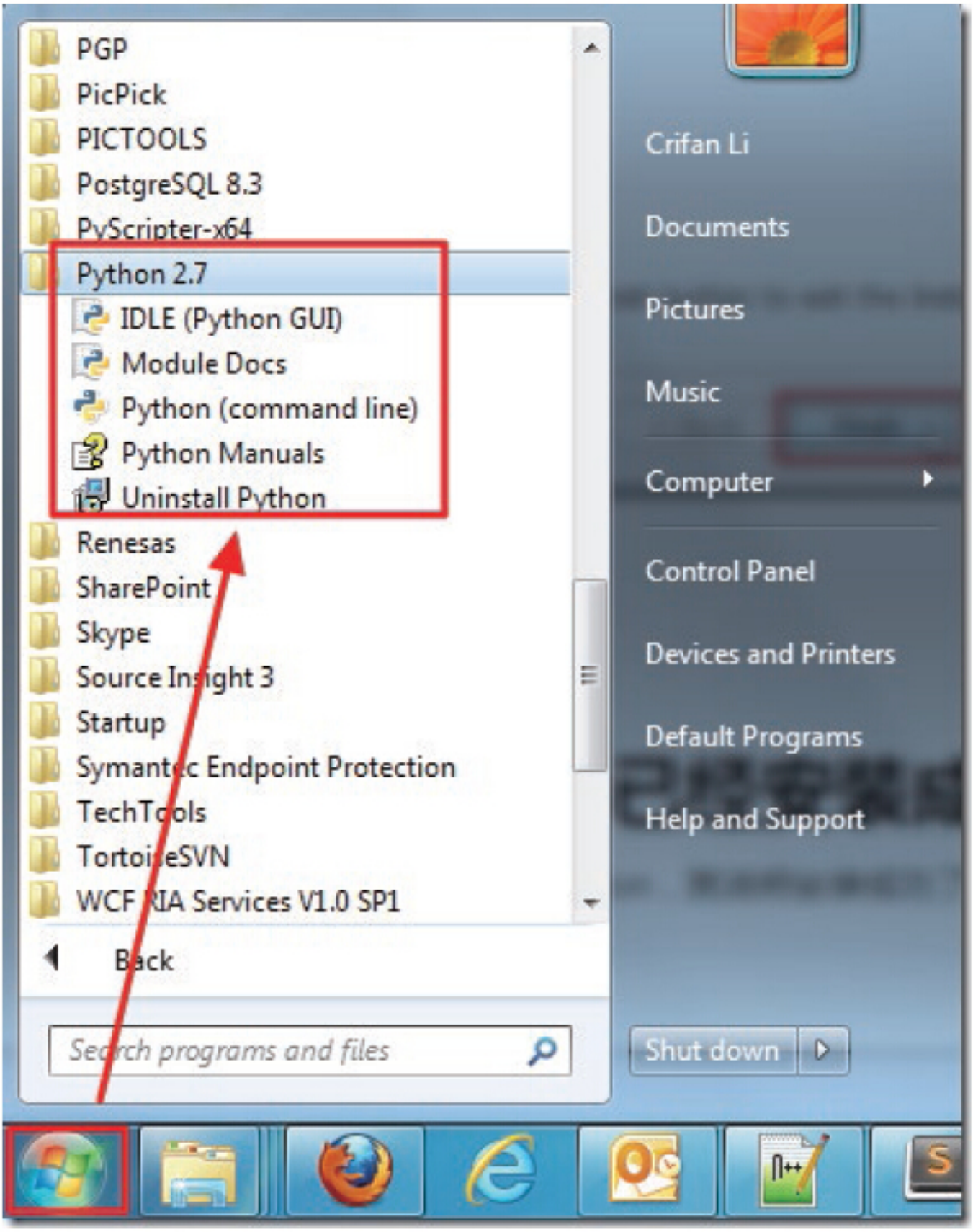


图2-8　Windows开始菜单中可找到Python

2）需要将Python安装路径添加到PATH环境变量中吗?

很多人提到，安装Python之后要把Python的安装路径添加到PATH环境变量中才能正常使用Python。而根据实际经验，不将安装路径添加至PATH环境变量中也是可以正常进行Python开发的。如果你非要去添加，当然也是可以的。具体操作步骤是：将此处Python的安装路径“D：\tmp\dev_install_root\Python27_x64\”添加到PATH环境变量中即可，如图2-9所示。

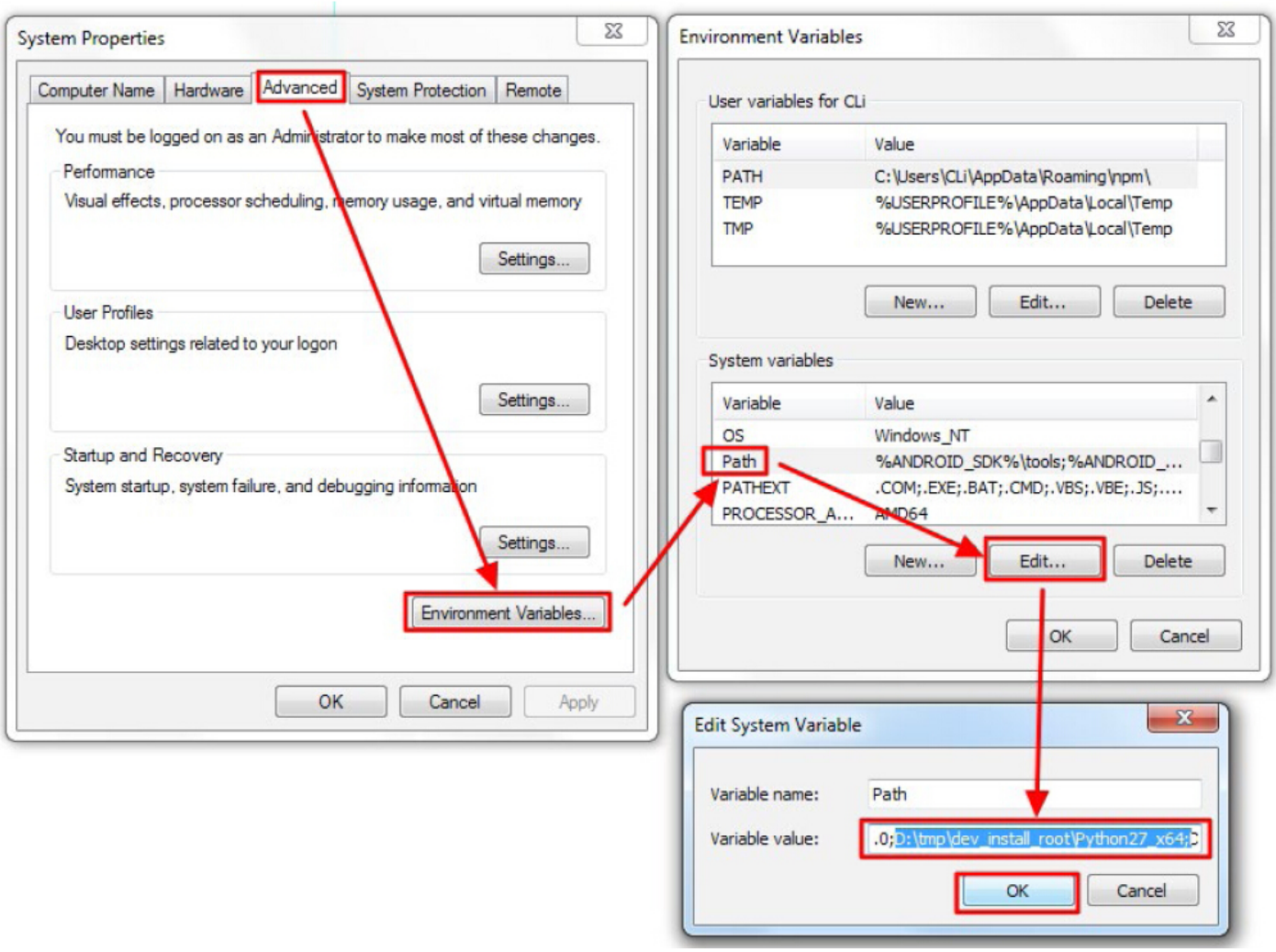


图2-9　将Python安装路径添加到PATH环境变量中

2.7　在Linux系统中安装Python

一般所说的Linux指的是Linux内核，而我们常说的Linux指的是该Linux类型的操作系统，是基于此Linux内核加上其他功能组件和模块制作出来的发行版。目前相对流行一些的Linux发行版有（较早的）RedHat（红帽子）和OpenSUSE，以及后来出现的Ubuntu等。

2.7.1　在Ubuntu中安装Python

先说如何在Ubuntu中安装Python，很简单，根本无需安装，因为Ubuntu本身已经自带了Python了，如图2-10所示。

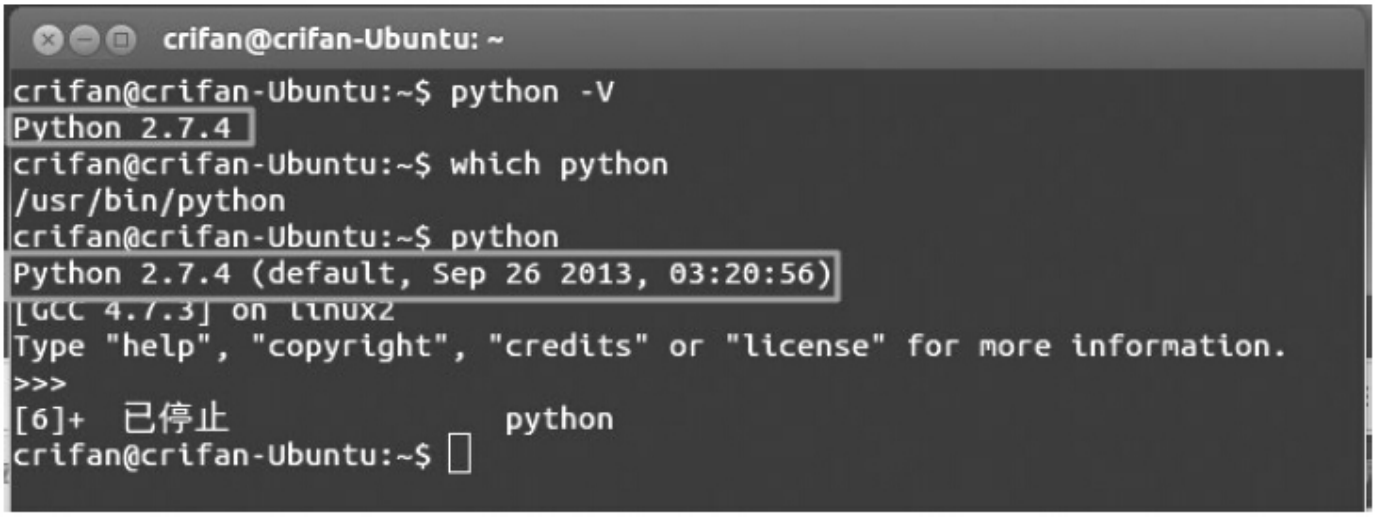


图2-10 Ubuntu中自带Python

2.7.2 为何不推荐初学者在Ubuntu中安装Python

接下来详细解释为何不推荐Python初学者去安装Python。如上所述，Linux类的系统，包括Ubuntu，都已经默认安装了一个某个版本的Python了。

而如果在此基础上再去安装另外一个版本相对新的Python将会出现以下情况。

1.对于Linux类的系统来说

由于Linux类的系统，包括Ubuntu中有很多内置的工具和服务都是Python代码写的，比如Ubuntu软件中心就依赖于Python。而如果破坏了原先自带的Python则很容易导致现有Linux系统中的相关工具和服务出现异常。为了保证Ubuntu等Linux系统的正常运行，最好不要卸载当前Ubuntu系统中自带的Python，就算是旧版本的Python，如之前的Python 2.7.4也建议保留。

并且，再安装一个更新版本的Python很容易引起混淆：如果当前新安装的Python和系统自带的Python没有设置好的话，很容易导致各种问题。

综上所述，最好不卸载系统默认安装的（或许版本相对有点旧的）Python，也不要随便安装新版本的Python。

2.对于有需要安装新版本Python的人来说

现实中的确会遇到一些情况，比如有些人为了用某些软件而确实需要用到其他某个更新版本的Python，这样就要在系统已安装了某个版本的Python的前提下再去安装另外一个版本，以及再去使用某些辅助的工具，比如pythonbrew，在需要时切换使用不同版本的Python。很显然，对于Python的普通使用者，如果没有这个需求则最好不要再另外安装其他版本的Python。

3.对于初学者来说

根据前面的解释就很容易明白了，对于Python初学者来说，一是为了不破坏Linux类系统的稳定性，不故意人为地制造出更多问题；二是本身也暂时没需求非要再安装另外某个版本的Python。所以结论很简单：使用Linux系统自带的Python就足够学习使用了，无需再安装另外版本的Python。

2.8 在Mac中安装Python

由于Mac系统的基本架构和Linux系统比较类似，换句话说，Mac系统的底层架构和Linux架构是一样的，此特性用于此处的讨论，如何在Mac上安装Python，其结论也和Linux系统中是一样的：Mac系统中也已经内置了Python了，如图2-11所示。对于初学者也无需再安装，直接拿来使用即可。

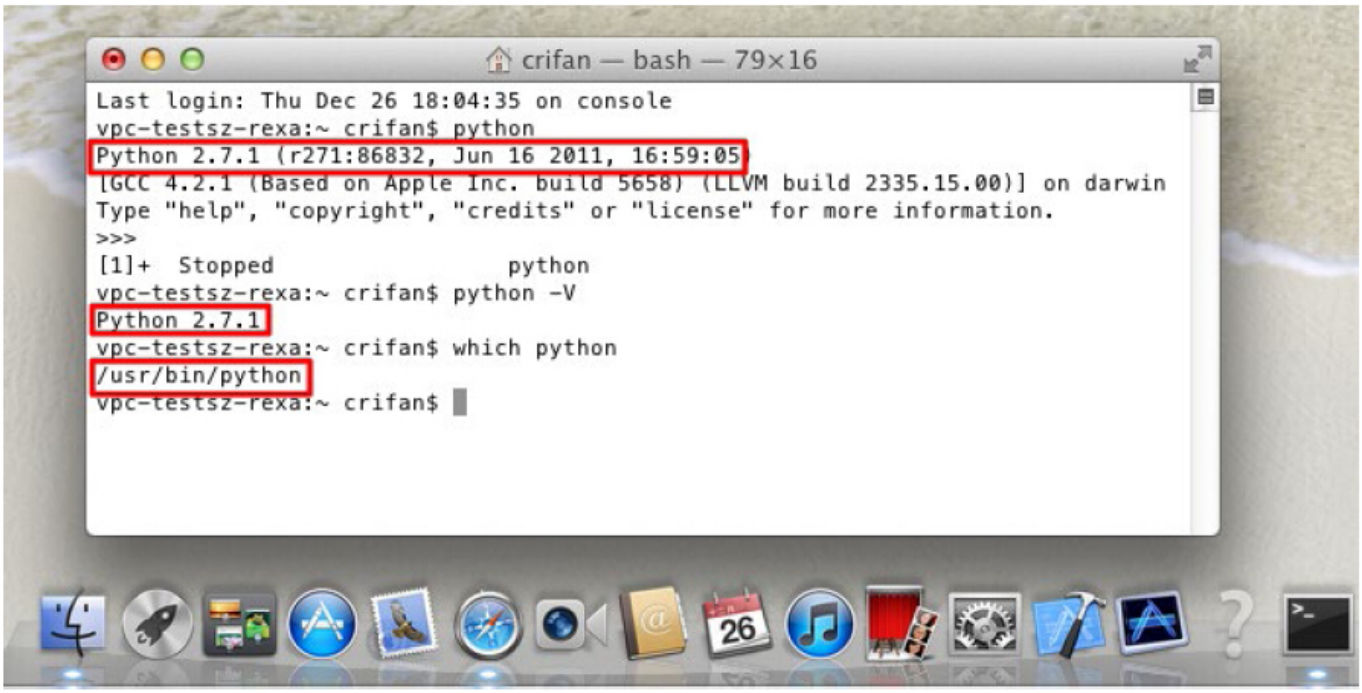


图2-11 Mac中也自带安装了Python

至此，对于如何在主流的操作系统中安装Python的问题就都已解释完毕，接下来就是介绍如何开发Python了。

第3章 选择合适的Python开发环境

接下来本章将介绍如何在多种不同平台下进行Python开发，期中涉及三种主流平台：

- Windows
- Linux
- Mac

下面先列出此处针对这几种不同平台的Python开发时所涉及的一些共同的特点和共用的东西。

3.1 不同平台下开发Python时共用的东西

不论是在Windows、Linux还是Mac下去开发Python都会有一些共用的东西，所以此处先要介绍一下这部分共用的东西。

1.共用的示例代码：用于演示如何开发Python

例3.1　用于演示如何开发Python的示例代码

为了在介绍Python开发环境时得以演示Python代码的运行，所以需要一段测试代码。此处采用下面这段代码：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-
"""
Function:
【整理】如何在Windows下开发Python（如何运行Python脚本）
http://www.crifan.com/how_to_do_python_development_under_windows_environment
Author:     Crifan Li
Version:    2012-12-06
"""
import platform;
pythonVersion = platform.python_version();
uname = platform.uname();
print "Just for demo how to do python development under windows:";
print "Current python version info is %s"%(pythonVersion);
print "uname=",uname;
```

如上代码的作用是打印Python版本信息和系统信息。

2.共用的文本编辑器：用于编写Python代码

除了演示不同平台的Python开发环境时都要用的Python示例代码之外，对于开发Python时所需要用到的用于写Python代码的文本编辑器方面，也有一些值得推荐的、跨平台的、功能好用的文本编辑器，如：

（1）Notepad++

Notepad++是个功能强大且易用的文本编辑器，其经典的称谓是Notepad的最佳替代品。虽然Notepad++很好用，但是也有个缺点，那就是目前只能运行于Windows平台。如果在Windows平台下开发Python的话，笔者还是极力推荐使用Notepad++这个文本编辑器。对于Notepad++，笔者也已经写好了专门的教程，以后有机会的话也会再单独发布出来的。

顺便提一下，Notepad++可以去Notepad++的官网的下载页面去下载，地址为http://notepad-plus-plus.org/download。

（2）Sublime

Sublime也是一款文本编辑器，以界面十分友好、功能也很强大而著称，且也有免费版可以使用。

Sublime可以用于上面所说的多种平台，包括Windows、Linux、Mac等。除了上面已经介绍在Windows平台下推荐Notepad++而不推荐Sublime之外，对于Linux和Mac都还是推荐使用Sublime去作为Python的文本编辑器。Sublime分两个大的版本：Sublime 2和Sublime 3，对应的官网下载页面为http://www.sublimetext.com/2和http://www.sublimetext.com/3。

3.2　不同平台下开发Python时的共同特点

此处先总结一下在不同的平台下开发Python时所涉及的一些共同的东西，尤其是一些共同的开发方式和方法。因为不论是在哪种平台下，Python的开发方式和逻辑都是相通的。Python的开发环境或者说开发方式，根据所用的工具和使用方式不同，可以

分为三类。

3.2.1 Python最原始的开发方式

最原始的开发Python的方式有如下三种。

1）找到合适的文本编辑器，编写Python代码，并保存为.py文件。

Python的代码本质上就是文本，就是一堆字符串，而对应的Python的py文件本质上就是文本文件。所以，从理论上来说只需要找个合适的文本编辑器就可以去编写Python代码了。

只不过对于各种文本编辑器来说，也存在好用和不好用之分。在不同的系统中，相对好用且适合用来开发Python的文本编辑器，如之前所推荐的：

- Windows中推荐用：Notepad++
- Linux和Mac中推荐用：Sublime

在不同的平台下就是选择对应的合适的文本编辑器，新建文件，写上对应的python代码，然后再去保存.py文件即可。

2）打开命令行终端环境，并且切换到对应的python脚本所在目录。

打开系统中的命令行环境：

- Windows中的cmd
- Linux和Mac中的Terminal（终端）

并且切换当前路径到脚本所在的目录。

3）在命令行终端环境下运行Python脚本（.py文件）。

在：

- Windows中的cmd
- Linux和Mac中的Terminal（终端）

中运行对应的xxx.py这个Python脚本。

如此就是一个最基本的、最原始的Python开发的流程了。而对于不同平台下的如何进行每一步的操作，后文会有详细解释。

3.2.2 利用Python的shell进行交互式开发

Python中有个shell，也叫作interactive shell，它提供了一个交互式的Python运行环境。即写一行代码输入后就可以立刻被运行，然后可以很直观地看到代码运行结果。Python的shell根据是否有GUI图形界面可分为如下两种。

1.无GUI版的：命令行（command line）版的Python shell

对于不同的系统：

- 在Windows中手动安装了Python后
- 在Linux和Mac都已默认安装了Python后

就都自带了此无GUI版本的命令行界面的Python shell了，所以无需额外安装而直接就可以拿来使用了。

2.有GUI版的：Python IDLE

Python shell加上图形界面后，就是那个IDLE了，所以：

- 在Windows中手动安装了Python后
- 在Linux中需要手动安装IDLE后
- 在Mac中默认安装的Python自带了IDLE后

就有了IDLE可以使用了。

在Ubuntu中安装Python的shell：IDLE

像Ubuntu的Linux类系统，默认是没有安装这个IDLE的，所以需要读者自己手动用如下命令去安装一下，如图3-1所示。

```
sudo apt-get install idle
```

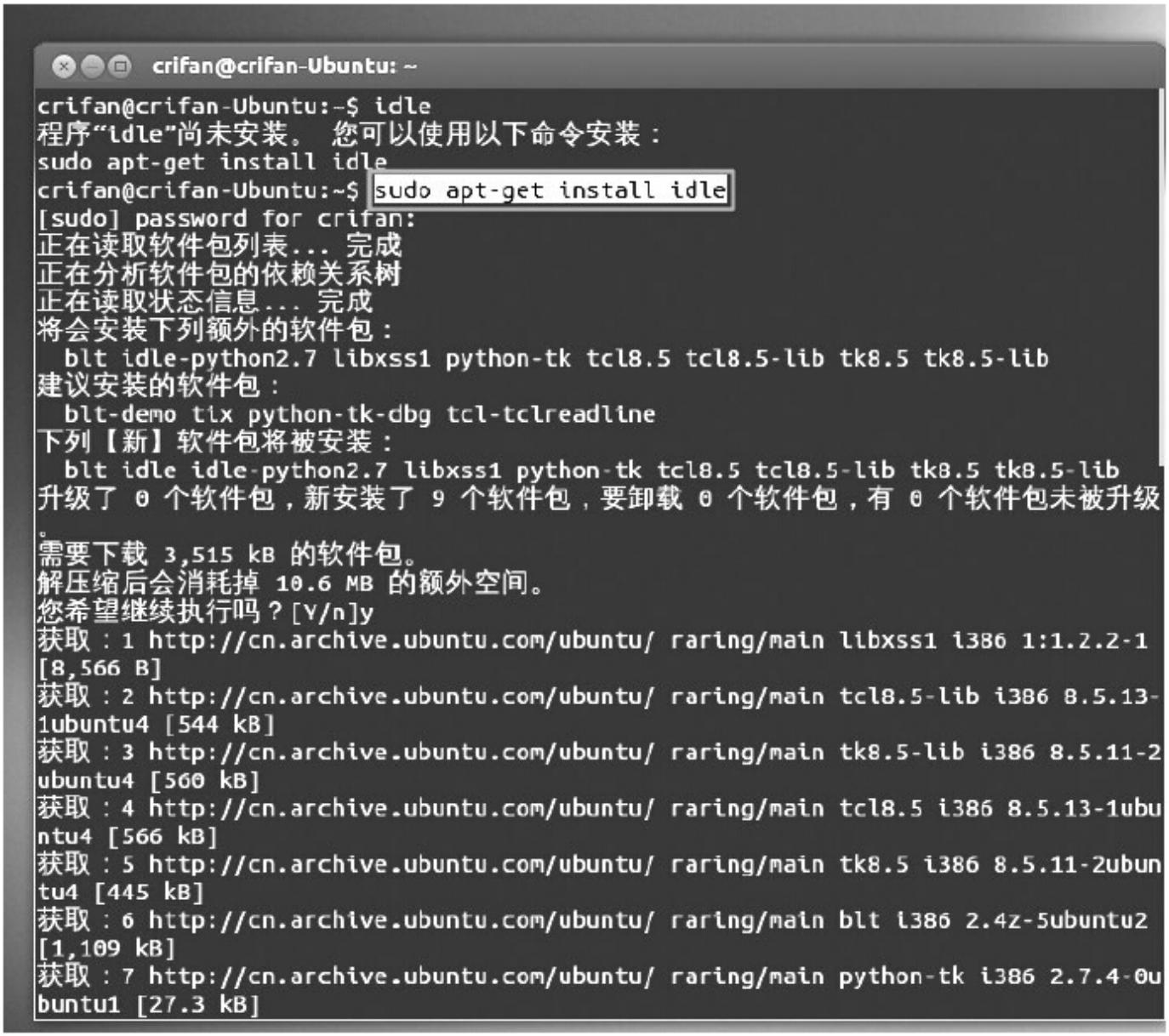


图3-1　Ubuntu中安装Python的IDLE

安装完毕后，就可以从终端中用命令idle去启动IDLE了，如图3-2所示。

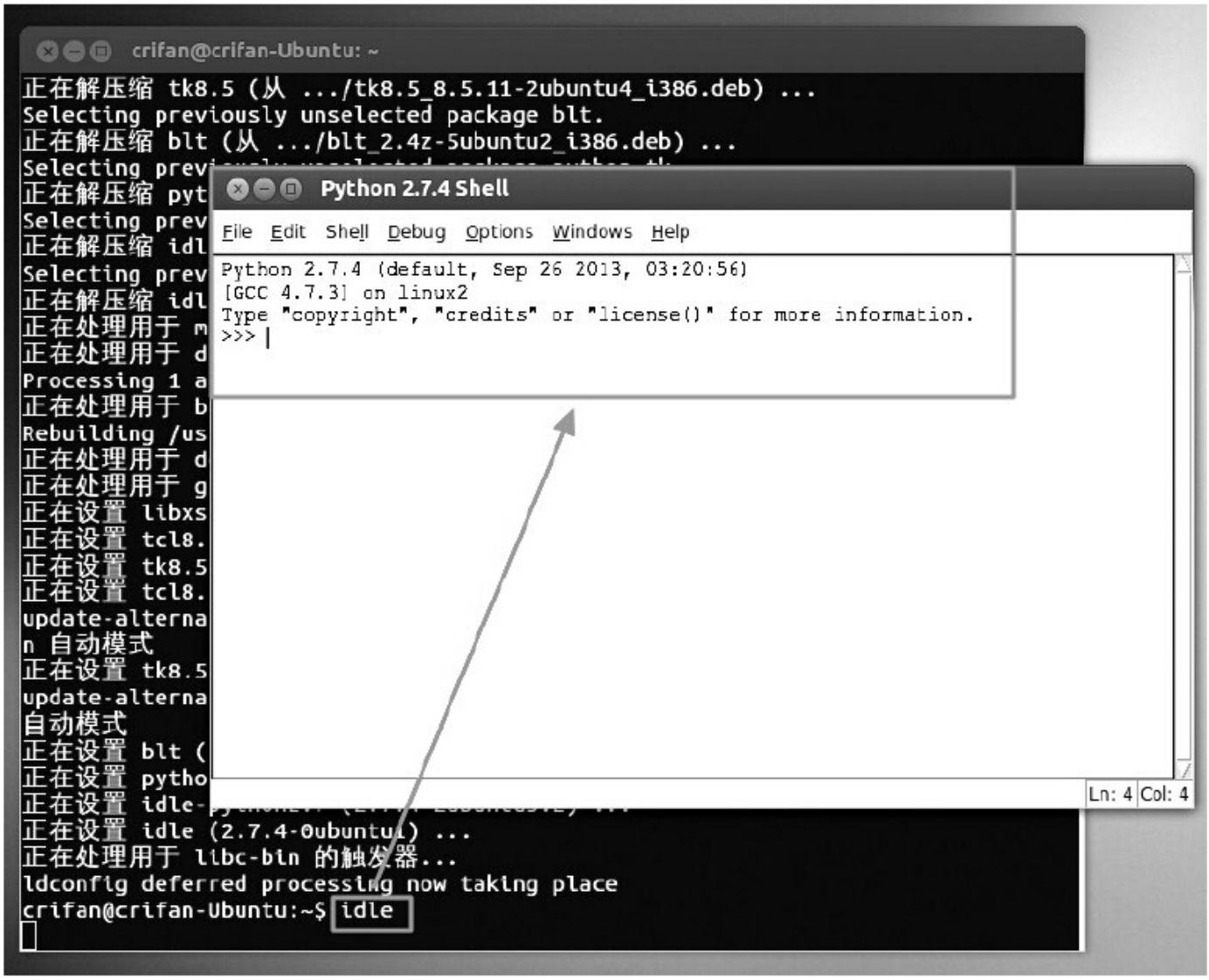


图3-2　Ubuntu中运行Python的IDLE

3.2.3　利用Python的IDE进行开发

什么是IDE

此处先简要解释一下IDE的基本概念：

IDE==Integrated Development Environment==集成开发环境

· 集成

所谓集成，就是把原先的一堆相对独立的、零散的东西组合到一起。

· 开发环境

所谓开发环境，一般是针对某种特定语言来说的。即在使用某种计算机语言进行软件开发的过程中涉及的一些开发环境，其中包含一些工具（编译器、链接器等）、外部条件（环境变量配置）等。

用公式表达即：

公式3.1　什么是IDE

IDE=集成开发环境=将开发相关的各种环境和工具都集成到一起

而Python的IDE即：

公式3.2　什么是Python的IDE

Python IDE=Python的集成开发环境=将和Python开发相关的各种工具集成在一起，另外再加上各种文件、代码、项目的组织和管理等方面的各种功能。

· Python代码编辑器：替代前面用的Notepad++或Sublime等文本编辑器

· Python的运行环境：模拟或替代Python的（命令行或GUI版本的交互式）shell

Python的IDE的目的是方便用户编写和运行Python代码，方便进行Python项目的开发。

为何会有Python的IDE

其实也很容易理解，之所以会有Python的IDE的诞生，是因为开发Python期间的写Python代码、调试Python代码、查找相关的函数的相关文档等相关操作，如果都是用相对基本的那些方式去操作的话，即用文本编辑器去编辑Python代码，写完代码后，再切换到命令行下去运行，会觉得很麻烦。尤其是大型项目的话，效率显得就更加低。且如果涉及文件很多的话，管理起来也很麻烦。另外再加上开发时所需要的一些额外的功能，比如调试复杂的Python代码，其是往往需要一步步跟踪调试后才能找到错误的根本原因等，此时上述原始的开发模式就更显得力不从心。

所以Python发展到现在，已经有了很多第三方的、别人开发的，可以用于或者专门用于Python开发的一些集成开发环境，即Python的IDE的出现。

3.3　Python的IDE

下面再详细解释一下Python的IDE的各个方面的内容。

3.3.1　Python的IDE和编辑器、终端等的关系

为了解释说明Python中的IDE和Python的原始开发环境中的文本编辑器、命令行终端（比如Windows的cmd）等之间的关系，下面找了其中一个相对典型的Python的IDE——PyScripter，下面通过图3-3和图3-4来对比性地解释。

通过上述对比图示的说明，就可以进一步理解命令行终端运行环境（如Windows的cmd）、Python的shell（command line版本、GUI版本：IDLE）、Python的IDE这三者之间的关系了。

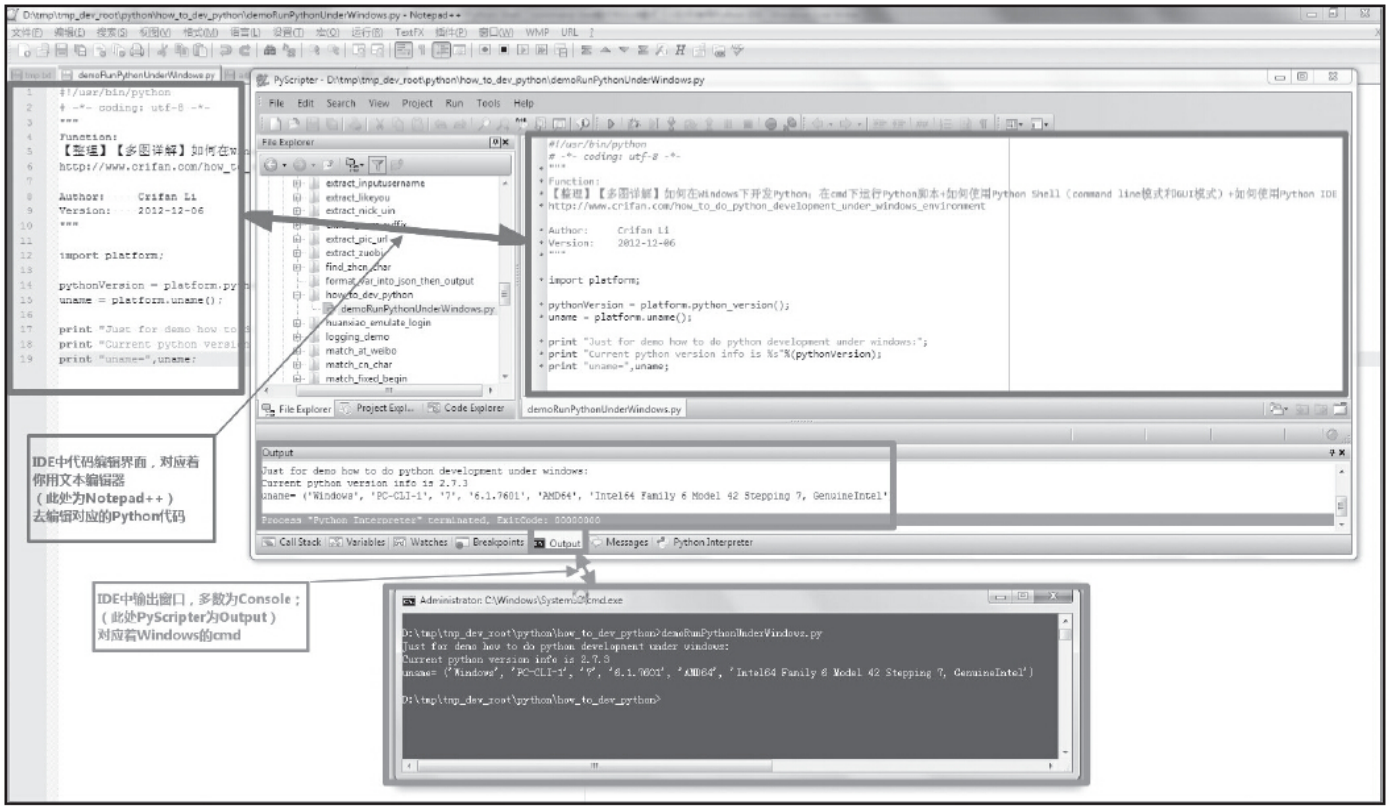


图3-3　文本编辑器：命令行终端与Pyscripter之间的映射关系

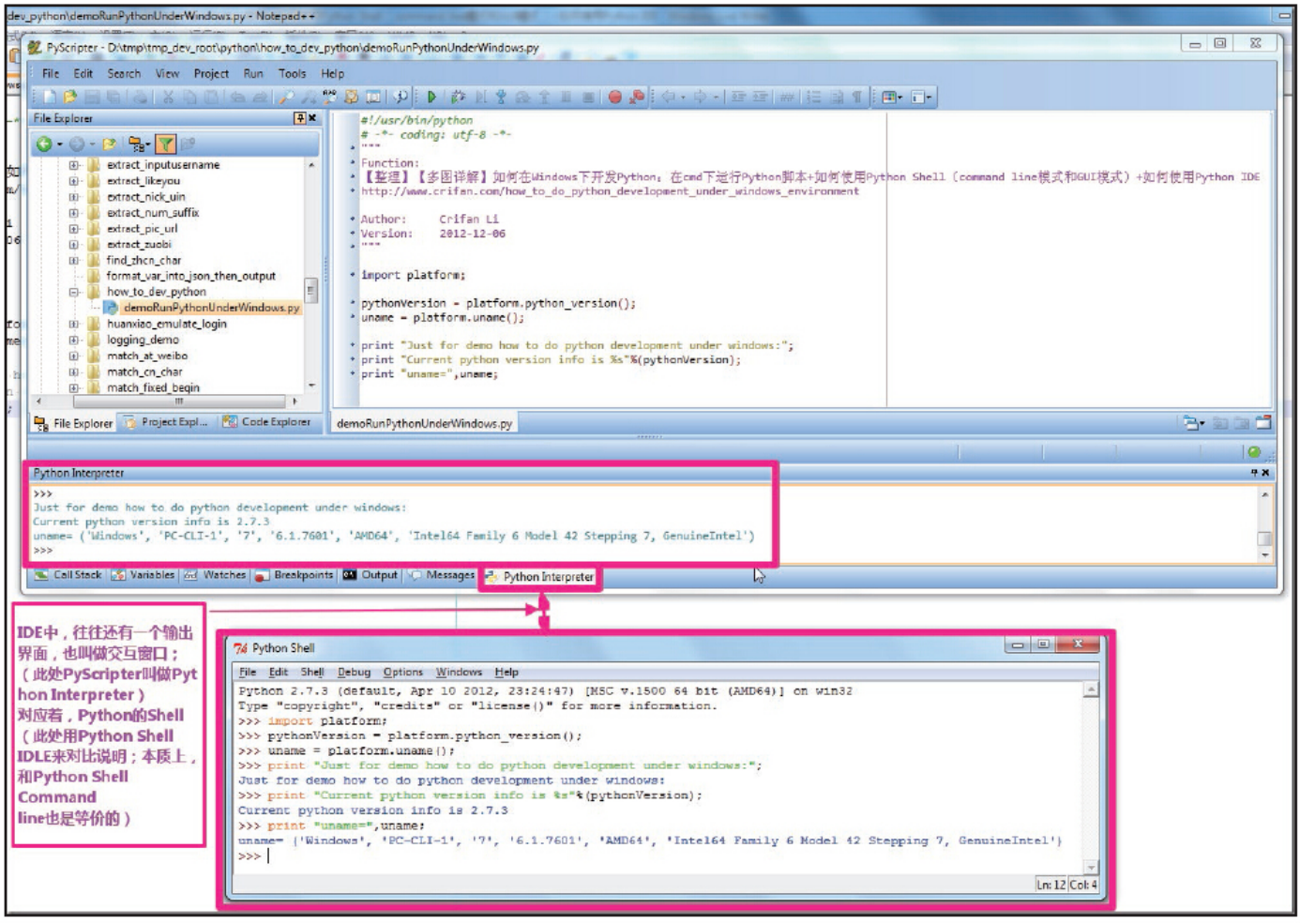


图3-4　交互式Python shell与Pyscripter之间的映射关系

3.3.2 Python的常见IDE

本节主要以Windows平台为例去解释各种常见的Python IDE。由于很多IDE是跨平台的，可以运行在除Windows之外的Linux和Mac系统中，比如Eclipse+PyDev。所以，如果读者的开发环境是Linux或Mac，本节的内容同样有很大的参考价值。

1.PyScripter

下面介绍PyScripter的这个Python的IDE的下载、安装和使用。

登录PyScripter主页（地址：http://code.google.com/p/pyscripter/）中的下载页面（地址：http://code.google.com/p/pyscripter/downloads/list）下载对应的安装文件。

比如针对笔者此处的64位Windows7的安装文件地址为http://pyscripter.googlecode.com/files/PyScripter-v2.5.3-x64-Setup.exe，下载完毕后安装即可。

接着在PyScripter中打开之前写好的示例代码，如图3-5所示。

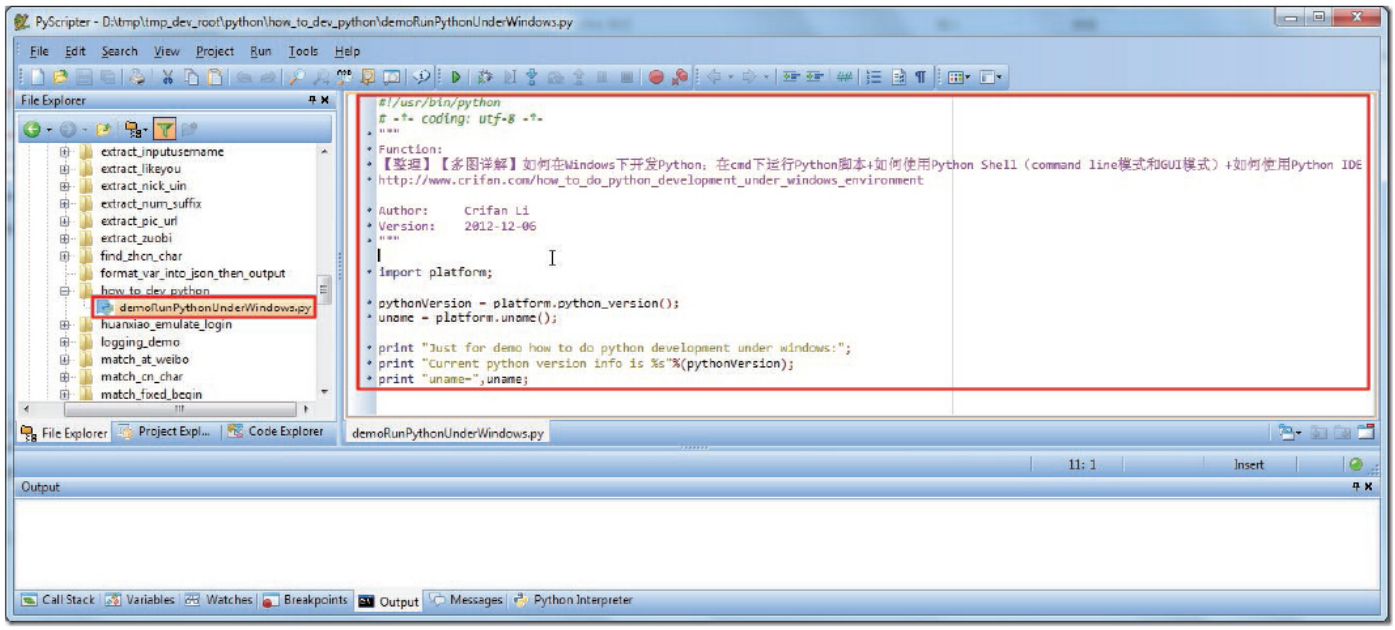


图3-5 在PyScripter中打开Python示例代码文件

然后执行Run→Run命令去运行代码，如图3-6所示。

当然，如果直接使用快捷键Ctrl+F9也是同样的效果。然后就可以看到PyScripter中如图3-7所示的Python Interpreter窗口中输出的结果了。

在此处也试试“从外部运行”，即执行Run→External Run命令，如图3-8所示。

可以看到输出结果是在PyScripter中的Output窗口中显示的，其等价于Windows中的cmd。总的来说，PyScripter还是比较好用的。

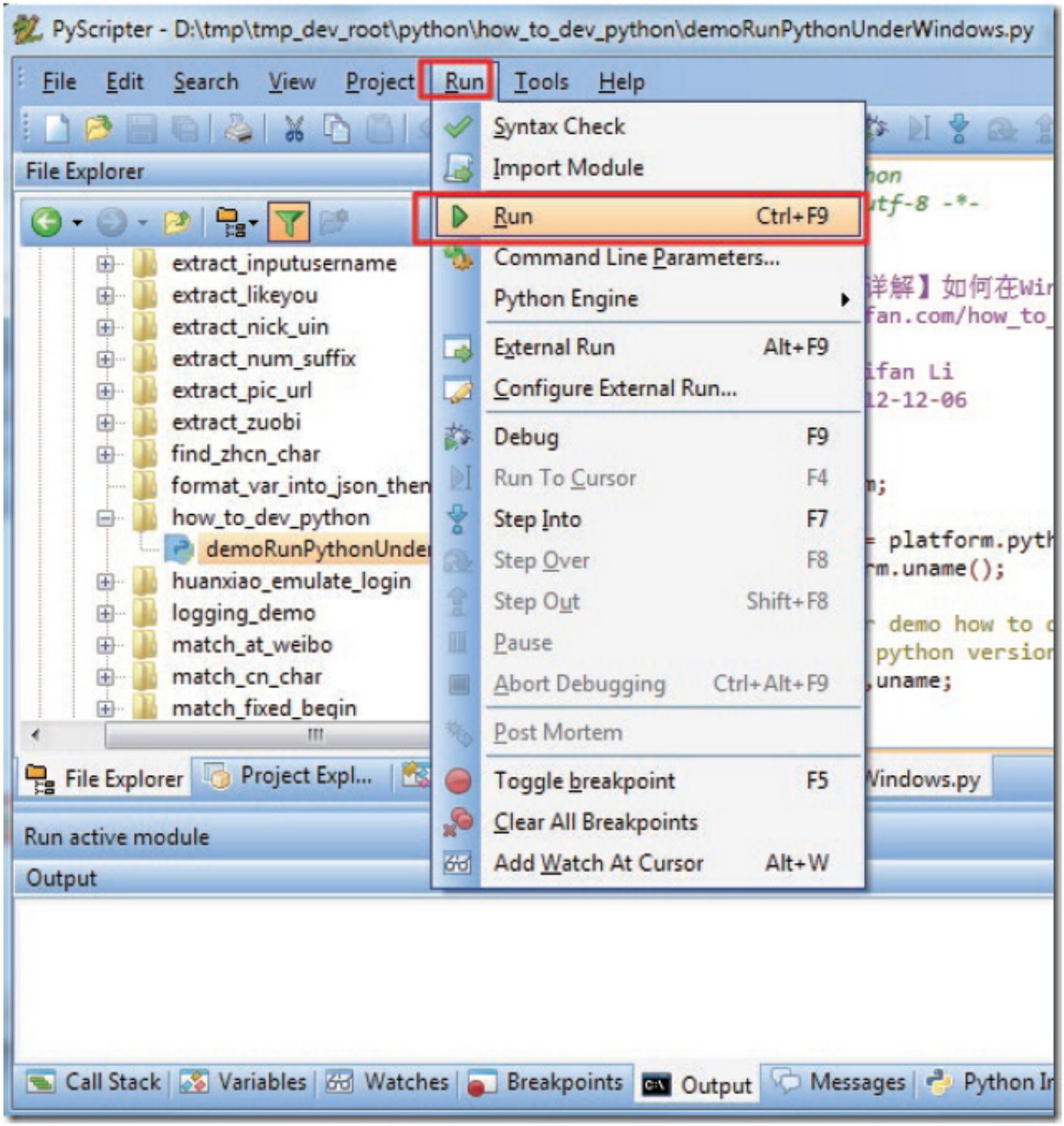


图3-6　在PyScripter中运行代码

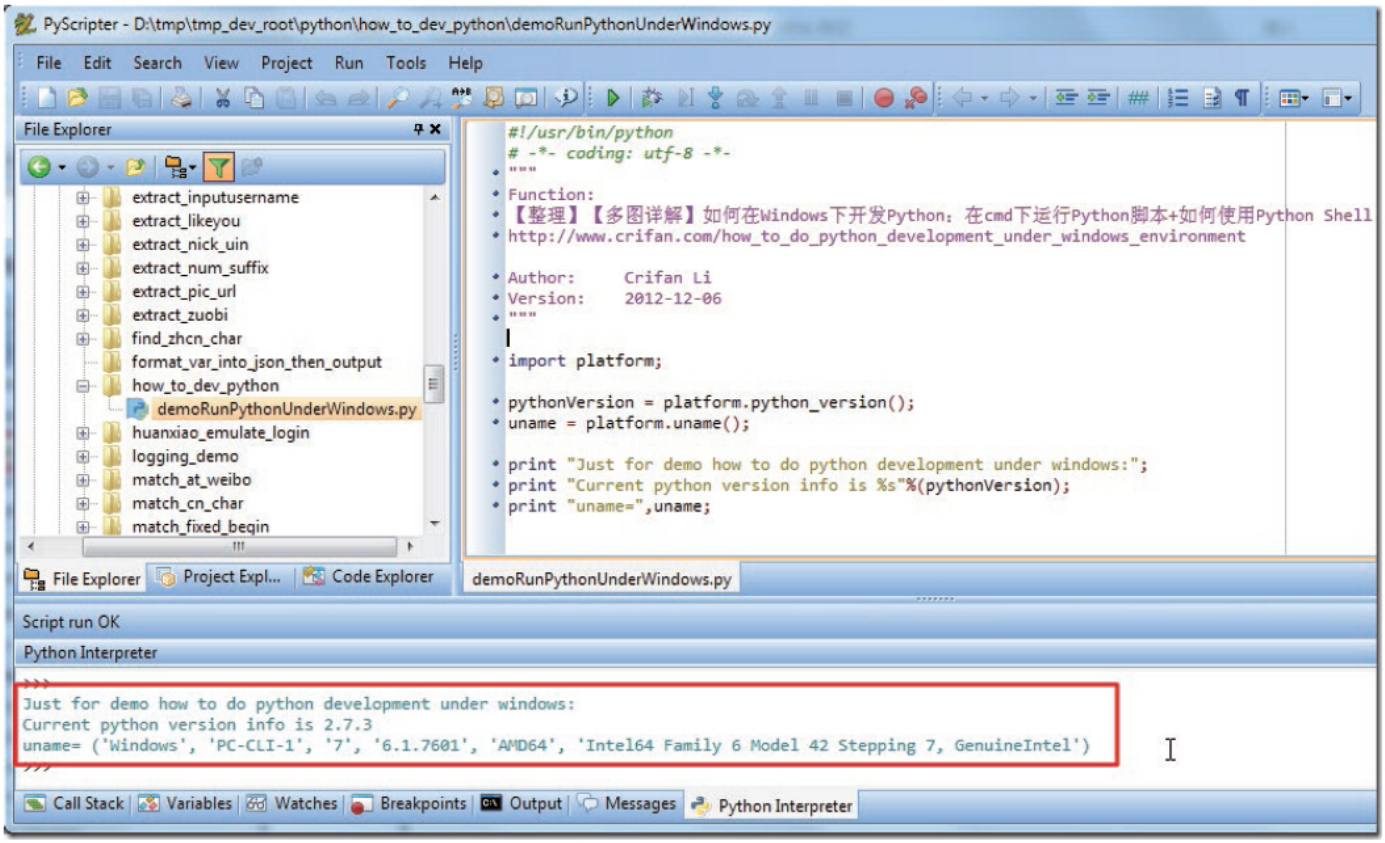


图3-7　PyScripter中的Python Interpreter窗口显示运行结果

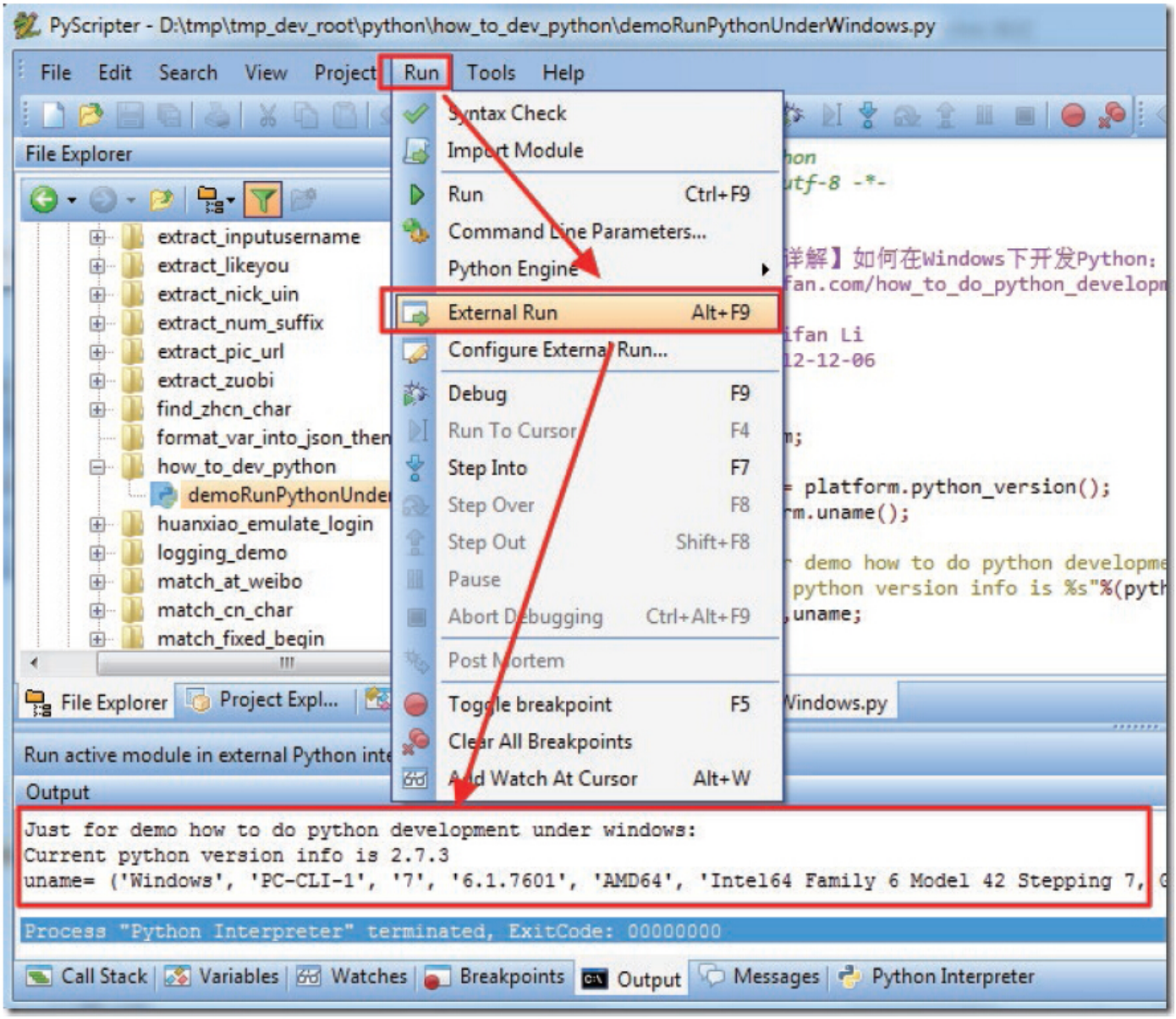


图3-8　PyScripter中的从外部运行代码

2.Eclipse+PyDev

Eclipse是一个通用的IDE框架，包括了基本的文件编辑和各种环境配置方面的功能和框架，支持通过插件进行功能扩展。而PyDev是一个专门用于Python开发的Eclipse的插件，所以此处可以通过Eclipse加上PyDev实现Python开发。下面介绍如何去在Eclipse中下载、安装和使用PyDev。

执行Help→Install New Software→Add命令，在弹出的对话框中添加对应的地址：http://pydev.org/updates，如图3-9所示。

然后选择对应的PyDev for Eclipse并继续安装，如图3-10所示。

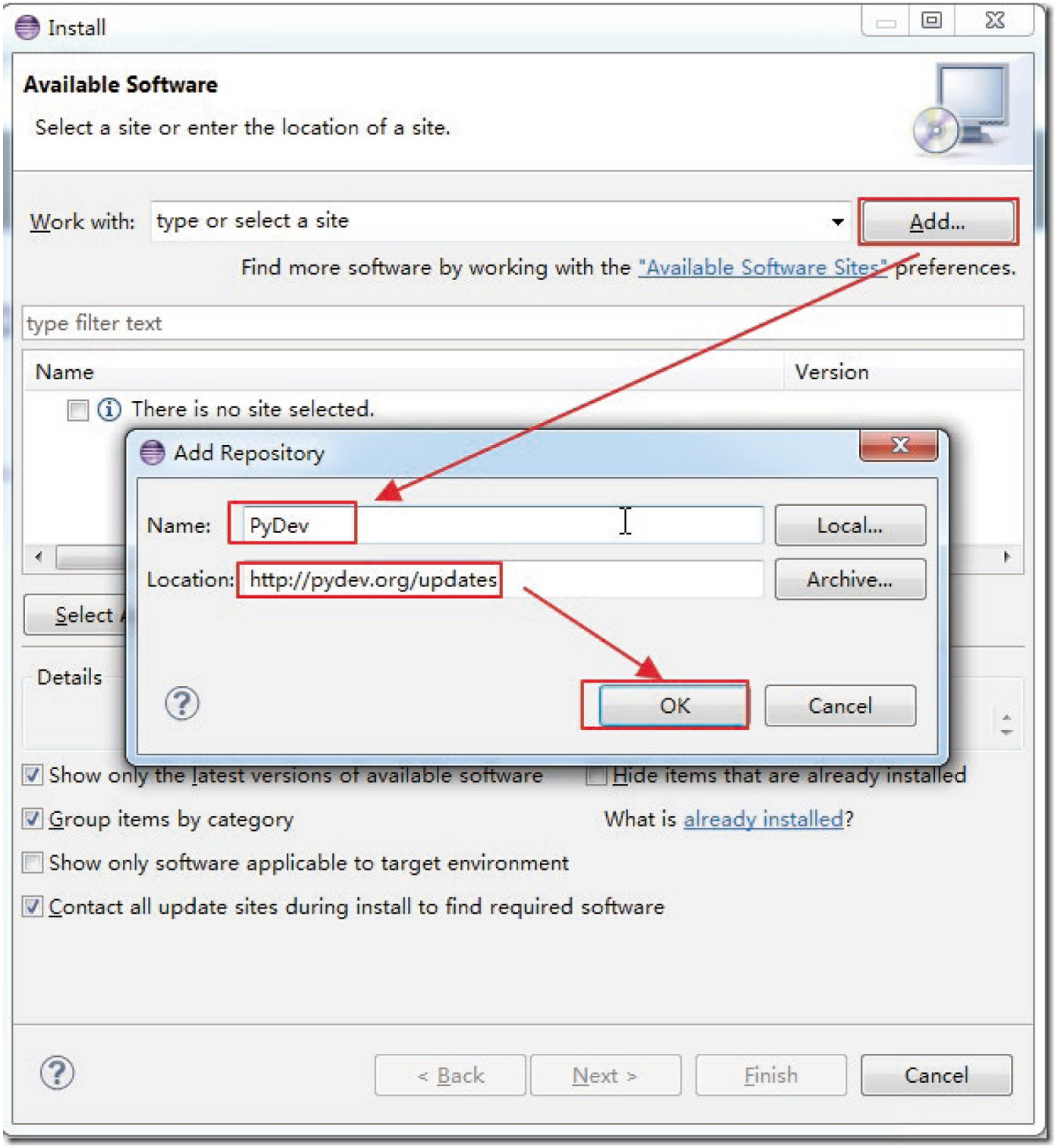


图3-9　在Eclipse中添加PyDev的更新地址

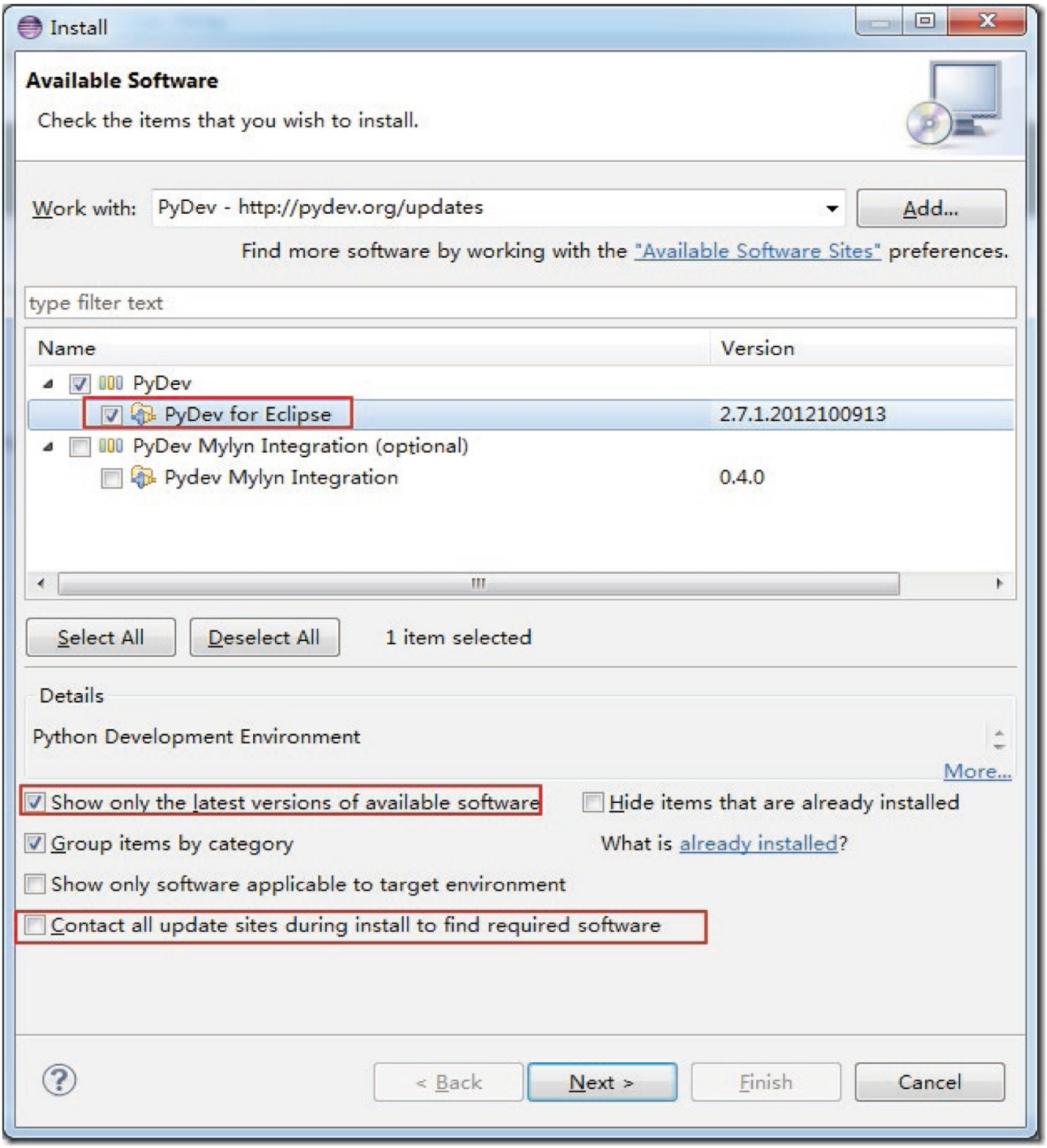


图 3-10　选择 PyDev for Eclipse 并继续安装

图3-10　选择PyDev for Eclipse并继续安装

安装完毕后重启Eclipse即可完成PyDev的安装。在使用PyDev之前需要对其进行恰当的配置，在Eclipse中执行Window→Preferences→PyDev→Interpreter–Python→Auto Config命令，如图3-11所示。

然后就可以自动找到当前系统中已安装的Python了，如图3-12所示。

接着就可以用PyDev去进行Python开发了。执行File→New→Other→PyDev→PyDev Project命令新建PyDev项目，如图3-13所示。

然后输入项目名、选择所用Python的版本等项目的基本信息，如图3-14所示。

Eclipse的视图切换到PyDev浏览模式

需要注意的是，如果此时是新建的Eclipse的话，那么默认显示出来的视图中是看不到PyDev的项目的。所以如果需要看到PyDev的界面则需要先去切换到PyDev的视图，执行Window→Show View→PyDev Package Explorer命令即可，如图3-15所示。

再单击一下Restore就可以切换到PyDev的视图了。

再去PyDev项目中新建一个pydevTest.py文件，如图3-16所示。

在新建的文件中输入测试代码，如图3-17所示。

再去运行Python代码，如图3-18所示。

选择对应的Python Run运行Python代码，如图3-19所示。

此时就可以在Eclipse的PyDev的Console中看到输出结果了，如图3-20所示。

PyDev支持代码补全提示功能

此处可以注意到PyDev是支持代码补全提示的功能的，如图3-21所示。

总体来说，使用PyDev去开发Python的效果还是不错的。

另外一个Python的ID：Aptana Studio 3

此处顺带提及一下，另外还有个IDE叫作Aptana Studio 3。该IDE其实就是基于Eclipse+PyDev的内核，然后专门设计用于Web领域开发的，使用它开发Python的方式和Eclipse+PyDev是一样的。

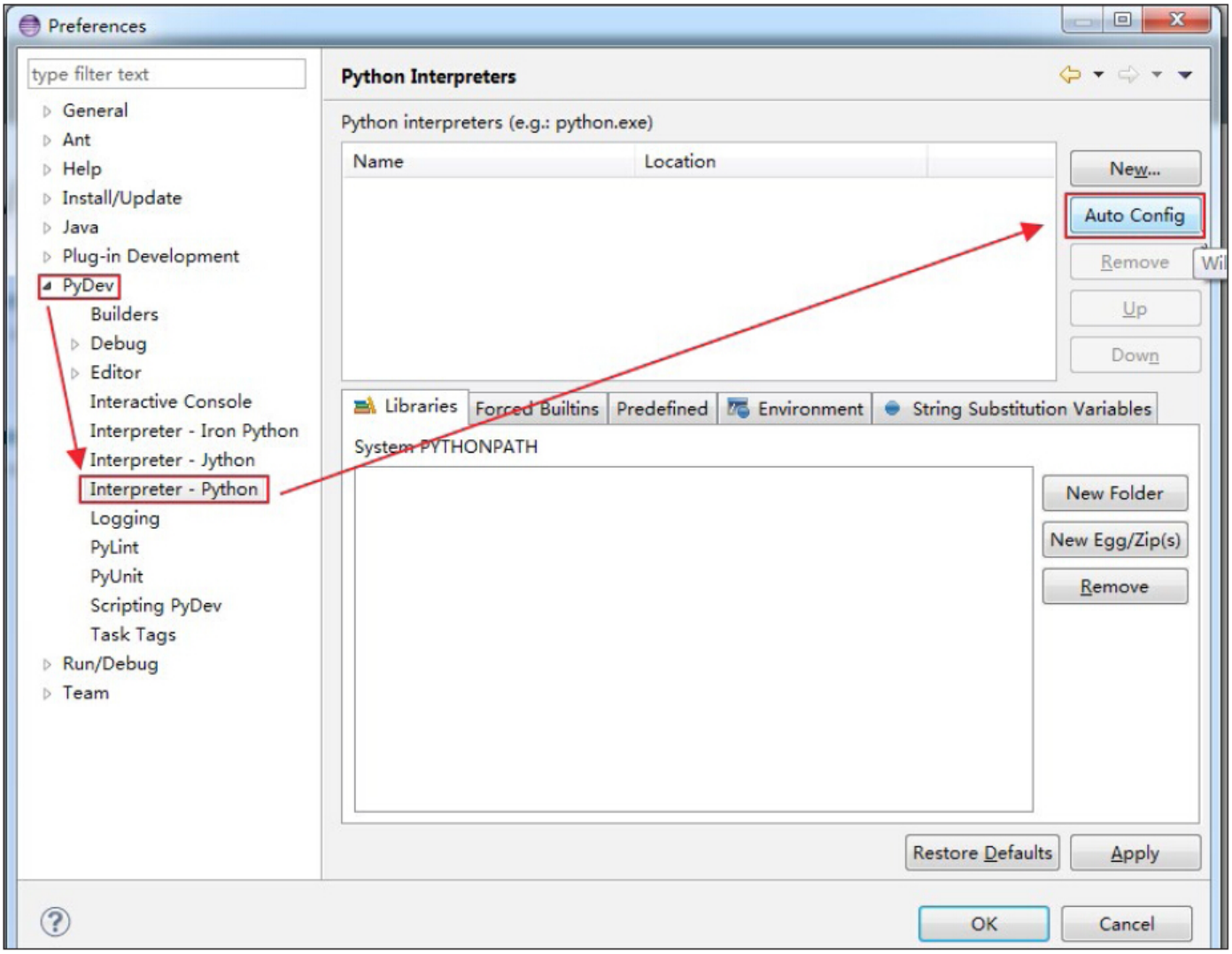


图3-11　Eclipse中自动配置PyDev

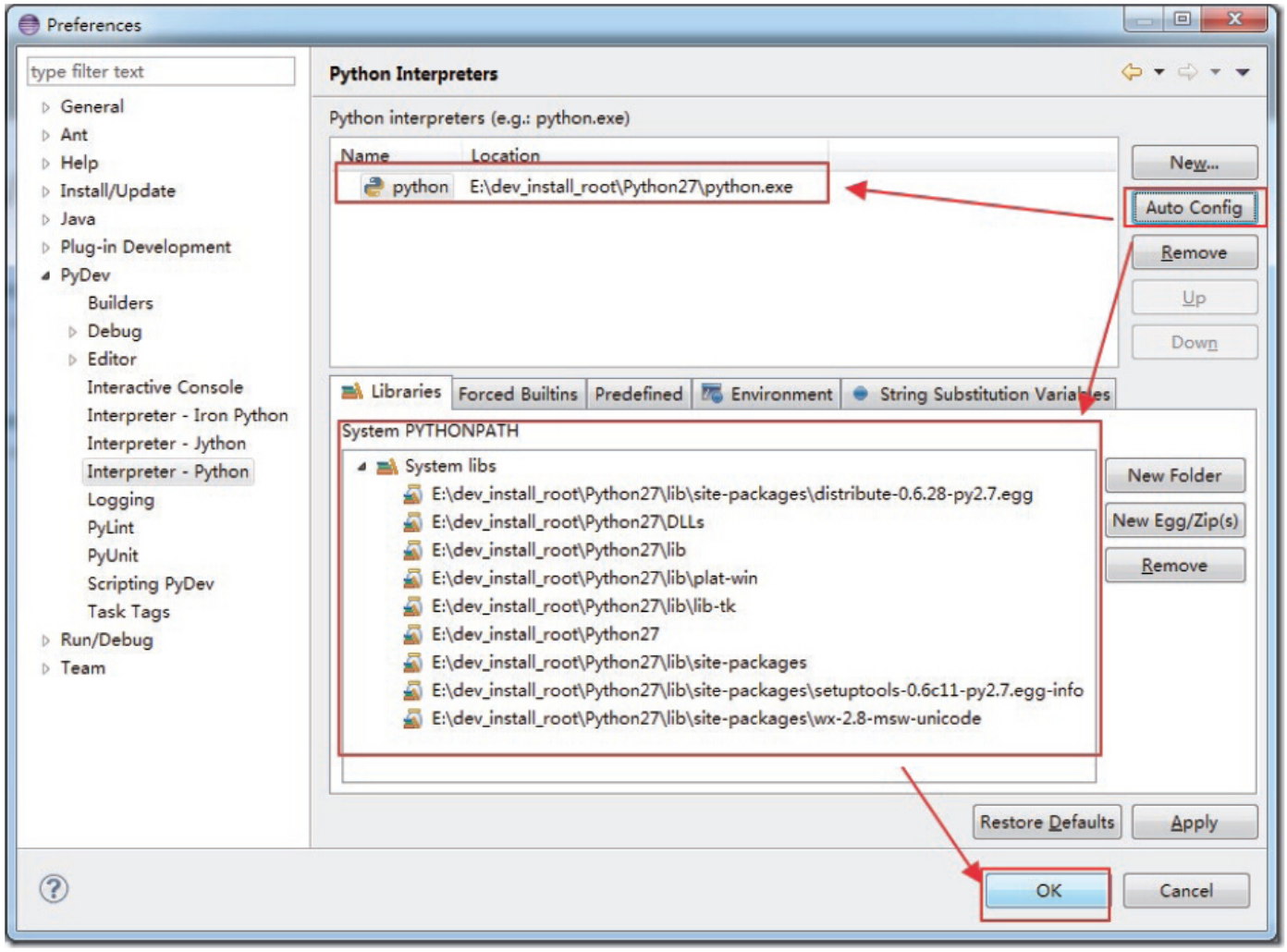


图3-12　PyDev自动找到已安装的Python

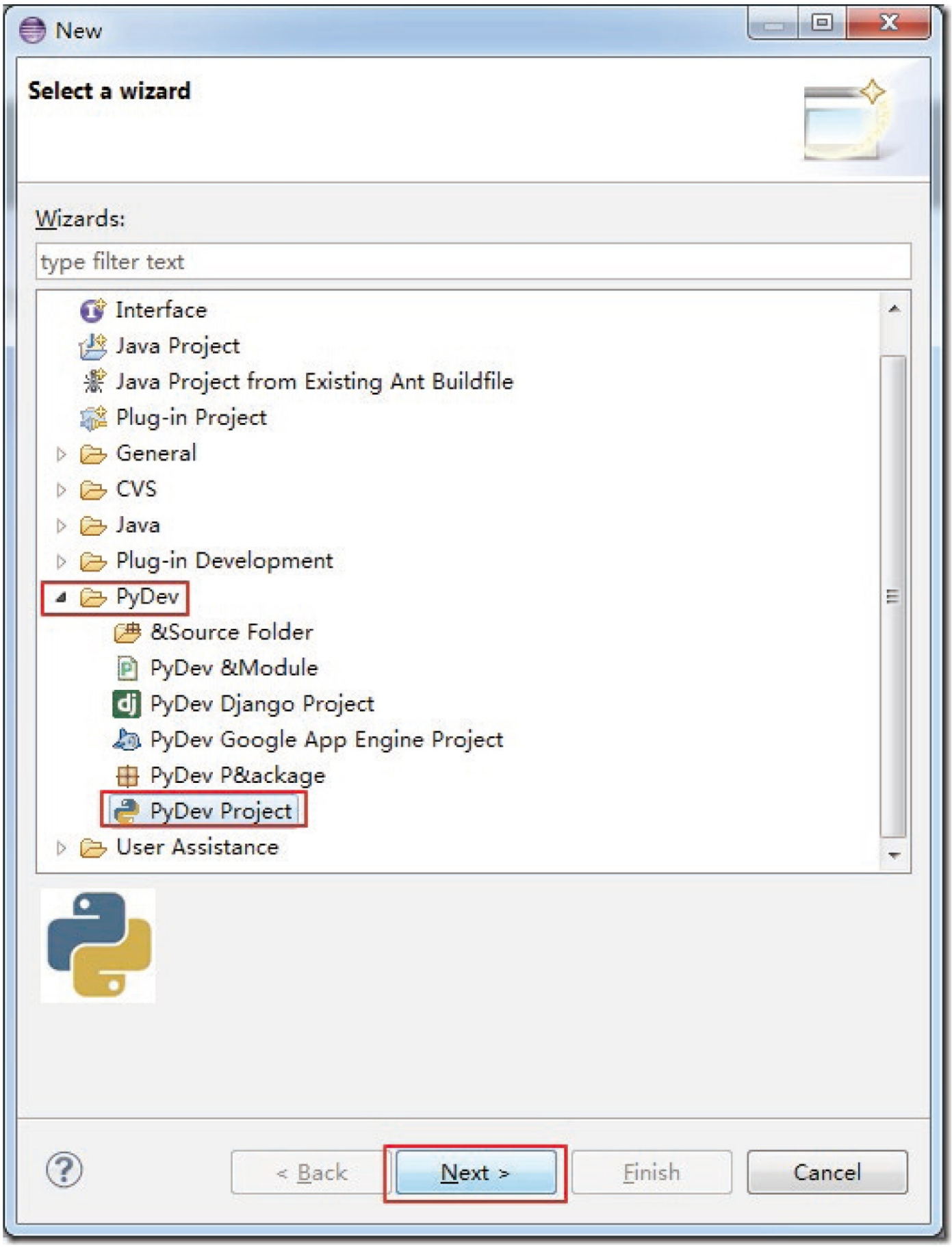


图3-13　Eclipse中新建PyDev项目

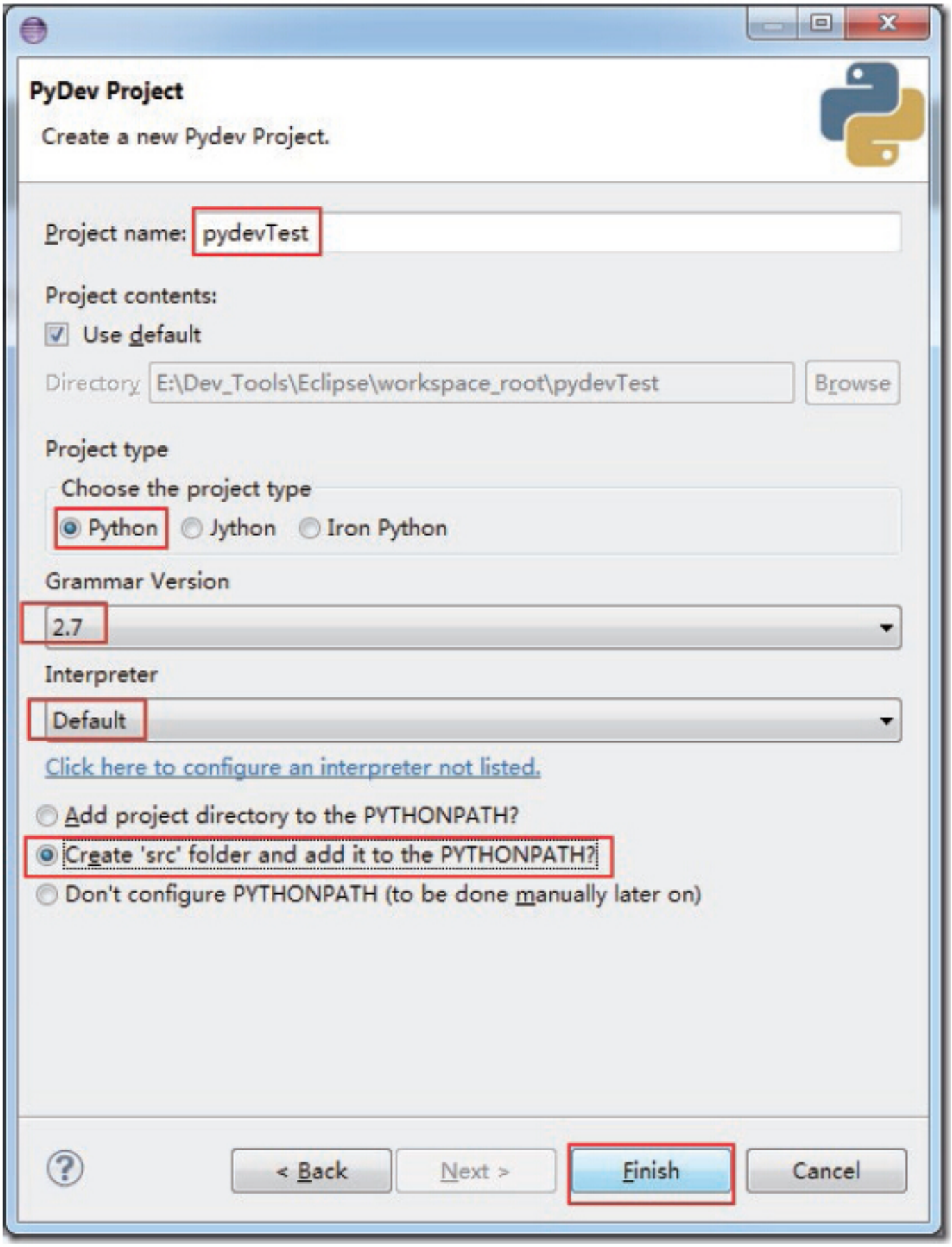


图3-14　配置PyDev项目信息

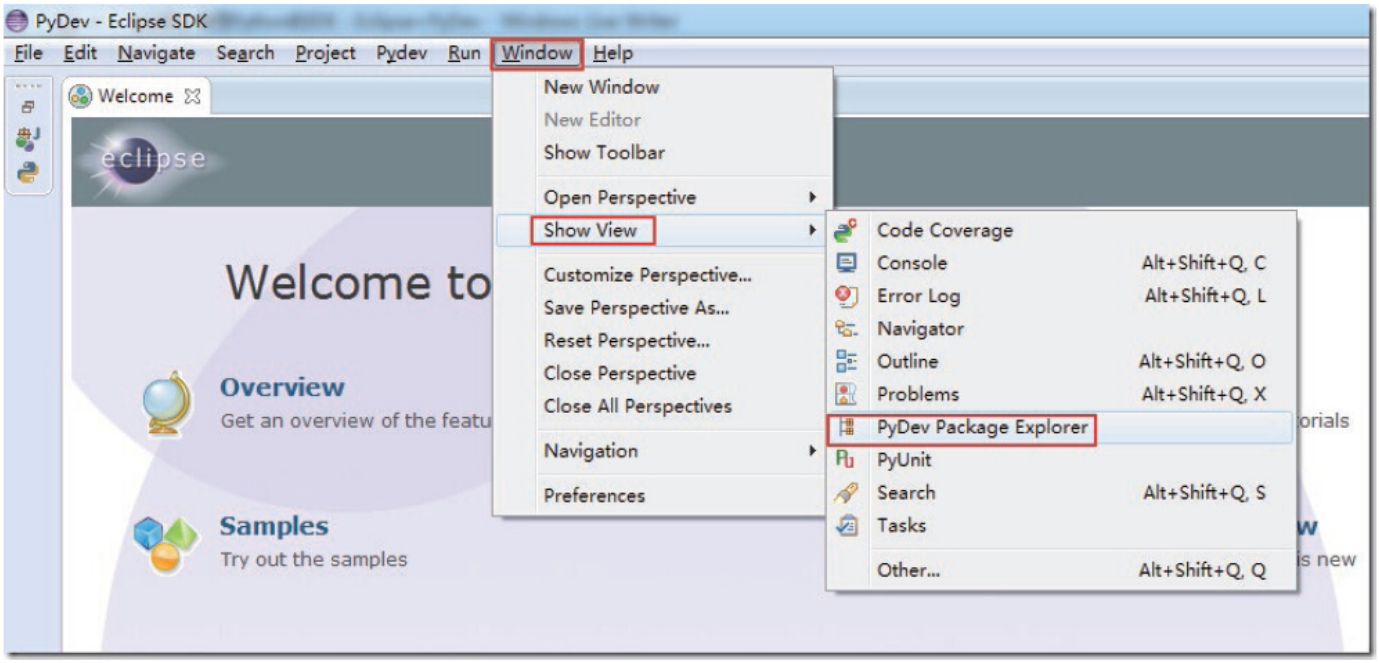


图3-15　切换到PyDev的视图

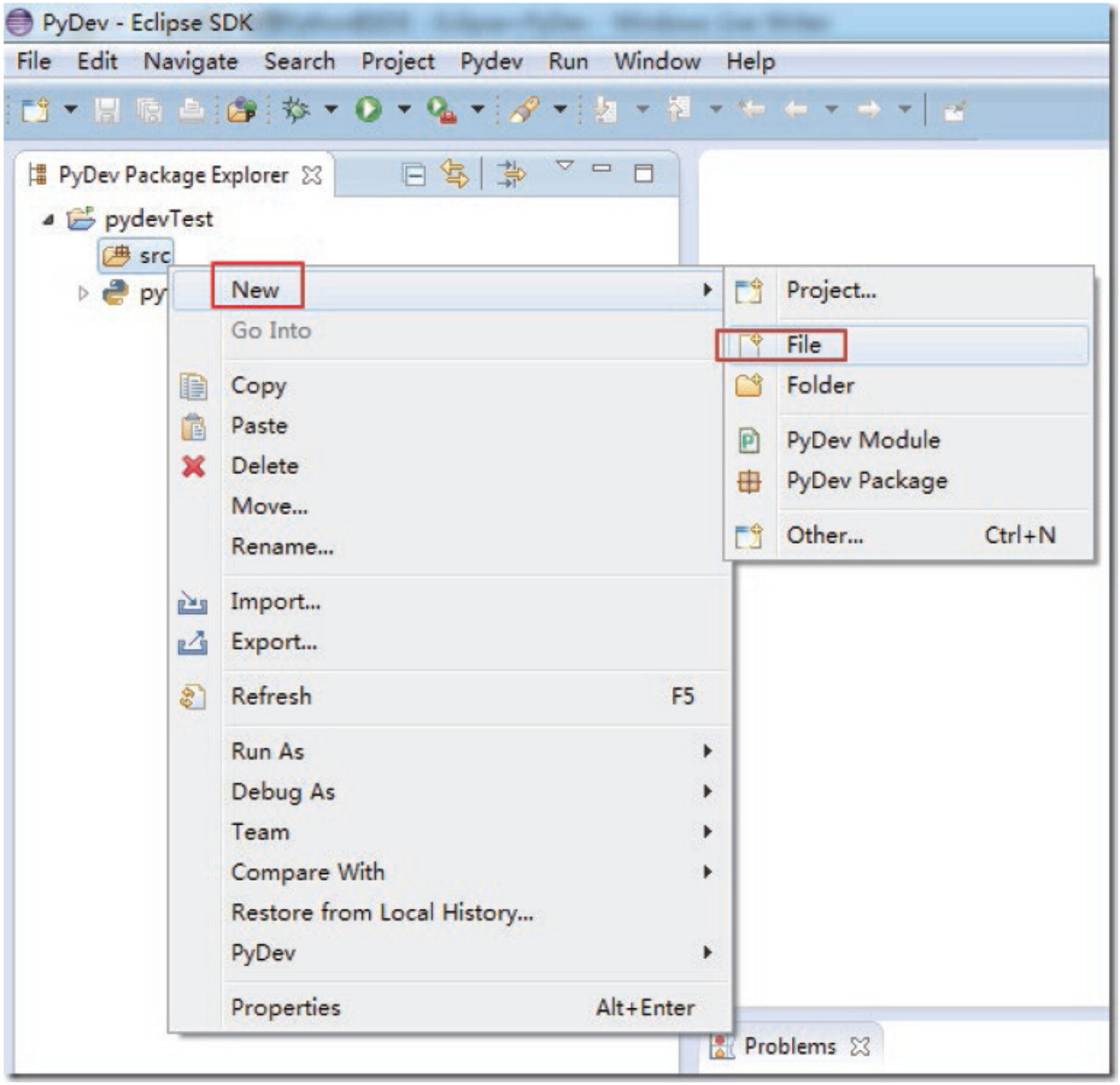


图3-16　PyDev中新建文件

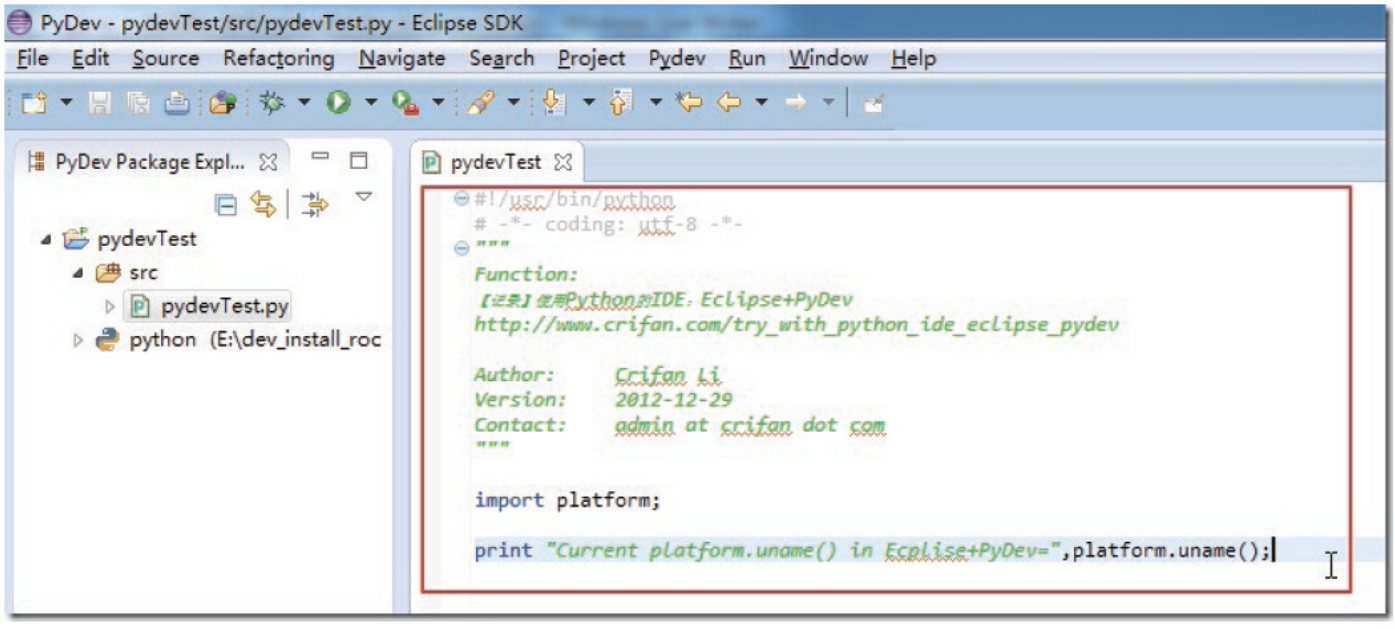


图3-17　在Python文件中输入测试代码

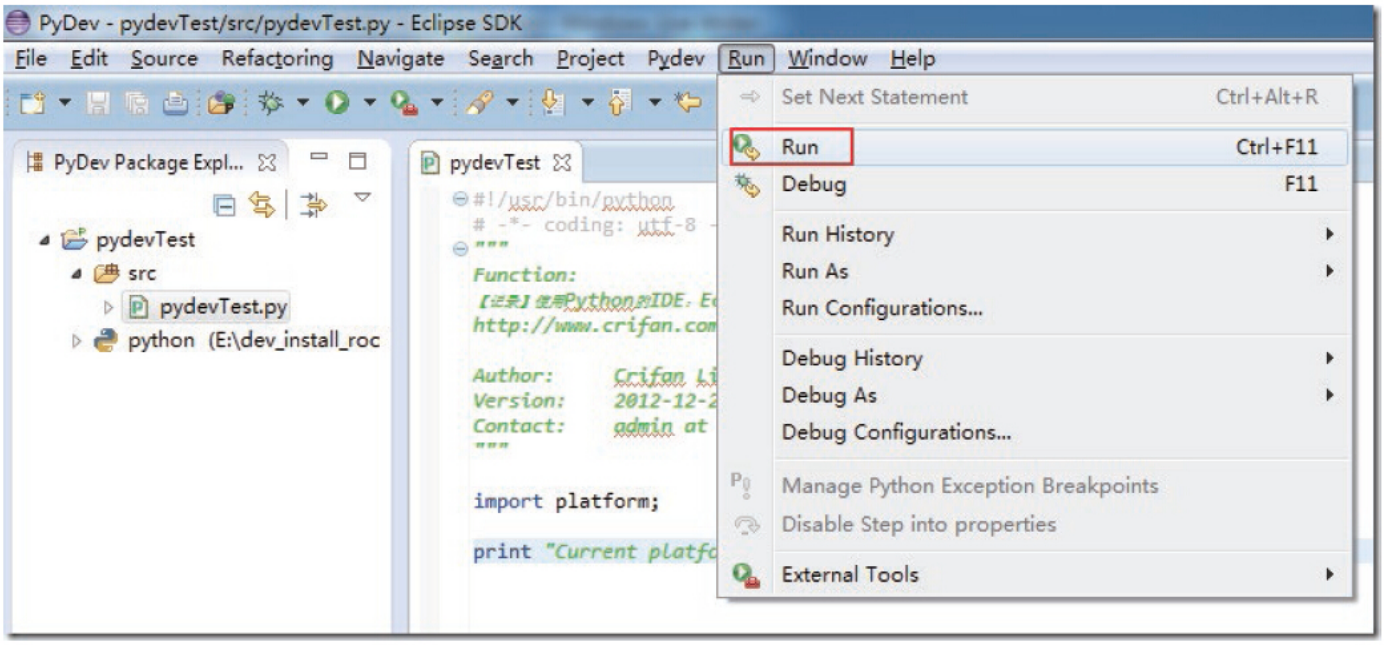


图3-18　在Eclipse中运行Python代码

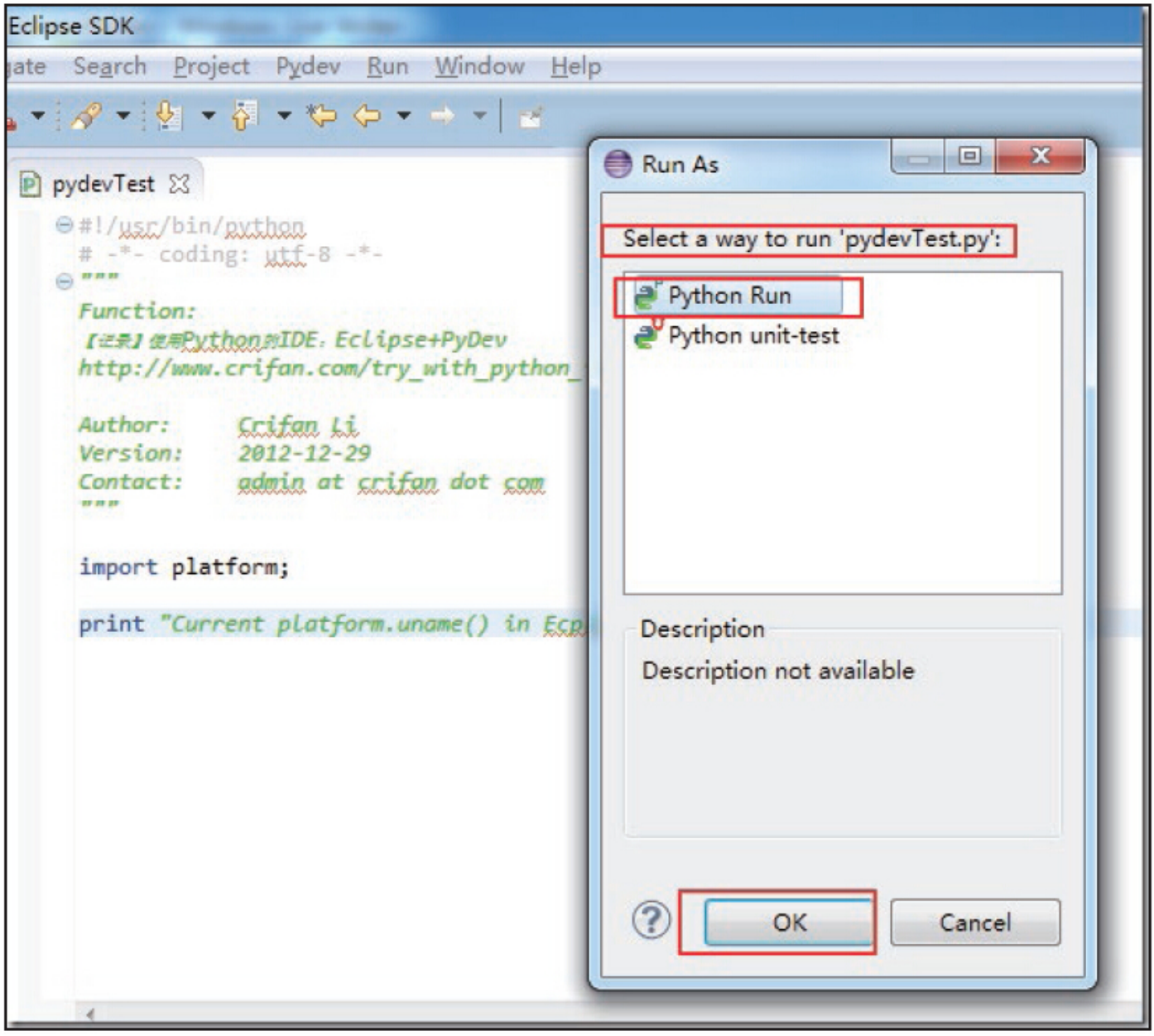


图3-19　选择Python Run去运行Python代码

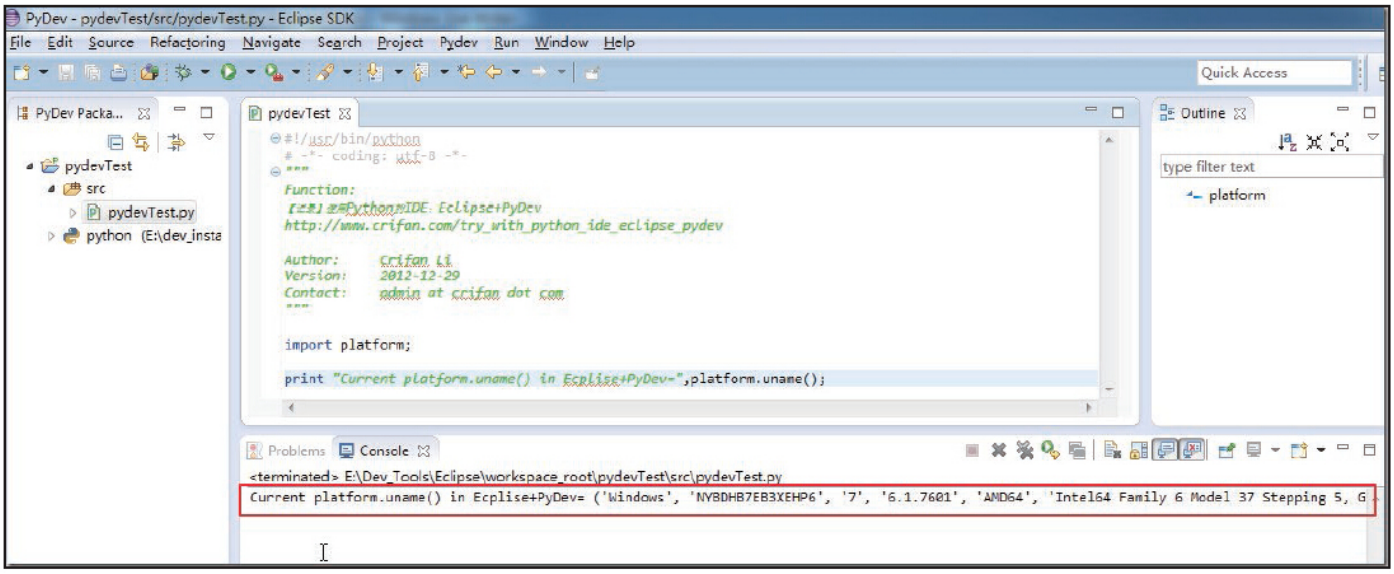


图3-20　Eclipse的PyDev的console中显示运行结果

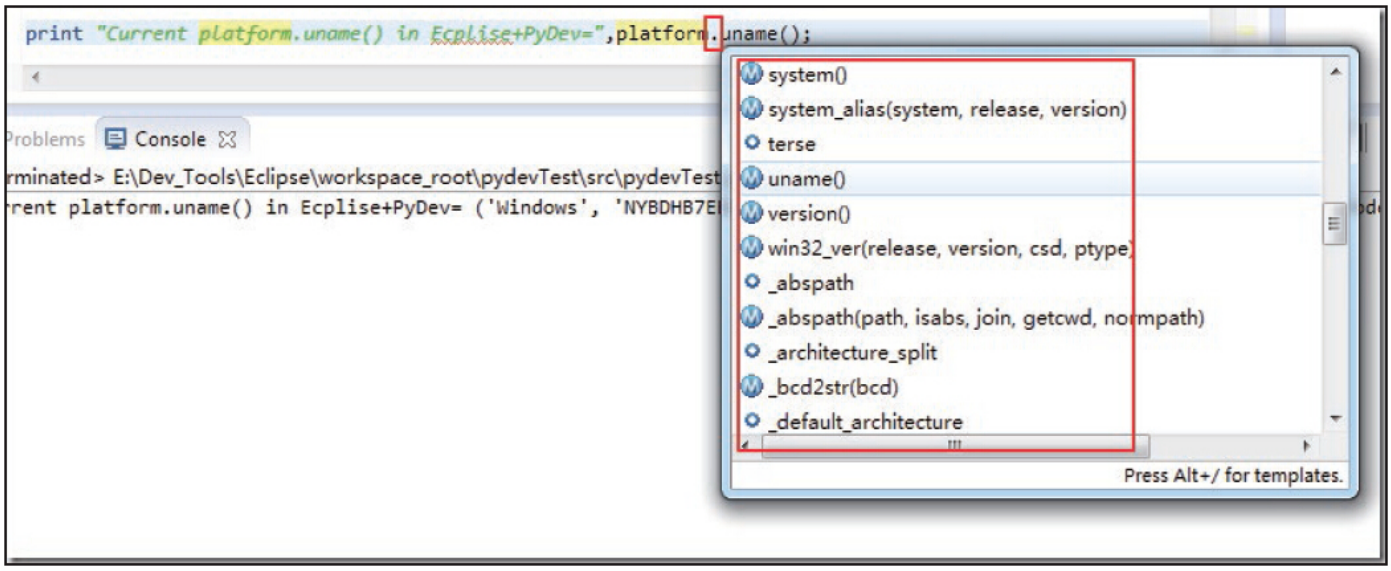


图3-21　PyDev支持代码补全

3.Ulipad

下面介绍如何下载和使用Ulipad这个Python的IDE。去Ulipad的主页中的下载页面（地址：http://code.google.com/p/ulipad/downloads/list）中下载自己所需的版本，当前Python版本是2.7，所以需要下载的是ulipad.4.1.py27.exe。

下载后双击安装即可，安装完毕后再去启动Ulipad。

无法启动Ulipad

如果安装完毕后无法启动Ulipad，则可以去给启动参数中加上-n参数，如图3-22所示。

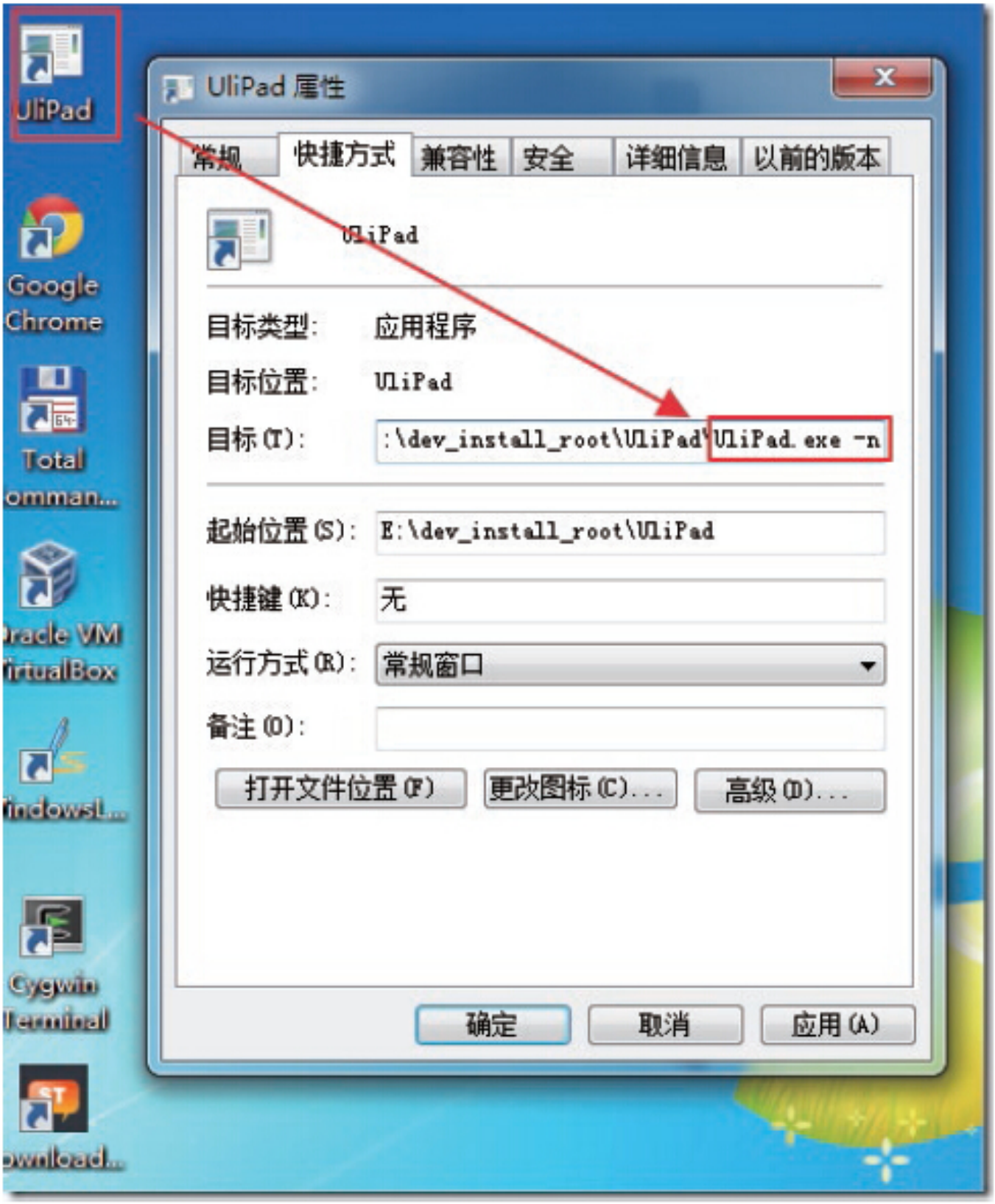


图3-22　给Ulipad加上-n启动参数

如此就可以正常启动Ulipad了。

启动后的Ulipad的主界面如图3-23所示。

然后打开之前我们的Python测试代码文件，如图3-24所示。

在使用Ulipad开发Python之前，需要先配置Ulipad，然后通过执行“编辑”→“参数”→Python→“设置Python解释器”命令指定使用哪个Python解释器，如图3-25所示。

改为自己当前Python的安装路径下面的python.exe和pythonw.exe，如图3-26所示。

然后再去运行Python代码，如图3-27所示。

最后在信息窗口中显示输出结果，如图3-28所示。

4.PyCharm

PyCharm是另外一个Python的IDE，且是收费的。PyCharm尤其适合用于Web开发，比如Django、GAE等。

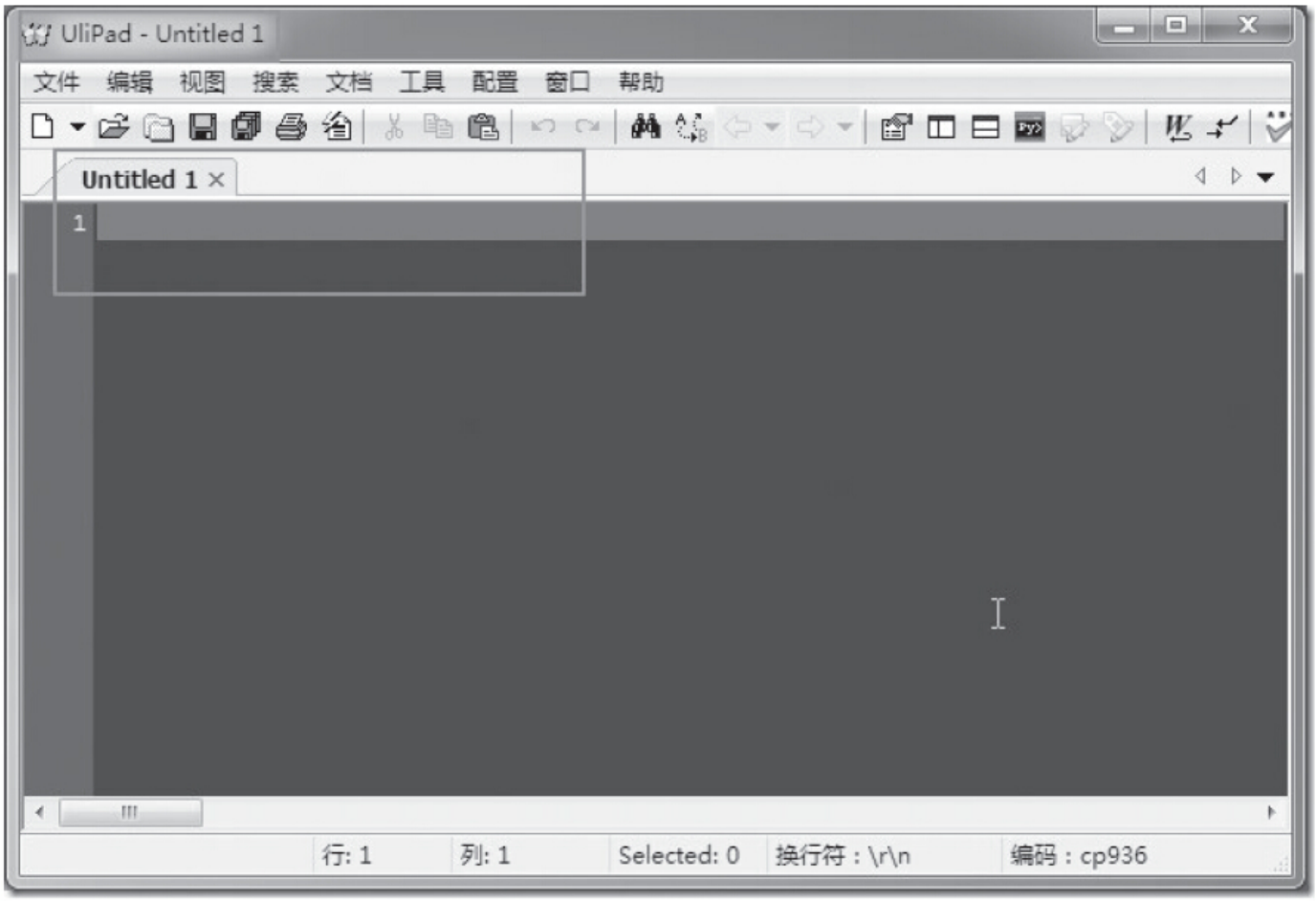


图3-23　Ulipad主界面

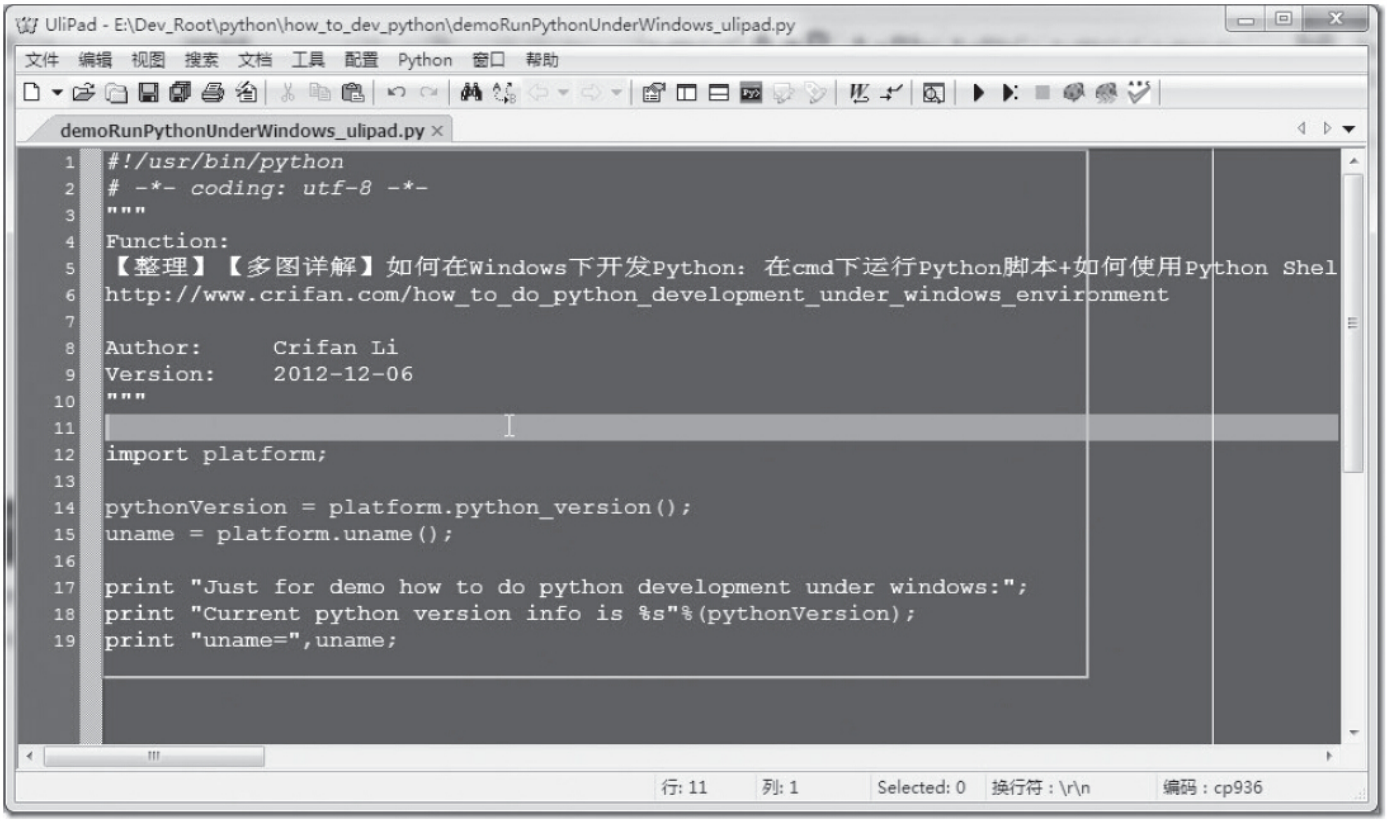


图3-24　Ulipad中打开Python测试代码文件

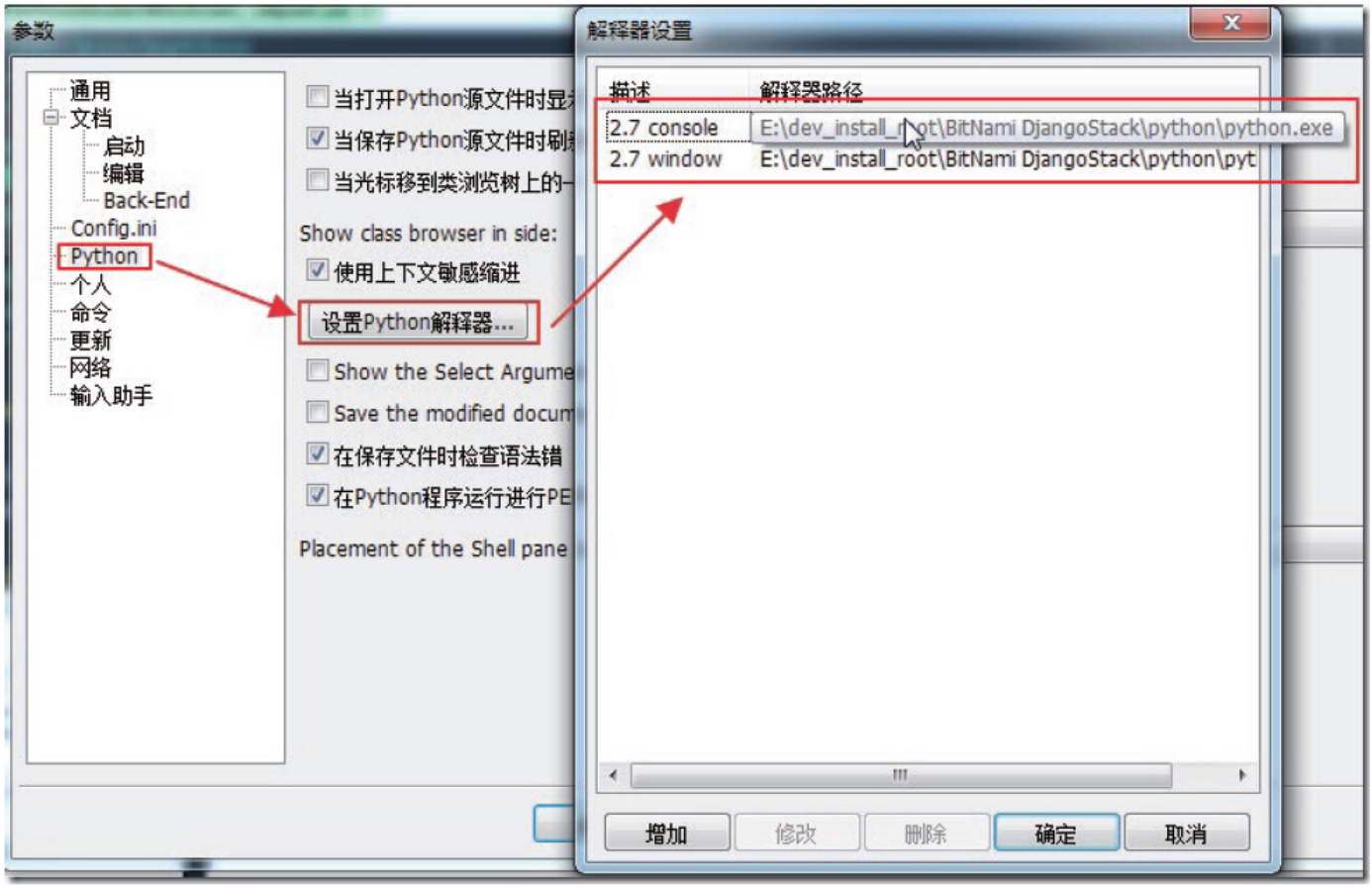


图3-25　Ulipad中设置Python解释器

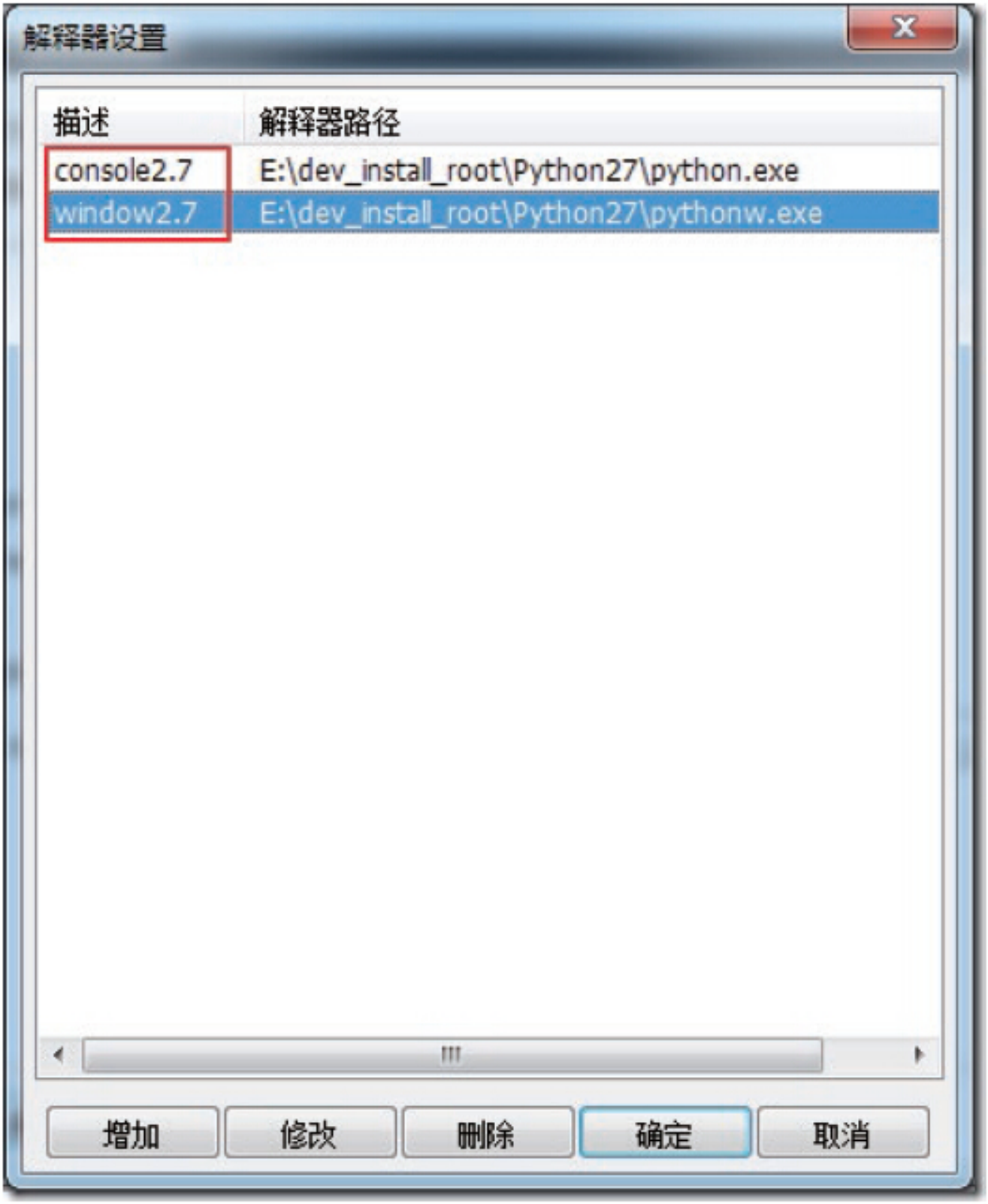


图3-26　Ulipad中设置Python的console和window解释器

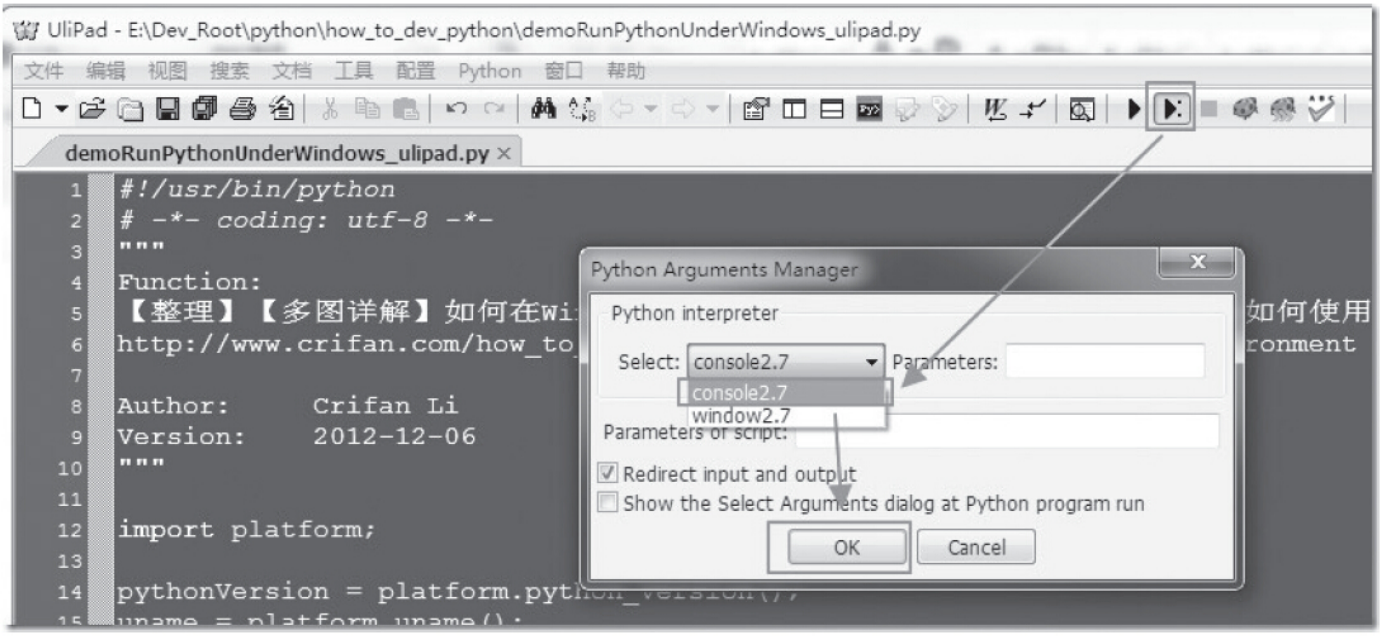


图3-27　Ulipad中运行并选择Python的console2.7

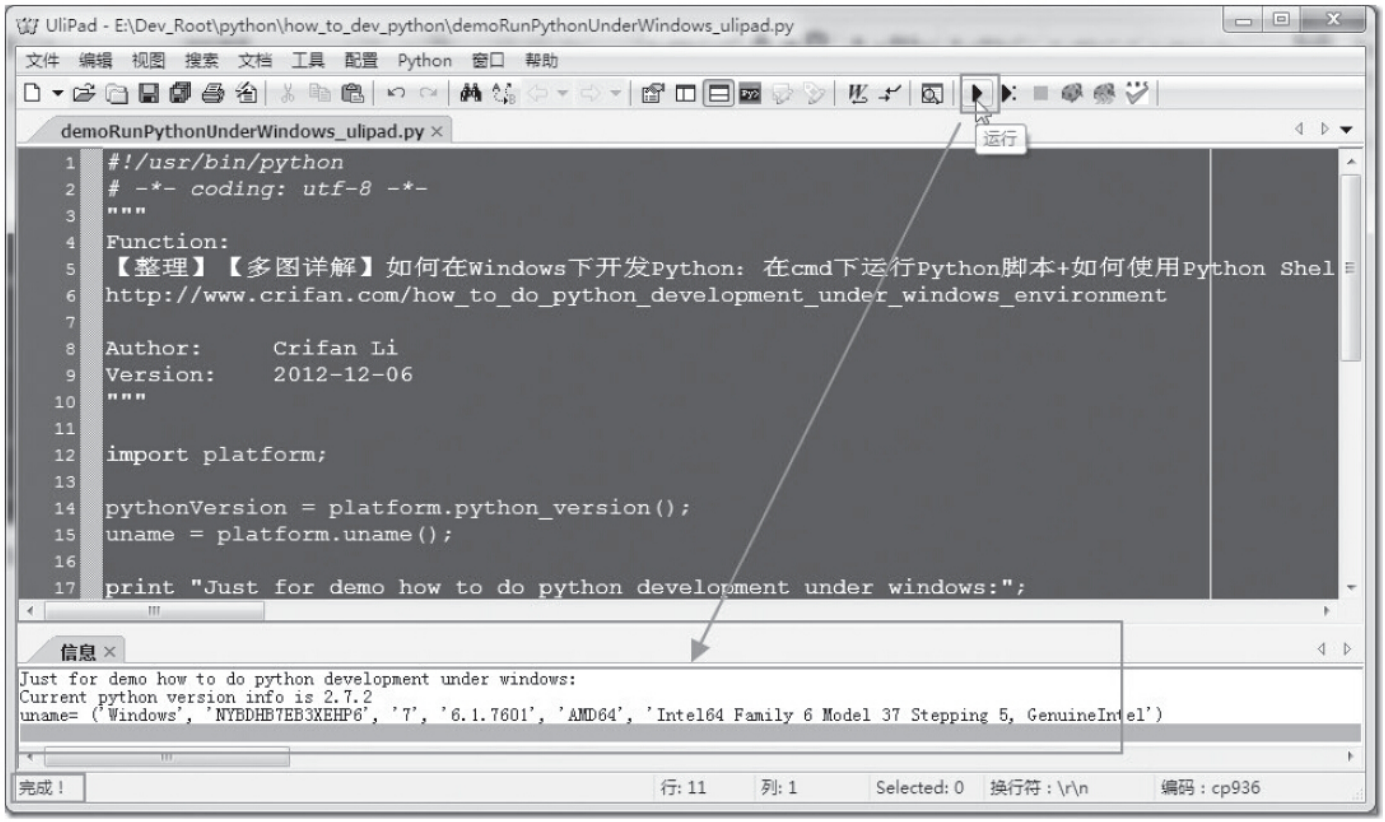


图3-28　Python代码运行结果在Ulipad的信息窗口中显示

5.Eric

为了专门用于QT开发，别人又设计出一个Python的IDE：Eric。更多详情可参考其官网：http://eric-ide.python-projects.org/index.html。

3.3.3　Python IDE常见问题及解答

1.IDE中打开了文件并不代表已经运行了文件

笔者之前遇到一个朋友，他在使用IDE期间由于对相关概念不了解而犯了一个典型的错误，如图3-29所示。

图3-29中在当前的IDE中打开了一个.py文件：balanced_color_module.py。如果读者概念很清晰的话，应该明白接着应该去菜单中运行代码，然后再去对应的console中（此处估计是Debug I/O）去查看运行的结果。而如果不清楚概念的话，则会不清楚如何才能查看程序运行结果。

此处另外很明显的一点是，此处的那个py文件（balanced_color_module.py），即使去运行该文件也不会有什么print输出信息。另外，此处没有打开对应的console而打开的是Python Shell，所以从逻辑上就是：其在Python Shell中输入的任何代码，本质上和当前IDE中打开的py文件没有任何关系。所以导致的结果就是，虽然在Python Shell中输入了一些代码：

```
color = {"red":0.2, "green":0.4, "blue":0.4}
balanced_color(color);
```

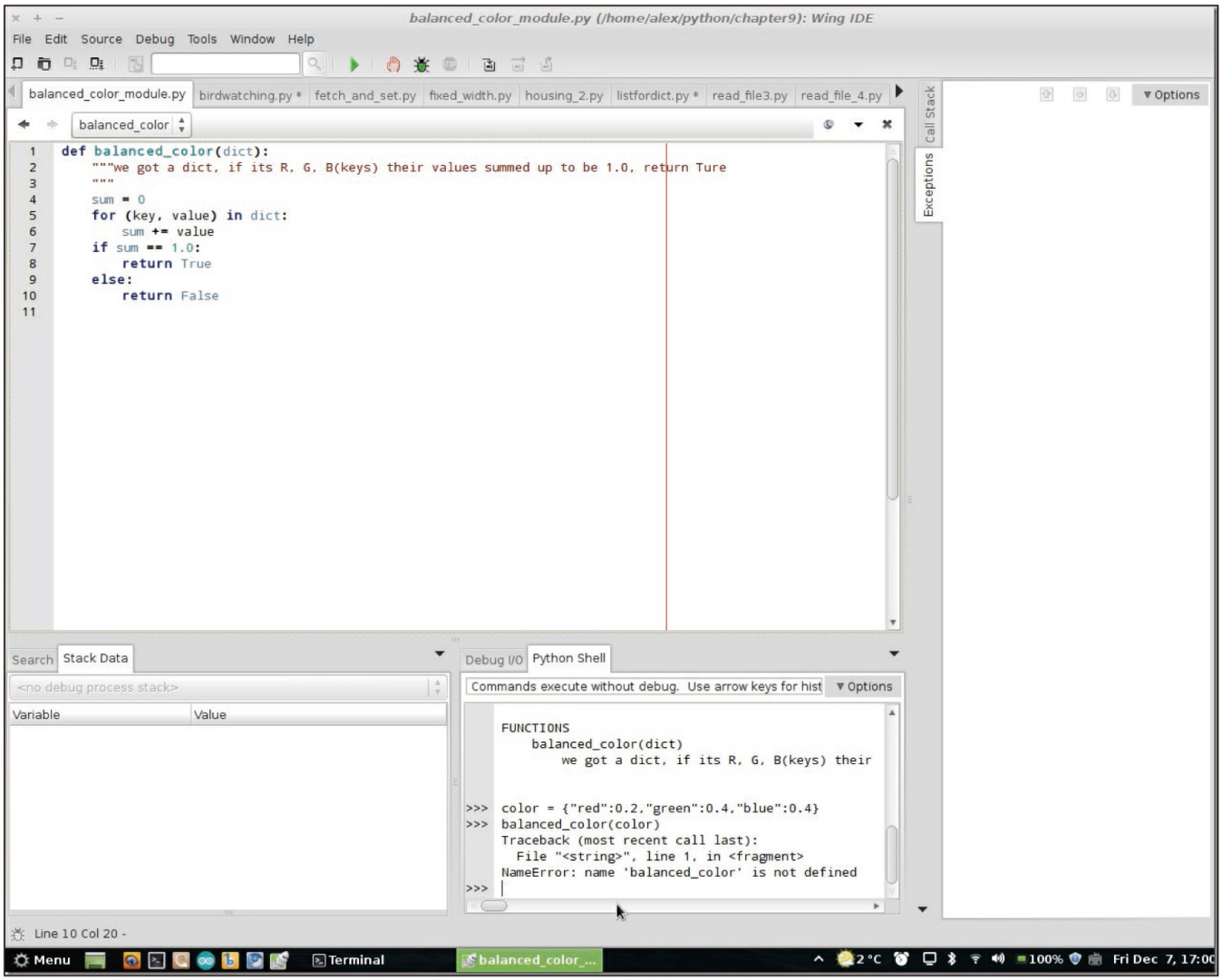


图3-29　因概念不清而犯的典型错误

但是结果却出错，找不到对应的balanced_color函数，此结果也是在常理之中、预料之内的了。因为此时其所做的事情是：使用了IDE，但是并没有使用IDE中集成的终端调试输出，却另外打开了Python Shell。而由于此处的IDE中打开的代码和Python Shell没有关系，所以才会运行代码出错，找不到对应的函数。而正确的逻辑是要么使用文本编辑器+终端运行Python代码，要么使用IDE去编码和调试。且确保自己的代码可以正常运行，能否正常输入结果。

2.需要确保有可以运行的Python起始部分的代码

某读者碰到另外一个问题：通过IDE中运行Python代码但是没有响应，如图3-30所示。

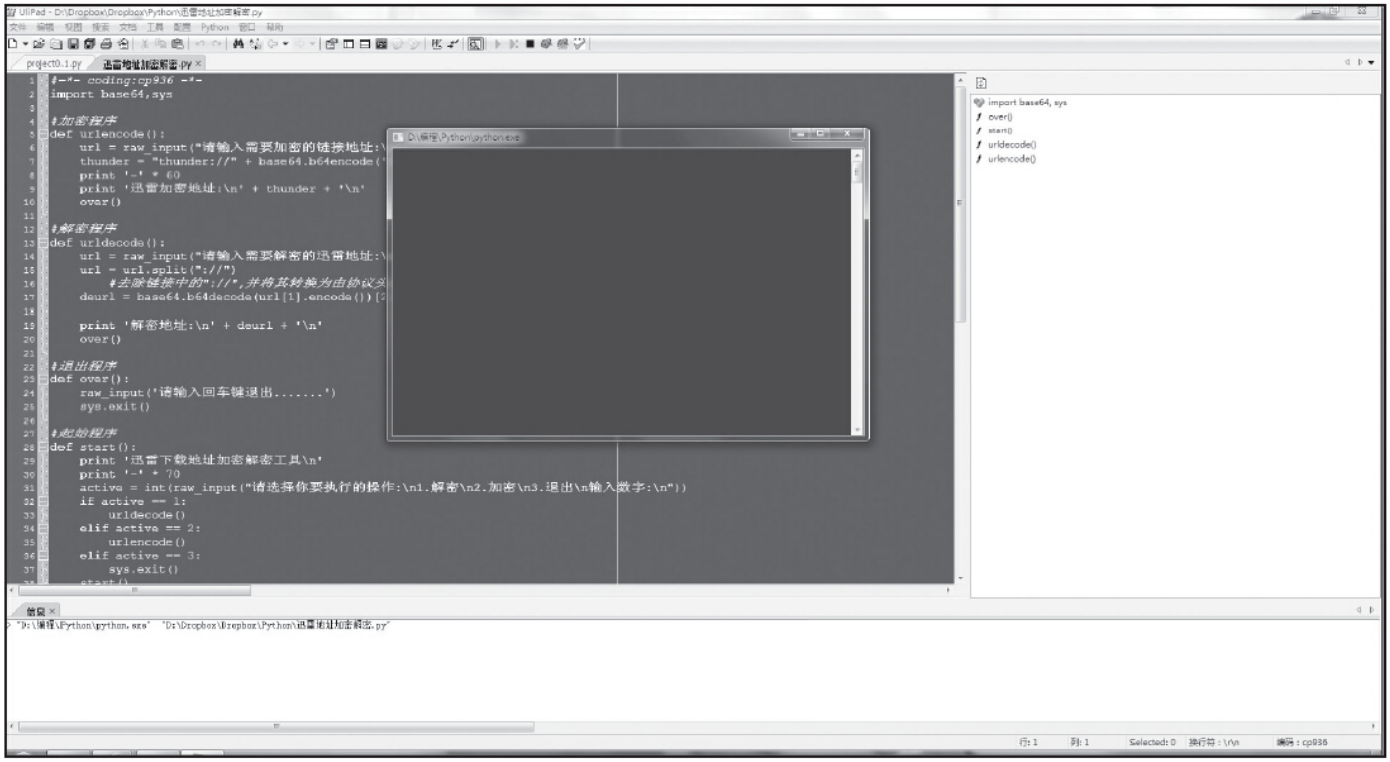


图3-30　运行Python后无响应

如上问题的本质上和IDE没有关系，但是不熟悉Python的人还是容易在IDE中犯这类问题。其错误原因是：没有起始代码可以运行。

解决办法是将上面最后一行的

```
    start();
```

去掉缩进而顶行写，变成：

```
start();
```

即可使Python执行到start函数了。当然最好的写法是将

```
start();
```

再改为

```
if __name__ == "__main__":
    start();
```

关于如此修改的原因和含义，请参见4.3节。

3.4 在Windows环境下进行Python开发

在明白了Python的基本的开发逻辑后，再来看看具体是如何在Windows下开发Python的。

3.4.1 最原始的Python开发方式

Windows中相对来说最原始和最基本的开发Python的方式有如下几种。

1）找个Windows中的文本编辑器，新建个.py文件，写上Python代码。

Windows中最常见的文本编辑器有Notepad，但是由于其功能太弱且不够好用，所以前面才推荐在Windows中使用Notepad++这个更好用的文本编辑器的。

为何推荐用Notepad++去写Python代码

此处推荐用Notepad++去做Python开发的原因，除了本身Notepad++作为文本编辑器好用之外，还有一个相对来说更重要的原因：Notepad++对于字符编码的支持更好。

具体解释一下就是，在进行Python开发期间，很多时候会因Python开发者对于Python文件的编码不太了解而导致出现很多字符编码方面的问题。

因此需要在文本编辑器中可以方便地查看当前文件的编码以及可以尝试用不同的编码去解析同一份内容，最好能方便地在不同编码格式之间转换，而Notepad++在这方面支持很好。

所以，为了更加深入地学习和真正地掌握Python就有必要从一开始学习Python时就选用合适的工具，以此尽量避免后期可能出现的字符编码方面的错误，以及在出现字符编码的问题时能够快速定位问题的根本原因并找到合适的解决办法。

下面就来具体解释如何使用Notepad++去开发Python。

打开Notepad++后，单击“新建”按钮后会出现一个新的文件，如图3-31所示。

然后将之前提到的示例代码1粘贴至此处，如图3-32所示。

此时细心的读者会注意到Python代码没有语法高亮显示，那是因为Notepad++不知道此处新建的空白文件中的内容是Python代码，没法帮你自动实现语法高亮。为了实现代码高亮则需要手动去设置一下，执行语言→P→Python命令即可，如图3-33所示。

此时就可以看到Python代码语法高亮的效果了，如图3-34所示。

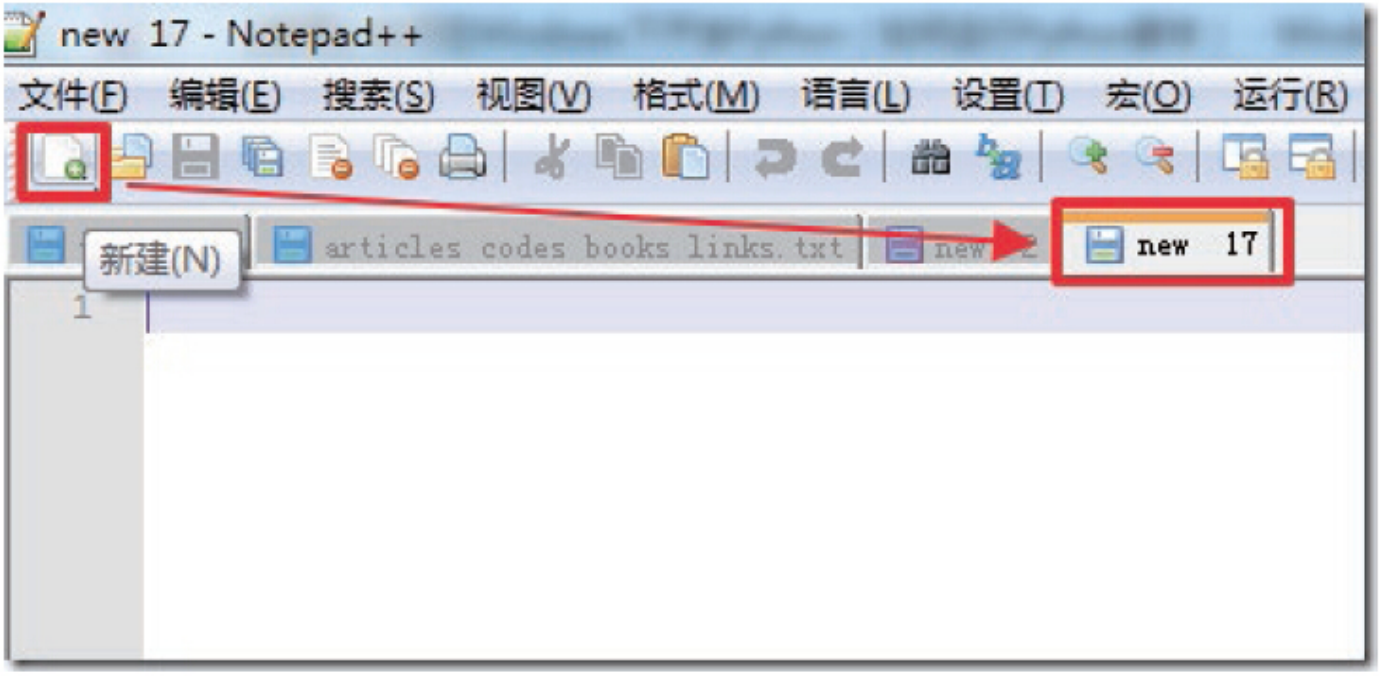


图3-31　新建文件

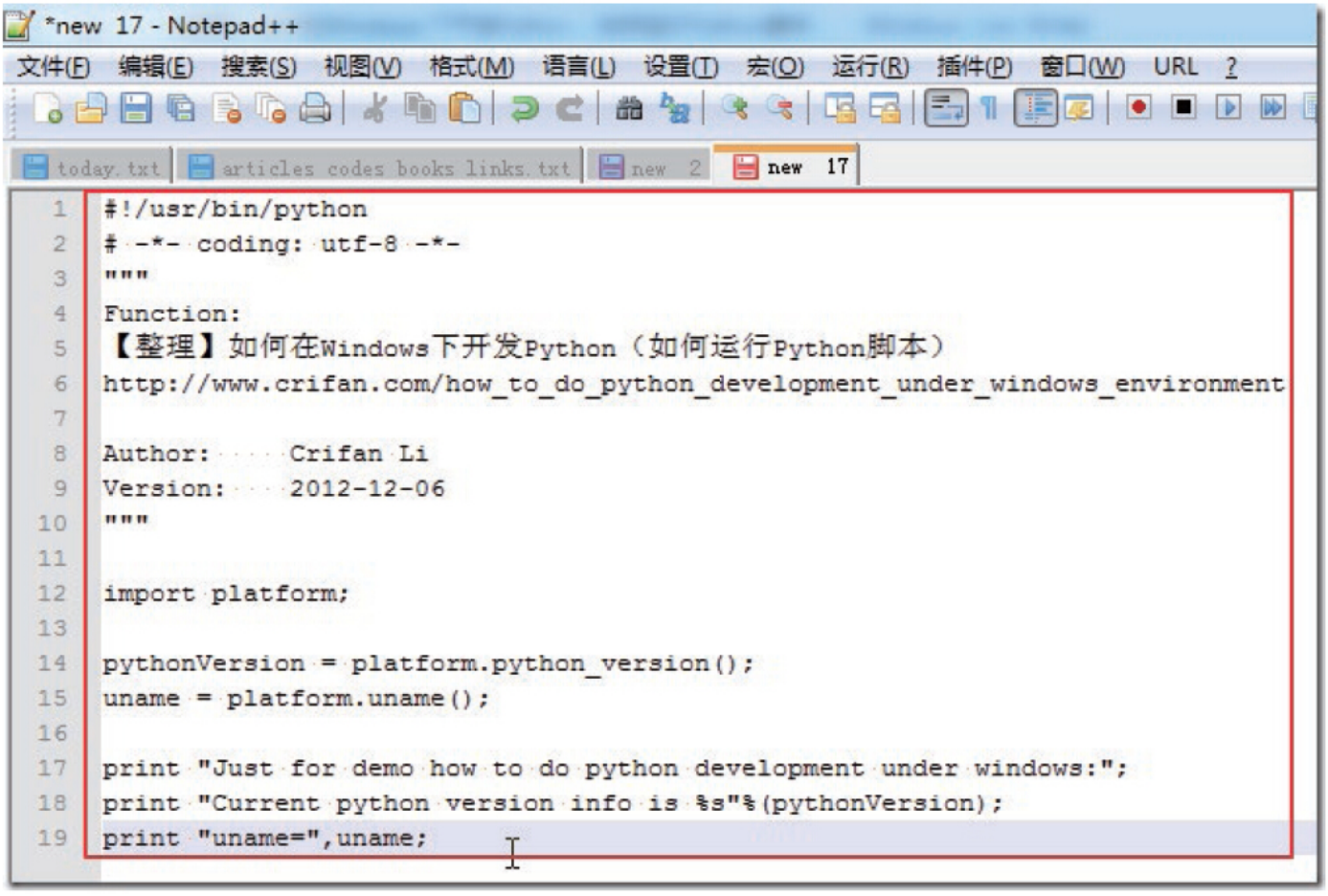


图3-32　将示例代码1粘贴至新建文件夹中

然后将代码文件保存到某个位置，执行“文件”→“另存为”命令，在弹出的对话框中输入要保存的文件名demoRunPythonUnderWindows。

其中可以看到Notepad++帮忙自动写好了.py后缀，那是因为我们之前设置了Python语法高亮。然后再选择对应的路径去保存，如图3-35所示。

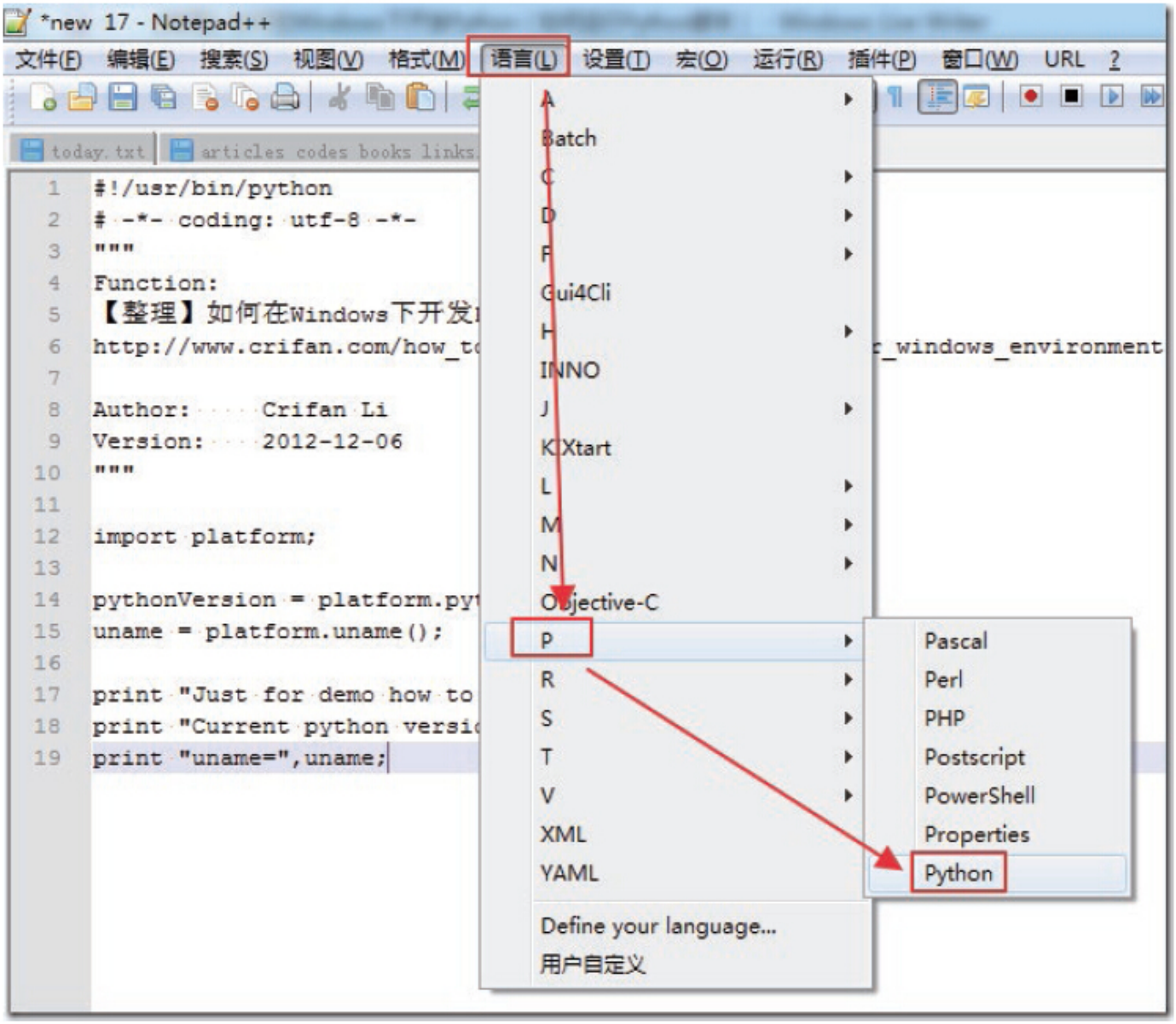


图3-33　手动实现代码高亮显示

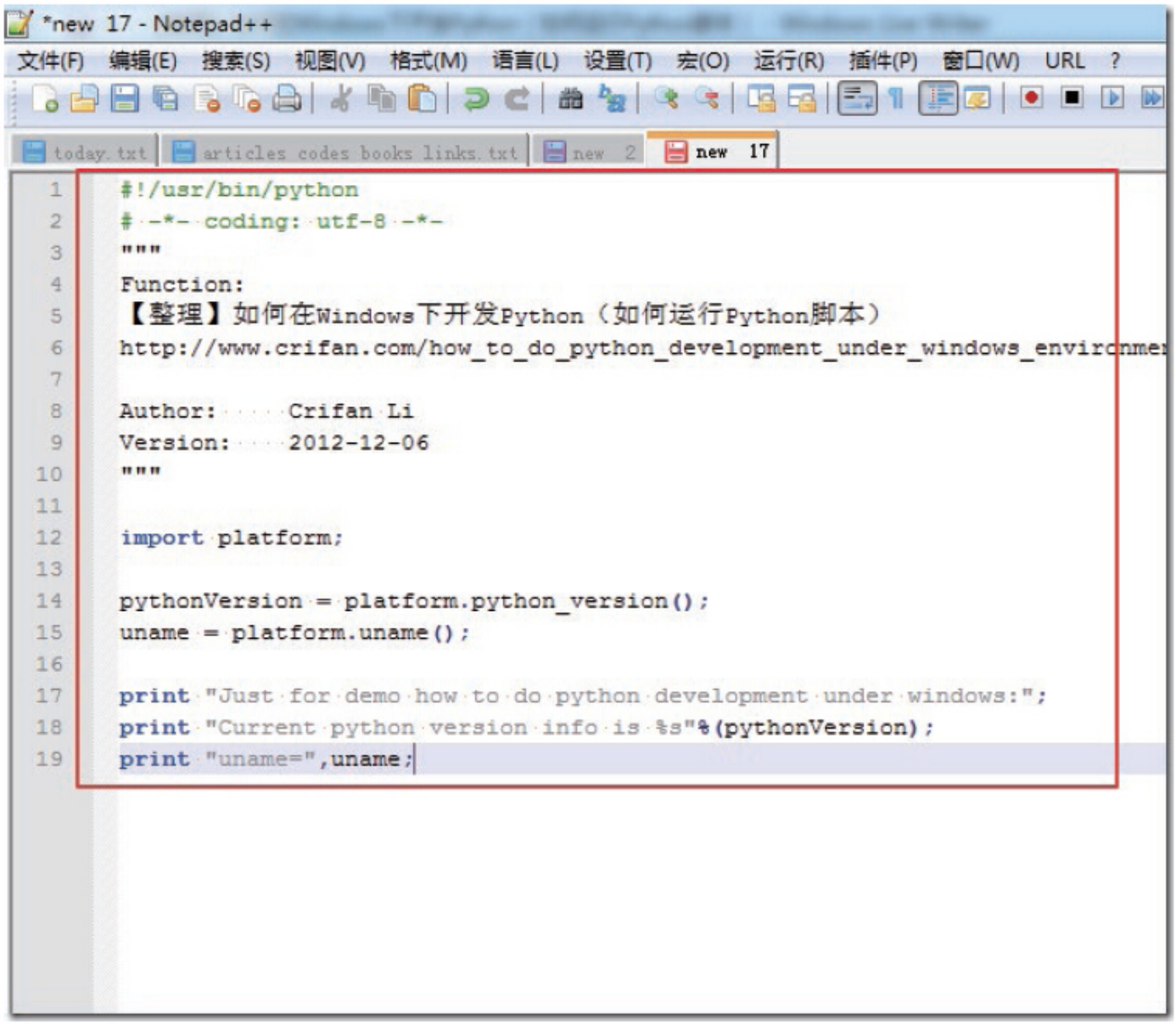


图3-34　Python代码语法高亮显示代码

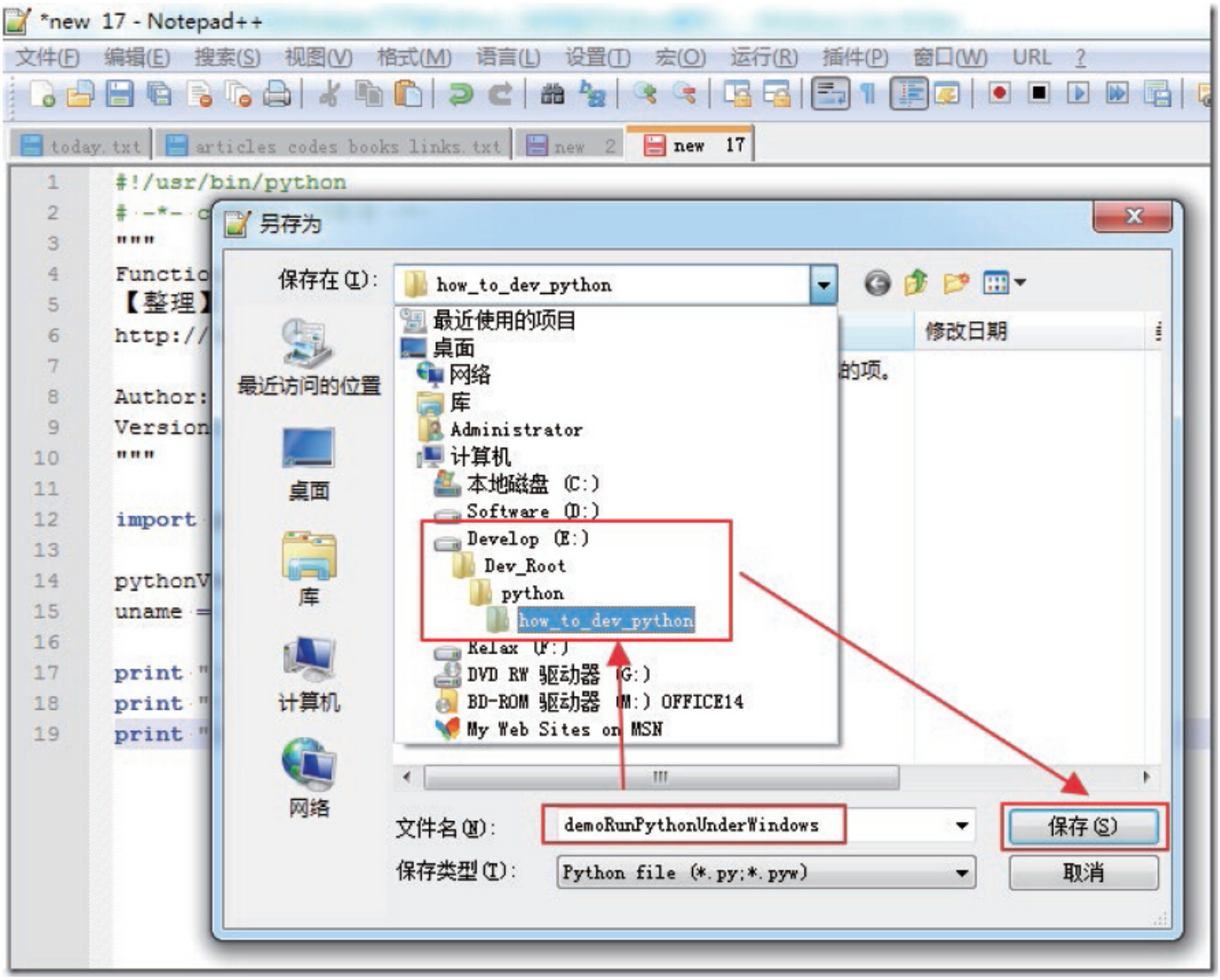


图3-35　选择对应路径保存

路径中不要包含（中文、空格等）特殊字符

对于保存文件来说，和之前解释的Python安装路径不要包含特殊字符类似，也需要注意保存的文件的路径中不要包含各种非ASCII的特殊字符，而只包含普通的英文字符，即字母数字下划线。比如，笔者此处的路径用的是"E:\Dev_Root\python\how_to_dev_python\"，否则很容易在后期开发期间出现一些异常现象，比如找不到文件、文件夹以及常见的中文路径无法识别等问题。

保存后效果如图3-36所示。

至此算是写好了Python代码。

2）打开Windows的cmd并切换到对应的Python脚本所在目录。

写完Python代码并保存好文件后，再将其打开可以运行Python代码的终端命令行界面，Windows的终端是cmd，然后再切换到对应的Python文件所在的文件夹。

Windows下cmd切换到对应的路径有如下两种办法。

· 手动打开cmd并切换到对应路径

通过打开Windows中的命令行工具cmd，如图3-37所示。

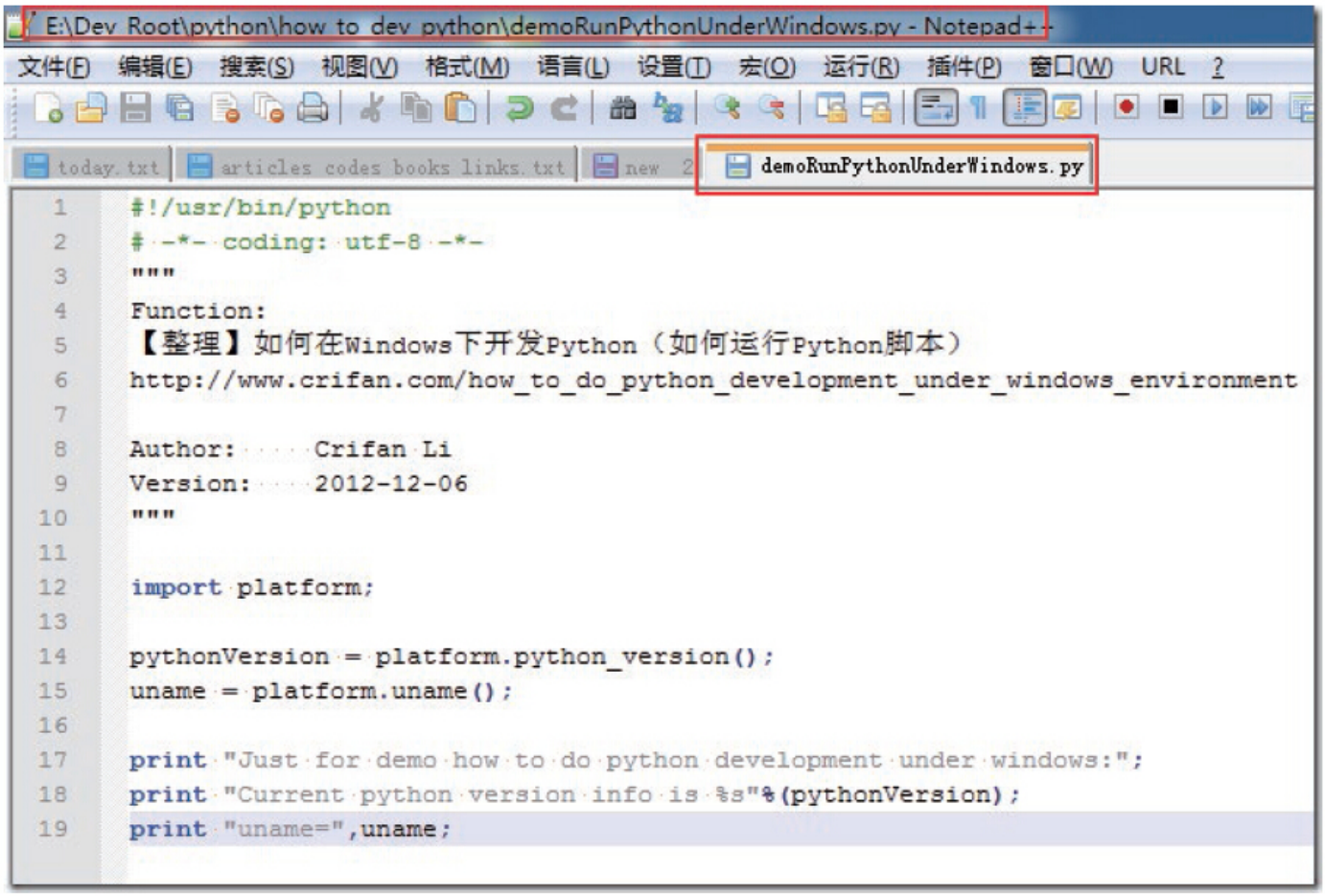


图3-36　选择路径完毕保存后效果

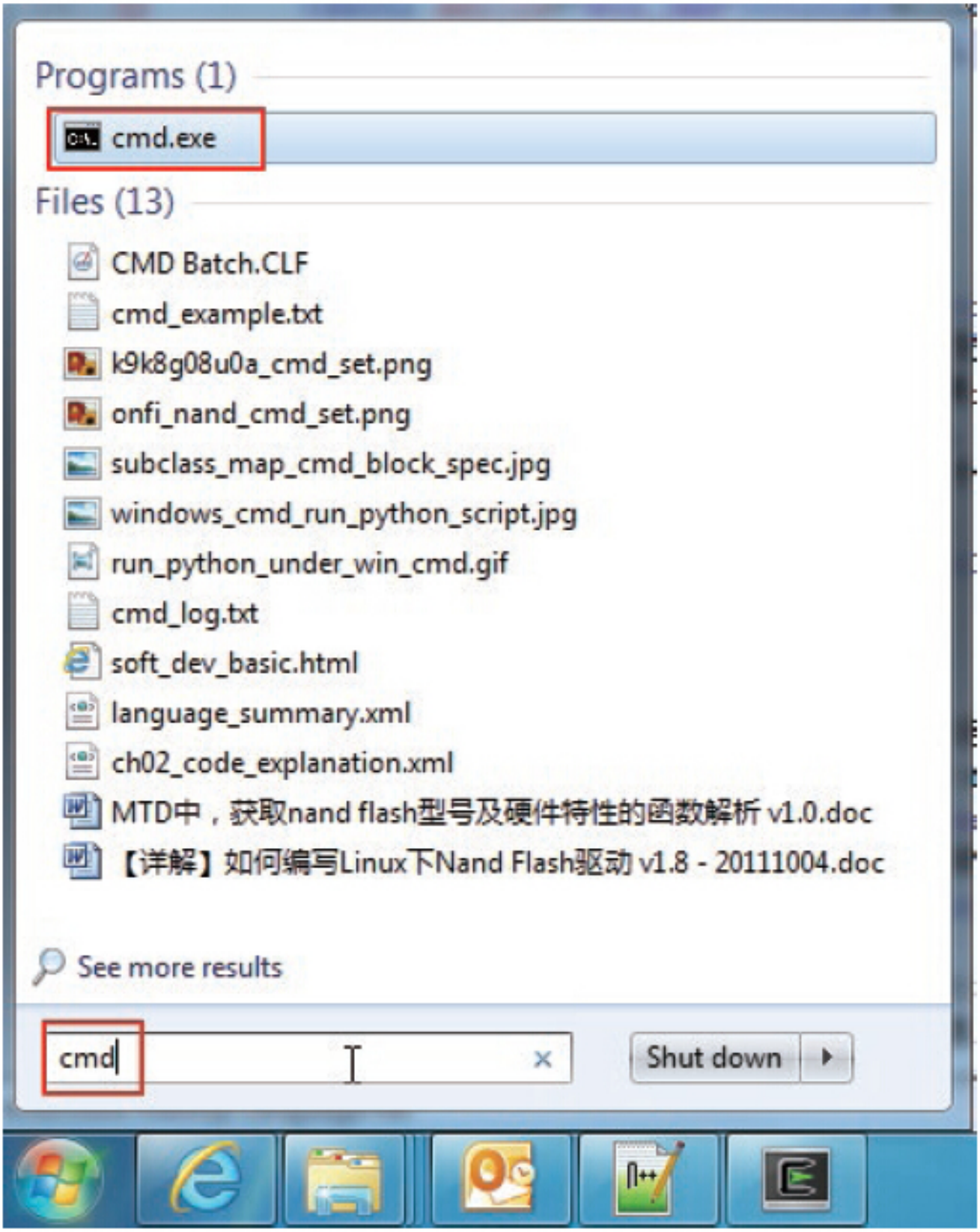


图3-37　打开Windows中命令行工具cmd

然后再用cd命令去切换到对应的目录“E:\Dev_Root\python\how_to_dev_python\”即可。

· 通过Notepad++的Open current dir cmd

此处采用一个更方便的办法，利用Notepad++中功能直接打开cmd并切换到对应路径，如图3-38所示。

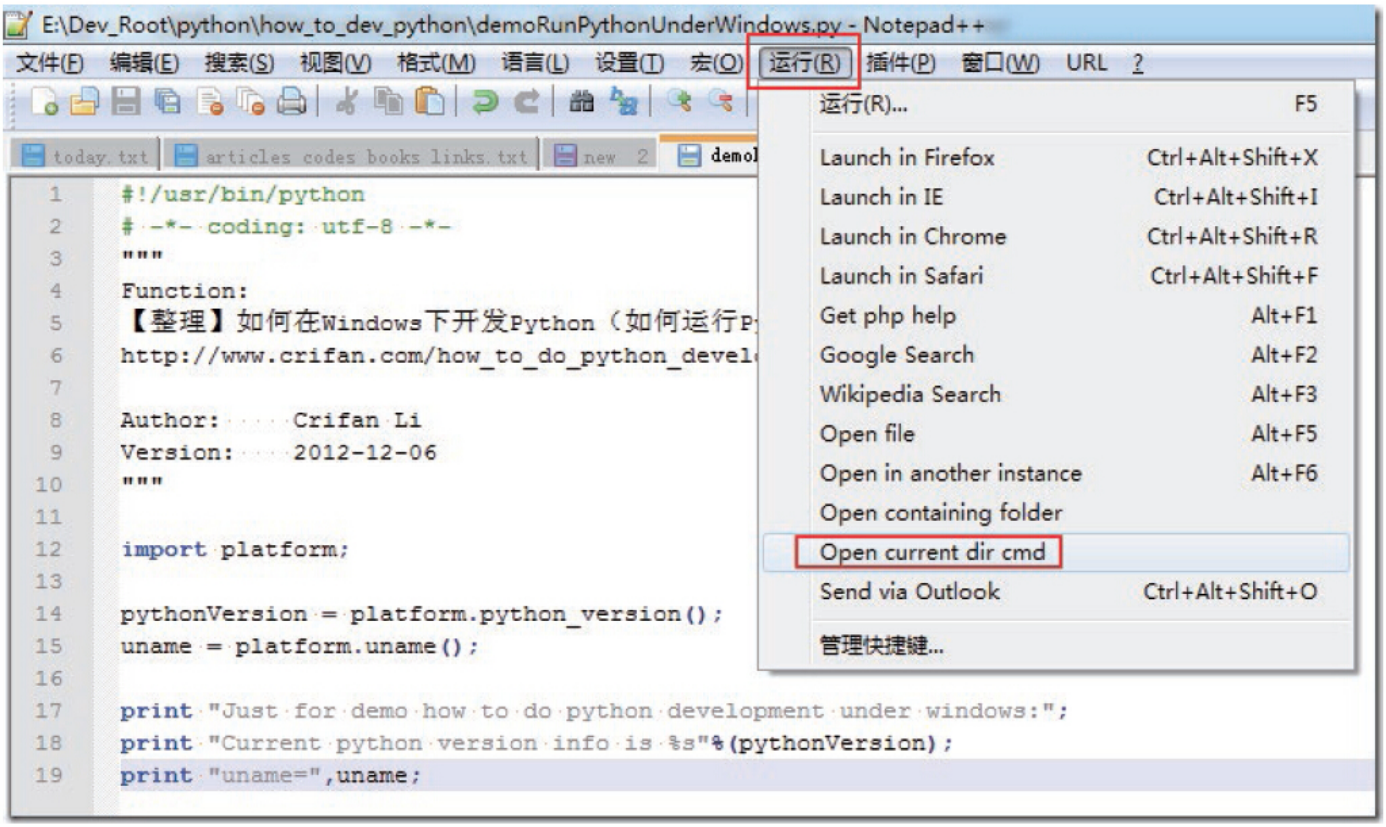


图3-38　直接打开cmd

然后就直接实现打开了cmd并且切换到当前文件所在路径了，如图3-39所示。

3）在cmd中运行Python脚本（.py文件）。

在cmd中输入Python脚本，即.py文件的完整的文件名。此处为demoRunPython-UnderWindows.py。

利用TAB键实现自动输入

在系统中输入文件名的正常思路是通过手动输入Python的完整的文件名，即对于此处的demoRunPythonUnderWindows.py去一个字母一个字母地输入。不过其实有个小技巧，可以利用Windows（和Linux中通用的）Tab键自动帮忙显示出当前目录的文件。

如果当前文件夹有多个文件，多次按Tab键即会在多个文件之间切换。此处只有一个文件，所以通过按一次Tab键就可以一下子就显示出对应的整个文件名了，如图3-40所示。

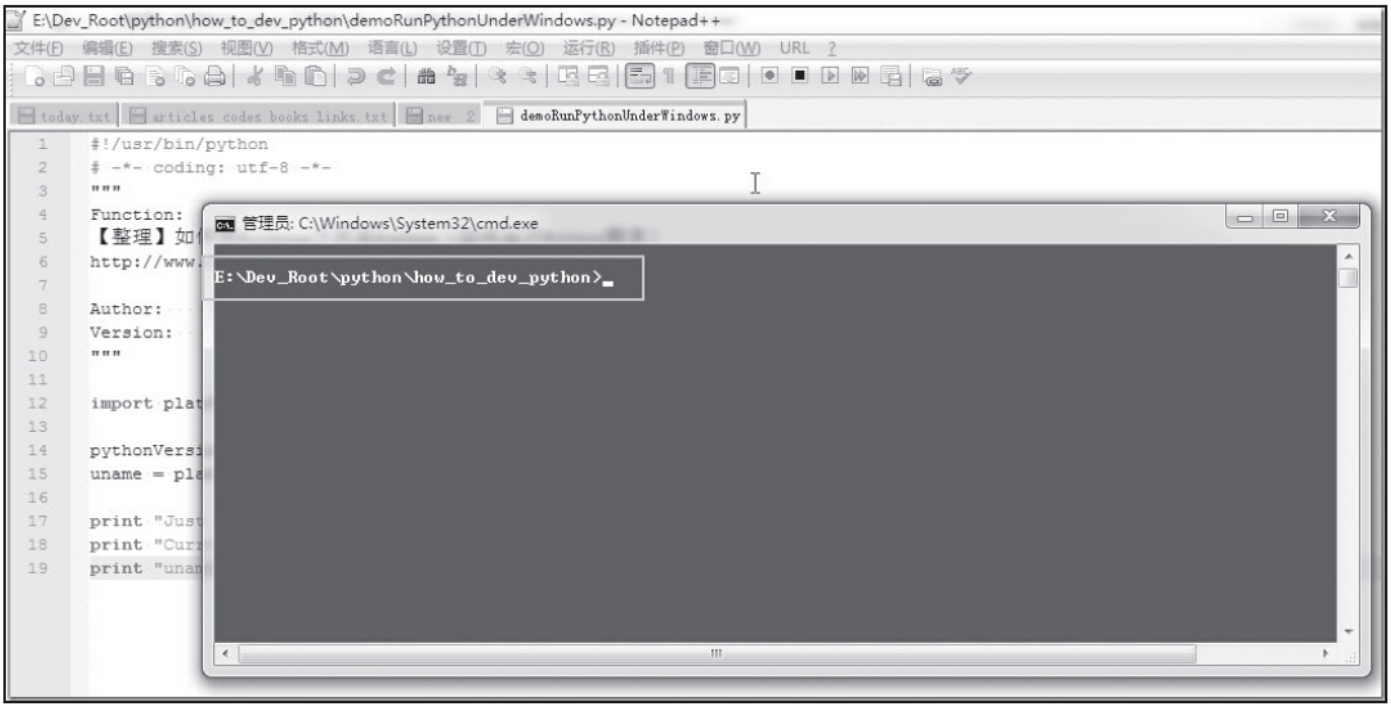


图3-39　切换至当前文件所在路径

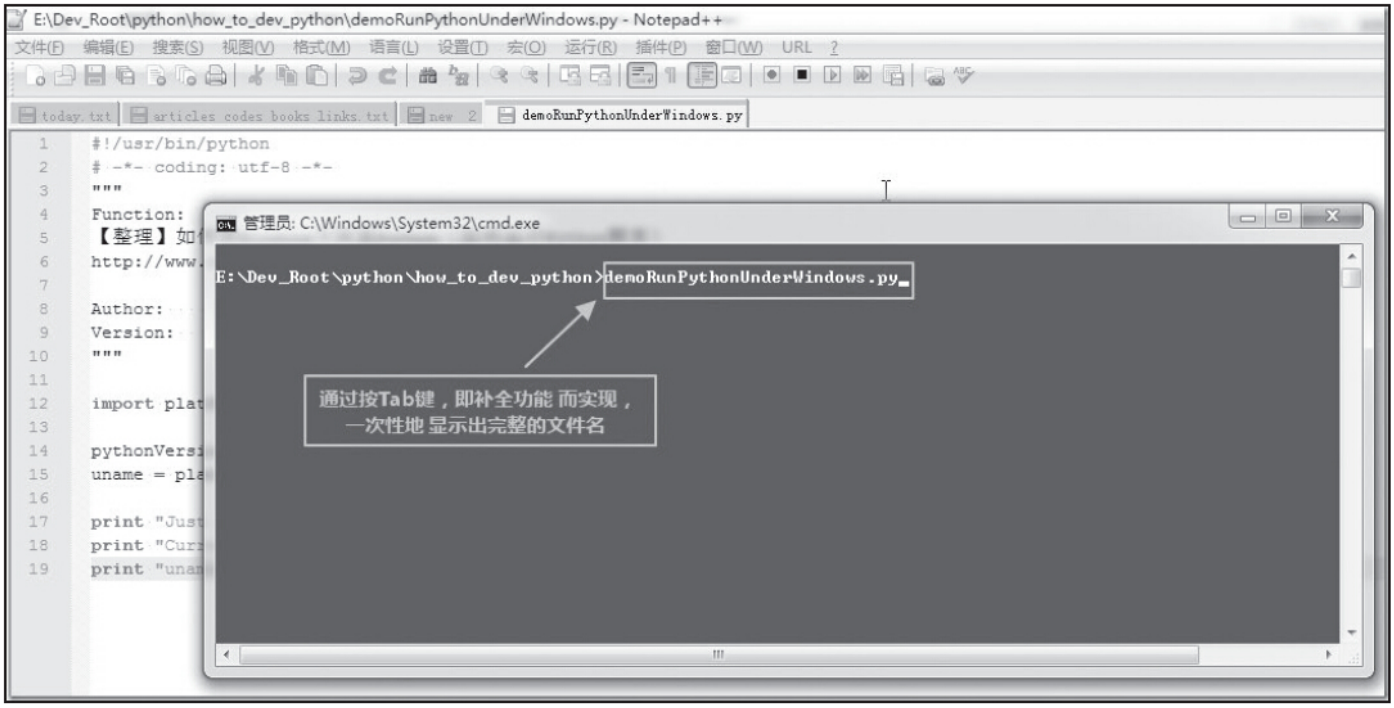


图3-40　显示文件名

按回车键即可运行对应的Python脚本，接着在cmd中也就可以看到输出的结果了，如图3-41所示。

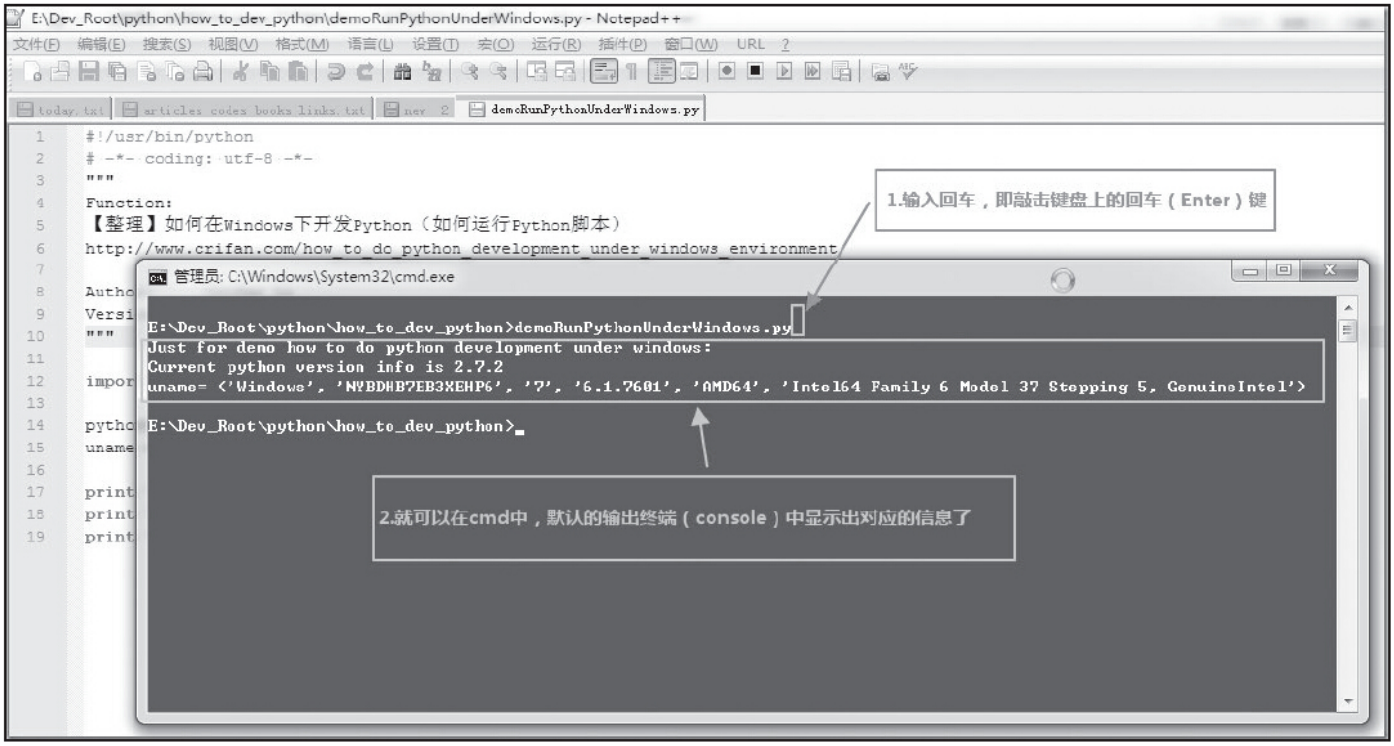


图3-41　输出结果

这就是在Windows的cmd中运行Python脚本去进行Python开发的完整流程。接下来继续开发Python的常见流程是：

· 继续修改Python代码，添加新的代码，然后保存python文件；

· 再回到cmd中，重新再次运行Python，以验证程序运行是否正常，是否获得了期望的结果；

· 如此反复，由少到多，一点点写出足够复杂的Python代码，实现最终所需的复杂功能。

3.4.2　用Python的shell进行交互式开发

如前所述，Python有个shell，接下来就是去介绍如何在Windows使用Python的shell。

1.命令行版本的Python Shell–Python（command line）

Windows下安装好Python后，读者可以在“开始”菜单中找到对应的command line版本的Python Shell，执行“开始”→Python 2.7→Python（command line）命即可，如图3-42所示。

其实，Windows 7中更方便的方式是直接在搜索框中搜索python，这样即可找到，如图3-43所示。

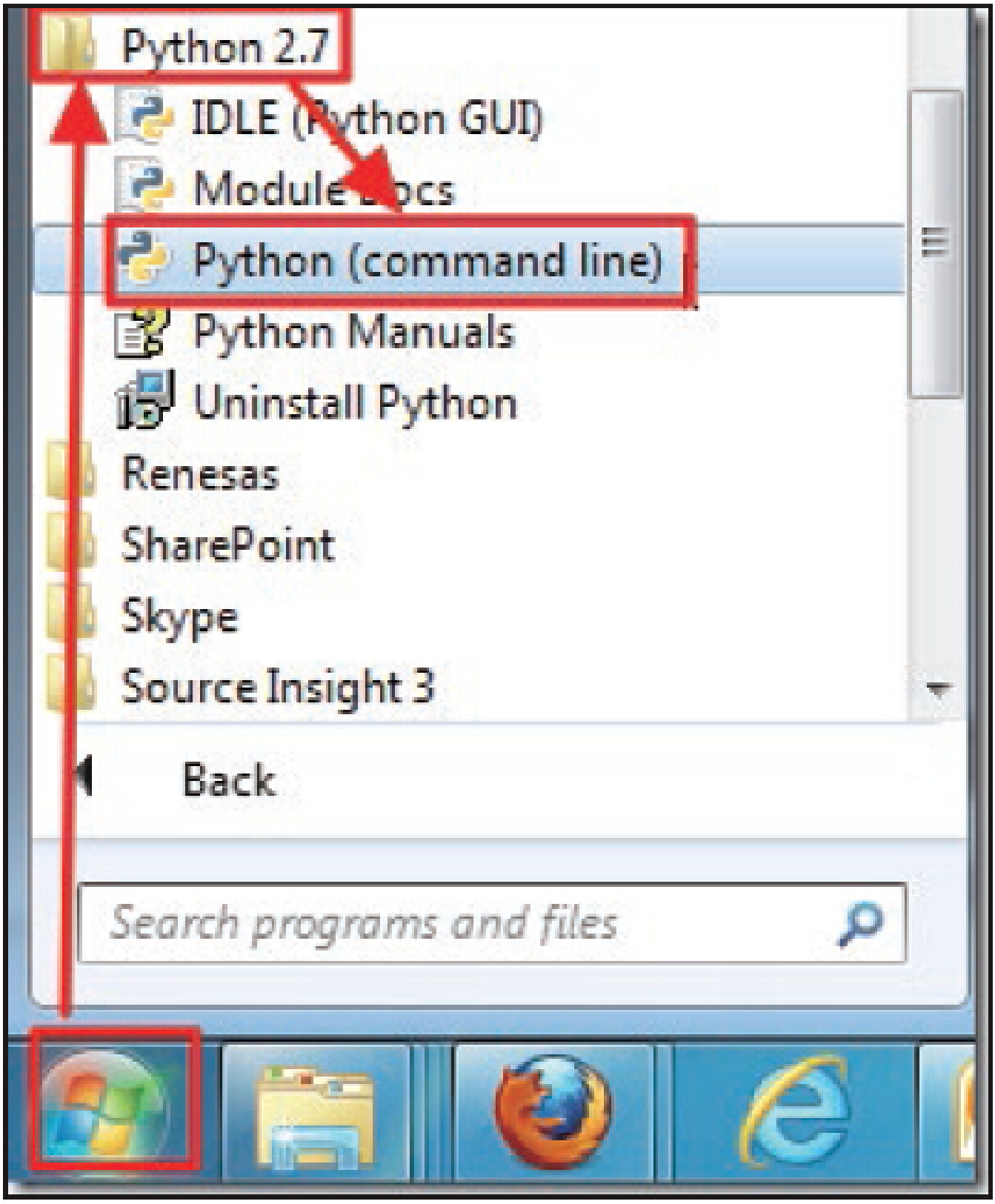
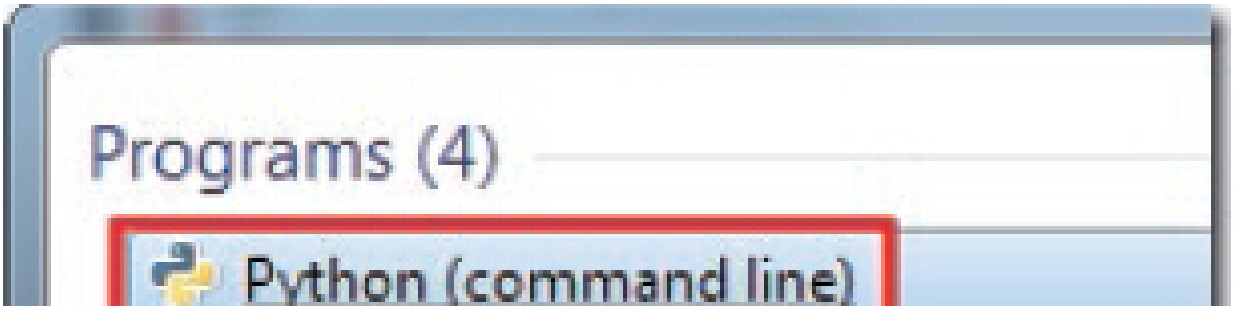


图3-42　执行命令找到对应command line版本的Python shell



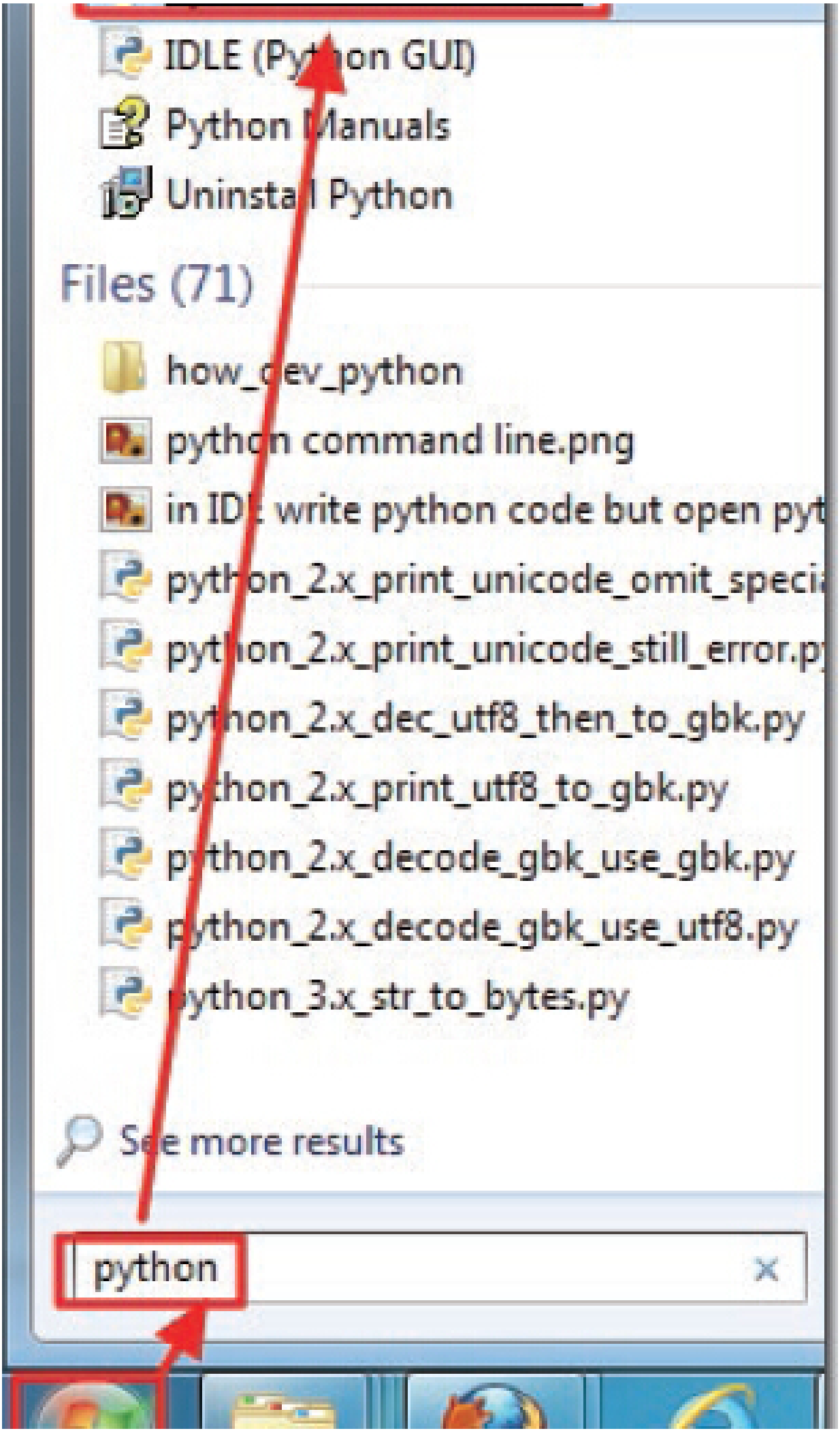

IDLE (Python GUI)
Python Manuals
Uninstall Python
Files (71)
how_dev_python
python command line.png
python_2.x_print_unicode_omit_speci
python_2.x_print_unicode_still_error.p
python_2.x_dec_utf8_then_to_gbk.py
python_2.x_print_utf8_to_gbk.py
python_2.x_decode_gbk_use_gbk.py
python_2.x_decode_gbk_use_utf8.py
python_3.x_str_to_bytes.py
See more results
python

图3-43　直接搜索

打开后如图3-44所示。

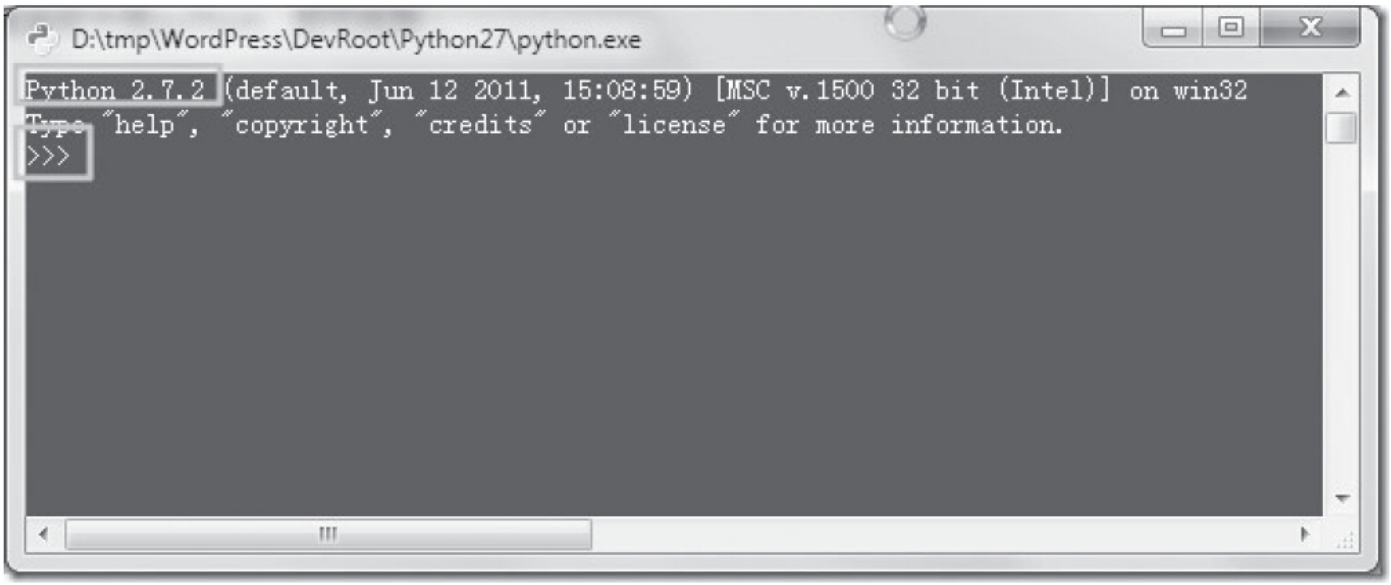


图3-44　直接搜索后界面

启动后会显示出当前Python的版本信息和系统信息。接着是三个大于号>>>，接下来就可以像之前所说的，就像在普通文本编辑器中输入Python代码一样，在此一行行输入代码，然后就可以显示对应的信息了，如图3-45所示。

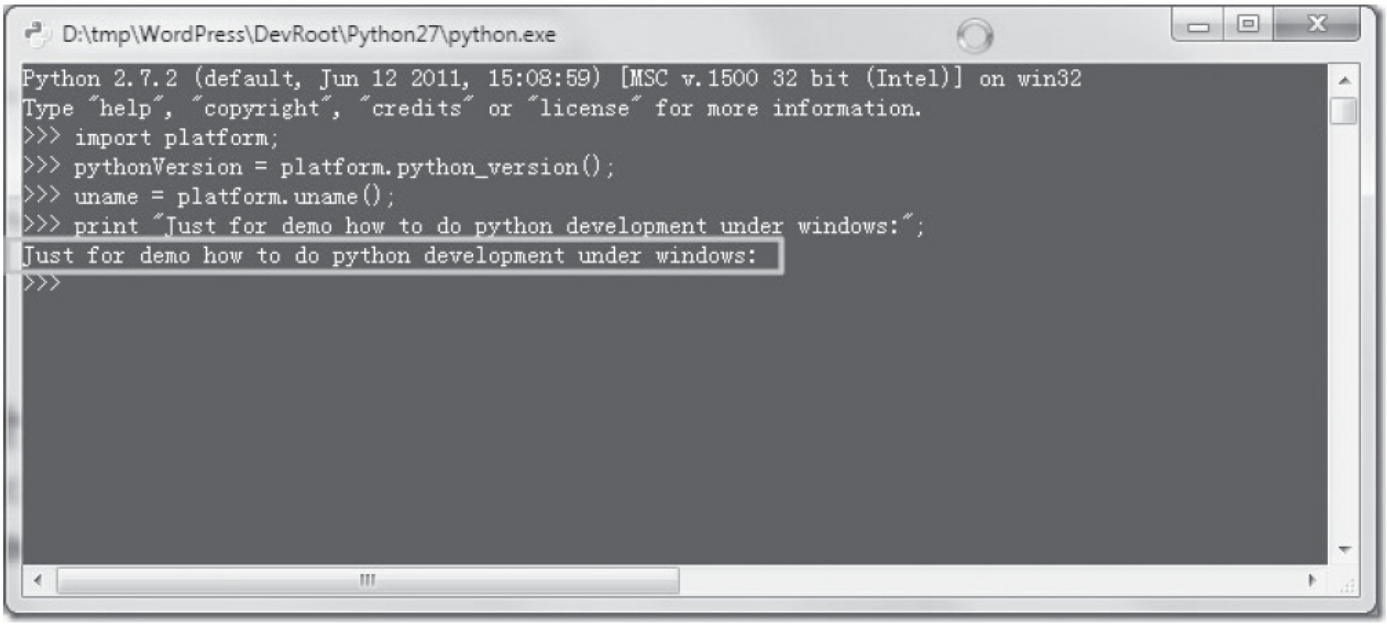


图3-45　显示对应信息

为何叫作交互式shell（interactive shell）

在上述过程中可以看到，当你输入对应的第一行print并回车时，此处命令行（command line）版本的Python shell中，就可以动态地、交互式地显示出对应的信息了。正由于此处shell中可以直接地、动态地、交互性式地显示出对应的信息，所以才被叫作Python的交互式的shell，简称Python shell。

将前面的代码输入完毕后结果显示如图3-46所示。

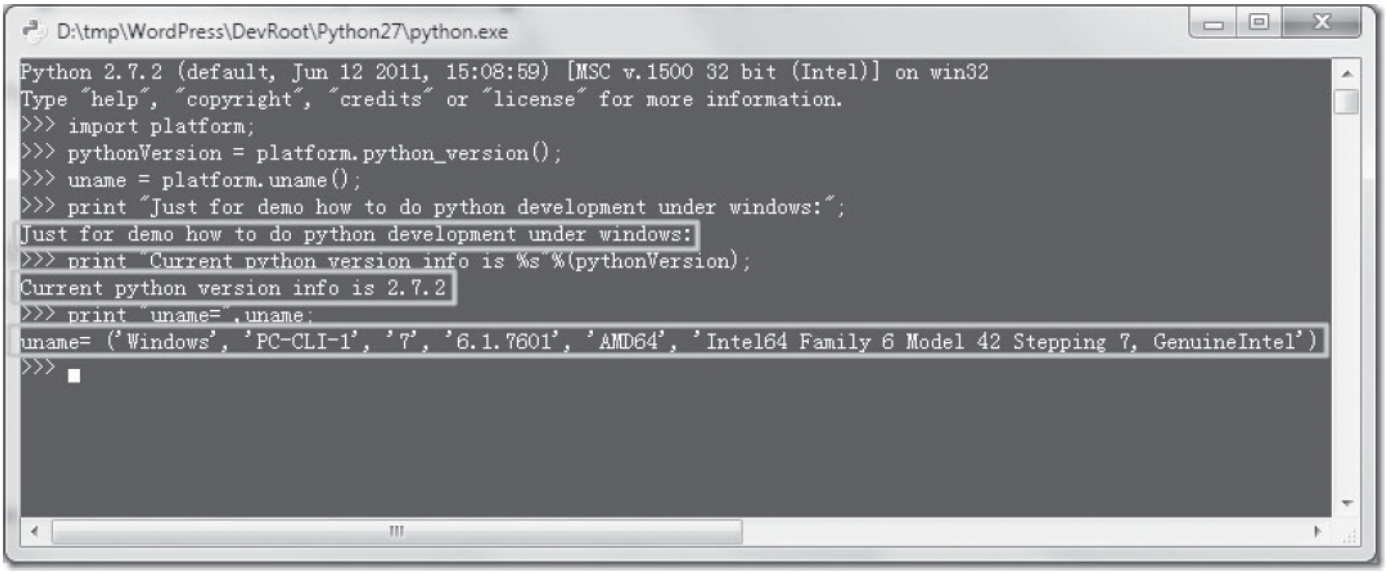
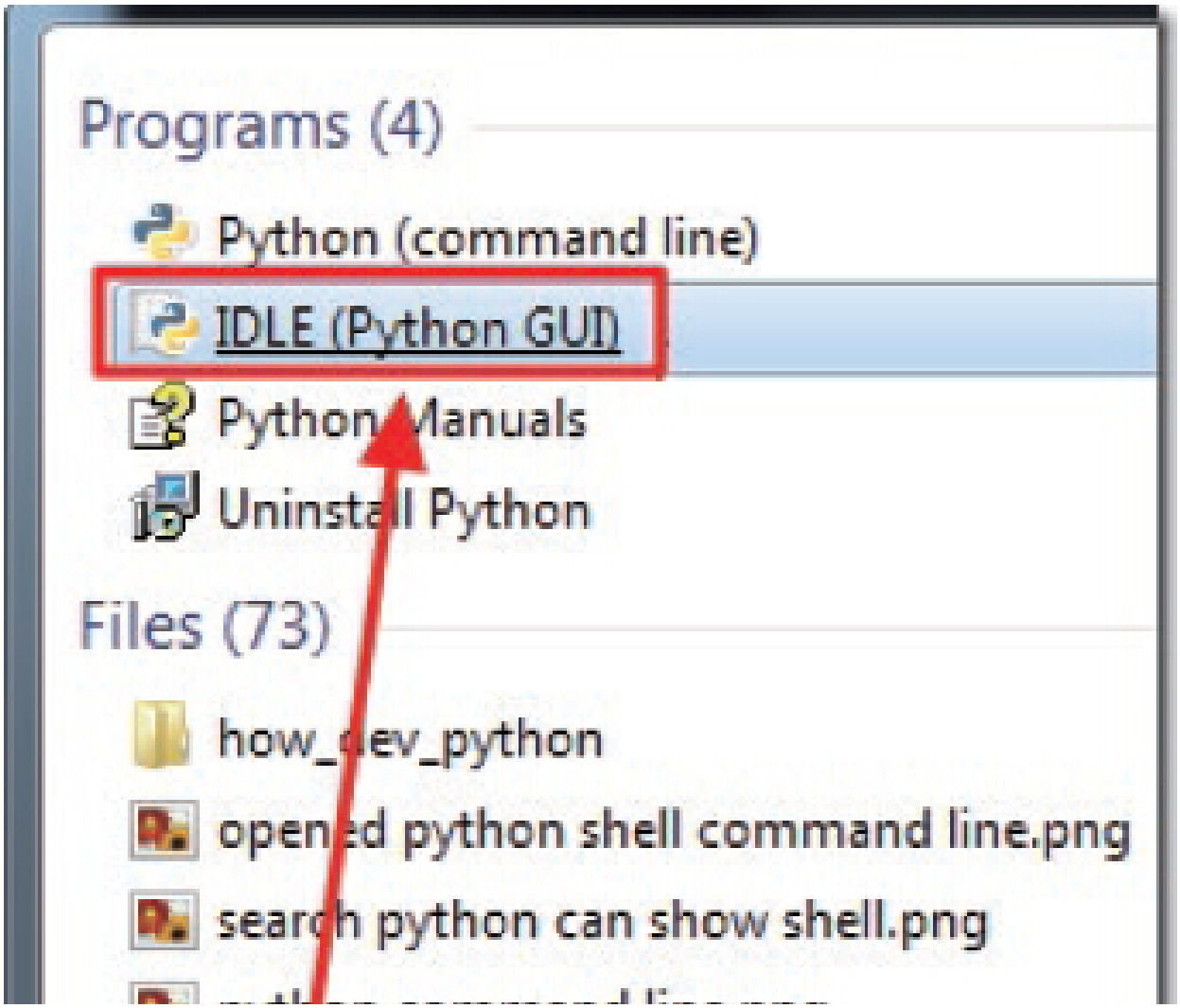


图3-46　代码输入完毕后效果

2.带图形界面的Python shell–IDLE（Python GUI）

下面将要介绍的同样是Python的shell，但是与command line版本相对应的带图形界面的GUI版本的shell，其打开方式和上面类似，可以通过直接搜索python而找到，如图3-47所示。



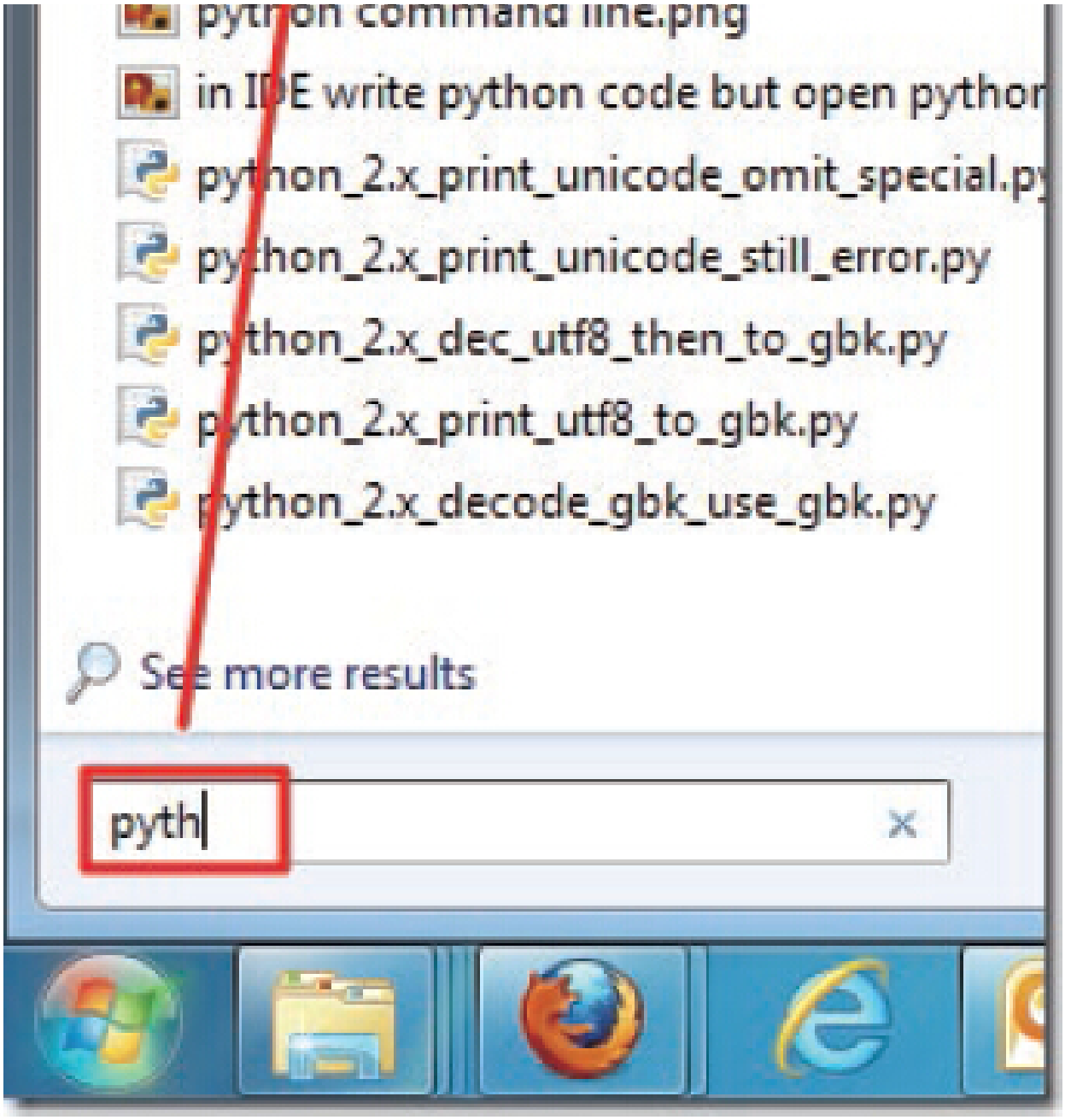


图3-47　直接搜索Python找到GUI版本的shell

从图3-47中可以看到其名字写的是IDLE。打开IDLE后如图3-48所示。

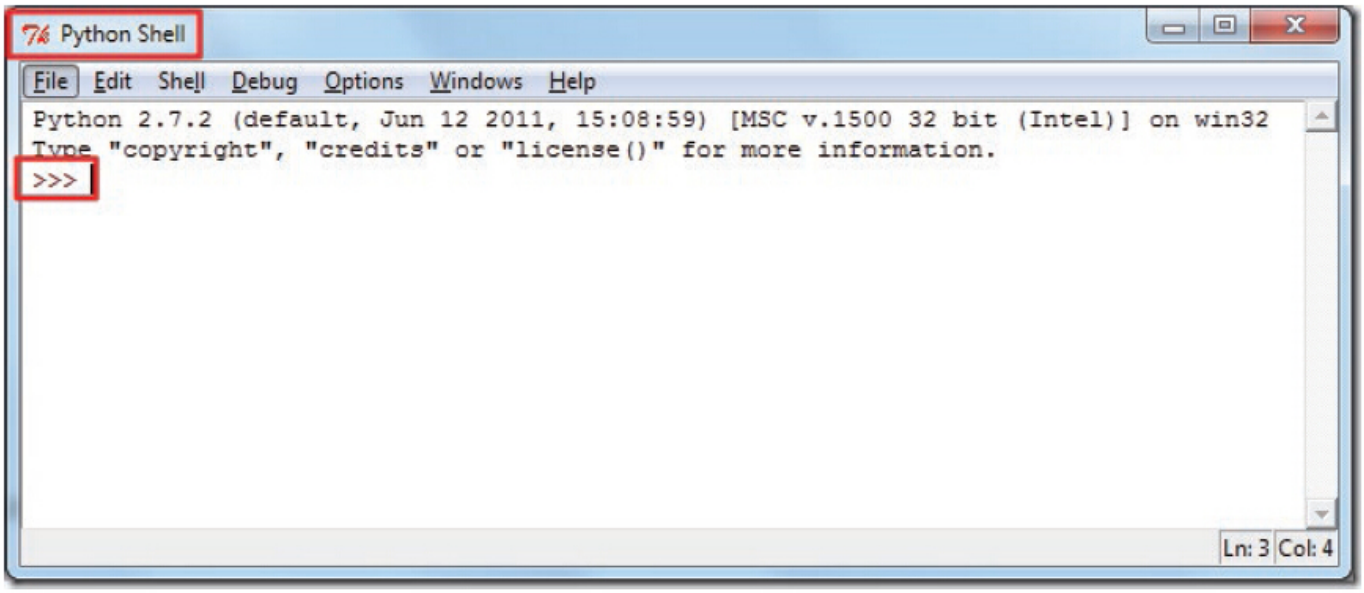


图3-48　打开IDLE后效果

IDLE左上角的红色TK图标表示IDLE是用Tkinter图形库开发的

左上角的红色图标看起来像是TK两个字母，对应的TK是Python的众多图形界面库的一种。而这个IDLE就是其他很多Python教程中，常用来作为Python的开发环境去教别人写Python代码的那个IDLE。关于Python中的Tkinter图形库的更多内容，可以参考6.2.1节。

输入前述的代码后，结果也是类似的，如图3-49所示。

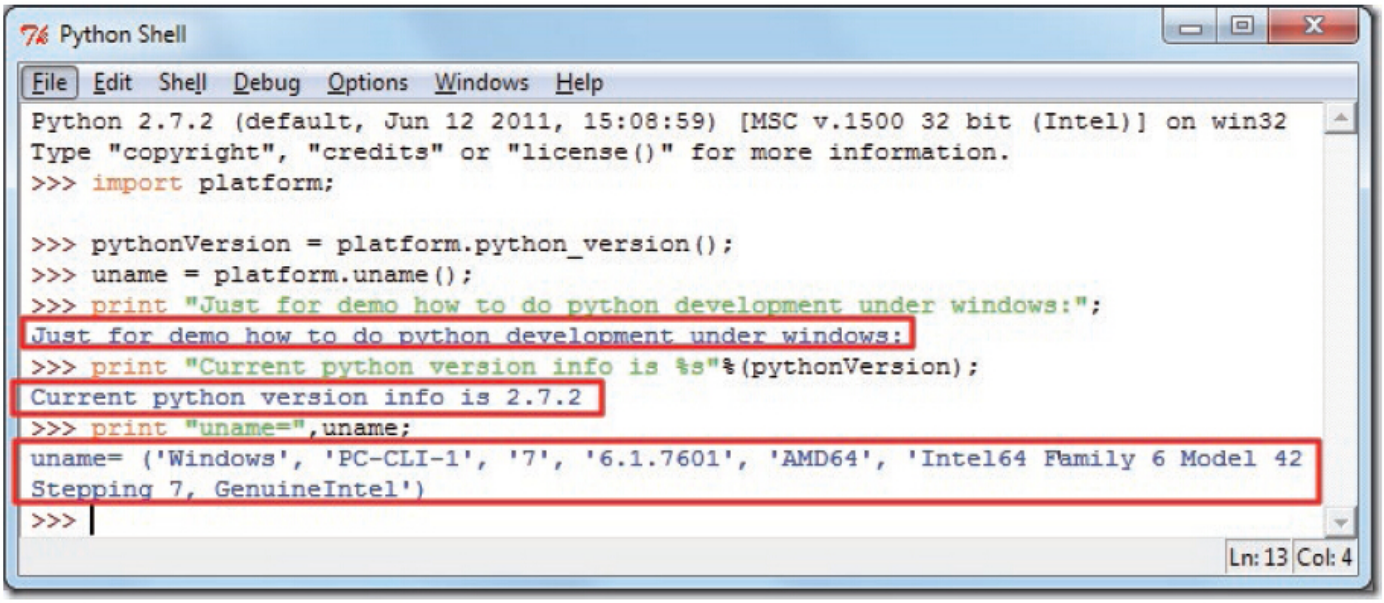


图3-49　输入前述代码后结果

IDLE中一次性粘贴多行代码再运行则会出错

IDLE的运行方式是交互式的，需要一行一行地输入后才能正常运行，因很多Python新手读者不熟悉而会导致犯这类错误：在他处复制Python示例代码，然后一次性粘贴多行代码到IDLE中，如图3-50所示。

然后按回车运行，如图3-51所示。

结果很明显会出错，此处是不能正常输出之前所期望的Python版本信息，而是出现了异常情况：只是显示此处的注释代码。

其原因就是因为此处最开始部分的代码就只是注释代码，对应的如果是其他的多行代码，则对应的就会出现其他对应的错误了，如图3-52所示。

而如果想要正确地运行代码，则应该与之前操作方式一样，正常的一行一行地输入代码并回车。此时才能执行所输入的该行代码，才能正常地显示出你所希望看到的信息，如图3-53所示。

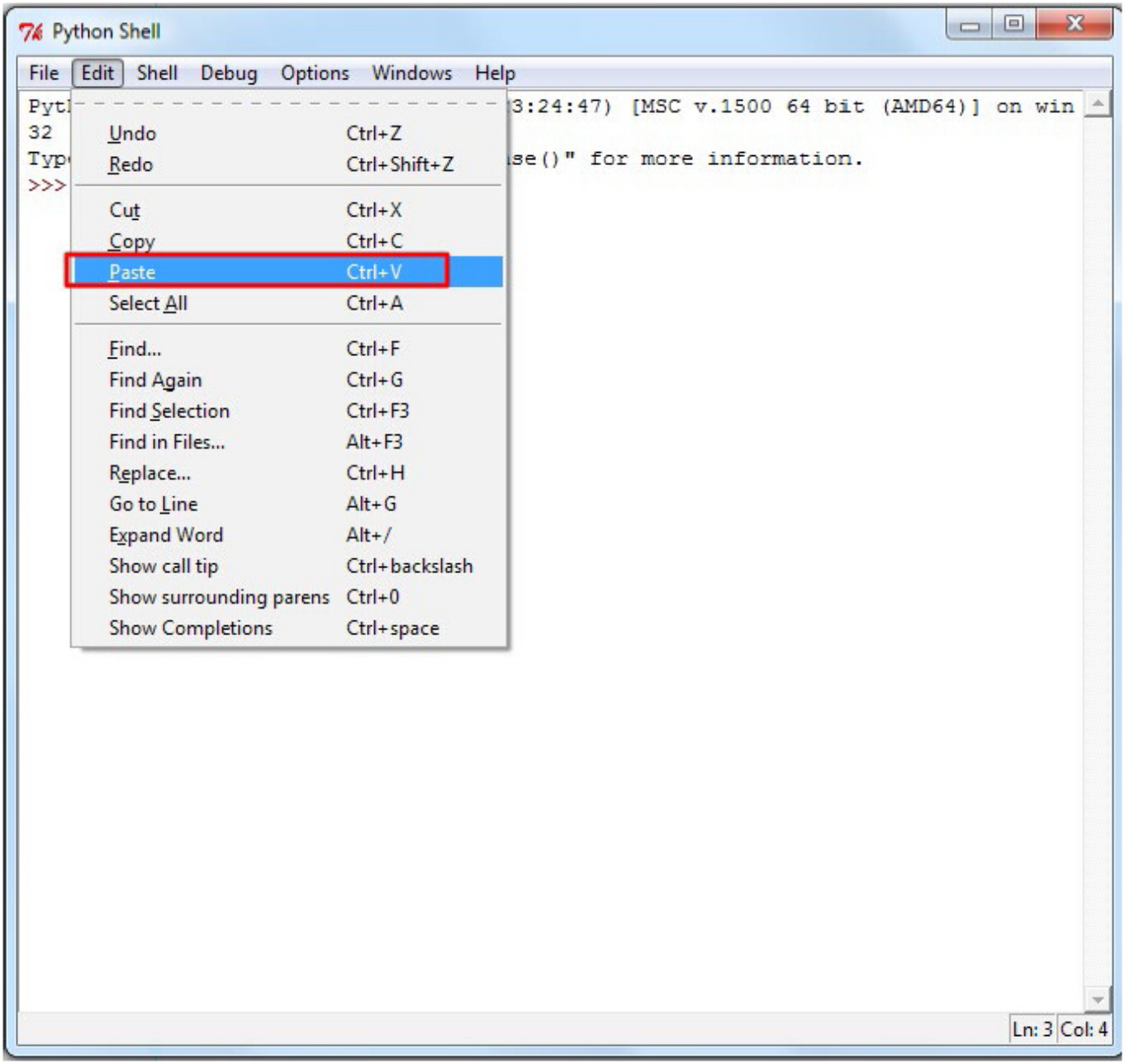


图3-50　将其他处的Python示例代码一次性粘贴

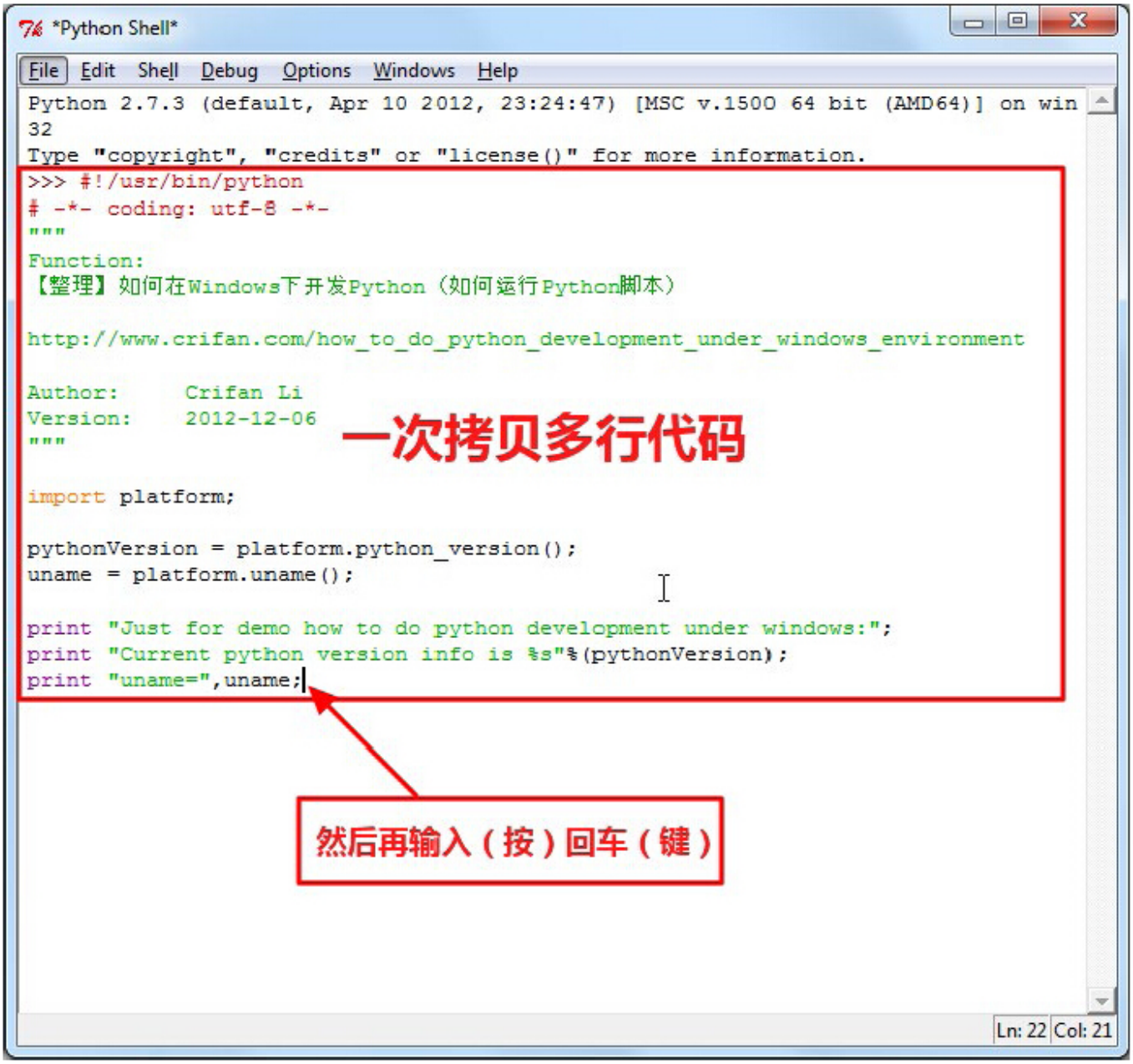

Python Shell
File Edit Shell Debug Options Windows Help
Python 2.7.3 (default, Apr 10 2012, 23:24:47) [MSC v.1500 64 bit (AMD64)] on win
32
Type "copyright", "credits" or "license()" for more information.
>>> #!/usr/bin/python
# -*- coding: utf-8 -*-
"""
Function:
【整理】如何在Windows下开发Python（如何运行Python脚本）
http://www.crifan.com/how_to_do_python_development_under_windows_environment
Author: Crifan Li
Version: 2012-12-06
"""
import platform;
pythonVersion = platform.python_version();
uname = platform.uname();
print "Just for demo how to do python development under windows:";
print "Current python version info is %s"%(pythonVersion);
print "uname=",uname;
一次拷贝多行代码
然后再输入（按）回车（键）
Ln: 22 Col: 21

图3-51　按回车键运行

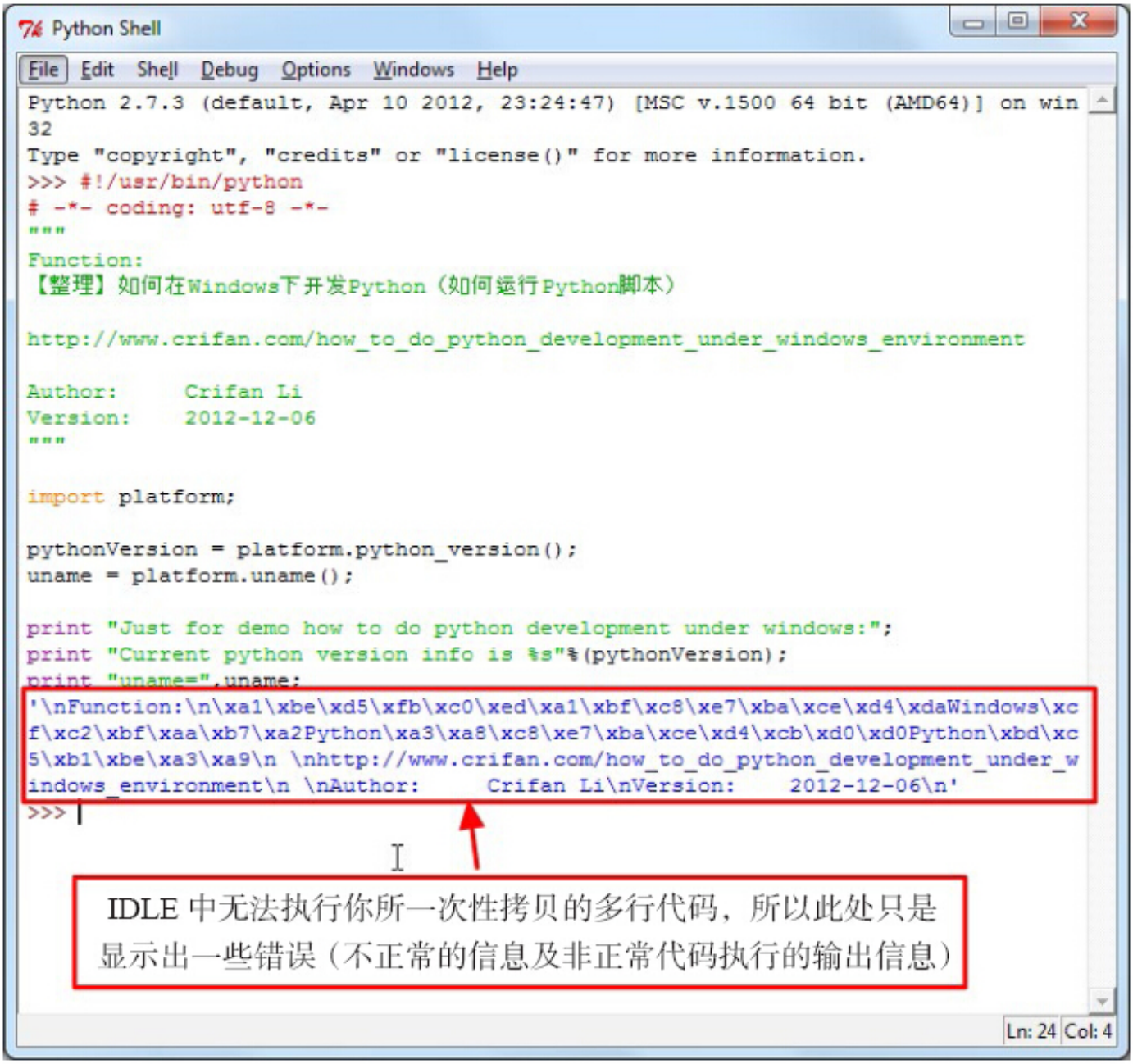

Python Shell
File Edit Shell Debug Options Windows Help
Python 2.7.3 (default, Apr 10 2012, 23:24:47) [MSC v.1500 64 bit (AMD64)] on win
32
Type "copyright", "credits" or "license()" for more information.
>>> #!/usr/bin/python
# -*- coding: utf-8 -*-
"""
Function:
【整理】如何在Windows下开发Python（如何运行Python脚本）
http://www.crifan.com/how_to_do_python_development_under_windows_environment
Author: Crifan Li
Version: 2012-12-06
"""
import platform;
pythonVersion = platform.python_version();
uname = platform.uname();
print "Just for demo how to do python development under windows:";
print "Current python version info is %s"%(pythonVersion);
print "uname=",uname;
'\nFunction:\n\xa1\xbe\xd5\xfb\xc0\xed\xa1\xbf\xc8\xe7\xba\xce\xd4\xdaWindows\xc
f\xc2\xbf\xaa\xb7\xa2Python\xa3\xa8\xc8\xe7\xba\xce\xd4\xcb\xd0\xd0Python\xbd\xc
5\xb1\xbe\xa3\xa9\n \nhttp://www.crifan.com/how_to_do_python_development_under_w
indows_environment\n \nAuthor: Crifan Li\nVersion: 2012-12-06\n'
>>>
IDLE 中无法执行你所一次性拷贝的多行代码，所以此处只是
显示出一些错误（不正常的信息及非正常代码执行的输出信息）
Ln: 24 Col: 4

图3-52　显示错误

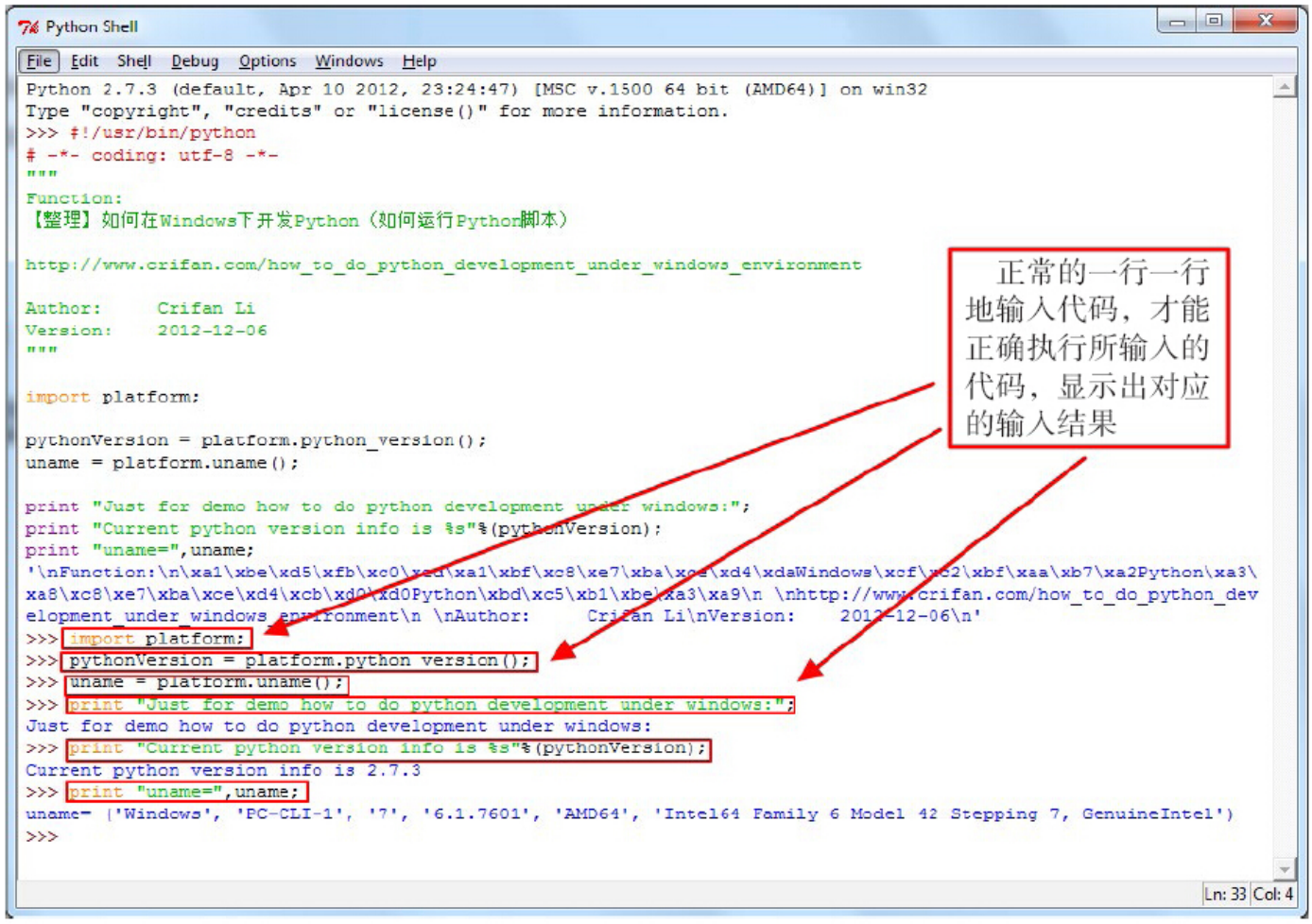


图3-53　显示对应的输入结果

Python shell的用途

不论是command line的shell还是GUI版本的IDLE，作为Python shell，在普通的Python开发实践中其实都很少有用到。因为很明显，Python shell可以很方便地立刻就看到代码执行的结果，所以相对比较适合用来测试、演示一些简单的代码的执行效果，但是对于长期的Python开发就显得不方便使用了。所以Python shell的用途可以总结为：对于Python的shell，不论是command line版还是GUI版，都比较适合偶尔要测试少量的Python代码的情况下去使用，而不适合长期的Python开发环境。

例3.2　举例：用Python的IDLE去做URL解码

比如笔者之前就借助IDLE去实现了一些小功能：在IDLE环境下使用Python的一些库函数实现一些对于url解码的功能，如图3-54所示。

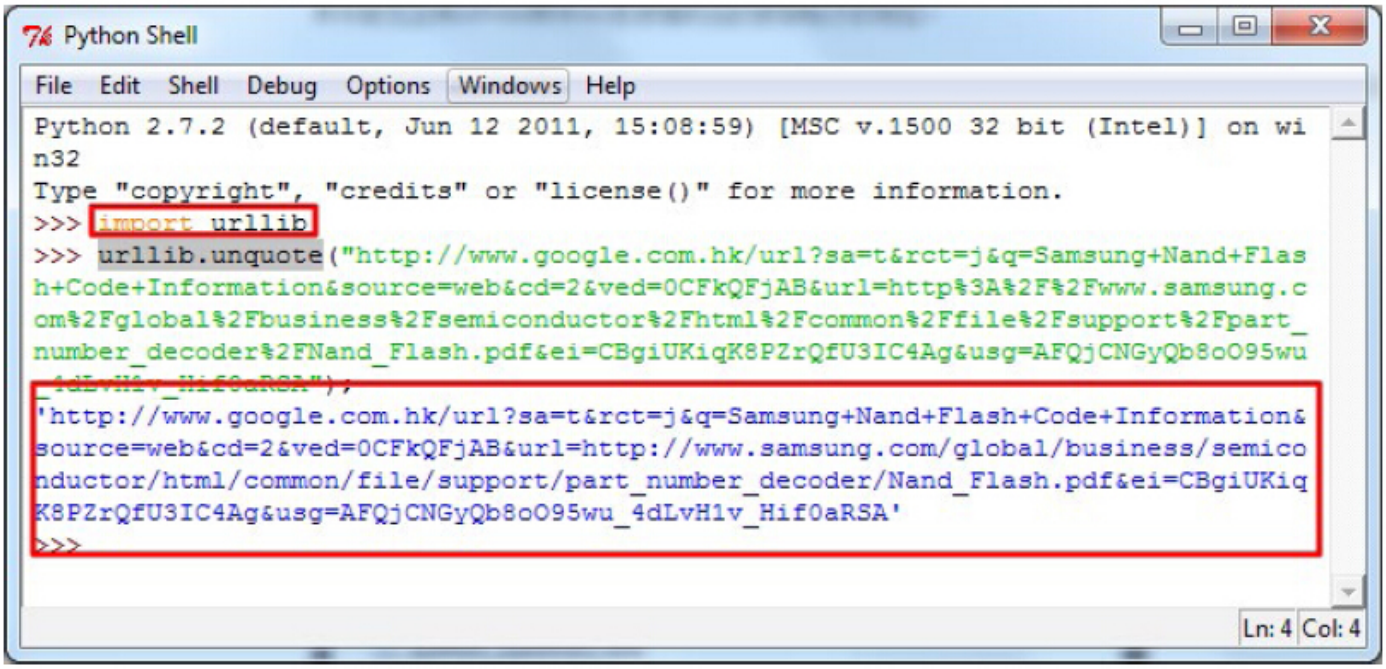


图3-54　IDLE中实现Google的URL解码

3.4.3　用Python的IDE进行开发

关于Windows中Python中的IDE如何使用，3.3.2节中已经介绍得非常详细了。此处就不再赘述。

3.5　在Linux环境下进行Python开发

其实在Linux中开发和运行Python脚本的过程与在Windows中很类似。唯一不同的是使用的一些与开发相关的工具和一些细节。比如Linux类系统中就无法直接使用笔者上面推荐的Notepad++文本编辑器。下面就来详细介绍如何在Linux类系统中开发Python。此处对于Linux系统的演示，需要选择某个具体的发行版来举例说明，本节选择相对最具代表性的Ubuntu系统为例来解释。

3.5.1　Python最原始的开发方式

1）Linux中用Sublime新建文件，写上Python代码，再保存为.py文件。

如之前所推荐的，Linux下面的文本编辑器推荐用Sublime，可以到Sublime主页下载对应的deb安装包，此时即可在Ubuntu中安装Sublime了。安装完毕后找到Sublime并运行，如图3-55所示。

图3-55　Ubuntu中查找并运行Sublime Text

然后粘贴上述示例代码后再保存为demo-RunPythonUnderUbuntu.py。注意此处的演示代码和最初给出的示例代码是一致的，不同的只是改动了个别字符，此细微改动对于演示无影响，如图3-56所示。

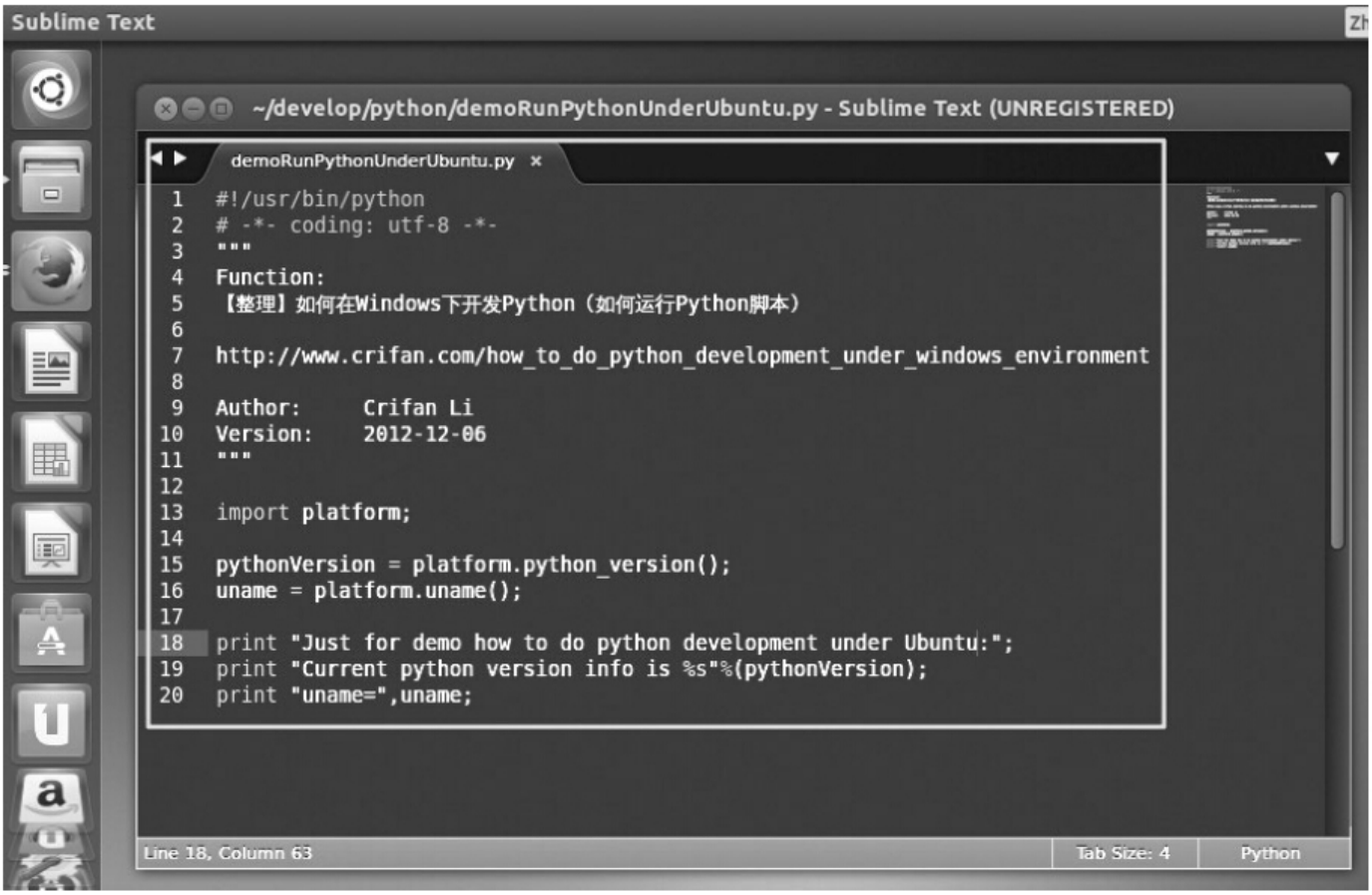


图3-56　Ubuntu中Sublime中的Python示例代码

2）打开Linux中的Terminal终端，并切换到对应的python脚本所在目录。

接着再到Ubuntu系统中打开Terminal终端，如图3-57所示。

图3-57　Ubuntu中搜索并打开Terminal终端

然后再切换到对应的Python脚本所在目录，如图3-58所示。

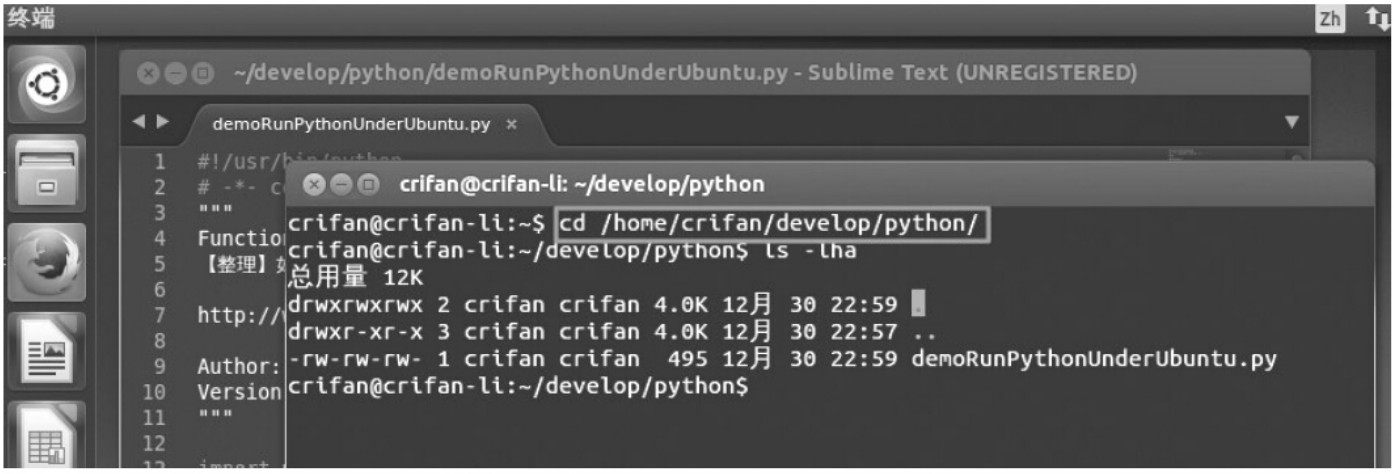


图3-58　Ubuntu中打开终端并切换到Python文件所在目录

3）在Ubuntu的Terminal环境中运行你的Python脚本（.py文件）。

最后在终端中运行Python脚本，如图3-59所示。

如此即可实现在Linux类的Ubuntu系统中实现运行Python代码，进行Python开发了。

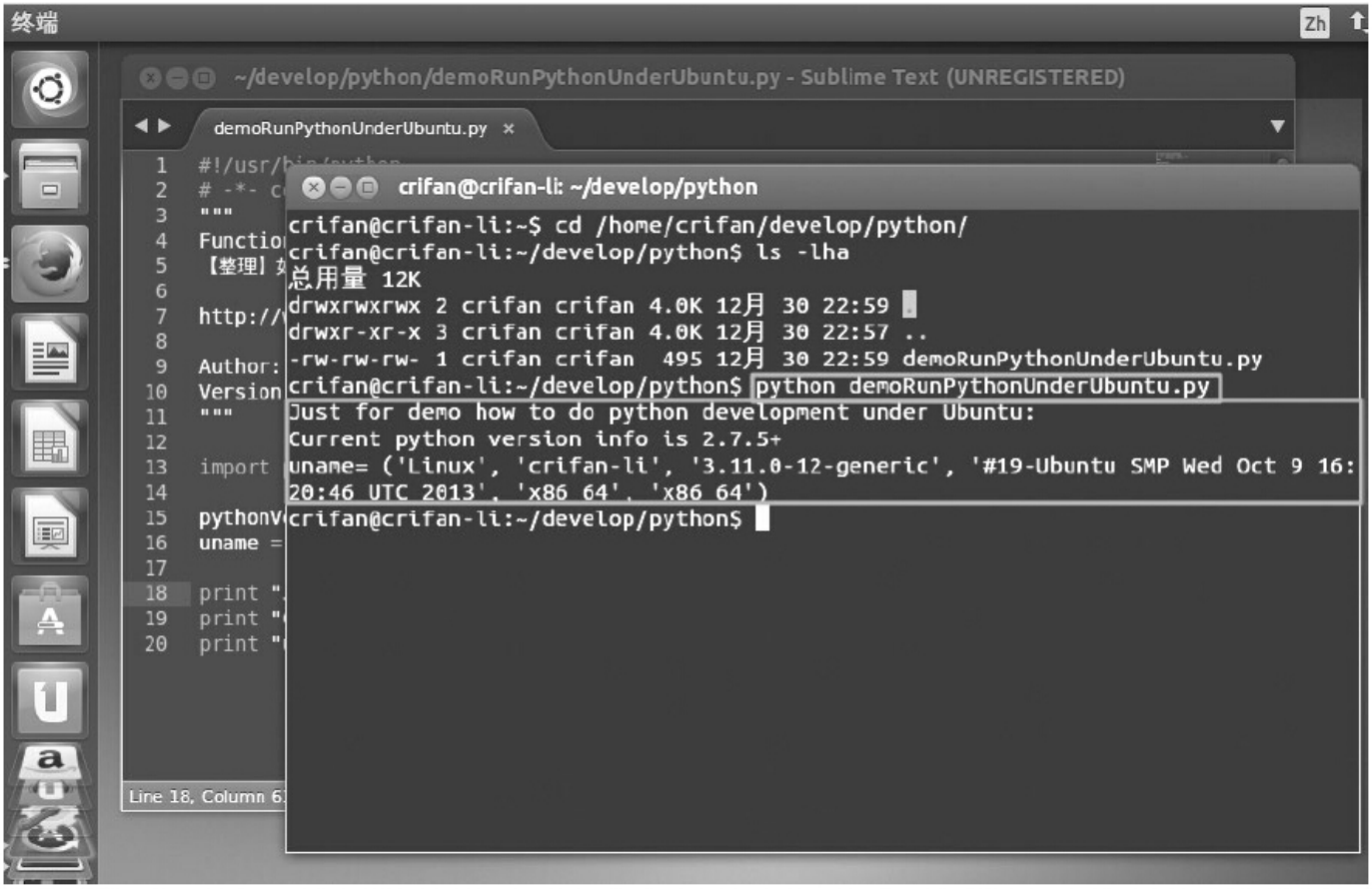


图3-59　在Ubuntu的Terminal中运行Python文件

3.5.2　用Python的shell进行交互式开发

Ubuntu中的基于命令行Python的shell的使用方式是直接在Terminal终端中打开即可，如图3-60所示。

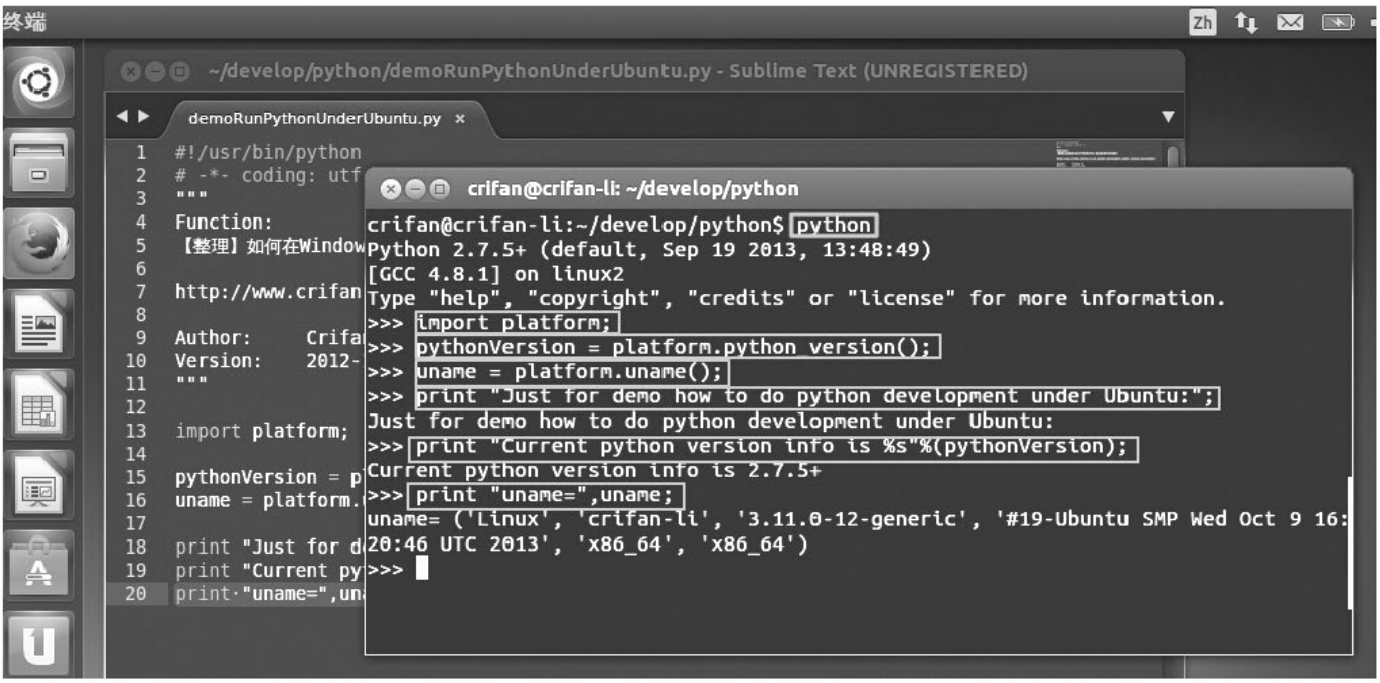


图3-60　在Ubuntu的Python的shell中开发Python

如此即可实现在Linux类的Ubuntu系统中用Python shell进行Python开发了。

3.5.3 用Python的IDE进行开发

此处，由于之前介绍的很多Python的IDE（如Eclipse+PyDev）是跨平台的，所以在Linux下使用Eclipse+PyDev这类IDE和Windows下的使用方式一样，此处不再赘述，有需要的读者可以参考3.3.2节了解Windows中如何使用该IDE。

3.6 在Mac环境下进行Python开发

3.6.1 Python最原始的开发方式

1）Mac中用Sublime新建文件，写上Python代码，再保存为.py文件。

如之前推荐的，Mac下面的文本编辑器也用Sublime。读者登录Sublime主页下载对应的dmg安装包后双击打开，然后即可找到其中的Sublime Text，如图3-61所示。

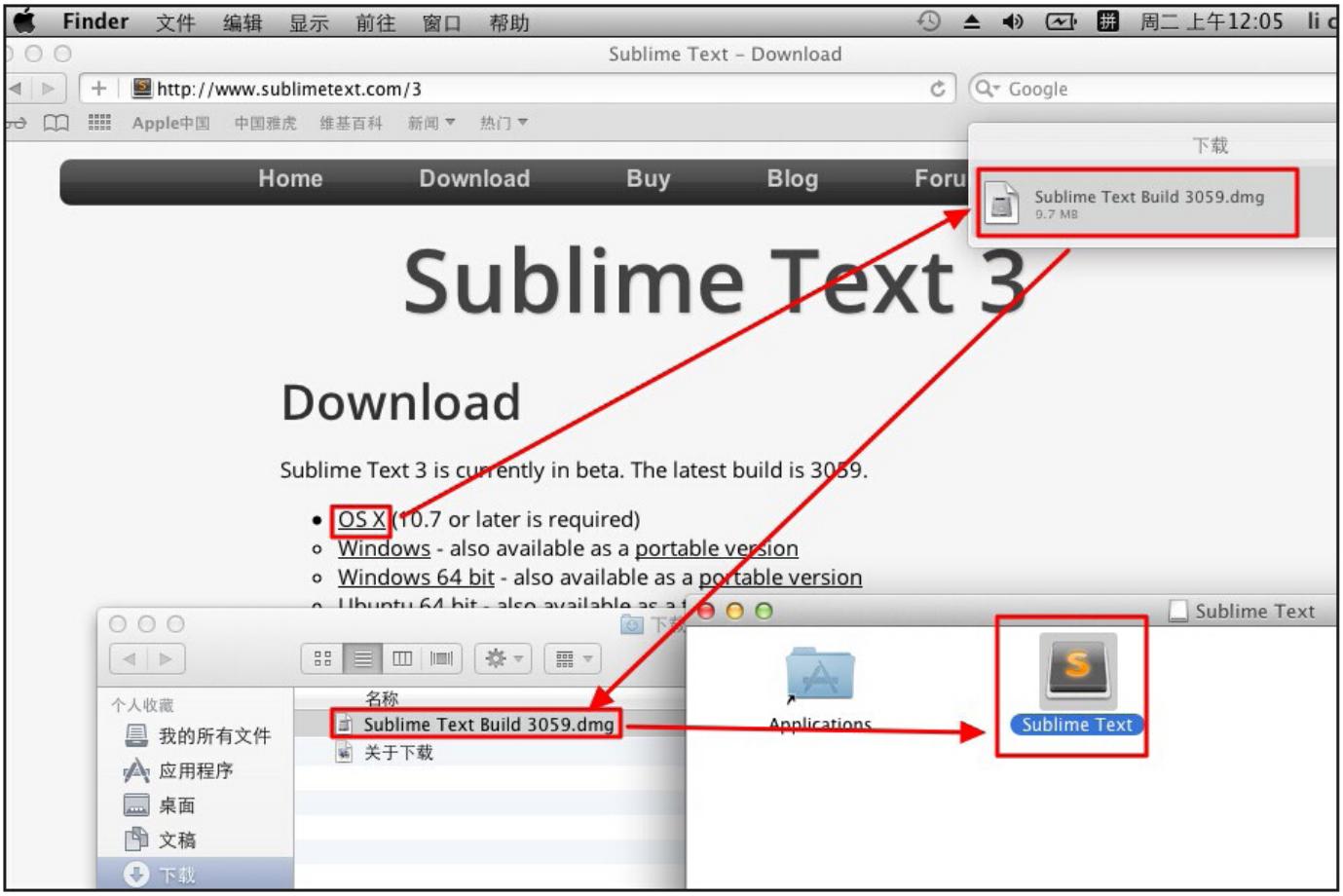


图3-61 下载并运行Sublime的dmg文件以找到Sublime Text

然后粘贴上述示例代码并保存为demoRunPythonUnderMac.py，如图3-62所示。

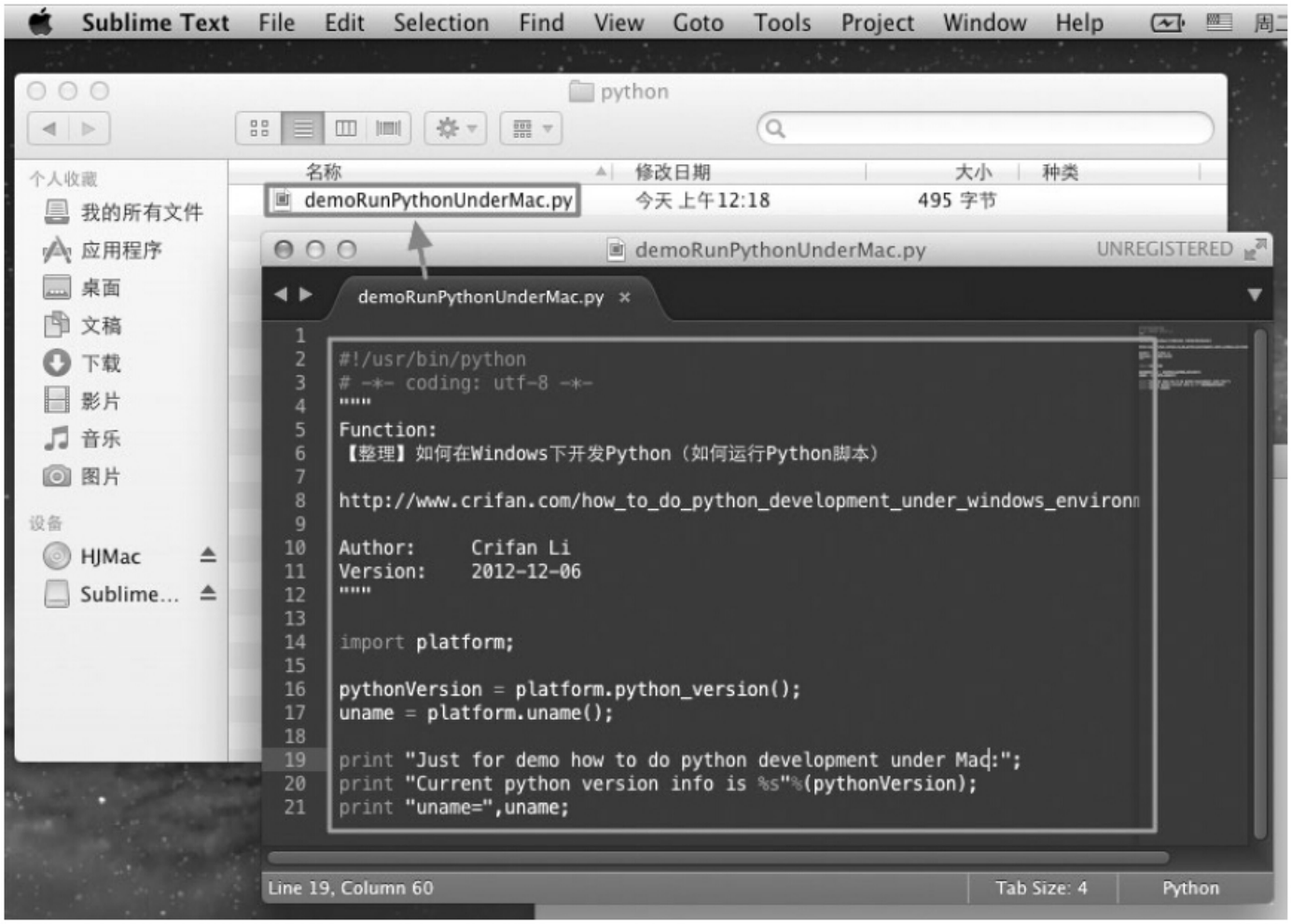


图3-62　Mac中保存后Python示例代码文件

2）打开Mac中的Terminal终端，并且切换到对应的Python脚本所在目录。

然后再到Mac系统中打开Terminal终端，如图3-63所示。

图3-63　在Mac的LaunchPad中的实用工具中找到Terminal终端

然后再切换到对应的Python脚本所在目录，如图3-64所示。

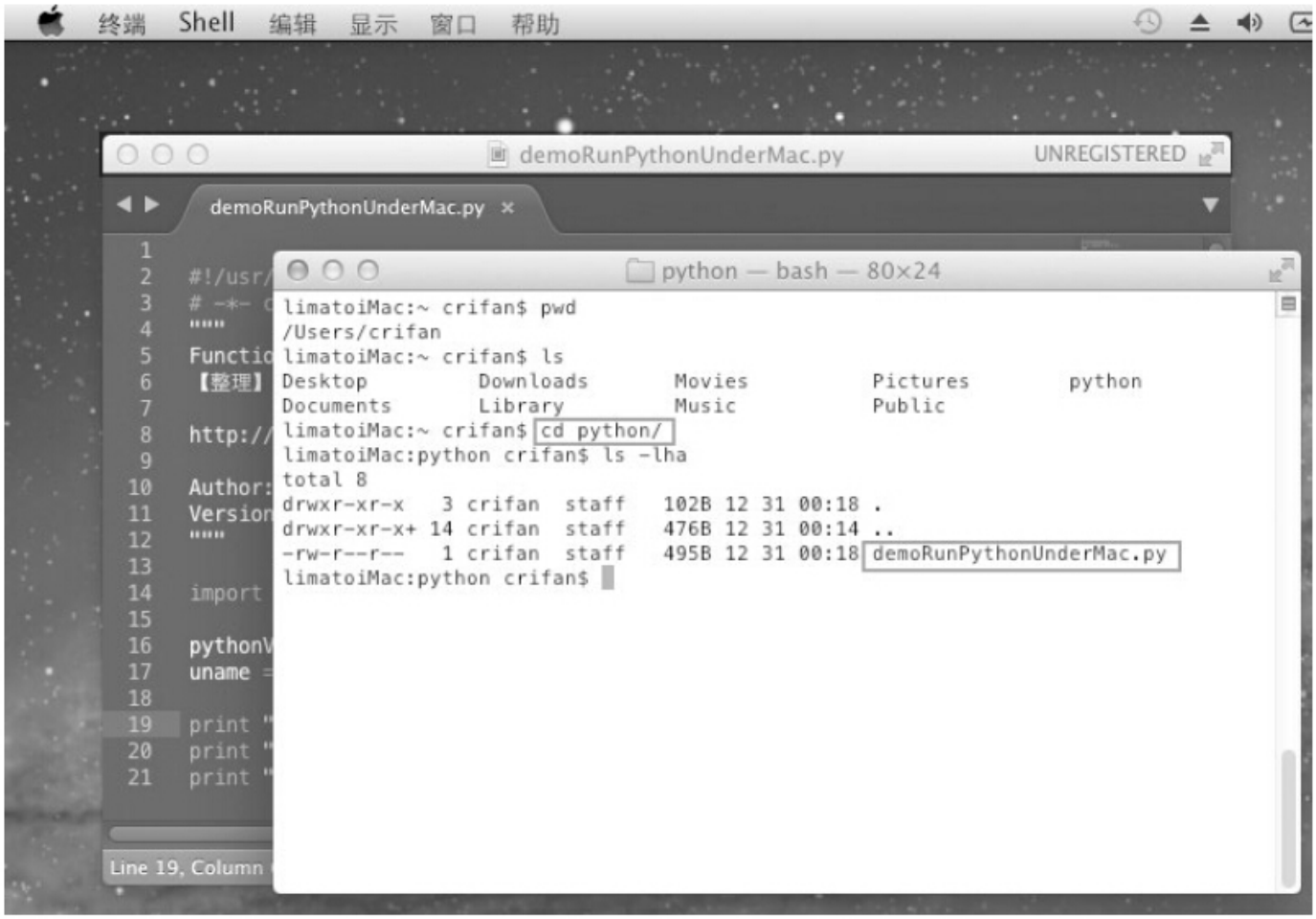


图3-64　Mac中打开终端并切换到Python文件所在目录

3）在Mac的Terminal环境中运行你的Python脚本（.py文件）。

然后就是在终端中运行Python脚本了，如图3-65所示。

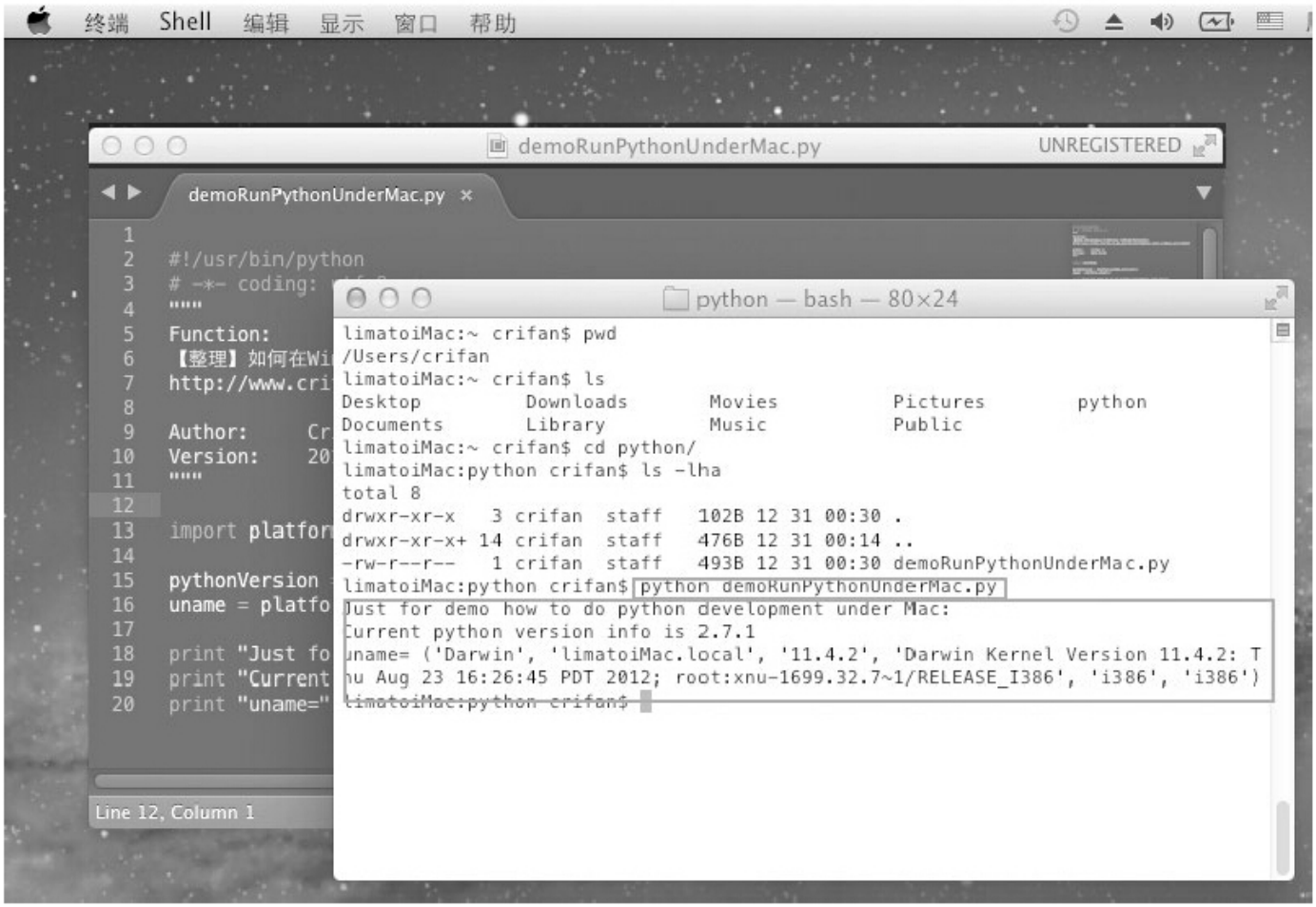


图3-65　在Mac的Terminal中运行Python文件

如此即可实现在Mac中运行Python代码，进行Python开发了。

另外，在此附上一个笔者之前写的Python脚本在Mac的终端下运行的效果，如图3-66所示。

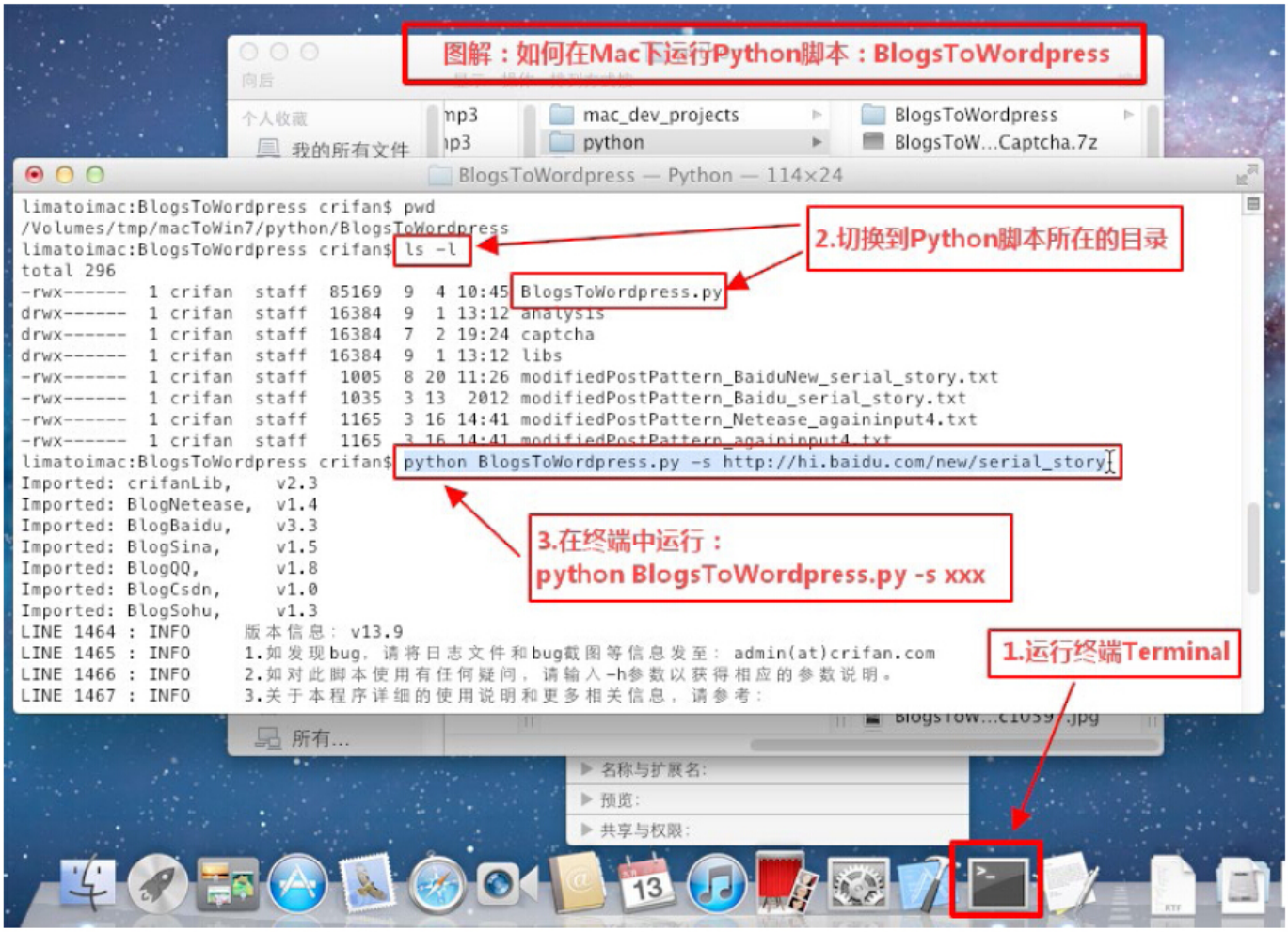


图3-66　在Mac下的Terminal中运行Python脚本：BlogsToWordpress

3.6　在Mac环境下进行Python开发

3.6.1　Python最原始的开发方式

1）Mac中用Sublime新建文件，写上Python代码，再保存为.py文件。

如之前推荐的，Mac下面的文本编辑器也用Sublime。读者登录Sublime主页下载对应的dmg安装包后双击打开，然后即可找到其中的Sublime Text，如图3-61所示。

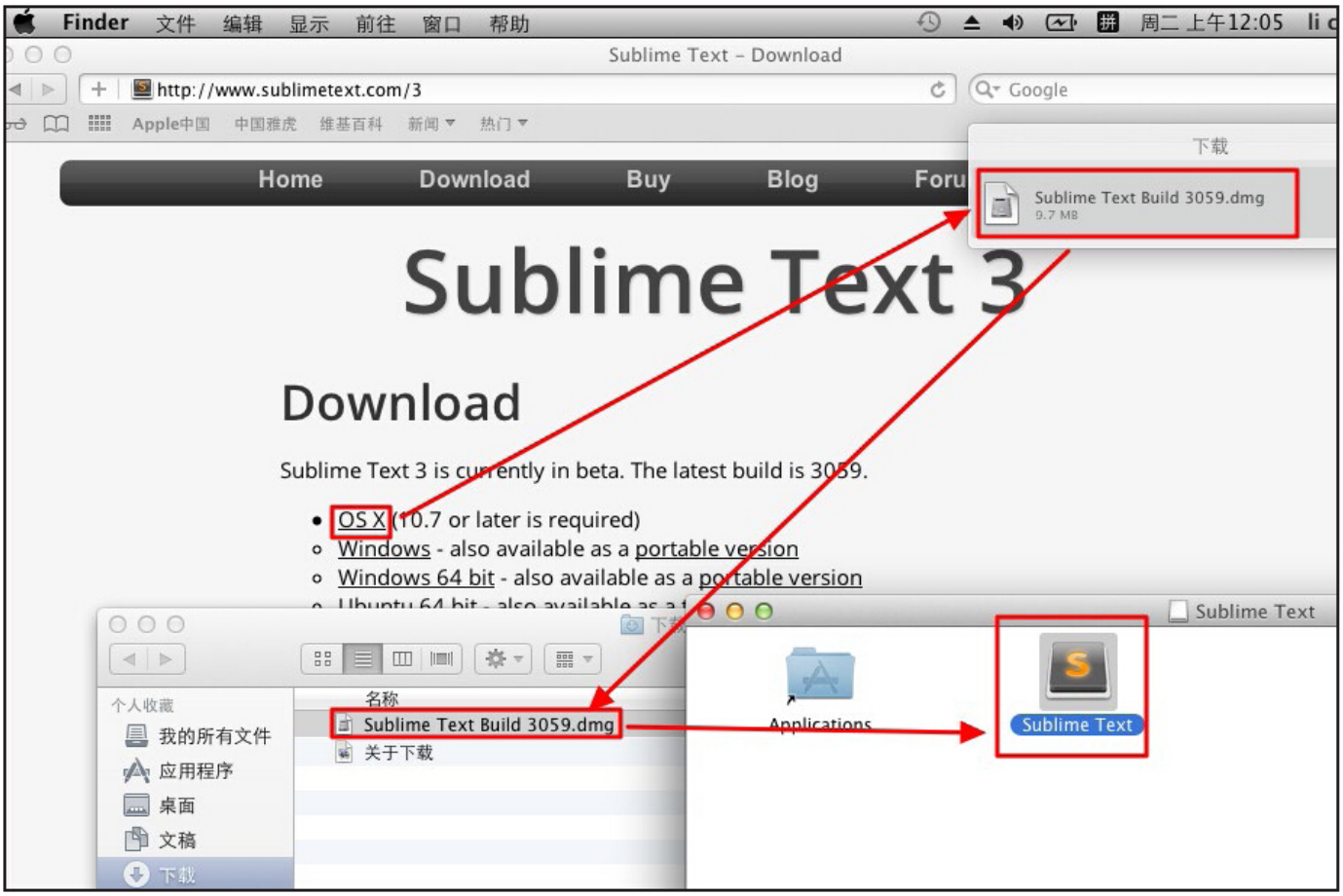


图3-61　下载并运行Sublime的dmg文件以找到Sublime Text

然后粘贴上述示例代码并保存为demoRunPythonUnderMac.py，如图3-62所示。

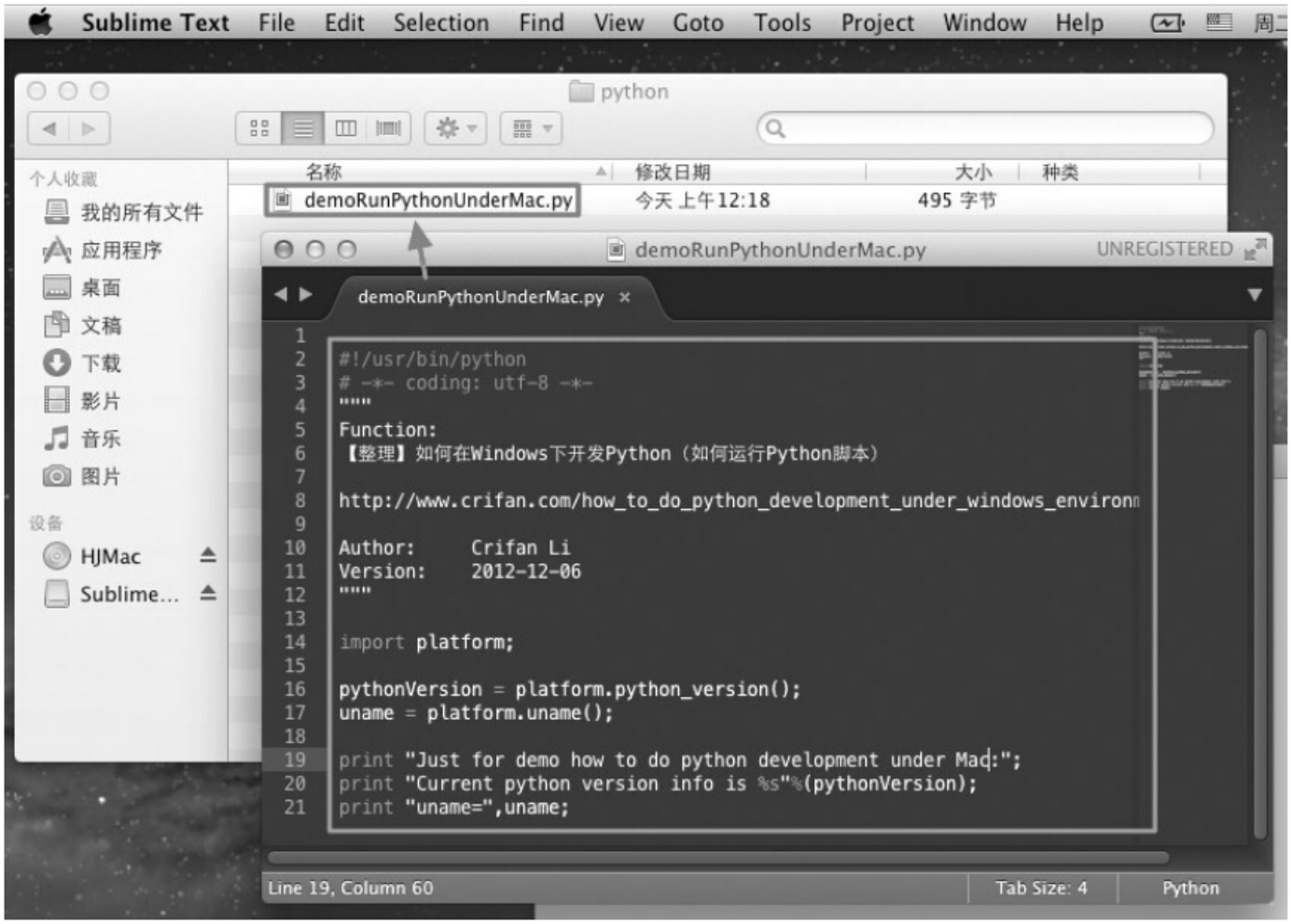


图3-62　Mac中保存后Python示例代码文件

2）打开Mac中的Terminal终端，并且切换到对应的Python脚本所在目录。

然后再到Mac系统中打开Terminal终端，如图3-63所示。

图3-63　在Mac的LaunchPad中的实用工具中找到Terminal终端

然后再切换到对应的Python脚本所在目录，如图3-64所示。

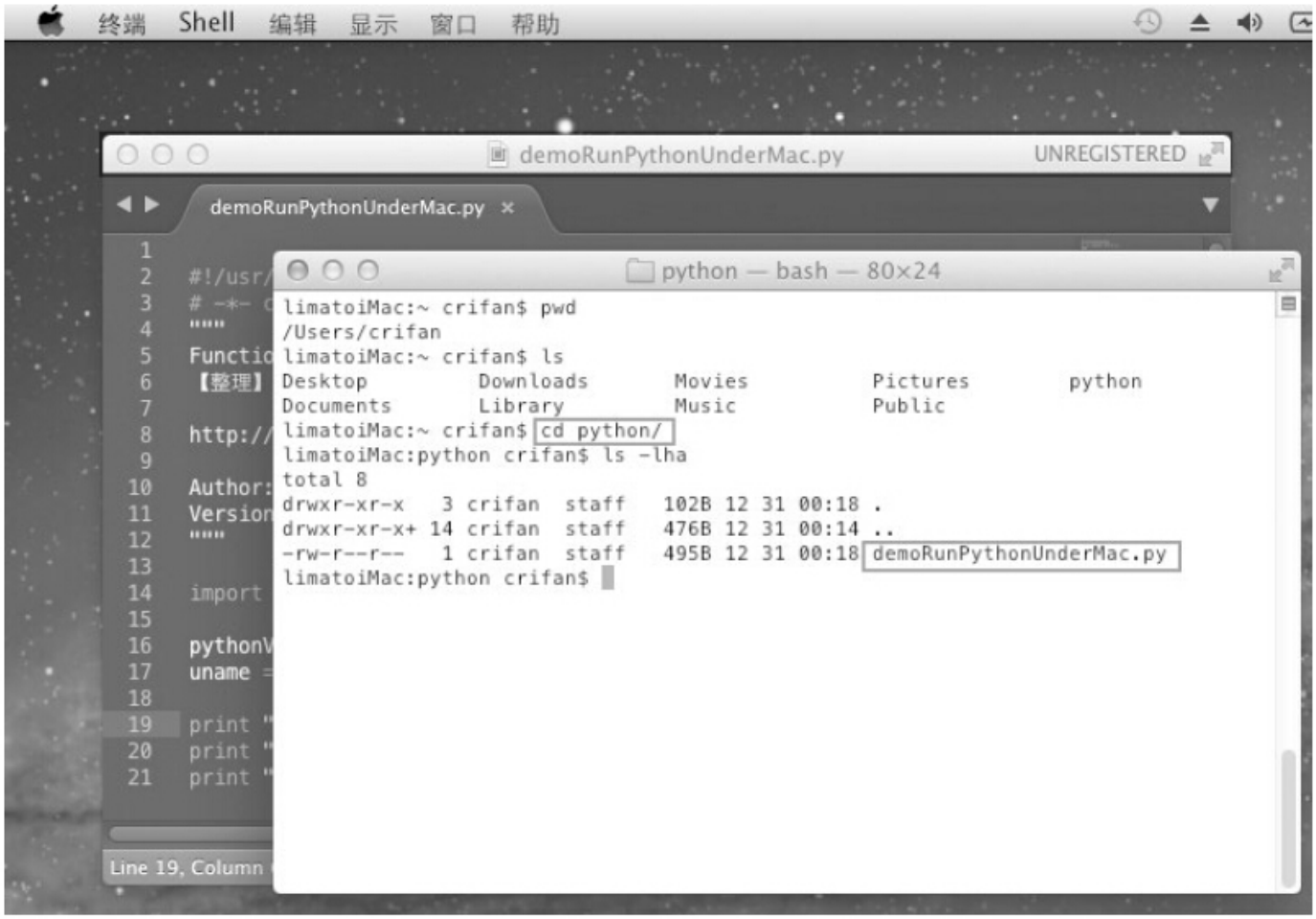


图3-64　Mac中打开终端并切换到Python文件所在目录

3）在Mac的Terminal环境中运行你的Python脚本（.py文件）。

然后就是在终端中运行Python脚本了，如图3-65所示。

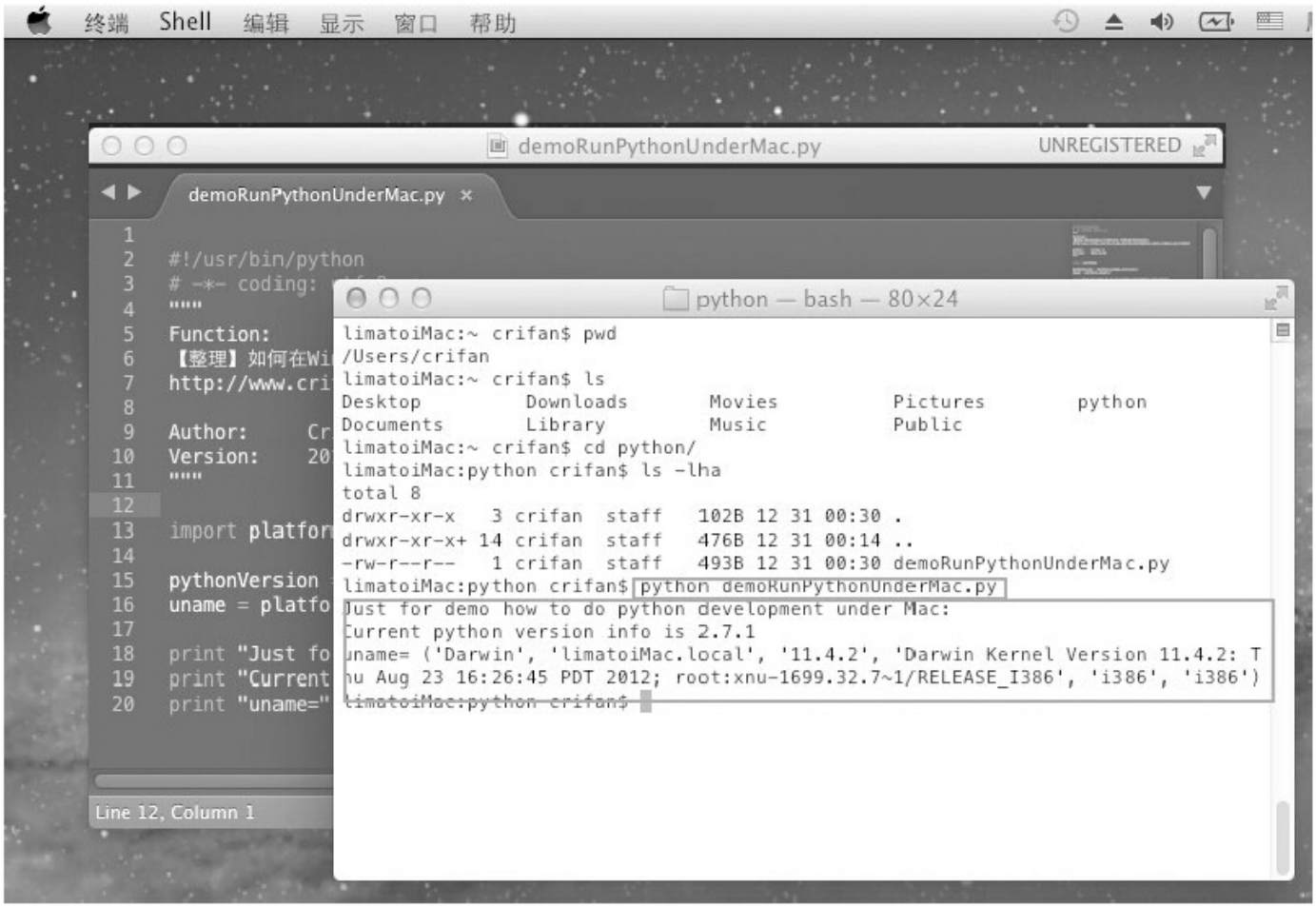


图3-65　在Mac的Terminal中运行Python文件

如此即可实现在Mac中运行Python代码，进行Python开发了。

另外，在此附上一个笔者之前写的Python脚本在Mac的终端下运行的效果，如图3-66所示。

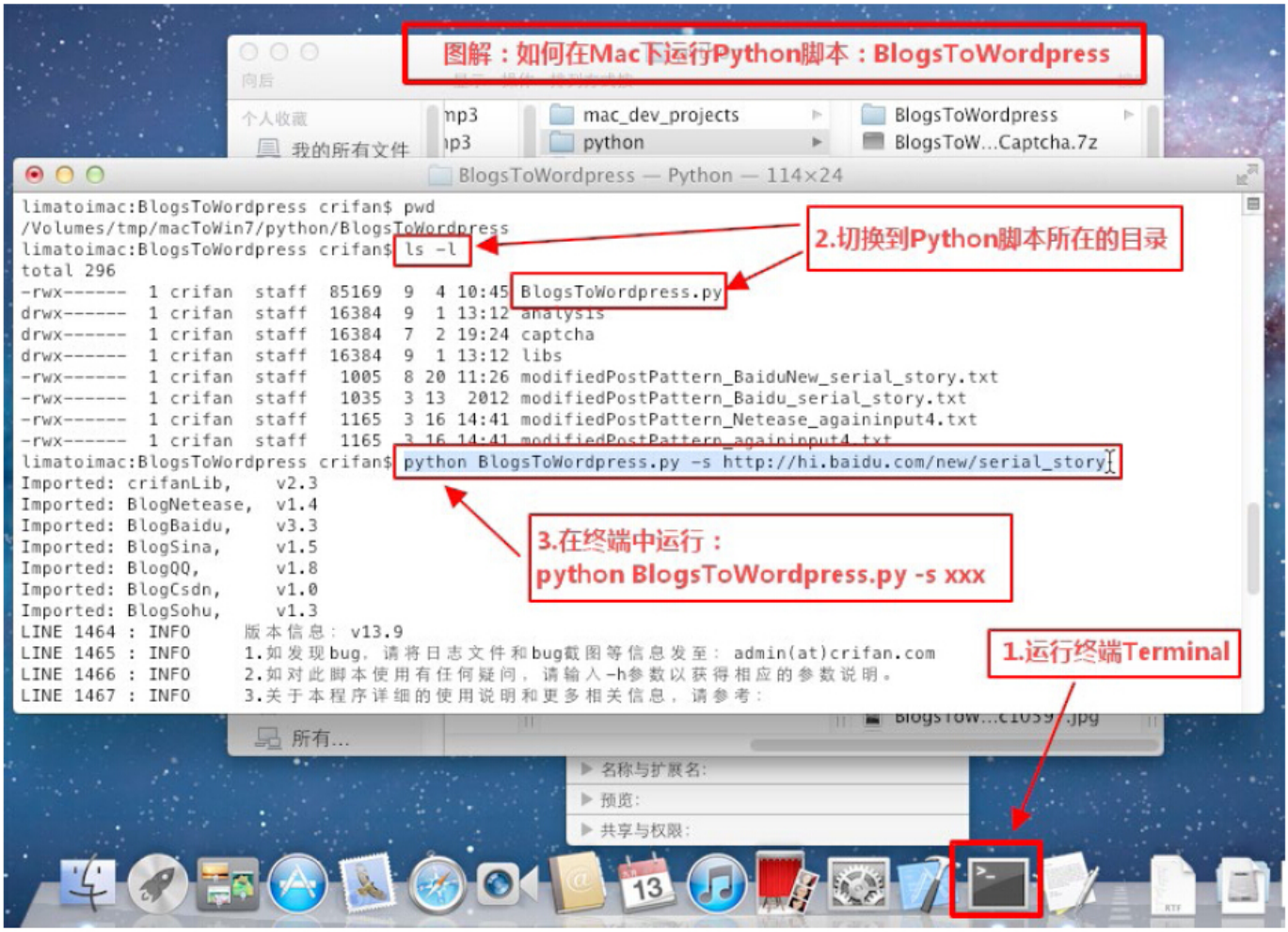


图3-66　在Mac下的Terminal中运行Python脚本：BlogsToWordpress

3.6.2　用Python的shell进行交互式开发

Mac中的基于命令行的Python的shell直接在Terminal终端中打开并使用即可，如图3-67所示。

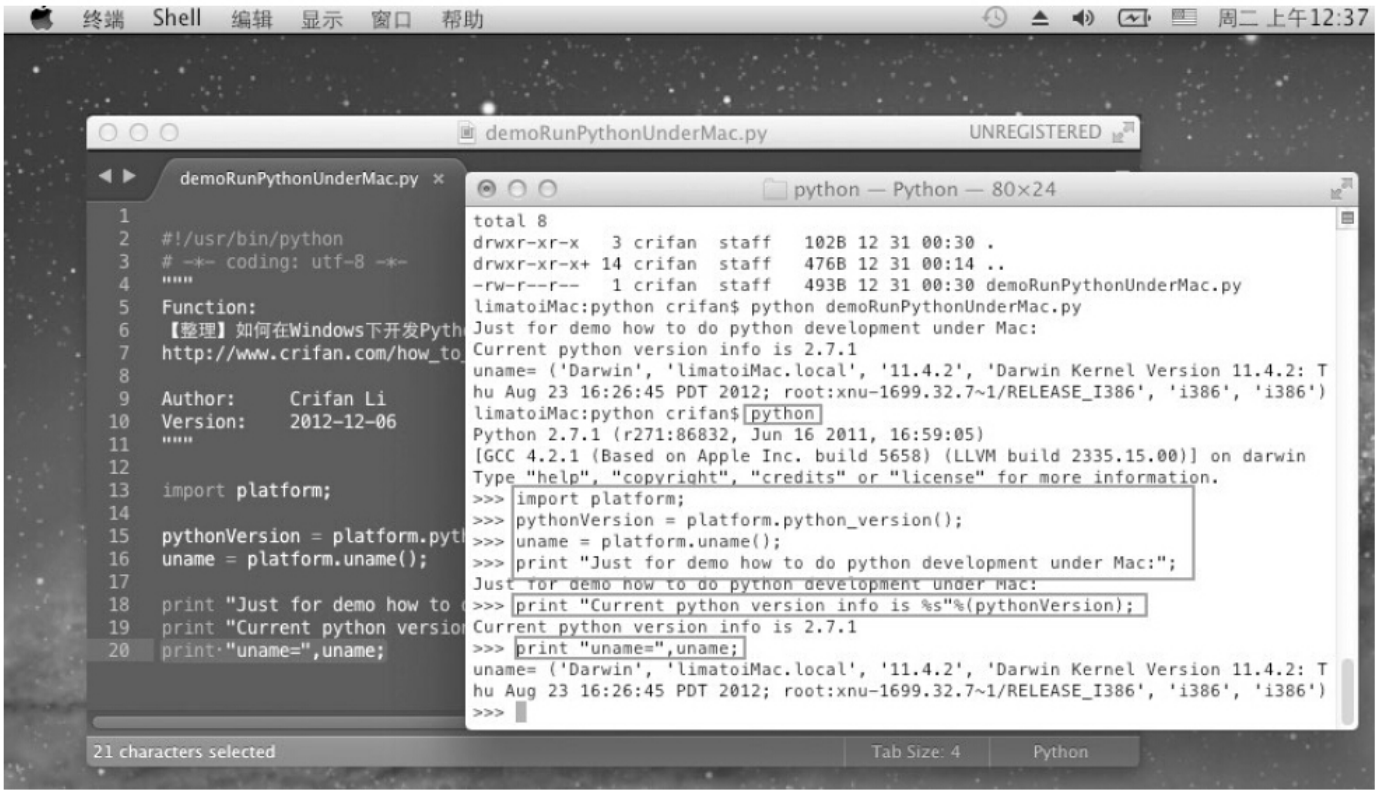


图3-67 在Mac的Python的shell中开发Python

如此即可实现在Mac中用Python的shell实现交互式开发。

3.6.3 用Python的IDE进行开发

同理，由于之前介绍的Eclipse+PyDev等Python的IDE是跨平台的，所以在Mac系统中也是可以使用这些跨平台的Python的IDE的。关于在Mac下如何使用这类Python的IDE可以参考Windows中如何使用该IDE。

至此，读者应该对于什么是Python的基本开发模式、什么是Python的shell、什么是Python的IDE，以及在多种平台不同环境下如何进行开发有了更加直观的认识了。

3.7 究竟应选用哪种环境开发Python

每个人到底应使用何种开发环境去开发Python？总的概括起来就是：各取所需，根据自己的需求去决定用什么开发环境。

针对初学者来说，笔者建议是使用第一种开发方式，即最原始的开发方式：用文本编辑器写Python代码并保存文件，然后到终端运行。

之所以如此推荐是因为包括Python在内的很多东西的学习本质上都是一个循序渐进的过程。而循序渐进的过程意味着你不能在没学会走路就开始去学跑，否则很容易摔跤，并且会摔得很痛，而且摔跤后爬起来也未必知道是什么原因导致你摔跤的。对于Python的学习也是如此，在还没有搞懂如何使用文本编辑器编写Python代码，没有学会在终端中运行Python代码，没有学会如何利用交互式的shell去调试Python代码时，直接就去用各种界面上看起来很炫的Python的IDE，就很可能导致你在使用IDE期间出现很多问题但却不知道错误原因。并且也容易在出现问题后，即使有人告诉你解决办法并帮你解决后，初学者仍无法透彻地理解问题

的根本原因，今后很可能再犯类似错误。

鉴于此，笔者才推荐初学者先学会基本的开发方式，在学习了更多的Python的基本知识，理解了基本的开发环境的原理和机制之后，再去使用Python的IDE去开发。到时候你才能真正做到理解并能充分利用IDE的功能，真正提高Python的开发效率。

即：

（1）开始先用文本编辑器加上终端

- Windows中的Notepad++加上cmd或Sublime加上cmd
- Linux中的Sublime加上Terminal或Gedit加上Terminal
- Mac中的Sublime加上Terminal

（2）以后再选用某个Python的IDE

- Windows中的PyScripter或Eclipse+PyDev或Ulipad
- Linux中的Eclipse+PyDev
- Mac中的Eclipse+PyDev

第4章　Python的基础知识

此处先介绍Python的最基本的语法知识，然后再介绍更多的Python的知识和概念。

4.1　SheBang和Python文件编码声明

很多人在开始接触Python时常会看到Python文件最开始的两行是这样的：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-
```

下面来解释这两行的详细含义。

4.1.1　#！/usr/bin/python

“#！”的特殊字符组合叫作SheBang，其出现的位置是文本文件的第一行的前两个字符。Linux类的系统中，发现文本文件出现SheBang时，系统就会去找SheBang后面的内容，尝试加载对应程序去解析此文件。所以此处的“#！/usr/bin/python”意思就是：Linux中运行此Python文件时，系统中就会尝试去加载/usr/bin/python去解析此Python文件。所以上述的含义总结起来

就是：对于常见的第一行是#！/usr/bin/python的Python文件在运行时，系统会尝试去找到对应的/usr/bin/python去解析并执行此Python文件。

4.1.2 Python文件编码声明

简单解释的话，“#-*-coding：utf-8-*-”就是用来指定文件编码的。而详细解释如下：根据官网的定义，之所以加上这一句，则是因为早在Python 2.1时，想要输入Unicode字符，只能用基于Latin-1的“unicode-escape”的方式输入，这就导致了对于其他非Latin-1的国家和用户想要输入Unicode字符，显得很繁琐和不方便。而我们最终希望的是所有的编程人员都根据自己的喜好和需要，以任意编码方式输入字符串。所以经过大家的讨论后最终采取的方案是允许在Python文件中通过文件开始处的字符串形式去声明自己的Python文件用何种编码。此声明放在注释代码内，且给出了该声明格式的严格定义。

1）如果没有此文件编码类型声明，则Python默认以ASCII编码去处理。所以，如果Python文件中没有编码声明而文件中又包含非ASCII编码的字符的话，Python解析器解析的Python文件自然就会报错了。

2）文件编码类型声明必须放在Python文件的第一行或第二行。换句话说，如果把文件编码类型声明放在其他行，则无法被识别。

3）编码声明所支持的格式有如下三种。

· 带等号的

```
# coding=<encoding name>
```

· 带冒号的

```
#!/usr/bin/python
# -*- coding: <encoding name> -*-
```

这种写法是最常见的，也是大多数编辑器都可以正确识别的。

· vim的配置

```
#!/usr/bin/python
# vim: set fileencoding=<encoding name> :
```

此种写法是专门用于vim的。注：vim是Linux类系统中使用的非常广泛的一种文本编辑器。

4）声明的格式的语法只要符合正则表达式：

```
"coding[:=]\s*([-\w.]+)"
```

就可以。所以我们可以根据此正则表达式的含义而写出各种合法的声明，比如：

```
coding:    utf-8
coding=utf-8
coding=        utf-8
encoding:utf-8
crifanEncoding=utf-8
```

5）如果你的Python文件本身编码是带BOM的UTF-8，即文件前三个字节是\xef\xbb\xbf，那么即使你没有声明文件编码也自动当做是UTF-8的编码。如果你声明了文件编码则必须是声明了（和你文件编码本身相一致的）UTF-8，否则由于声明的编码和实际编码不一致，则自然会报错。

根据上面的解释，下面可以给出一些常见的Python文件声明的例子。

（1）合法的Python文件编码声明

有如下几种不同风格的写法。

1）Emacs风格的，带声明了解释器的（注释中的）文件编码声明。

```
#!/usr/bin/python
# -*- coding: latin-1 -*-
import os, sys
http://www.hzcourse.com/resource/readBook?path=/openresources/teach_ebook/uncompressed/15066/OEBPS/Text/...
#!/usr/bin/python
# -*- coding: iso-8859-15 -*-
import os, sys
http://www.hzcourse.com/resource/readBook?path=/openresources/teach_ebook/uncompressed/15066/OEBPS/Text/...
#!/usr/bin/python
# -*- coding: ascii -*-
import os, sys
http://www.hzcourse.com/resource/readBook?path=/openresources/teach_ebook/uncompressed/15066/OEBPS/Text/...
```

2）不带编码声明解释器的，直接用纯文本形式的文件编码声明。

```
# This Python file uses the following encoding: utf-8
import os, sys
http://www.hzcourse.com/resource/readBook?path=/openresources/teach_ebook/uncompressed/15066/OEBPS/Text/...
```

3）文本编辑器也可以有多种（其他的）定义编码的方式。

```
#!/usr/local/bin/python
# coding: latin-1
import os, sys
http://www.hzcourse.com/resource/readBook?path=/openresources/teach_ebook/uncompressed/15066/OEBPS/Text/...
```

很明显该写法没有使用-*-而是直接用了coding加上编码值。

4）不带编码声明的，默认当做ASCII处理。

```
#!/usr/local/bin/python
import os, sys
http://www.hzcourse.com/resource/readBook?path=/openresources/teach_ebook/uncompressed/15066/OEBPS/Text/...
```

（2）非法的Python文件编码声明举例

1）少了coding：前缀。

```
#!/usr/local/bin/python
# latin-1
import os, sys
http://www.hzcourse.com/resource/readBook?path=/openresources/teach_ebook/uncompressed/15066/OEBPS/Text/...
```

2）编码声明不在第一行或第二行。

```
#!/usr/local/bin/python
#
# -*- coding: latin-1 -*-
```

```
import os, sys
http://www.hzcourse.com/resource/readBook?path=/openresources/teach_ebook/uncompressed/15066/OEBPS/Text/...
```

3）不支持的，非法的字符编码（字符串）声明。

```
#!/usr/local/bin/python
# -*- coding: utf-42 -*-
import os, sys
http://www.hzcourse.com/resource/readBook?path=/openresources/teach_ebook/uncompressed/15066/OEBPS/Text/...
```

4.2 Python中的缩进

因为Python中的缩进不仅仅是美观问题，而且还决定了代码的逻辑层次，即决定了代码的含义，所以相对来说很重要。因此本节在介绍Python的基本的变量定义等内容之前，需要详细地介绍Python中的缩进。

4.2.1 其他语言的缩进只影响代码的美观

在解释Python中的代码缩进之前先来看看其他语言，例如C语言中普通的代码缩进的含义和效果。我们已经知道，代码风格良好的C语言代码需要有必要的缩进，例如：

```
#include "stdio.h"
int main(void){
    printf("Demo indent in C file\n");
    return 0;
}
```

这些必要的缩进使得代码逻辑上层次更清晰，对应着此处的printf和return等语句都是main函数中的内容，所以会有一级的缩进。而如果代码风格不好，缺少必要的缩进，比如如下代码所示故意写成看起来很错乱的缩进的效果。

```
#include "stdio.h"
    int main(void){
printf("Demo indent in C file\n");
            return 0;
}
```

此时虽然看起来很不好，但是实际上不影响代码的逻辑。换句话说，代码仍然是可以正常编译、正常执行的，如图4-1所示。

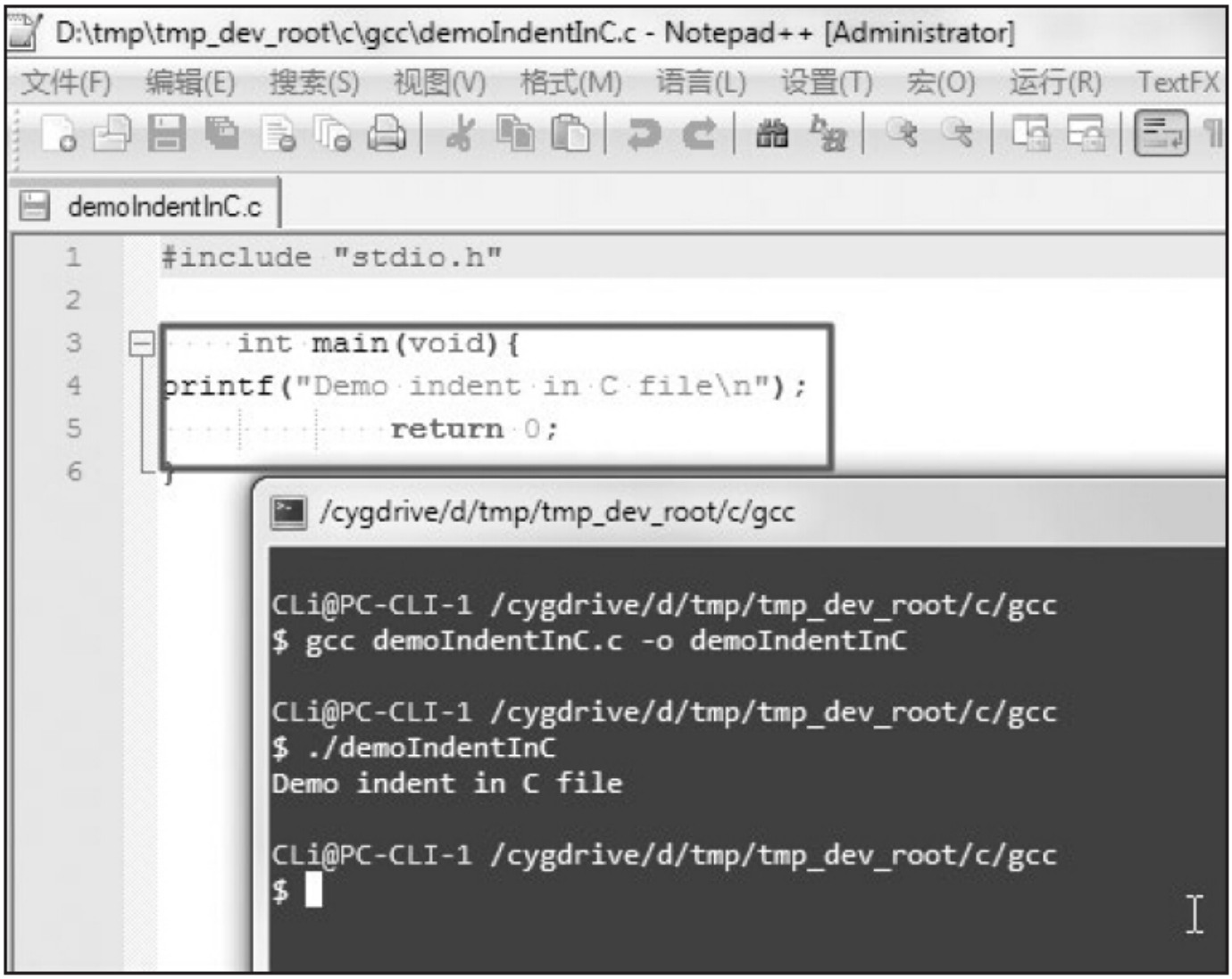


图4-1　缩进很乱的C代码仍然能够正常编译和运行

4.2.2　Python的缩进会影响代码的逻辑

1.Python代码缩进错误举例

接下来说说Python中代码的缩进。为了说明Python的缩进是影响代码逻辑的，此处先通过几个例子来说明Python代码中缩进所导致的错误。

（1）IndentationError：unexpected indent

运行如下代码：

```
# -*- coding: utf-8 -*-
import pickle
import math
   ipath = "D:/123"
   fileobj= open(ipath, 'rb')
   pdata= pickle.load(fileobj)
   fileobj.close()
```

结果出现如下错误：

```
D:\tmp\tmp_dev_root\python\pick_dump_error>pickle_dump.py
  File "D:\tmp\tmp_dev_root\python\pick_dump_error\pickle_dump.py", line 4
    ipath = "D:/123"
    ^
IndentationError: unexpected indent
```

出错的原因是因为Python解析器发现该代码缩进有问题所以才报错的。而错误的具体原因是在代码“import math”之后没有出现有效的函数定义等内容，而直接出现一个缩进加上一个赋值语言“ipath=”D：/123“”，而此种缩进的前面，既不是函数定义的形式：

```
def someFunction():
    xxx
    xxx
```

也不是其他的形式，所以这样的代码从语法上来说就不正确。所以Python解析器不知道这段代码是属于哪个范围的，因为无法解析这样的代码而报错的。

（2）代码执行无结果

比如这样的代码：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-
"""
Function:
【教程】详解Python中代码缩进（Indent）
http://www.crifan.com/tutorial_python_indent
Author:     Crifan Li
Version:    2012-12-24
Contact:    admin at crifan dot com
"""
def indentDemo():
    print "Just demo for python code indent";
#if __name__ == "__main__":
    indentDemo();
```

执行后的结果是空的：

```
D:\tmp\tmp_dev_root\python\python_indent>python_indent.py
D:\tmp\tmp_dev_root\python\python_indent>
```

看起来是什么输出都没有，这和我们期望的结果明显不符，因为我们希望它可以输出那段print语句内容的。其原因在于上述代码中的“indentDemo()；”是属于函数indentDemo中的代码，而不是此脚本所能执行出来的代码。而想要达到我们要的效果，即能够执行到对应的函数indentDemo，可以改为：

```
def indentDemo():
    print "Just demo for python code indent";
if __name__ == "__main__":
    indentDemo();
```

或：

```
def indentDemo():
    print "Just demo for python code indent";
#if __name__ == "__main__":
indentDemo();
```

都可以达到我们所要的效果，输出对应的信息为：

```
D:\tmp\tmp_dev_root\python\python_indent>python_indent.py
```

```
Just demo for python code indent
D:\tmp\tmp_dev_root\python\python_indent>
```

2.代码缩进的含义

解释完常见的由于缩进问题所导致的结果以及其中的原因和解决办法之后，再来详细解释Python中代码缩进的含义：Python中是通过代码的缩进来决定代码的逻辑的。通俗的说，Python中的代码的缩进不仅仅是为了好看，更是决定了代码的含义，决定了上下行代码之间的关系，用英文说法就是决定了代码的context。所以如果缩进弄错了就会导致程序出错，执行结果变成不是自己想要的了。关于代码缩进主要有这几方面的含义。

（1）关于第一行要执行的代码

很多初学者在写了Python代码但运行后没有输出想要的结果，很多都是由于缩进有误而导致的。就拿上面的错误例子中“代码执行无结果”来说，实际上Python解释器去执行你的代码的逻辑如图4-2中的描述所示。

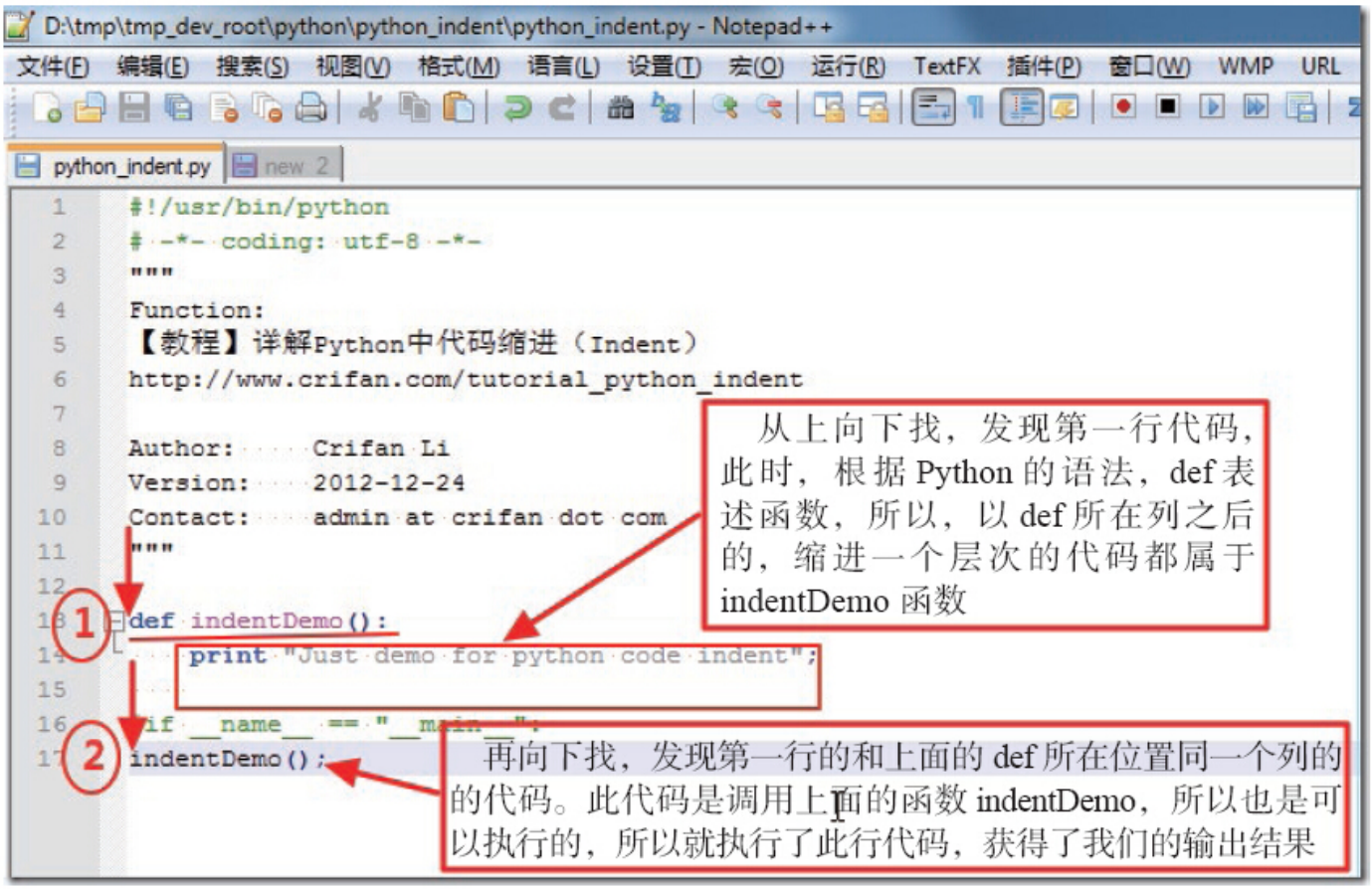


图4-2　Python从能找到的第一行的代码开始运行

如果没有找到可以运行的代码就直接退出，就不会输出任何信息了，如图4-3所示。

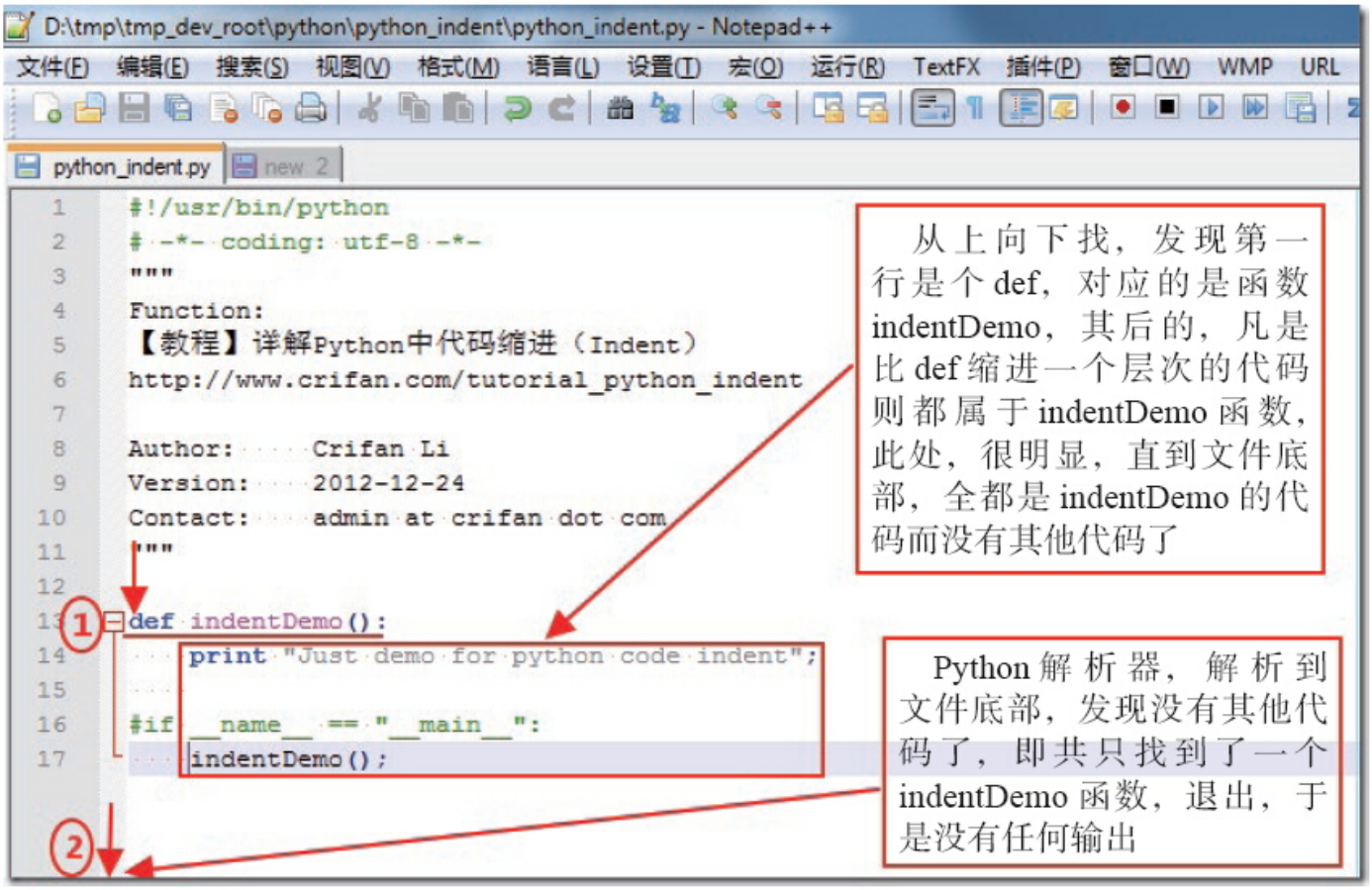


图4-3　Python解析器找不到可以运行代码所以退出

如果有__name__是否为__main__的判断，则会去判断是否为真，为真的话再去运行对应的代码段。此处应该调用执行indentDemo函数，如图4-4所示。

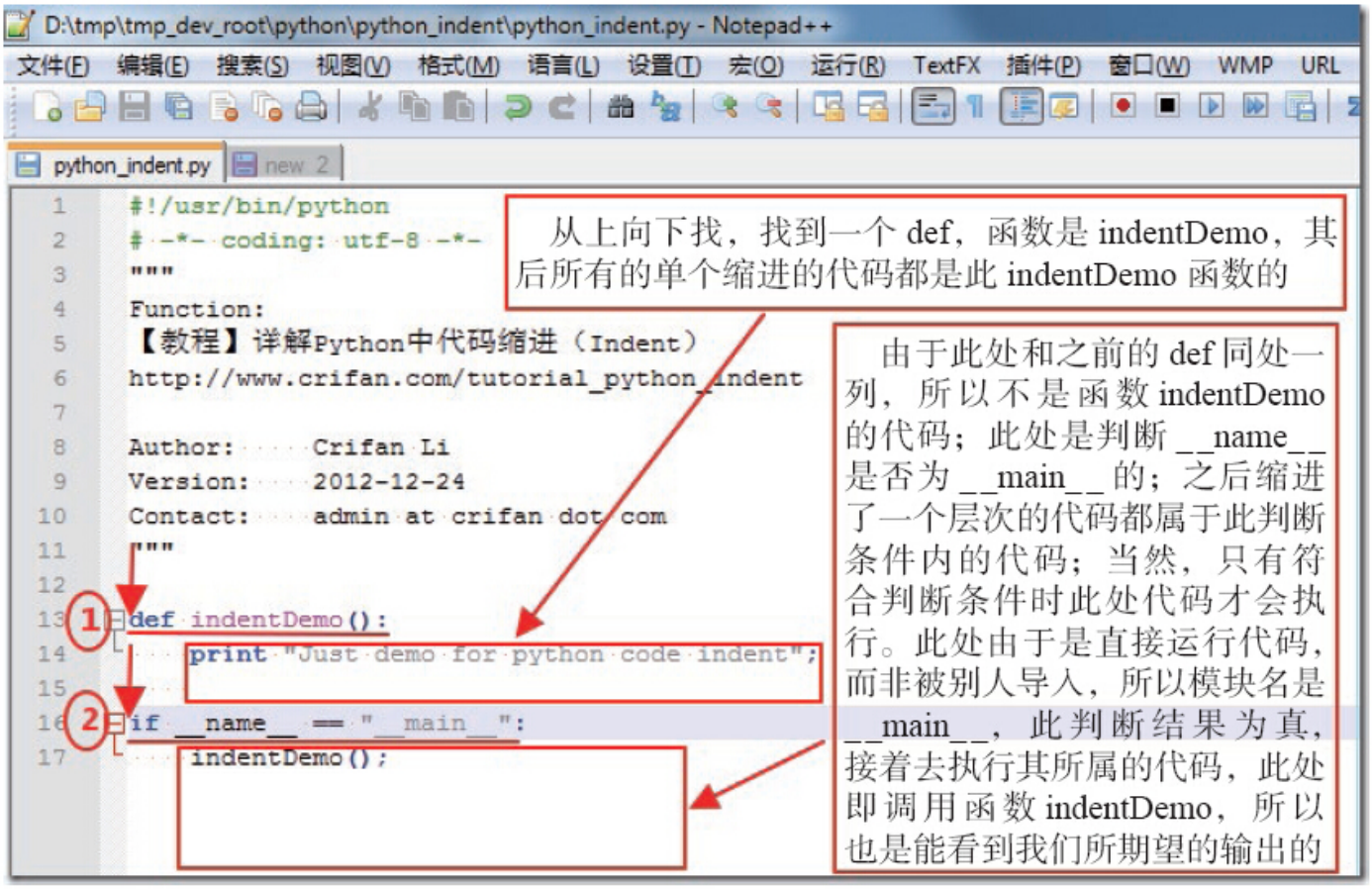


图4-4　去判断__name__是否为__main__然后再去执行对应的代码段

其中关于__name__和__main__的含义，可参见4.3节的解释。

（2）代码块

对应的上述几个图解中，def indentDemo后面的代码也就因此成为代码块，如图4-5所示。

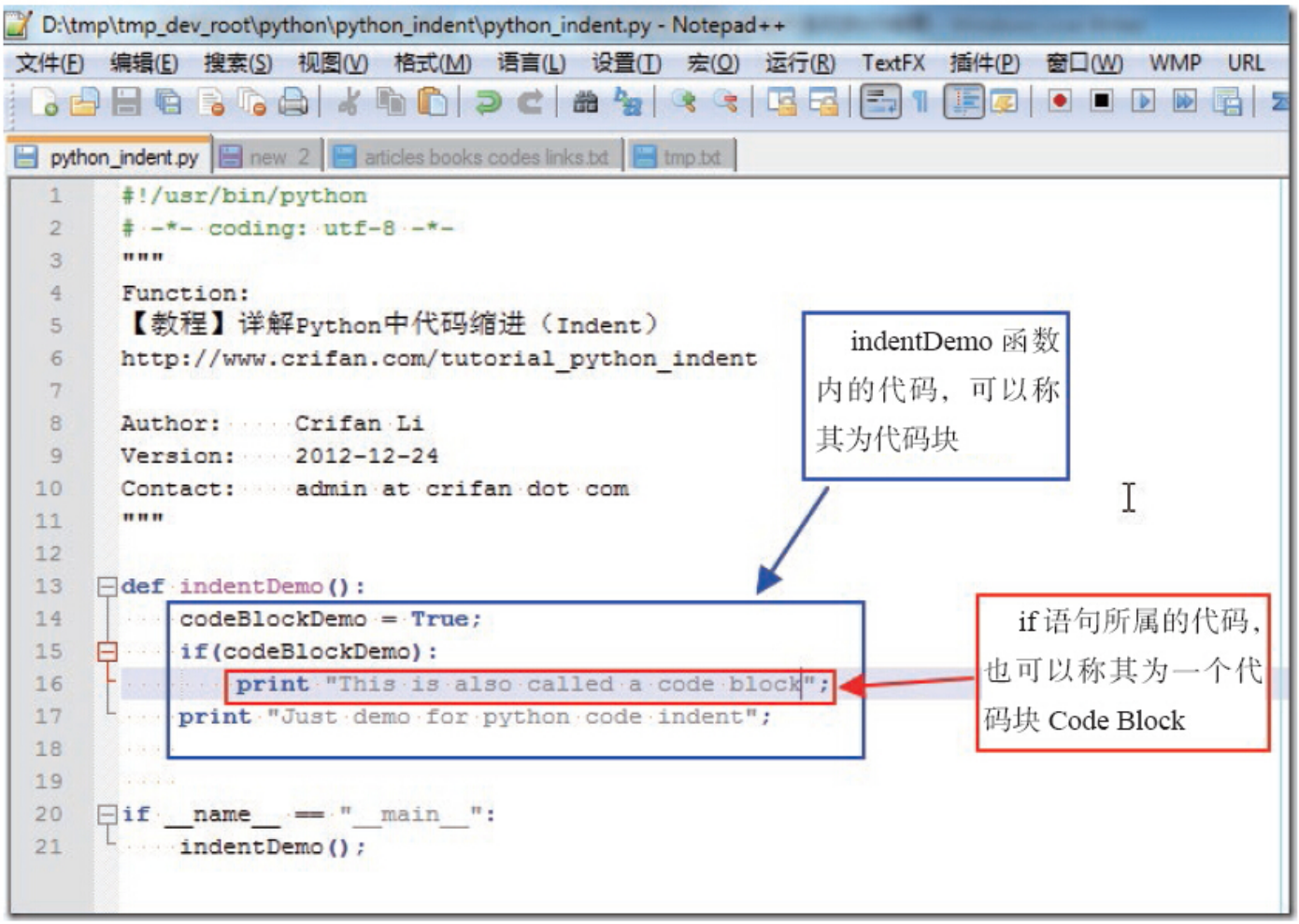


图4-5　Python中的代码块

说白了就是一个逻辑上的概念，可以简单理解为等价于其他语言中的一个函数内的代码或一个if判断内的代码。

总结起来就是：Python中的代码缩进，不仅仅是为了代码看起来好看，不仅仅是为了看起来逻辑上更加清晰，更重要的是决定了代码的内在逻辑关系，进而影响代码的执行结果。而其他很多语言，比如C、C#、Java等的代码缩进只是为了代码“看起来”的逻辑更加清晰，但是不影响代码的内在逻辑关系，不影响代码的执行结果。

4.3　Python中__name__和__main__的含义

我们在自己写Python代码和看别人的Python代码时，常常会遇到这样的代码：

```
def main():
    http://www.hzcourse.com/resource/readBook?path=/openresources/teach_ebook/uncompressed/15066/OEBPS/Text/...
if _?_name == "__main__":
    main();
```

其中的main函数也可以是其他任意你自己所写的函数的名字。此处就出现了两个很常见的东西：__name__和__main__，接下来就来解释__name__和__main__的含义。

4.3.1 __name__详解

先来说__name__的含义，根据Python官方手册（见图4-6）中以及整理了别处的含义解释，最终总结出__name__的含义是：

· 如果是放在Modules模块中，就表示是模块的名字；

· 如果是放在Classs类中，就表示类的名字。

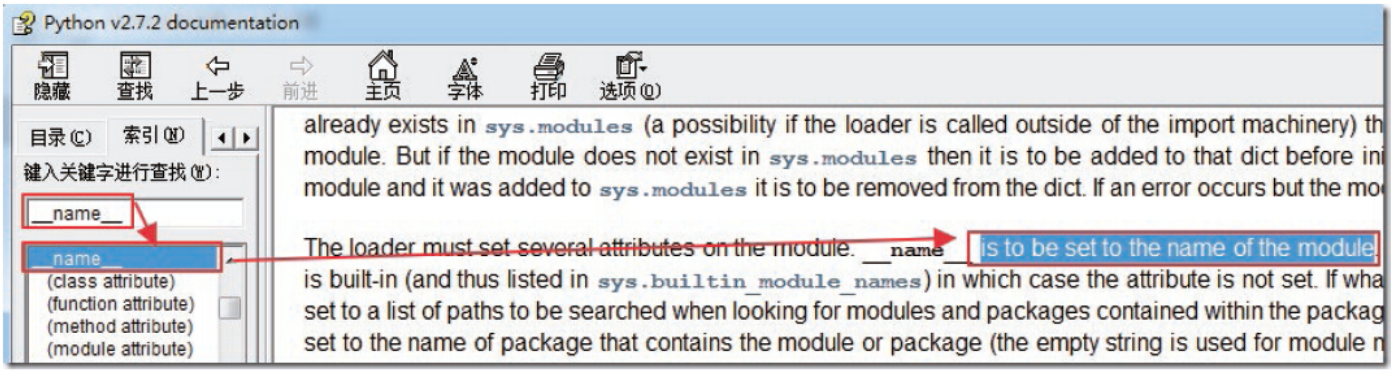


图4-6　Python手册中__name__的含义

4.3.2 __main__详解

在解释__main__之前先介绍一个背景知识：我们之前所了解的，Python代码可以直接一行行写出来然后再去运行，但是这只是针对我们的小程序来说的，实际上我们更多时候所见到的是Python代码设计成更加通用的可以被重复调用的模块的形式，此时Python函数和功能都是写在对应的函数里面的。而所写的Python脚本文件在被当做模块时，第一次被导入（import）后，系统会自动为其创建一个域名空间（namespace），其中模块都是有自己的不同的名字的。而对于此处的脚本的主模块的名字，始终都叫作__main__。

解释完背景知识，再去Python手册中找到对应的解释，如图4-7所示。

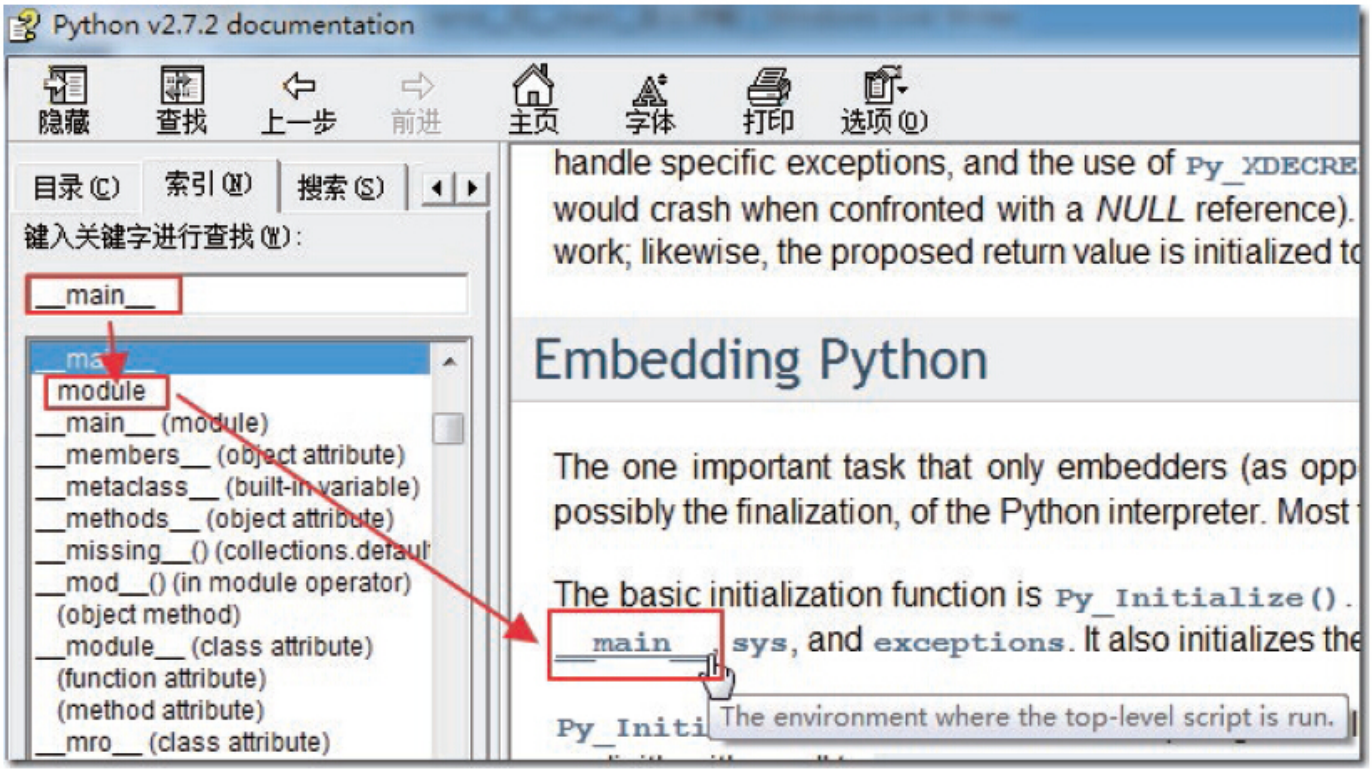


图4-7　Python手册中__main__的含义

将上述官方解释整理后，再加上别处的解释，最终可总结出__main__的含义：

· 如果当前Python文件被当做模块调用时，此__name__的名字就是当前Python脚本所对应的模块名，就不是__main__了，因此此时就不去执行main函数。

· 而如果当前Python文件被当做普通的Python脚本文件直接被执行时，此时__name__就始终都是__main__，所以此时就去执行main函数。

4.3.3　__name__和__main__搭配使用的目的

上述将__name__和__main__搭配使用的目的是当一个模块不是当做模块被调用而是被直接执行时，就运行事先写好的那些测试代码，用于测试当前模块的功能。所以我们可以用这套机制去实现添加模块测试代码或者其他方面的功能。下面用一个示例来说明具体效果。

作为普通的Python代码，我们是可以这么写的：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-
"""
 【整理】Python中的__name__和__main__含义详解
http://www.crifan.com/python_detailed_explain_about__name__and__main__
Version:    2012-11-17
Author:     Crifan
Contact:    http://www.crifan.com/contact_me/
"""
def square(x):
    return x * x
print "test: square(42) ==",square(42);
```

对应的运行结果也是很正常的，调用函数square得到执行结果：

```
E:\Dev_Root\python\__name__and__main__>__name___and___main__.py
test: square(42) == 1764
```

相对而言更加模块化的可以被重复调用的写法则可以改为对应的模块文件是mymath.py，里面有我们实现的函数square：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-
"""
【整理】Python中的__name__和__main__含义详解
http://www.crifan.com/python_detailed_explain_about__name__and__main__
Version:    2012-11-17
Author:     Crifan
Contact:    http://www.crifan.com/contact_me/
"""
def square(x):
    return x * x
print "test: square(42) ==",square(42);
```

然后其他Python文件__name__and__main__.py中导入此mymath模块，然后使用其square函数：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-
"""
 【整理】Python中的__name__和__main__含义详解
http://www.crifan.com/python_detailed_explain_about__name__and__main__
Version:    2012-11-17
Author:     Crifan
Contact:    http://www.crifan.com/contact_me/
"""
import mymath;
print "In caller, test for mymath: square(12)=",mymath.square(12);
```

运行效果为：

```
E:\Dev_Root\python\__name__and__main__>__name__and__main__.py
test: square(42) == 1764
In caller, test for mymath: square(12)= 144
```

此处就同时出现了原本用于mymath.py中测试square函数的打印结果：

```
test: square(42) == 1764
```

很明显是作为模块的调用者所不希望看到的、也不会去关心的内容。此时我们希望的是：作为模块mymath.py本身，希望有自己的相关的调试的代码用于调试自己模块函数、演示如何使用，但是又不希望在被别的、本模块的调用者执行到、看到，此时就可以用上述的__main__来实现了，将mymath.py改为：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-
"""
 【整理】Python中的__name__和__main__含义详解
http://www.crifan.com/python_detailed_explain_about__name__and__main__
Version:    2012-11-17
Author:     Crifan
Contact:    http://www.crifan.com/contact_me/
"""
def square(x):
    return x * x
if __name__ == "__main__":
    print "test: square(42) ==",square(42);
```

此时：

· 作为mymath.py本身，自己独立运行时可以运行到此处的代码，用于调试和验证自己的函数square执行是否正确：

```
E:\Dev_Root\python\__name__and__main__>mymath.py
```

```
test: square(42) == 1764
```

· 同时，作为mymath.py的调用者__name__and__main__.py，在import mymath时也不会看到对应的代码执行的结果：

```
E:\Dev_Root\python\__name__and__main__>__name__and__main__.py
In caller, test for mymath: square(12)= 144
```

其中的__main__就是作为模块mymath.py本身，而作为脚本自己去运行的话，对应的模块名就是上面所解释的始终叫作__main__。

这点上述代码已经验证过了。因为mymath.py中的__name__就是对应的内置的变量，通过if去判断的确是__main__，然后才会去执行到对应的模块的测试代码的。

如果被当做一个模块被他人调用时对应的模块mymath.py的模块名就是mymath，这点我们可以来验证一下，将__name__and__main__.py改为：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-
"""
 【整理】Python中的__name__和__main__含义详解
http://www.crifan.com/python_detailed_explain_about__name__and__main__
Version:    2012-11-17
Author:     Crifan
Contact:    http://www.crifan.com/contact_me/
"""
import mymath;
print "In caller, test for mymath: square(12)=",mymath.square(12);
print "As module be imported, mymath module name=",mymath.__name__;
```

然后运行就可以看到输出的结果是mymath了：

```
mymath了：
E:\Dev_Root\python\__name__and__main__>__name__and__main__.py
In caller, test for mymath: square(12)= 144
As module be imported, mymath module name= mymath
```

也可以通过另外一个更加精简的例子来说明此问题：

```
#!/usr/bin/python
# Filename: using_name.py
if __name__ == "__main__":
    print "This program is being run by itself"
else:
    print "I am being imported from another module"
```

4.4 Python中的面向对象编程

作为支持面向对象的编程语言Python，当然也是有很多特性是支持对象方面的编程的。Python中和面向对象有关的一些最基础的内容就要属self和__init__了，下面就来详细解释一下它们的含义。

4.4.1 self和__init__的含义

在Python中的类Class的代码中常看到函数中的第一个参数都是self，且Class中的函数里面访问对应的变量（读取或者写入）以及调用对应的函数时都是：

· self.valueName

· self.function()

的形式。先来解释Python中的self的含义，我们都知道，self这个英文单词意思表示自己、本身，而此处有几种潜在含义：

· 这里的自己指的是实例Instance本身。

· 说到“自己”这个词都是和相对而言的“其他”而说的。而此处的其他则指的是类Class和其他变量，比如局部变量、全局变量等。

此处的self是个Object对象，是当前类的实例。因此对应的含义是：

· self.valueName

valueName表示self对象，即实例的变量，与其他的Class的变量、全局的变量、局部的变量是相对应的。

· self.function()

function表示是调用的是self对象，即实例的函数，与其他的全局的函数是相对应的。

而关于Python中为何要有self则是因为在类的代码（函数）中需要访问当前的实例中的变量和函数的，即访问Instance中的：

· 对应的变量（property）：Instance.ProperyName，去读取之前的值和写入新的值

· 调用对应函数（function）：Instance.function()，即执行对应的动作

而需要访问实例的变量和调用实例的函数当然需要对应的实例Instance对象本身。而Python中就已经规定，函数的第一个参数必须是实例对象本身，并且建议，把该参数的名字写为self，现已是约定俗成的写法。所以我们需要self，需要用到self。

而如果没有用到self，即代码中去掉self后，那种写法所使用的变量实际上不是你所希望的，即不是真正的实例中的变量和函数，而是去访问到了其他部分的变量和函数了。甚至会出现由于没有合适的初始化实例变量而导致后续无法访问的错误。下面就通过代码来演示，如果去掉self或者没有合理的使用self会出现哪些错误。

1）如果没有在__init__中初始化对应的实例变量的话，将导致后续引用实例变量出错。

下面用代码去说明，如果没有在类Class的最初的__init__函数中正确地初始化实例变量则会导致后续没有变量可用，因而出现AttributeError的错误：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-
"""
--------------------------------------------------------------------------------
Function:
 【整理】Python中：self和init__的含义 + 为何要有self和__init__
http://www.crifan.com/summary_the_meaning_of_self_and___init___in_python_and_why_need_them
Author:     Crifan
Verison:    2012-11-27
--------------------------------------------------------------------------------
"""
#注：此处全局的变量名，写成name只是为了演示而用
#实际上，好的编程风格应该写成gName之类的名字，以表示该变量是Global的变量
name = "whole global name";
class Person:
    def __init__(self, newPersionName):
        #self.name = newPersionName;
```

```
        #1.如果此处不写成self.name
        #那么此处的name，只是__init__函数中的局部临时变量name而已
        #和全局中的name无关
        name = newPersionName;
        #此处只是为了代码演示而使用了局部变量name
        #不过需要注意的是，此处很明显，由于接下来的代码也没有利用到此处的局部变量name
        #于是就导致了此处的name变量实际上被浪费了，根本没有利用到
    def sayYourName(self):
        #此处由于找不到实例中的name变量，所以会报错：
        #AttributeError: Person instance has no attribute 'name'
        print 'My name is %s'%(self.name);
def selfAndInitDemo():
    persionInstance = Person("crifan");
    persionInstance.sayYourName();
################################################################################
if __name__=="__main__":
    selfAndInitDemo();
```

从上述代码可见，由于在类的初始化（实例化）的__init__函数中没有给self.name设置值而使实例中根本没有name这个变量，导致后续再去访问self.name就会出现AttributeError的错误。对应的如果写成self.name，意思就正确了，就是初始化时给实例中新增加并且正常设置了正确的值newPersionName了，所以后续再去通过self.name就可以访问到当前实例中正确的变量name了。相应的正确写法的代码如下：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-
"""
--------------------------------------------------------------------------------
Function:
 【整理】Python中：self和init__的含义 + 为何要有self和__init__
http://www.crifan.com/summary_the_meaning_of_self_and___init___in_python_and_why_need_them
Author:     Crifan
Verison:    2012-11-27
--------------------------------------------------------------------------------
"""
#注：此处全局的变量名写成name只是为了演示而用
#实际上，好的编程风格应该写成gName之类的名字，以表示该变量是Global的变量
name = "whole global name";
class Person:
    def __init__(self, newPersionName):
        #此处正确的，通过访问self.name的形式，实现了：
        #1.给实例中，增加了name变量
        #2.并且给name赋了初值，为newPersionName
        self.name = newPersionName;
    def sayYourName(self):
        #此处由于开始正确地初始化了self对象，使得其中有了name变量，所以此处可以正确访问name
值了，可以正确地输出了：
        #My name is crifan
        print 'My name is %s'%(self.name);
def selfAndInitDemo():
    persionInstance = Person("crifan");
    persionInstance.sayYourName();
################################################################################
if __name__=="__main__":
    selfAndInitDemo();
```

2）虽然代码中的函数中使用对应的变量是可以运行的，但是实际上使用的却不是实例中的变量。

有时候，虽然你写的代码可以运行，但是代码中使用到的变量，由于没有加self，实际上用到的不是实例的变量而是其他的变量。此类问题主要和Python中的变量的作用域有关，但是此处例子中也和是否使用self有关：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-
"""
--------------------------------------------------------------------------------
Function:
 【整理】Python中：self和init__的含义 + 为何要有self和__init__
http://www.crifan.com/summary_the_meaning_of_self_and___init___in_python_and_why_need_them
Author:     Crifan
Verison:    2012-11-27
--------------------------------------------------------------------------------
"""
#注：此处全局的变量名写成name只是为了演示而用
#实际上，好的编程风格应该写成gName之类的名字，以表示该变量是Global的变量
name = "whole global name";
class Person:
    name = "class global name"
```

```
    def __init__(self, newPersionName):
        #self.name = newPersionName;
        #此处，没有使用self.name
        #而使得此处的name，实际上仍是局部变量name
        #虽然此处赋值了，但是后面没有被利用到，属于被浪费了的局部变量name
        name = newPersionName;
    def sayYourName(self):
        #此处，之所以没有像之前一样出现：
        #AttributeError: Person instance has no attribute 'name'
        #那是因为，虽然当前的实例self中，没有在__init__中初始化对应的name变量，实例self中没有对应的name变量
        #但是由于实例所对应的类Person，有对应的name变量,所以也是可以正常执行代码的
        #对应的，此处的self.name，实际上是Person.name
        print 'My name is %s'%(self.name); # -> class global name
        print 'name within class Person is actually the global
name: %s'%(name); #-> whole global name
        print "only access Person's name via
Person.name=%s"%(Person.name); # -> class global name
def selfAndInitDemo():
    persionInstance = Person("crifan");
    persionInstance.sayYourName();
    print "whole global name is %s"%(name); # -> whole global name
################################################################################
if __name__=="__main__":
    selfAndInitDemo();
```

从如上代码中可见，此处开始__init__中没有给self实例初始化对应的name，而后面的函数sayYourName中虽然可以调用到self.name，也没有出现AttributeError错误。但是实际上此处的值不是所期望的，不是传入的name，即crifan，而是类中的name的值，即class global name。

再来解释解释__init__的含义。首先来看init单词本身，其含义是初始化。而__init__的含义也如同其单词本意一样用来初始化，但是给谁初始化呢？很明显是给对应的类Class本身初始化。而关于Python中为何要有__init__，其主要有两方面的作用。

（1）支持带参数的类的初始化

这个用法，比较类似于其他语言中的对于Class初始化时，可以在运行时传递不同的参数，对应着多个构造函数。用代码演示如下：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-
"""
--------------------------------------------------------------------------------
Function:
 【整理】Python中：self和init__的含义 + 为何要有self和__init__
http://www.crifan.com/summary_the_meaning_of_self_and___init___in_python_and_why_need_them
Author:     Crifan
Verison:    2012-11-27
--------------------------------------------------------------------------------
"""
class Person:
    def __init__(self, newPersionName):
        #在开始初始化新的类Class的示例Instance时，给对应的、不同的Instance设置不同的人名
（Person name）
        self.name = newPersionName;
    def sayYourName(self):
        #不同的Person的示例，调用同样的方法时，说出自己的名字，结果都是对应着自己的各自不同的
名字
        print 'My name is %s'%(self.name); #My name is crifan
def initDemo():
    persionInstance = Person("crifan");
    persionInstance.sayYourName();
################################################################################
if __name__=="__main__":
    initDemo();
```

其中，就是针对Person这个类，不同的示例在初始化时都传递一个对应的参数，这样不同的Person就都有了自己的不同的名字了。

（2）实现类本身相关内容的初始化

当一个Class稍微复杂一点的时候，或者内部函数需要用得到的时候，往往都需要在别人实例化、使用你这个类之前做一些基本的与自己的类有关的初始化方面的工作。而这部分工作往往就放到__init__函数中去了。换句话说，你要用人家的类中的变量和函

数之前，总要给人家一个机会做点准备工作，然后才能为你服务吧。这个概念相对还是很好理解的，就不多赘述。因为上面的代码也可以当做这方面的示例代码，因此不再贴出其他示例代码了。而不同的人应该有不同的名字，而给不同的人设置不同的名字，也是需要在调用sayYourName之前就先初始化好的。

4.4.2 初学者不要从最开始就太关注面向对象

其实，关于Python中的面向对象这部分内容，如果你和笔者的情况类似，只是用Python去实现一些自己需要的功能，则不太建议从最开始学习Python时就太关注面向对象这方面的内容。因为很可能你在使用Python去实现功能的时候，可以不用面向对象这方面的东西，就可以达到你的目的。而等你对于Python有了比较深入的了解之后，加上本身具有一定的面向对象方面的基本知识，比如知道面向对象中各种关系，然后再去学习Python中关于面向对象方面的语法。也只有那样，才能真正慢慢地熟悉和掌握Python面向对象方面的知识。而作为Python初学者，在最开始时把更多精力放在了解Python的基本语法和常见的库如何使用上，才更有利用学习和使用Python去解决实际问题。

4.5 Python中的变量

4.5.1 基本变量的声明和定义

关于Python中的变量的定义、初始化、使用等方面的基本知识，通过和其他语言，尤其是典型的C语言去对比，可以更好地说明问题。下面先给出一些典型的变量，在C语言中的定义、初始化和使用的例子。

C测试代码为：

```
#include "stdio.h"
int main(void){
    int demoInt;
    float demoFloat;
    char* demoStr;
    demoInt = 123;
    demoFloat = 12.34;
    demoStr = "hello crifan";
    printf("Demo variables in C:\n");
    printf("demoInt=%d, demoFloat=%f, demoStr=%s\n", demoInt, demoFloat, demoStr);
    return 0;
}
```

上述代码运行效果如图4-8所示。

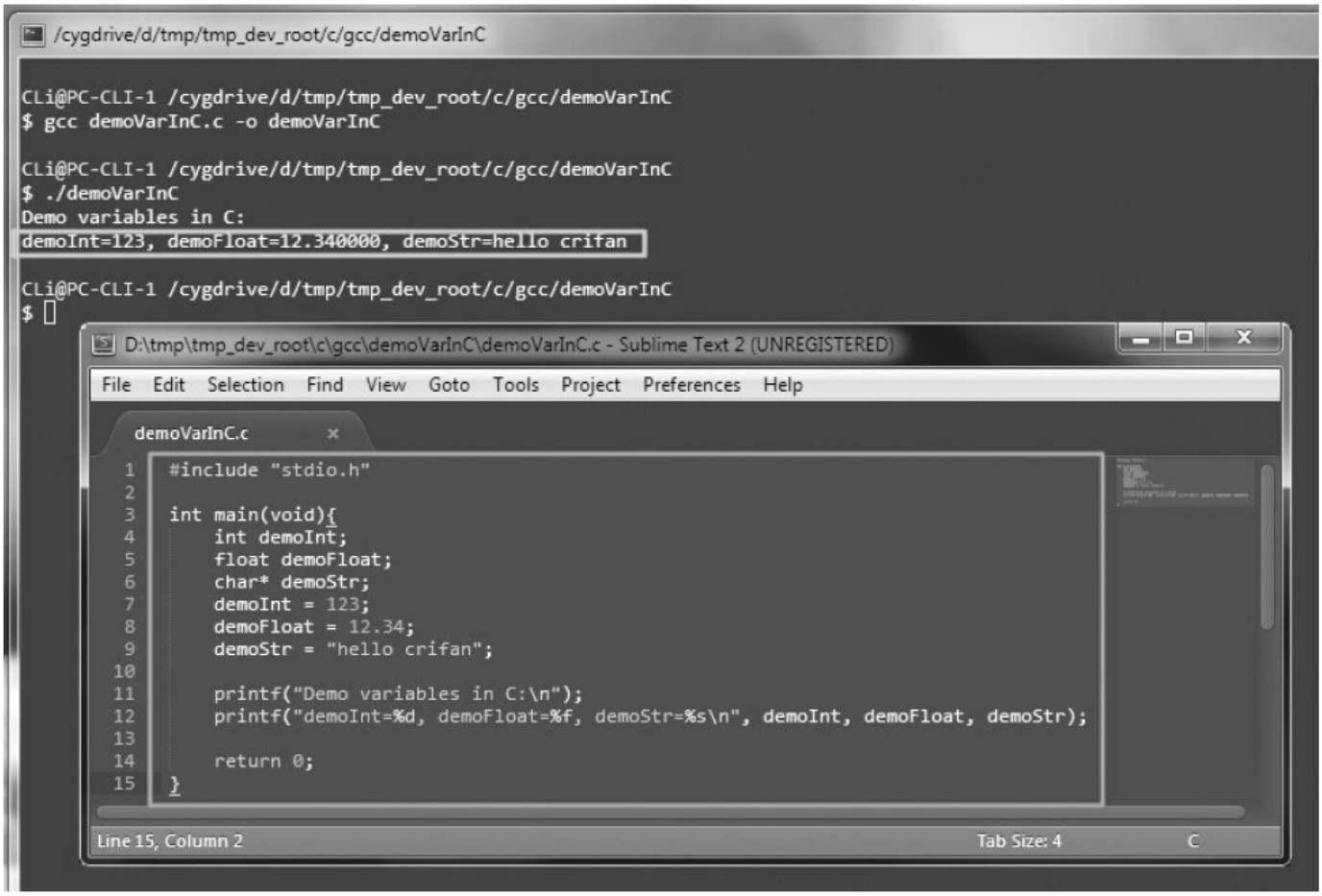


图4-8　测试变量的C代码在Cygwin中的运行效果

从图4-8中可以看到，其中变量如下。

（1）先定义变量

比如demoInt，先是定义为int类型。

（2）再给变量赋值

再去给demoInt赋值为123。

（3）再使用变量

然后在printf函数中调用变量demoInt，打印输出变量demoInt。

其中当然也可以把变量定义和赋值，即初始化，放在一起。

然后再对比给出Python中的写法：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-
"""
Function:
演示在Python中的变量的定义和使用
Author:     Crifan Li
Version:    2013-12-31
"""
import platform;
demoInt = 123;
demoFloat = 12.34;
demoStr = "hello crifan";
print "Demo variables in Python"
print "demoInt=%d, demoFloat=%f, demoStr=%s"%(demoInt, demoFloat, demoStr)
```

对应的运行效果如图4-9所示。

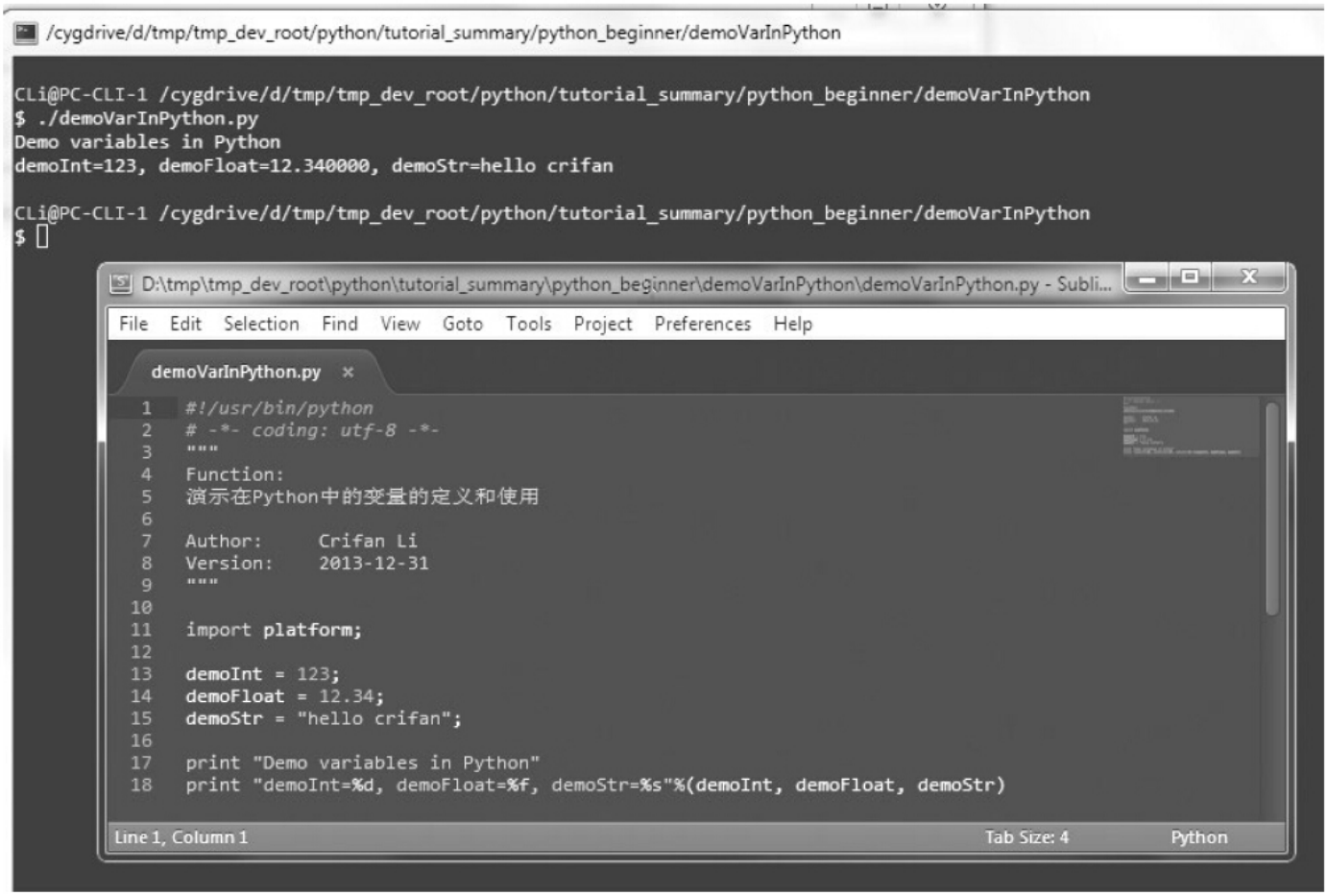


图4-9　测试变量的Python代码在Cygwin中的运行效果

其中可见变量都是：

（1）直接定义加上赋值

比如demoInt，无需事先定义，直接可以拿来赋值为123。

（2）再使用变量

然后在print函数中调用变量demoInt，打印输出变量demoInt。

所以结论是：Python中的变量，无需定义而可以直接使用。

4.5　Python中的变量

4.5.1　基本变量的声明和定义

关于Python中的变量的定义、初始化、使用等方面的基本知识，通过和其他语言，尤其是典型的C语言去对比，可以更好地说明问题。下面先给出一些典型的变量，在C语言中的定义、初始化和使用的例子。

C测试代码为：

```
#include "stdio.h"
int main(void){
    int demoInt;
    float demoFloat;
    char* demoStr;
    demoInt = 123;
    demoFloat = 12.34;
    demoStr = "hello crifan";
    printf("Demo variables in C:\n");
    printf("demoInt=%d, demoFloat=%f, demoStr=%s\n", demoInt, demoFloat, demoStr);
    return 0;
}
```

上述代码运行效果如图4-8所示。

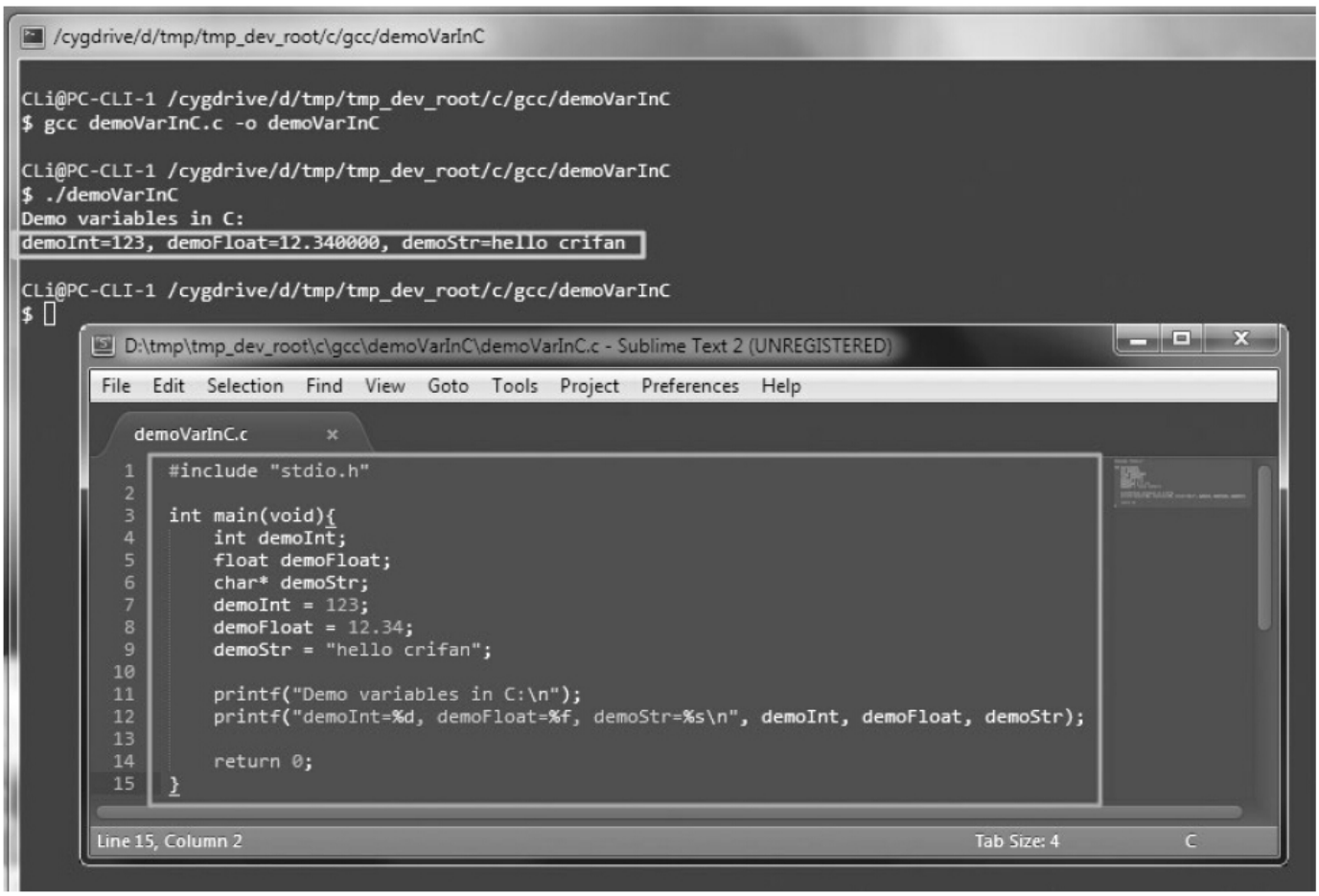


图4-8　测试变量的C代码在Cygwin中的运行效果

从图4-8中可以看到，其中变量如下。

（1）先定义变量

比如demoInt，先是定义为int类型。

（2）再给变量赋值

再去给demoInt赋值为123。

（3）再使用变量

然后在printf函数中调用变量demoInt，打印输出变量demoInt。

其中当然也可以把变量定义和赋值，即初始化，放在一起。

然后再对比给出Python中的写法：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-
"""
Function:
演示在Python中的变量的定义和使用
Author:     Crifan Li
Version:    2013-12-31
"""
import platform;
demoInt = 123;
demoFloat = 12.34;
demoStr = "hello crifan";
print "Demo variables in Python"
print "demoInt=%d, demoFloat=%f, demoStr=%s"%(demoInt, demoFloat, demoStr)
```

对应的运行效果如图4-9所示。

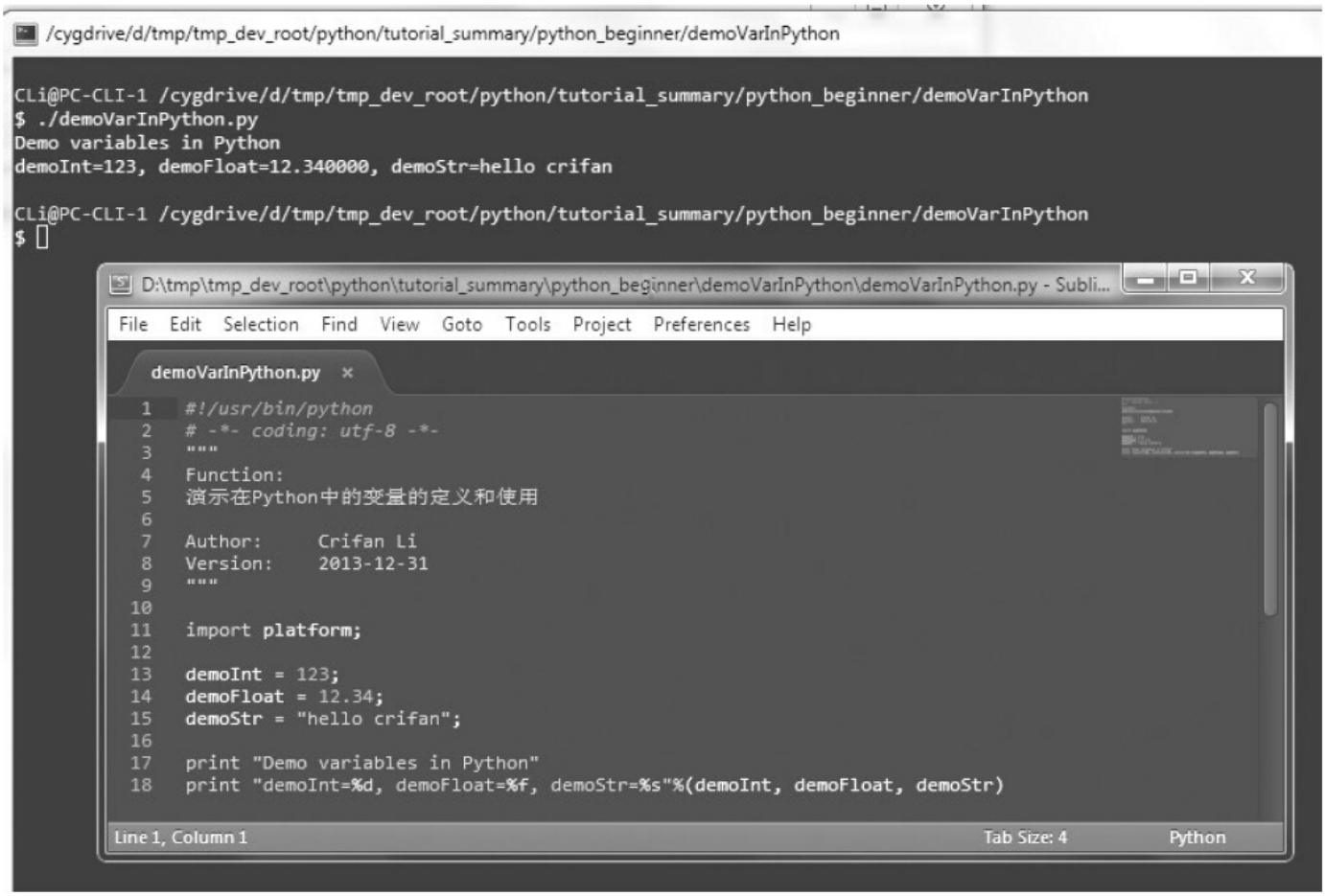


图4-9　测试变量的Python代码在Cygwin中的运行效果

其中可见变量都是：

（1）直接定义加上赋值

比如demoInt，无需事先定义，直接可以拿来赋值为123。

（2）再使用变量

然后在print函数中调用变量demoInt，打印输出变量demoInt。

所以结论是：Python中的变量，无需定义而可以直接使用。

4.5.2 变量的作用域

变量的作用域的英文名叫作variable scope。下面通过代码来解释Python中变量的作用域：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-
"""
---------------------------------------------------------------------------
Function:
 【整理】Python中：self和init__的含义 + 为何要有self和__init__
http://www.crifan.com/summary_the_meaning_of_self_and___init___in_python_and_why_need_them
Author:     Crifan
Verison:    2012-11-27
---------------------------------------------------------------------------
"""
#注：此处全局的变量名写成name只是为了演示而用
#实际上，好的编程风格应该写成gName之类的名字，以表示该变量是Global的变量
name = "whole global name";
class Person:
    name = "class global name"
    def __init__(self, newPersionName):
        #self.name = newPersionName;
        #此处，没有使用self.name
        #而使得此处的name，实际上仍是局部变量name
        #虽然此处赋值了，但是后面没有被利用到，属于被浪费了的局部变量name
        name = newPersionName;
    def sayYourName(self):
        #此处，之所以没有像之前一样出现：
        #AttributeError: Person instance has no attribute 'name'
        #那是因为，虽然当前的实例self中，没有在__init__中初始化对应的name变量，实例self中没有对应的name变量
        #但是由于实例所对应的类Person有对应的name变量，所以也是可以正常执行代码的
        #对应的，此处的self.name实际上是Person.name
        print 'My name is %s'%(self.name); # -> class global name
        print 'name within class Person is actually the global name: %s'%(name); #-> whole global name
        print "only access Person's name via Person.name=%s"%(Person.name); # ->
class global name
def selfAndInitDemo():
    persionInstance = Person("crifan");
    persionInstance.sayYourName();
    print "whole global name is %s"%(name); # -> whole global name
################################################################################
if __name__=="__main__":
    selfAndInitDemo();
```

Python中变量的作用域如图4-10所示。

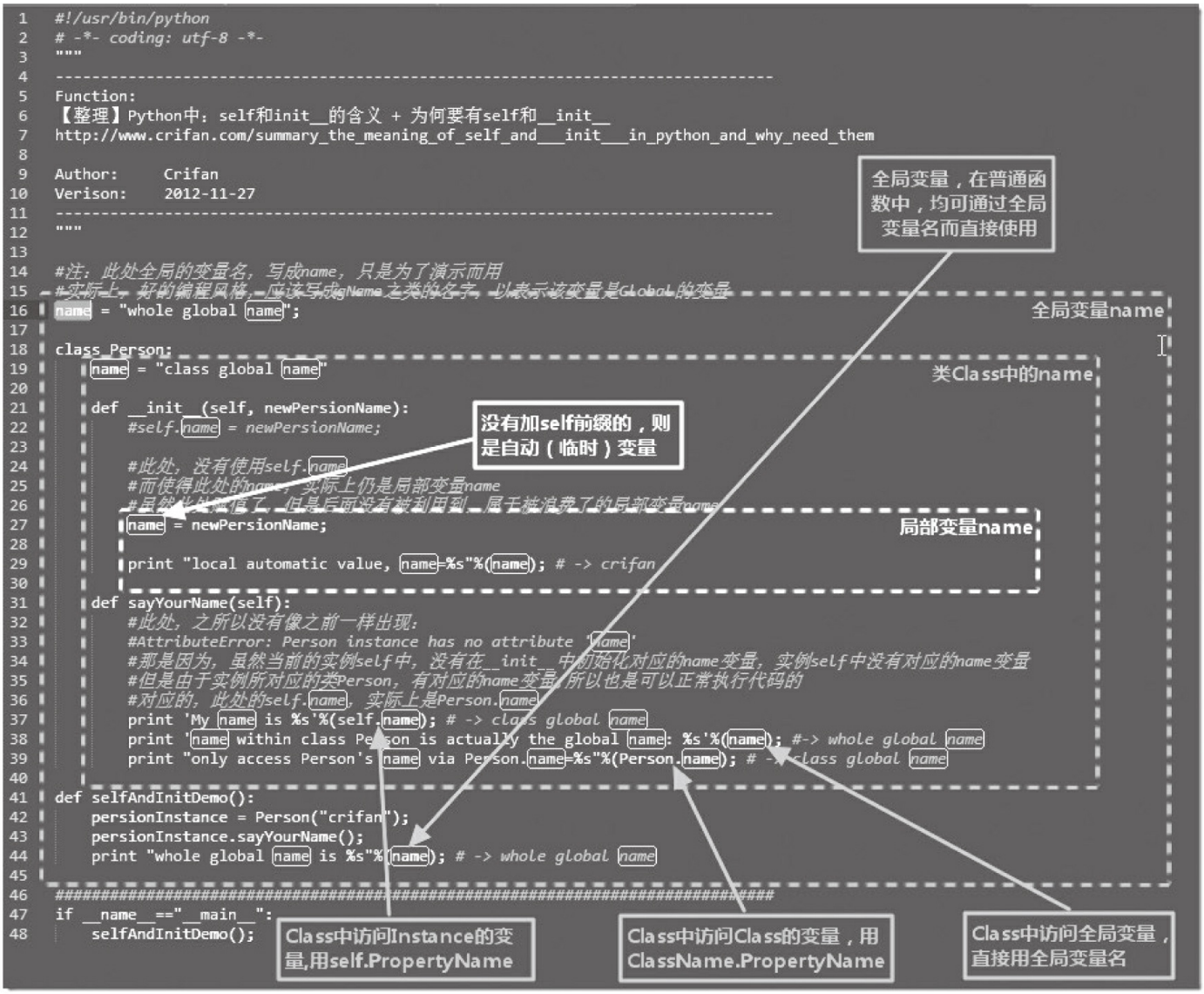


图4-10　Python中变量的作用域

接着再用文字详细解释其含义。

（1）全局变量

不论是其他普通函数中还是Class类中，全局变量（Global variable）都是可以通过对应的变量名而直接引用的。

（2）局部变量

对于函数内，包括类Class的函数内，普通的变量都是自动临时变量。

变量的优先级关系

普通局部变量（automatic variable）和全局变量的作用域最核心的区别在于：内部变量的优先级大于外部变量。此处就是（函数内的）局部变量的优先级大于（外部的）全局变量。换句话说：

· 如果函数内部出现了和全局变量同名的变量，则说明是局部变量。

· 如果函数内，没有同名的局部变量，则对应的变量是全局变量。

这个变量的优先级关系不仅针对与Python语言，对于几乎所有语言，如C、C++、C#等都适用。

所以，上面示例中的__init__中的name不是全局变量，而

```
name = "whole global name";
```

中的name是局部变量。其中，此处是属于那种Python和/或其他特殊语言所特有的，无需声明变量就可以直接使用的情况。即局部变量name并没有声明而直接拿过来使用。通过：

```
name = newPersionName;
```

在初始化时设置为对应的名字，此处即crifan。

（3）类的变量

类（Class）的变量在类的范围内只能通过ClassName.PropertyName或者说ClassName.VariableName去访问当前类ClassName的变量PropertyName。例子中就是通过Person.name去访问类Person中的变量name的值的。

（4）示例的变量

示例（Instance）中的变量理论上是用InstanceObject.PropertyName访问的。而此处由于Python中默认的约定俗成的写法，把类的函数的第一个参数的名字定义为Instance变量，且名字叫作self，所以就变成self.PropertyName了。所以上述在类的函数中想要访问Instance的变量的话，就是通过self.name去访问类Person的示例self中的变量name了。

如此，只有真正地搞懂变量的作用域才能写出正确的含义的代码，才不会由于变量作用域的错误理解而导致变量的值不正确。

4.6 Python中的分支结构

用计算机语言去写程序时常会遇到分支判断，Python也不例外。所以Python中也有对应的if/elif/else等结构。不过没有switch/case的结构，因为switch/case的结构也是可以通过if/else模拟出来的。下面写几个常见的if/else的写法，用于演示基本的语法。

1.只有if无else

语法为：

```
if condition:
    code-block
```

举例：

```
x = 123
if x > 0:
     print "x的值大于0"
```

2.有if也有else

语法为：

```
if if-condition:
    if-body
else:
    else-body
```

举例：

```
loveLanguage = "Python"
if (loveLanguage == "Python"):
    print "我喜欢的编程语言是Python"
else:
    print "我喜欢的编程语言不是Python"
```

3.除了有if和else还有elif

很明显elif是else if的简写。语法为：

```
if if-condition:
    if-code-block
elif condition-1:
    code-block-1
elif condition-2:
    code-block-2
elif condition-n:
    code-block-n
else:
    code-block-else
```

举例：

```
loveLanguage = "Python"
if (loveLanguage == "Python"):
    print "我喜欢的编程语言是Python"
elif(language == "C#"):
    print "我喜欢的编程语言是C#"
elif(language == "Java"):
    print "我喜欢的编程语言是Java"
else:
    print "我喜欢的编程语言是除了Python，C#，Java之外的某种语言"
```

4.7 Python中的函数

写程序要有良好的习惯，其中一个习惯就是当代码量相对增多，某些代码属于一个功能集合的时候，就应该去将这部分代码单独组织到函数里面。下面介绍Python中的函数的语法和调用。

Python中的函数的语法为：

```
def functionname( parameters ):
   "function_docstring"
   function_suite
   return [expression]
```

举例：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-
"""
Function:
演示在Python中的函数
Author:     Crifan Li
```

```
Version:    2013-12-31
"""
#函数的定义
def printInputStr(inputStr):
    "此函数只是为了演示，打印输入的字符串"
    print inputStr
    return
#函数的调用
printInputStr("第一次调用函数。")
printInputStr("再次调用函数")
```

运行效果如图4-11所示。

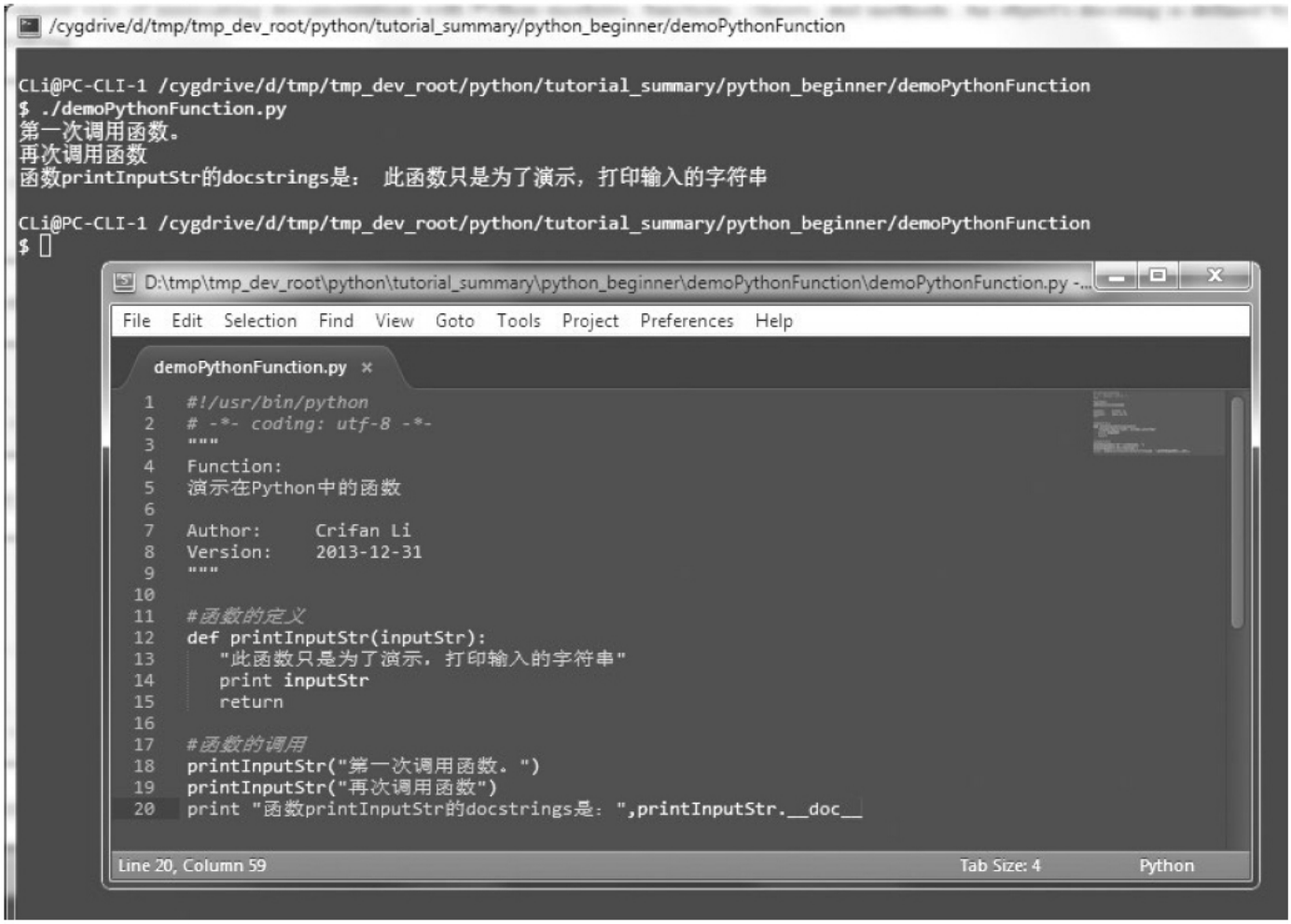


图4-11　演示Python函数和Docstrings并在Cygwin中运行

如此就是基本的函数的写法了。其中的Docstrings中文可以理解为Python代码中的函数解释说明的字符串，用于生成相关文档时使用。对于Docstrings的功能，除了上述的函数作为对象之外，其内置就拥有__doc__属性，其值就是上述的docstrings。另外，docstrings典型的用法是用其他工具，比如Doxygen从Python源代码中生成对应的代码的文档。其中就可以将对应的docstrigns提取出来，用于作为对应函数的描述内容，即从Python源代码中的docstrings生成源码所对应的文档的注释内容。

第5章　一些有趣的Python小实验

前面已经介绍了Python的基础知识，接下来去使用和延伸之前的基础知识，用Python实现一些实用和有趣的功能。

5.1 用Python查看系统平台信息

虽然Python属于上层语言，很少用于直接操作底层的硬件，但并不表示不可以用来和硬件搭配去实现自己的所需要的功能。比如之前就有人采用Python语言加上FPGA去实现定点平方根的运算，取代了传统的Verilog和VHDL语言进行硬件设计，从而很大程度地提高了后期复杂的软件算法的验证效率。不过这类相对复杂的和硬件相关的Python应用，毕竟用到的人不是很多。下面就来介绍一下比较简单的，适合有点硬件基础和电脑配置知识方面的Python初学者，去使用Python语言查看当前的系统和配置信息。

在windows 7中运行如下代码：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-
"""
Function:
Check current platform and hardware info
Author:     Crifan Li
Version:    2013-02-13
Contact:    http://www.crifan.com/about/me/
"""
import platform
def checkPlatformInfo():
    uname = platform.uname()
    print "uname=",uname
    arch = platform.architecture()
    print "arch=",arch
    machine = platform.machine()
    print "machine=",machine
    node = platform.node()
    print "node=",node
    platformInfo = platform.platform()
    print "platformInfo=",platformInfo
    processor = platform.processor()
    print "processor=",processor
    system = platform.system()
    print "system=",system
    version = platform.version()
    print "version=",version
if __name__ == "__main__":
    checkPlatformInfo();
```

此时即可输出相关的系统和硬件信息，如图5-1所示。

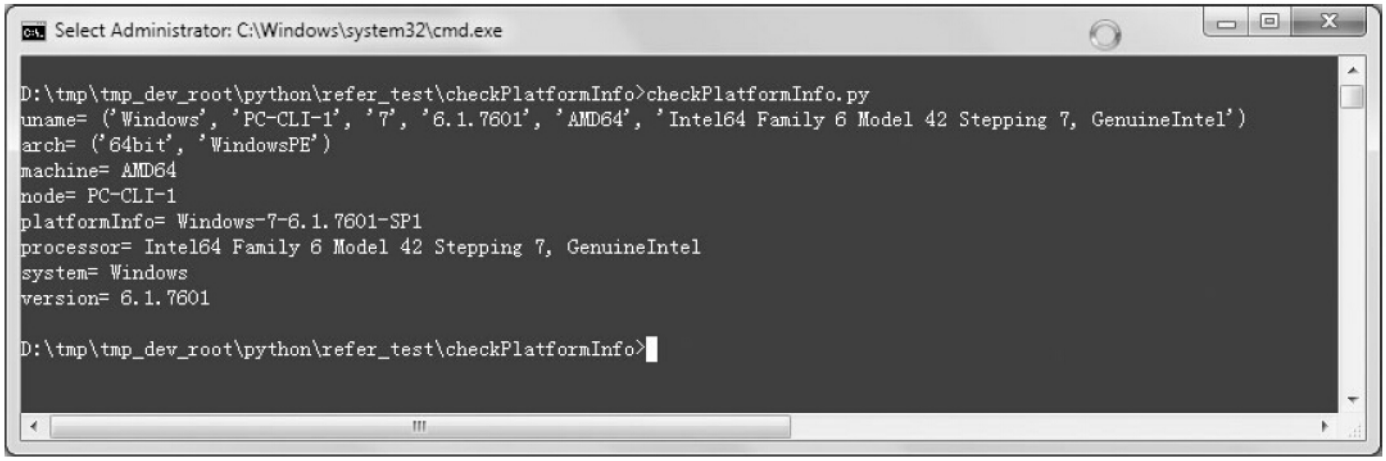


图5-1 Python检测出平台和硬件信息

将上述信息转化为我们常听到的对应电脑系统信息的描述语言就是：这台电脑的CPU是64位的Intel，操作系统是带SP1的内部版本号为6.1.7601的Windows 7。

如此就实现了通过Python代码去查看你想要获得的当前电脑的具体的软硬件配置信息了。

5.2 Python处理谐波和信号变换

Python可以被用来进行各种方面的应用，其中包括用于处理数字信号等方面的应用。接下来将演示使用Python生成各种所需要的波形。我们都知道正常的正弦波形如图5-2所示。

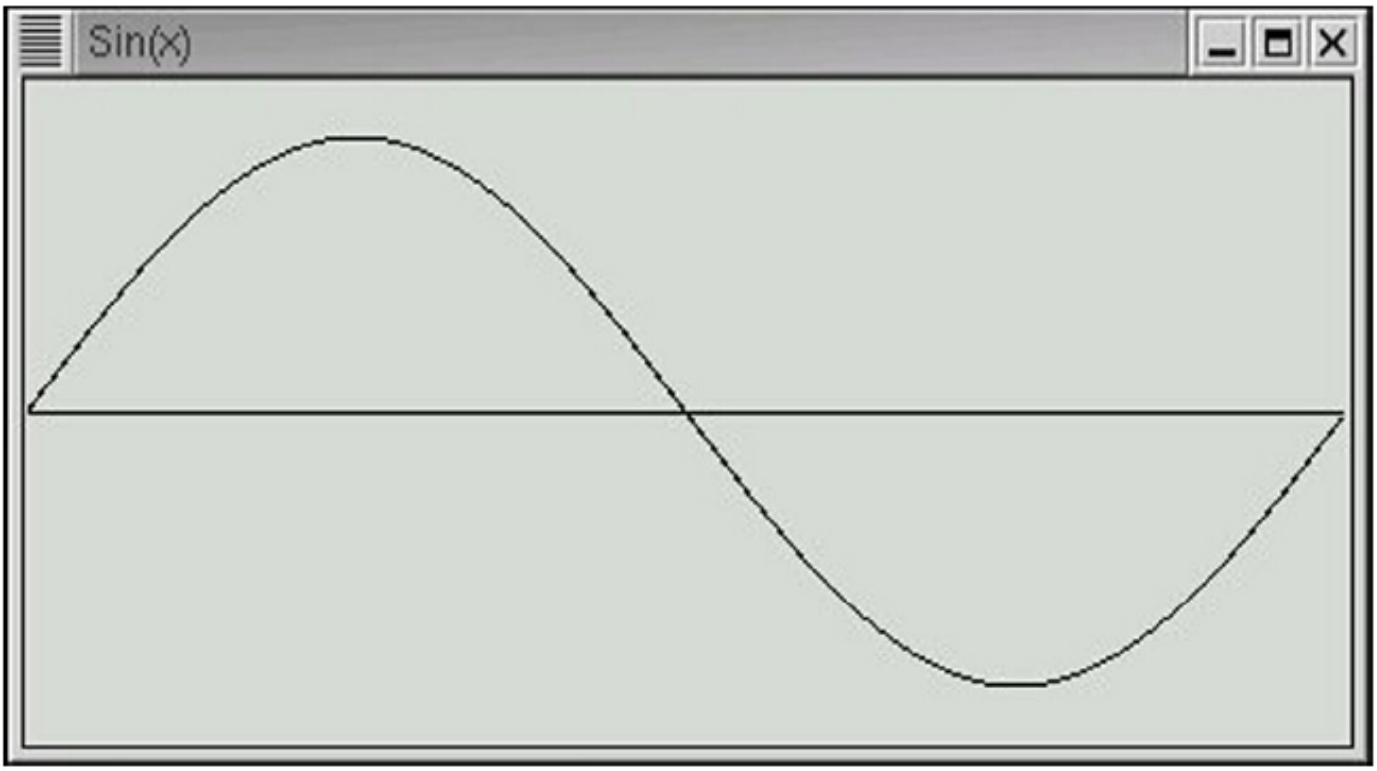


图5-2 Python显示出正弦波形

接着去写一个wave波形的Python库文件wave.py，代码如下：

```
#   w a v e . p y
#
import sys
from Tkinter import *
from math    import *
class wave :
def __init__ (self, points=400, formula=None) :
        self.data = [0.0]*points
        self.points= points
        if formula :
                for p in range(points) :
                        x = p*pi*2/points
                        self.data[p] = eval(formula)
def __add__ (self, other) :
       target = wave(points=self.points)
       for i in range(self.points) :
                target.data[i] = self.data[i] + other.data[i]
       return target
def __mul__ (self, other) :
       target = wave(points=self.points)
       if type(other) == type(5) or type(other) == type(5.0) :
                for i in range(self.points) :
                        target.data[i] = self.data[i] * other
       else :
                for i in range(self.points) :
                        target.data[i] = self.data[i] * other.data[i]
       return target
def __sub__ (self, other) :
       target = wave(points=self.points)
       for i in range(self.points) :
                target.data[i] = self.data[i] - other.data[i]
       return target
```

```
    def integral(self) :
           ans = 0.0
           for pt in self.data : ans = ans+pt
           return ans*2*pi/self.points
    def plot (self, title="", pixHeight=None, maxY=None, others=[]) :
           if not pixHeight : pixHeight = self.points*2/3   # Pleasant ratio
           pixWidth = self.points
           # find max and min data to scale
           if not maxY :
                     maxY = max (max(self.data), -min(self.data))
           offset = pixHeight/2
           scale = offset/maxY
           win = Tk()
           win.title (title)
           canvas = Canvas(win,width=pixWidth,height=pixHeight)
           # create zero line
           canvas.create_line(0,offset,pixWidth,offset)
           canvas.pack()
           self.plotOne (canvas, pixWidth, scale, offset)
           for i in range(len(others)) :
                     others[i].plotOne (canvas, pixWidth, scale, offset)
           if sys.platform == "win32" : win.mainloop()
    def plotOne (self, canvas, pixWidth, scale, offset) :
           for x in range(pixWidth) :
                     y = offset - self.data[x] * scale
                     if x : canvas.create_line(x-1,yprev,x,y)
                     yprev = y
    def fft (self) :
           work = self * 1   # Harmonics will be stripped from this
           for harm in range(1,10) :
                     formula="-sin(%d*x)" % harm
                     area = (wave(formula=formula)*work).integral()
                     amplitude = area/-pi
                     if amplitude > .000001 :
                                print "Harmonic=",harm, " Amplitude=%.04f" % amplitude
                      takeAway = wave(formula="sin(%d*x) * %f" % (harm,amplitude))
                      work = work-takeAway
    def test() :
    p1 = wave(formula="sin(x)/1")
    p3 = wave(formula="sin(3*x)/3")
    p5 = wave(formula="sin(5*x)/5")
    mys = p1+p3+p5
    mys.fft()
    if __name__ == "__main__" : test()
```

然后用测试代码去测试生成不同的波形。

（1）复杂的谐波

可以利用上述wave再加上测试代码：

```
    import wave
    a = wave.wave(formula="sin(x)")        # fundamental
    b = wave.wave(formula=".5*sin(2*x)")   # harmonic
    a.plot(maxY=1.2, pixHeight=200, title="Sin(x) and .5sin(2*x)", others=[b])
```

即可生成对应的谐波了，如图5-3所示。

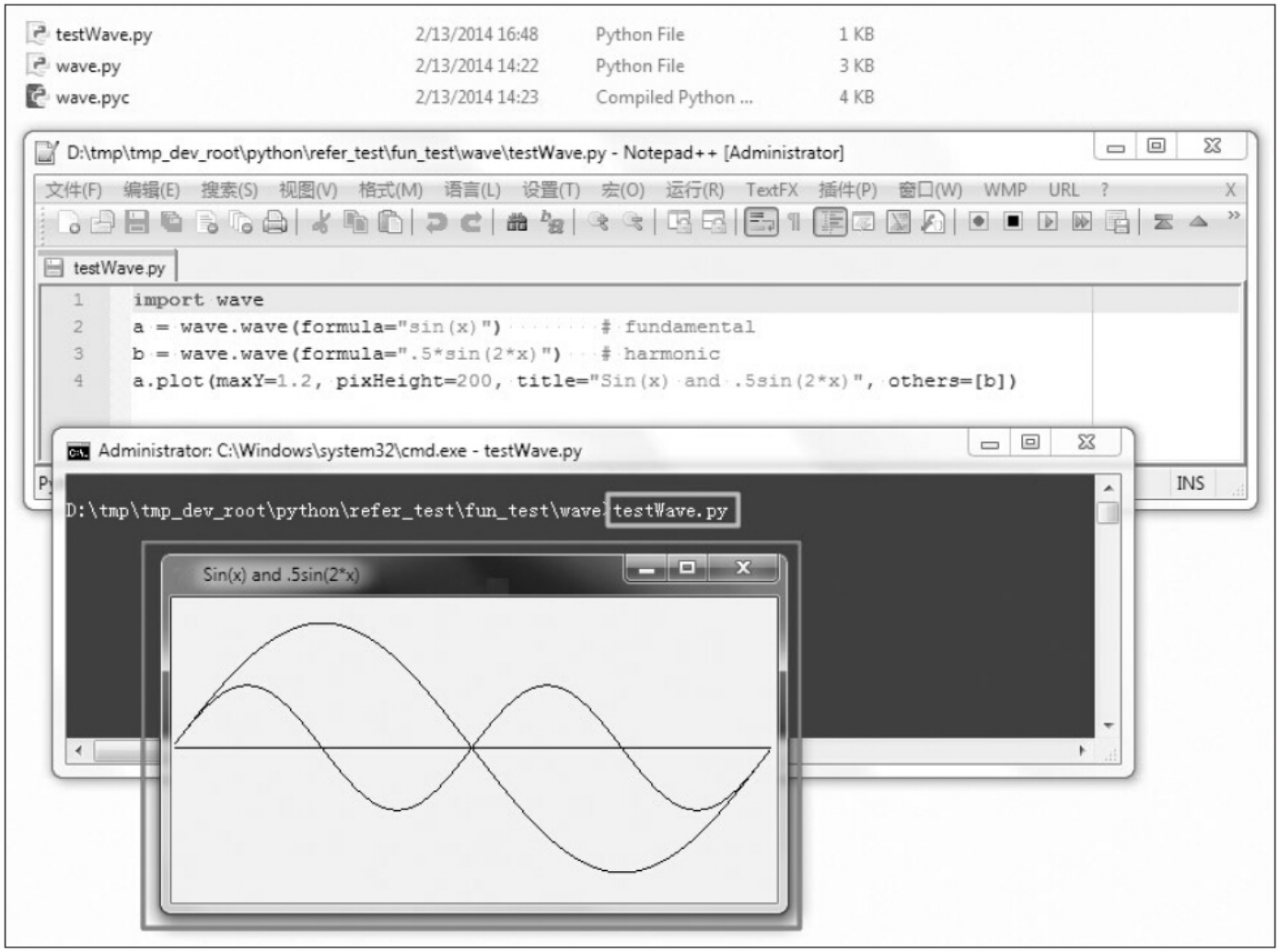


图5-3　Python显示出复杂的谐波

（2）更复杂点的波形

再用如下代码：

```
import wave
a = wave.wave(formula="sin(x)")         # fundamental
b = wave.wave(formula=".5*sin(2*x)")    # harmonic
c = a + b
c.plot(maxY=1.5, pixHeight=200, title="Sin(x) + .5sin(2*x)")
```

去生成更加复杂一点的波形，如图5-4所示。

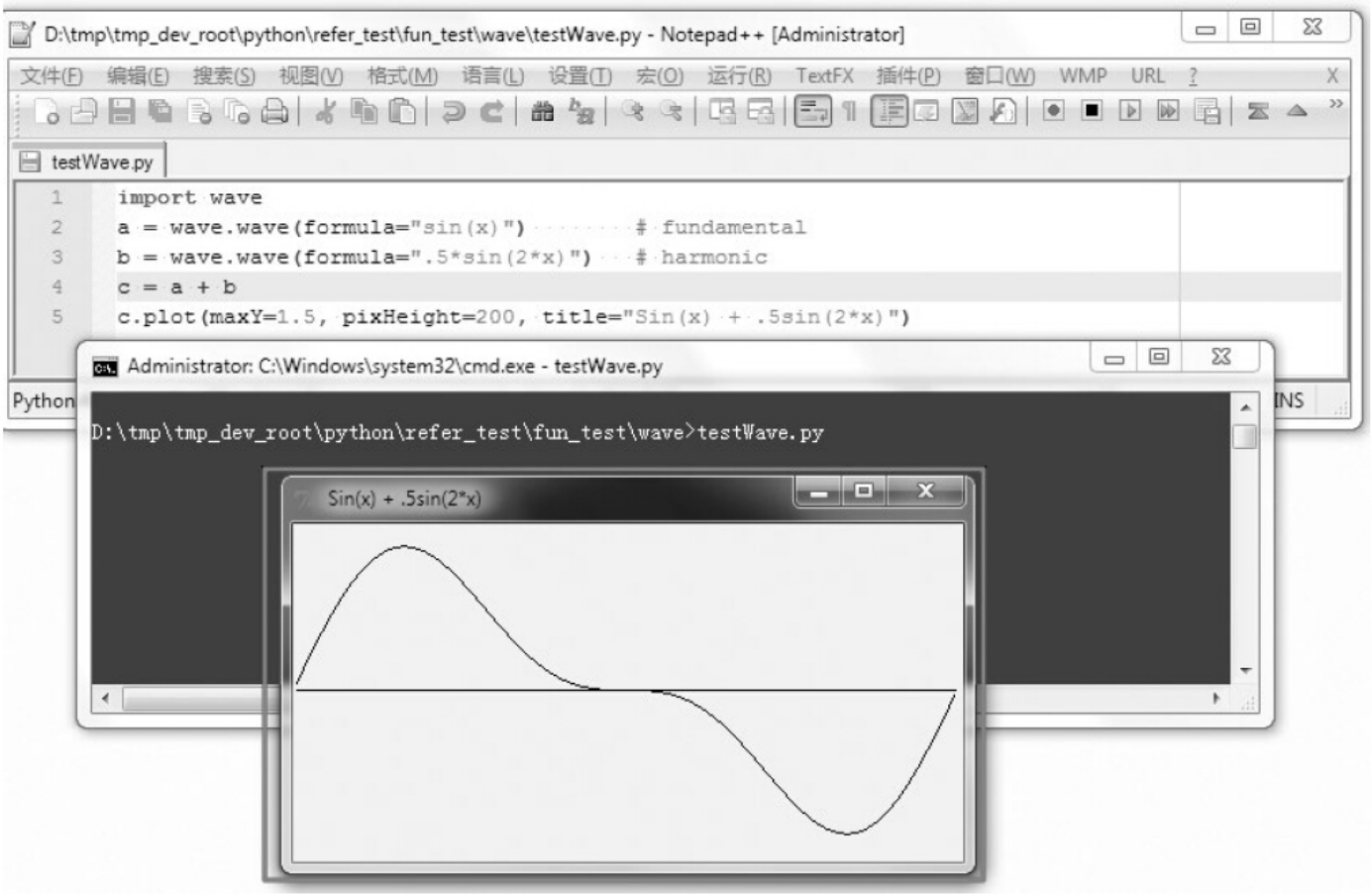


图5-4　Python显示出更加复杂的谐波

所以结论很明显：如果你具有了相关领域内的知识，即可利用丰富的Python库去实现你的想法，验证你的思路是否正确了。

5.3　更多有用且有趣的Python语法

除了上述介绍的一些有趣的Python应用方面的例子之外，下面再来介绍一下Python本身所支持的一些有用且有趣的语法和功能。

5.3.1　Python中交换不同的变量值

对于其他语言来说，交换两个变量的值的做法和思路一般是：

```
temp = a
a = b
b = temp
```

而在Python中可以用更加精简和明晰的方式去实现此变量交换：

```
b, a = a, b
```

运行效果如下：

```
>>> a = 1
>>> b = 2
>>> b, a = a, b
>>> a
2
>>> b
1
>>>
```

此处的语法叫作多变量赋值。并且也支持更多个变量同时赋值，效果如下：

```
>>> a = 1
>>> b = 2
>>> c = 3
>>> b, c, a = a, b, c
>>> a
3
>>> b
1
>>> c
2
>>>
```

由此可见，Python中这类精简高效的语法对于实现同样的功能来说，可以在很大程度上提高工作效率。

5.3.2 Python中集合类的变量的切片

Python中对于集合类型的变量支持通过索引取得其中的内容叫作slice——切片。下面通过字符串的例子，来解释这个切片的用法和效果。

字符串的索引编号可以用图5-5所示的示例来解释。

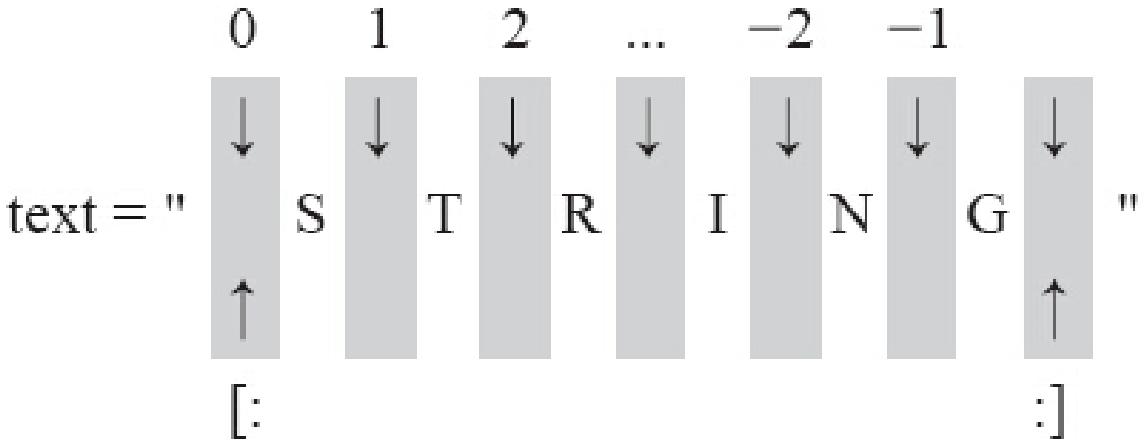


图5-5 Python中字符串的切片示例

而通过不同的索引去获得的值，可参考表5-1。

表5-1 Python中字符串的不同索引获得不同的切片值

普通写法	等价于	输出结果
text[1:4]		"TRI"
text[:5]	text[0:5]	"STRIN"
text[:-1]	text[0:-1]，text[0:len(text)-1]	"STRIN"
text[-4:]	text[-4:len(text)]	"RING"
text[2]	text[2:3]	"R"
text[:]	text[0:len(text)]	"STRING"
text[::-1]		"GNIRTS"
text[::2]		"SRN"

其中text[：：-1]中的-1表示步进长度，-1表示每次反向从最后向最前检索获得对应的字符，所以总体效果就是取反字符串中的字符。而对应的text[：：2]就很明显是每次移动2个字符，依次取得对应的位置的字符，所以结果是索引号是0、2、4的S、R、N了。

与此类似的，Python中的list类型的变量也是可以通过编号索引获得对应的内容的：

```
>>> listVar = [1,2,3,4,5]
>>> listVar[::-1]
[5, 4, 3, 2, 1]
>>> listVar[::2]
[1, 3, 5]
```

Python中的这类甚至支持负数作为检索标号即可取得对应内容，不用像其他很多编程语言中那样需要写很多的辅助函数获得子字符串或子列表的方式，极大地提高了程序员编码的效率。

5.3.3 Python中的for循环和枚举器

对于已有的一个列表来说：

```
>>> items = 'zero one two three'.split()
>>> print items
['zero', 'one', 'two', 'three']
```

其他语言中实现循环枚举的功能：

```
index = 0
for item in items:
    print index, item
    index += 1
```

而Python中的写法更加精简和高效：

```
for i in range(len(items)):
        print i, items[i]
```

并且还有更加精简的写法：

```
>>> items = 'zero one two three'.split()
>>> for (index, item) in enumerate(items):
print index, item
0 zero
```

```
1 one
2 two
3 three
```

其中enumerate可以直接生成（index，item）类型的tuple变量。

如此，实现同样的循环时，Python的代码比其他语言显得更加简洁明了。

5.3.4 Python中的条件性赋值

Python中对于多个条件去判断并赋值，其写法可以非常精简：

```
x = 3 if (y == 1) else 2
```

复杂的写法有：

```
x = 3 if (y == 1) else 2 if (y == -1) else 1
```

效果如下：

```
>>> y = 0
>>> x = 3 if (y == 1) else 2
>>> x
2
>>> x = 3 if (y == 1) else 2 if (y == -1) else 1
>>> x
1
```

如此，Python的语法的设计方面的优势就更加明显了，语法精简且更加符合人们的思维。

第6章 常见Python应用实例

前面介绍了Python可以做什么事情，并且举了些例子说明Python的强大之处，但是关于具体如何在实际中应用Python代码实现各种应用却没有提及。所以此处就针对于常见的一些Python的应用领域，给出一些简单的应用案例供读者学习和参考。

6.1 Python在网络方面的应用

1.网页抓取和内容提取

Python在网络方面的应用很多，其中比较常见的一类应用就是使用Python去实现网页抓取。网页抓取方面的应用，俗称为爬虫。下面就通过一个简单的例子来说明如何用Python代码去实现爬虫，并实现网页内容的解析，从中提取需要的内容。

假设我们此处的需求是抓取歌曲网站Songtaste中的主页http://www.songtaste.com/user/351979/，并提取其中笔者的网名crifan，如图6-1所示。

接下来就是去解释如何用Python代码实现这个需求。此处使用Python抓取网页的内容的代码相对不复杂，所以可以直接写出对应的获得网页的html源码的代码：

```
#--------------------------------import--------------------------------------
import urllib2;
#----------------------------------------------------------------------------
def main():
    userMainUrl = "http://www.songtaste.com/user/351979/";
    req = urllib2.Request(userMainUrl); #http request
    resp = urllib2.urlopen(req); #get http response
    respHtml = resp.read(); # get response body html
    print "respHtml=",respHtml; # you should see the ouput html
##############################################################################
if __name__=="__main__":
    main();
```

如此就可以通过此Python脚本去获得该url对应的html源码了。此时你会发现，此处获得的HTML网页源码和你打开上述网页http://www.songtaste.com/user/351979/所对应的HTML源码应该是基本一样的。具有一定的HTML的基础读者此时可以看出，想要提取的那个英文单词crifan的相关部分HTML源码是：

图6-1　抓取信息

```
<div class="icon_col">
        <h1 class="h1user">crifan</h1>
http://www.hzcourse.com/resource/readBook?path=/openresources/teach_ebook/uncompressed/15066/OEBPS/Text/...
 </div>
```

从已有网页的HTML源码中提取对应的某个字符串内容，常见的方法有两种。

（1）用正则表达式从网页提取所需内容

在具有一定的正则表达式方面的知识的前提下，根据上述的相关部分的html就可以写出对应的正则表达式去提取所需内容了：

```
#-------------------------------import-------------------------------------
import urllib2;
import re;
#--------------------------------------------------------------------------
def main():
    userMainUrl = "http://www.songtaste.com/user/351979/";
    req = urllib2.Request(userMainUrl);
    resp = urllib2.urlopen(req);
    respHtml = resp.read();
    print "respHtml=",respHtml; # you should see the ouput html
    #<h1 class="h1user">crifan</h1>
    foundH1user = re.search('<h1\s+class="h1user">(P<h1user>.+)</h1>', respHtml);
    print "foundH1user=",foundH1user;
    if(foundH1user):
         h1user = foundH1user.group("h1user");
         print "h1user=",h1user;
##############################################################################
if __name__=="__main__":
    main();
```

如此就可以打印出我们所需要的字符串crifan了。

（2）用Python第三方专门解析网页的BeautifulSoup提取所需内容

另外一种办法就是使用专门用于解析HTML的库函数去提取所需内容。BeautifulSoup就是一种相对比较常用的，专门用于解析HTML和XML的Python的第三方的库。接下来详细介绍一下如何使用BeautifulSoup去实现从HTML网页中提取所需要的信息。

首先下载对应的某个版本的BeautifulSoup，此处推荐使用http://www.crummy.com/software/BeautifulSoup/bs3/download/3.x/BeautifulSoup-3.0.6.py，下载后再另存为BeautifulSoup.py，并放到和你的当前Python同目录的位置，然后使用如下脚本：

```
songtasteHtmlEncoding = "GB2312";
soup = BeautifulSoup(respHtml, fromEncoding=songtasteHtmlEncoding);
#<h1 class="h1user">crifan</h1>
foundClassH1user = soup.find(attrs={"class":"h1user"});
print "foundClassH1user=%s",foundClassH1user;
if(foundClassH1user):
     h1userStr = foundClassH1user.string;
     print "h1userStr=",h1userStr;
```

也可以提取出来对应所需的字符串：crifan。

上述两种办法都可以实现上述的目的。此处再将两种方案的代码，放在同一个Python文件中变成完整的示例代码供参考：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-
"""
--------------------------------------------------------------------------------
 【版本信息】
版本:       v1.0
作者:       crifan
 【详细信息】
用于:
 【教程】抓取网并网页中所需要的信息 之 Python版
http://www.crifan.com/crawl_website_html_and_extract_info_using_python/
的示例代码。
--------------------------------------------------------------------------------
"""
#-------------------------------import-------------------------------------
import urllib2;
import re;
from BeautifulSoup import BeautifulSoup;
#--------------------------------------------------------------------------
```

```
def main():
    userMainUrl = "http://www.songtaste.com/user/351979/";
    req = urllib2.Request(userMainUrl);
    resp = urllib2.urlopen(req);
    respHtml = resp.read();
    #print "respHtml=",respHtml; # you should see the ouput html
    print "Method 1: Use python re to extract info from html";
    #<h1 class="h1user">crifan</h1>
    foundH1user = re.search('<h1\s+class="h1user">(P<h1user>.+)</h1>', respHtml);
    print "foundH1user=",foundH1user;
    if(foundH1user):
        h1user = foundH1user.group("h1user");
        print "h1user=",h1user;
    print "Method 2: Use python third lib BeautifulSoup to extract info from html";
    songtasteHtmlEncoding = "GB2312";
    soup = BeautifulSoup(respHtml, fromEncoding=songtasteHtmlEncoding);
    #<h1 class="h1user">crifan</h1>
    foundClassH1user = soup.find(attrs={"class":"h1user"});
    print "foundClassH1user=%s",foundClassH1user;
    if(foundClassH1user):
        h1userStr = foundClassH1user.string;
        print "h1userStr=",h1userStr;
################################################################################
if __name__=="__main__":
    main();
```

2.与复杂的网络提取相关的基本知识

演示完简单的网页抓取和内容提取后，再来说说实际中更加复杂的网页抓取、模拟登录等方面所涉及的一些相对基本的知识。

（1）HTML的charset字符编码

如果读者之前看到HTML的源码的话，常常会看到其中开头部分有类似于这样的HTML代码：

```
<meta http-equiv="content-type" content="text/html;charset=gb2312">
```

其中的charset的值是gb2312，这个就是用于指定当前HTML网页的字符串的编码。这时候就需要读者对字符编码知识有所了解，要了解在处理网页内容时，需要用对应的字符编码去解析字符串，然后才能进行后续处理，比如提取某个需要的字符串之类操作，否则就会产生乱码等问题。

（2）网页的自动跳转

有些网址，假设为A地址，用浏览器打开后，你会发现最终加载并显示出来的地址是B地址，此时就是发生了自动跳转。即访问A地址，对应的网页服务器的内部根据相应规则而跳转到地址B中去了。所以如果在处理网页抓取等情况时，也要先明白内部过程中所涉及的自动跳转等逻辑，然后才能用代码实现和模拟此自动跳转的过程，或者是禁止发生自动跳转。总之是需要搞懂网页执行的逻辑，再去用代码实现，从而达到自己的目的。

（3）HTTP的POST

普通的urllib2.urlopen默认是GET，如果加上post数据就变成了HTTP的POST了。HTTP的POST就是按照一定的格式去向服务器发送一定的数据，以此实现提交某些数据给服务器的目的。一般在处理模拟登录网站等过程中基本都会遇到POST操作。此部分的内容相对复杂，需要大量的背景理论知识以及足够多的实践才能很好地实现对应的这方面的处理。

（4）网页抓取和模拟登录等遇到的Cookie

与上面提到的POST类似，在模拟登录等方面往往还会遇到Cookie等方面的内容。Cookie的英文原意是小甜点，此处是指浏览器会在你自己的电脑上存储一定的数据，用于记录你之前的各种信息。Cookie的最经典的应用就是我们在使用淘宝购物时遇到的情况：在浏览了几个物品后，每次将商品加入了购物车的动作背后所发生的技术处理就是浏览器会去存储或更新某个Cookie文件，用于保存当前加入购物车的相关产品的信息，比如加入的数量、商品的ID等。

如上所提及的一些和网络处理有关的内容只是简要介绍一下，使得读者对于这方面的知识有个简单的概念。而想要真正实现复杂的网页抓取、模拟登录方面的功能，还会涉及额外更多更复杂的内容，此处由于篇幅所限无法详述。以后有机会的话也会考虑再去写教程，去详细解释这方面的原理和实现，供需要的人参考。

6.2 Python在图形界面方面的应用

介绍完Python在网络方面的应用，接着介绍Python在开发图形界面程序方面的应用。

6.2.1 Python的常见GUI图形库

Python中GUI图形界面方面的库根据来源可分为：

1.内置的图形库

（1）标准的Tcl/Tk的库：Tkinter

Tkinter是Python的内置的GUI方面的库，也是相对使用比较广泛的一种Python的GUI的库。

（2）Tcl/Tk延伸出来的库

· ttk——Tk themed widgets

· Tix——Extension widgets for Tk

2.第三方的图形库

（1）PyGTK

（2）PyQt

（3）wxPython

6.2.2 Python的GUI图形库：PyQt

在众多的Python的GUI图形库中，图形效果相对来说比较好的，要数鼎鼎大名的Python版本的QT库PyQt了。

在解释PyQt的下载安装和使用之前先要解释QT是什么：最开始的QT是Trolltech开发的而后被Nokia收购了，所以现在也叫作Nokia的Qt，它是一个C++库的大集合。Qt本身有多个版本：Qt2、Qt3、Qt4、Qt5等。再来解释PyQt是什么：PyQt是Riverbank Computing Limited（河岸计算机化有限公司）为C++框架开发的、跨平台的Qt的GUI/XML/SQL的Python捆绑包。其中针对于Qt2和Qt3的即为Python版本的Qt2和Qt3。再来解释PyQt4是什么：PyQt4是相对于PyQt之外的，专门针对Qt4、Qt5等的，另外一个Python捆绑包。即Python版本的Qt4、Qt5等。但是需要注意的是PyQt4并不包含Qt本身。PyQt4中包含有440左

右个类库、6000多个函数，实现了对应的GUI、XML处理、网络通信、SQL数据库、网络浏览等方面的功能，对应着其中众多的QtCore、QtGui、QtNetwork、QtXml、QtSvg、QtOpenGL、QtSql等模块。

接下来再来解释如何用PyQt4写一个简单的GUI程序：从PyQt的主页的下载页面http://www.riverbankcomputing.co.uk/software/pyqt/download下载和当前环境匹配的版本。笔者此处的是64位Windows7，所以下载的是PyQt-Py2.7-x64-gpl-4.9.6-1.exe，接着双击安装即可。在安装期间可以看到它包含很多模块，如图6-2所示。

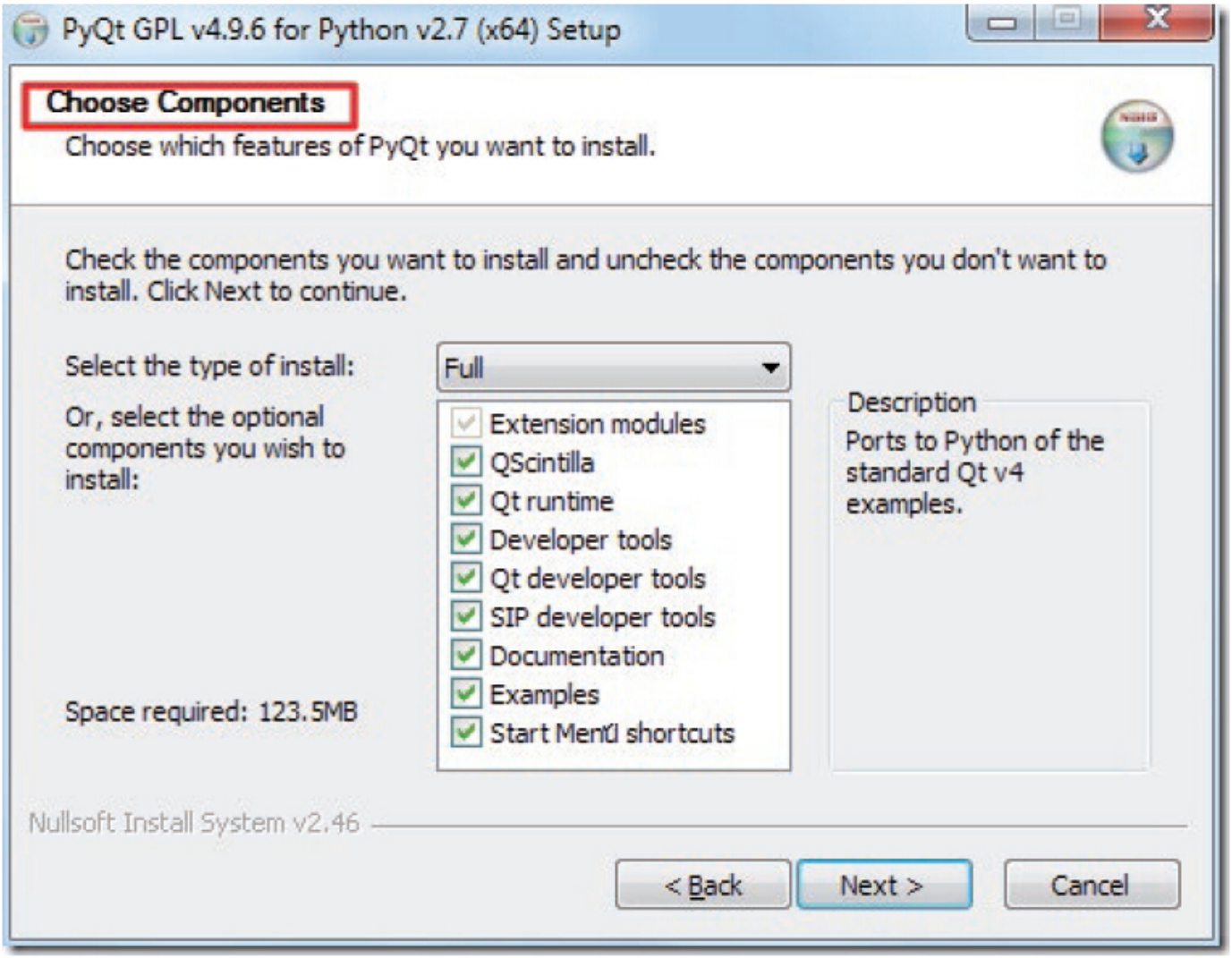


图6-2　安装PyQt：选择不同模块

然后就可以使用PyQt的示例代码：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-
"""
Function:
    【记录】折腾Python的PyQt4模块
http://www.crifan.com/python_module_pyqt4
Author:     Crifan Li
Version:    2013-01-04
Contact:    admin at crifan dot com
"""
#!/usr/bin/python
# -*- coding: utf-8 -*-
"""
ZetCode PyQt4 tutorial
In this example, we create a simple
window in PyQt4.
author: Jan Bodnar
website: zetcode.com
last edited: October 2011
```

```
"""
import sys;
from PyQt4 import QtGui;
def pyqtDemo():
    app = QtGui.QApplication(sys.argv);
    w = QtGui.QWidget();
    w.resize(250, 150);
    w.move(300, 300);
    w.setWindowTitle('Simple');
    w.show();
    sys.exit(app.exec_());
if __name__ == "__main__":
    pyqtDemo();
```

运行后可以看到对应的标题为simple的简单窗口，如图6-3所示。

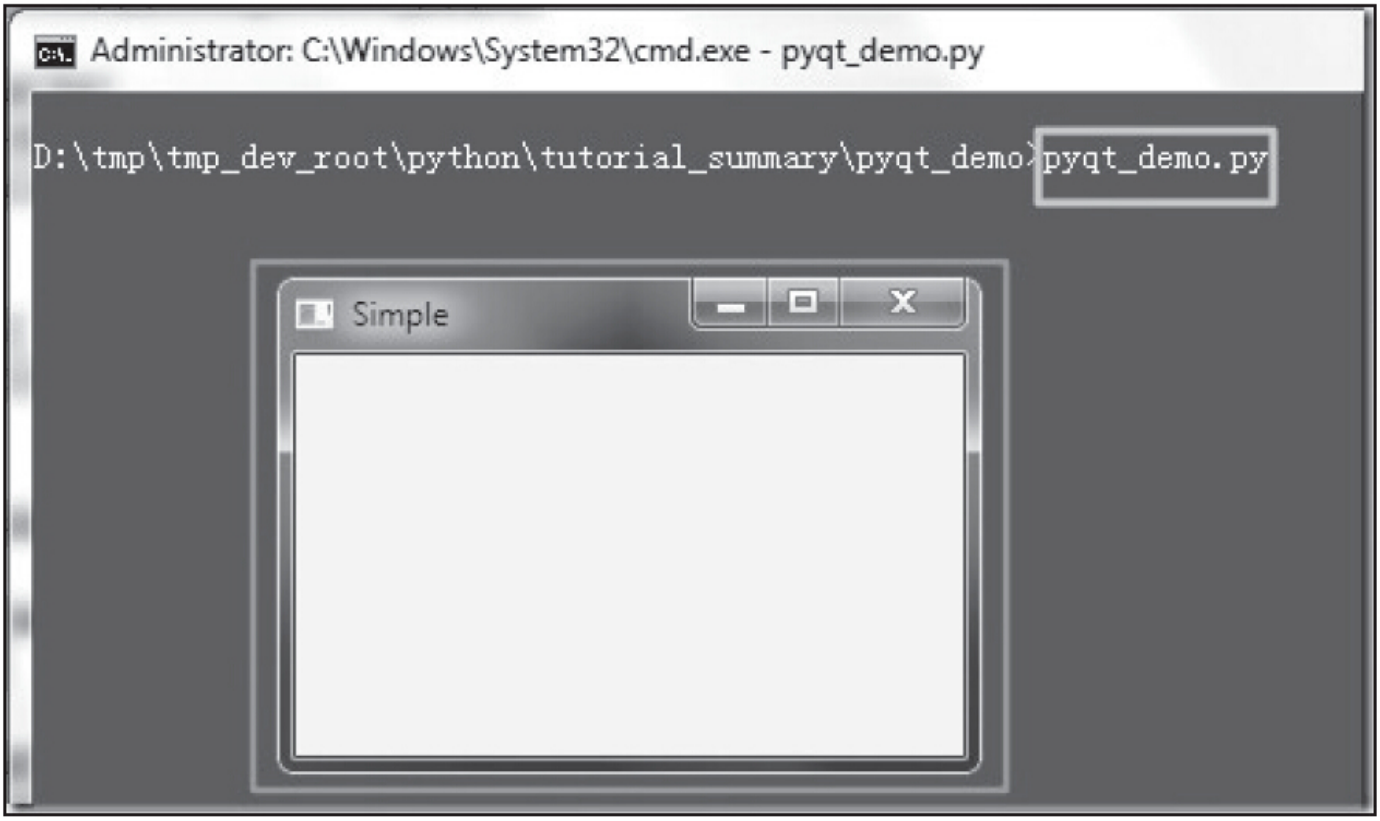


图6-3　PyQt示例：标题为simple的简单窗口

再去试试另外一个PyQt的例子：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-
"""
Function:
 【记录】折腾Python的PyQt4模块
http://www.crifan.com/python_module_pyqt4
Author:     Crifan Li
Version:    2013-01-04
Contact:    admin at crifan dot com
"""
import sys
from PyQt4 import QtGui
class Example(QtGui.QWidget):
    def __init__(self):
        super(Example, self).__init__()
        self.initUI()
    def initUI(self):
        QtGui.QToolTip.setFont(QtGui.QFont('SansSerif', 10))
        self.setToolTip('This is a <b>QWidget</b> widget')
        btn = QtGui.QPushButton('Button', self)
        btn.setToolTip('This is a <b>QPushButton</b> widget')
        btn.resize(btn.sizeHint())
        btn.move(50, 50)
        self.setGeometry(300, 300, 250, 150)
        self.setWindowTitle('Tooltips')
        self.show()
```

```
def pyqtDemoTooltip():
    app = QtGui.QApplication(sys.argv)
    ex = Example()
    sys.exit(app.exec_())
if __name__ == '__main__':
    pyqtDemoTooltip()
```

运行后的效果是一个带Button按钮和动态提示Tooltip的窗口，如图6-4所示。

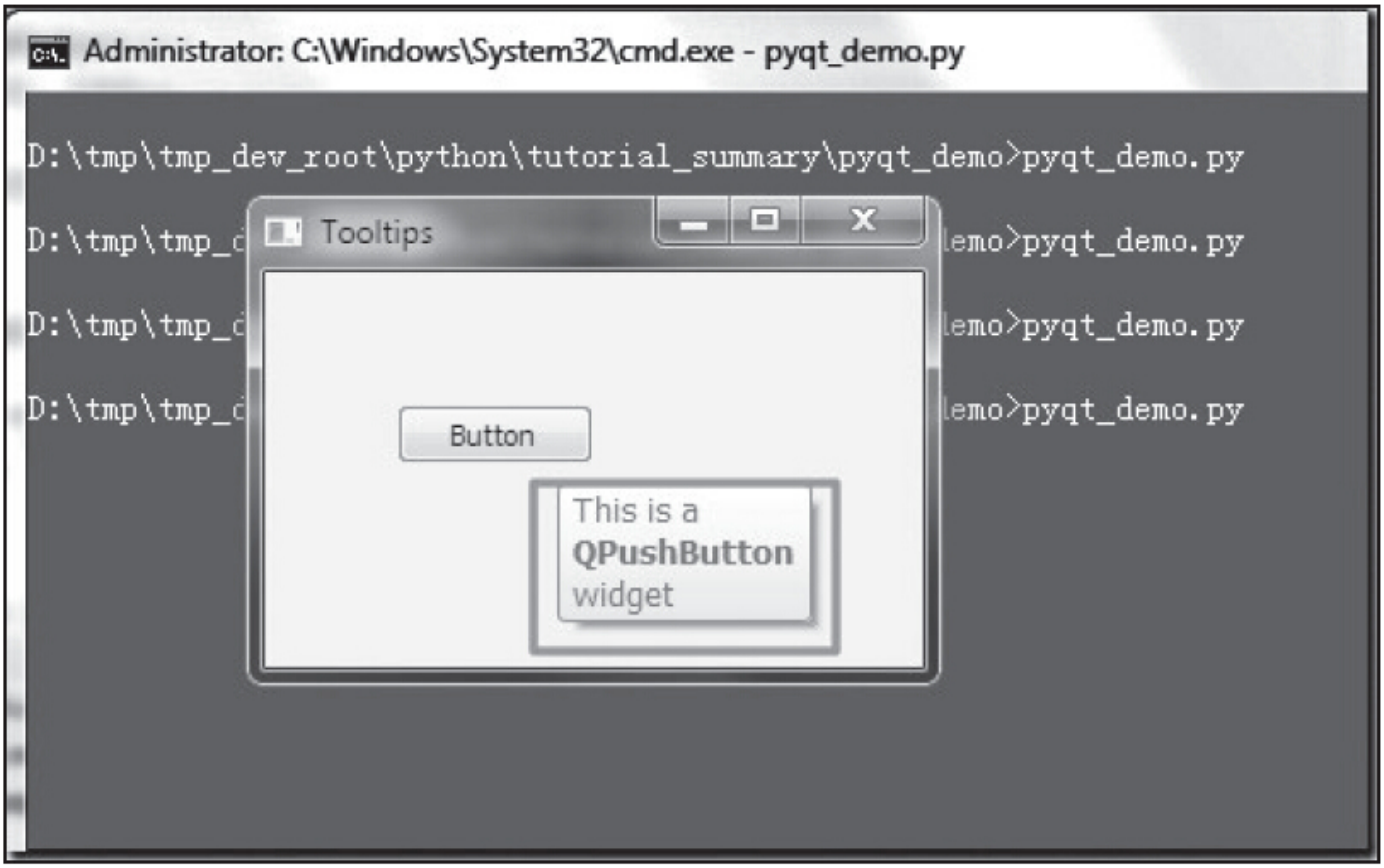


图6-4　PyQt示例：带Tooltip和Button的窗口

由图6-4可以看到PyQt的效果还是不错的。对于有兴趣继续深入研究PyQt去实现更加复杂功能读者，接下来需要做的事情就是参考官网文档和示例代码，去写自己的代码了。

6.3　Python在数据库方面的应用

Python支持数据库方面的开发。对于数据库方面的支持，使用度相对广泛的要数MySQL数据库，对应的Python的MySQL的支持有个叫作MySQLdb的Python的第三方库。下面就来演示如何使用MySQLdb和Python去操作数据库。

先登录网址http://pypi.python.org/pypi/MySQL-python/下载最新1.4版本的MySQLdb的安装文件MySQL-python-1.2.4.win32-py2.7.exe，下载后双击安装。此处需要提示一点：笔者此处是64位Windows7与该32位版本不匹配所以无法安装，最后只能通过下载源码，然后手动编译后再去安装及使用测试代码：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-
"""
---------------------------------------------------------------------------
Function:
  【记录】为Python安装MySQL数据库模块：MySQLdb
http://www.crifan.com/python_install_module_mysql_mysqldb/
```

```
Autor:  Crifan Li
Date:   2013-04-22
-------------------------------------------------------------------------------
"""
#--------------------------------import-----------------------------------------
import MySQLdb
def python_mysql_test():
    """
    try to use mysql in python
    """
    try:
conn=MySQLdb.connect(host='localhost',user='root',passwd='xxx',db='wordpress', port=3306)
        print "conn=",conn;
        cur=conn.cursor()
        print "cur=",cur;
        cur.execute('select * from wp_users')
        cur.close()
        conn.close()
    except MySQLdb.Error,e:
         print "Mysql Error %d: %s" % (e.args[0], e.args[1])
################################################################################
if __name__=="__main__":
    python_mysql_test();
```

测试连接本地数据库，则可以显示成功连接：

```
D:\tmp\tmp_dev_root\python\tutorial_summary\python_mysql_test>python_mysql_test.py
conn= <_mysql.connection open to 'localhost' at 2a46ad8>
cur= <MySQLdb.cursors.Cursor object at 0x0000000002A54C50>
```

接着通过数据库工具PhpMyAdmin去查看对应的数据库中对应的数据项，如图6-5所示。

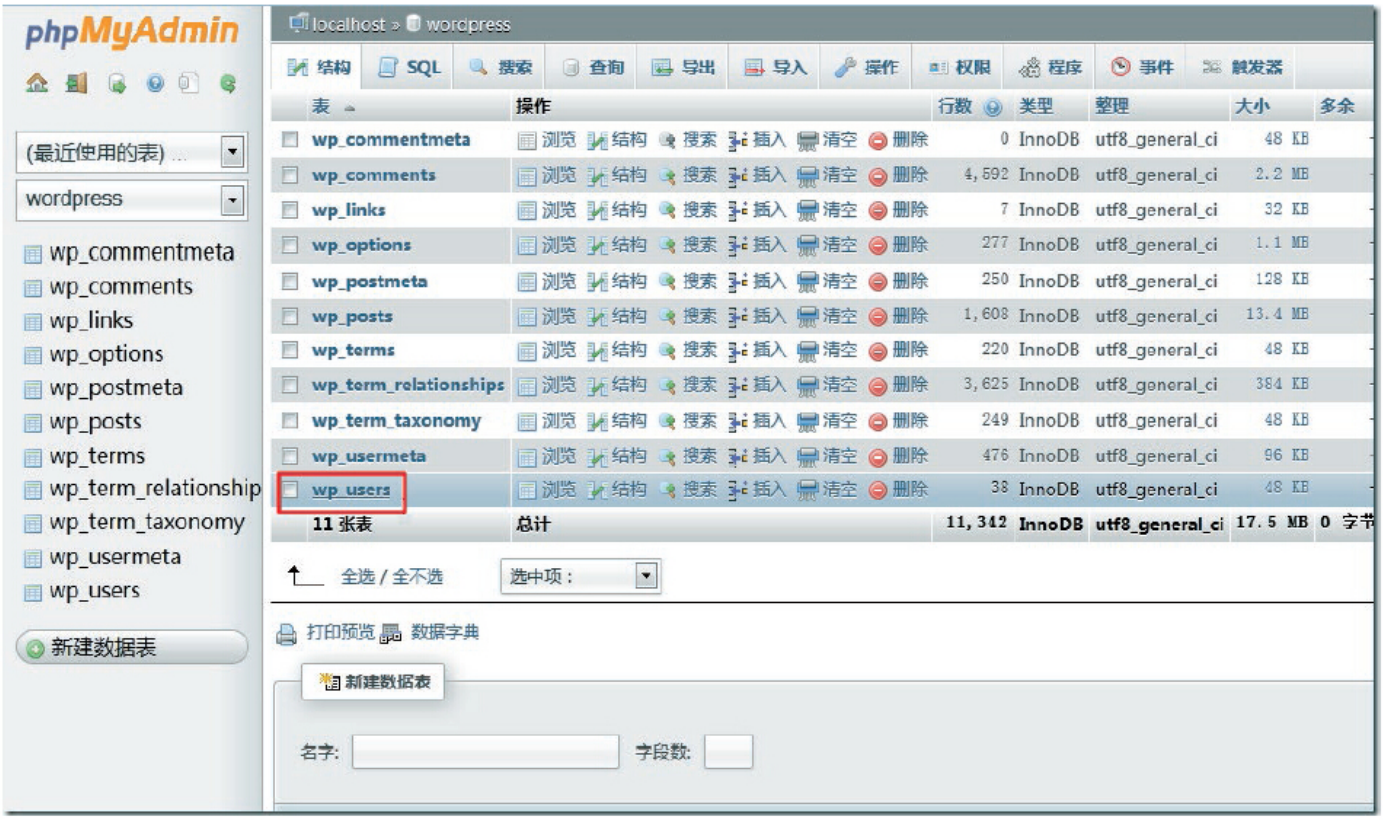


图6-5　使用PhpMyAdmin查看数据库中的数据项wp_users

由图6-5可知的确可以正常使用MySQLdb连接数据库，接着使用测试代码去新建数据项：

```
#!/usr/bin/python
# -*- coding: utf-8 -*-
"""
-------------------------------------------------------------------------------
Function:
  【记录】为Python安装MySQL数据库模块：MySQLdb
http://www.crifan.com/python_install_module_mysql_mysqldb/
```

```
Autor:  Crifan Li
Date:   2013-04-22
-------------------------------------------------------------------------------
"""
#--------------------------------import-------------------------------------
import MySQLdb
def python_mysql_test():
    """
    try to use mysql in python
    """
    try:
conn=MySQLdb.connect(host='localhost',user='root',passwd='xxx',db='test',
port=3306, charset='utf8')
        print "conn=",conn;
        cur=conn.cursor()
        print "cur=",cur;
        createdDb = cur.execute('create database if not exists pythonMysql')
        print "createdDb=",createdDb;
        selectedDb = conn.select_db('pythonMysql')
        print "selectedDb=",selectedDb;
        createdTable = cur.execute('create table test(id int,info varchar(20))')
        print "createdTable=",createdTable;
        value=[1,'hi rollen']
        insertedValue = cur.execute('insert into test values(%s,%s)',value)
        print "insertedValue=",insertedValue;
        values=[]
        for i in range(20):
            values.append((i,'hi rollen'+str(i)))
        insertedValues = cur.executemany('insert into test values(%s,%s)',values)
        print "insertedValues=",insertedValues;
        updatedInfo = cur.execute('update test set info="I am rollen" where id=3')
        print "updatedInfo=",updatedInfo;
        committed = conn.commit()
        print "committed=",committed;
        closedCur = cur.close()
        print "closedCur=",closedCur;
        closedConn = conn.close()
        print "closedConn=",closedConn;
    except MySQLdb.Error,e:
         print "Mysql Error %d: %s" % (e.args[0], e.args[1])
##############################################################################
if __name__=="__main__":
    python_mysql_test();
```

执行代码后输出信息如下：

```
D:\tmp\tmp_dev_root\python\tutorial_summary\python_mysql_test>python_mysql_test.py
conn= <_mysql.connection open to 'localhost' at 2be06a8>
cur= <MySQLdb.cursors.Cursor object at 0x00000000028B7BE0>
createdDb= 1
selectedDb= None
createdTable= 0
insertedValue= 1
insertedValues= 20
updatedInfo= 1
committed= None
closedCur= None
closedConn= None
```

然后再去数据库中确认数据项是否创建成功，如图6-6所示。

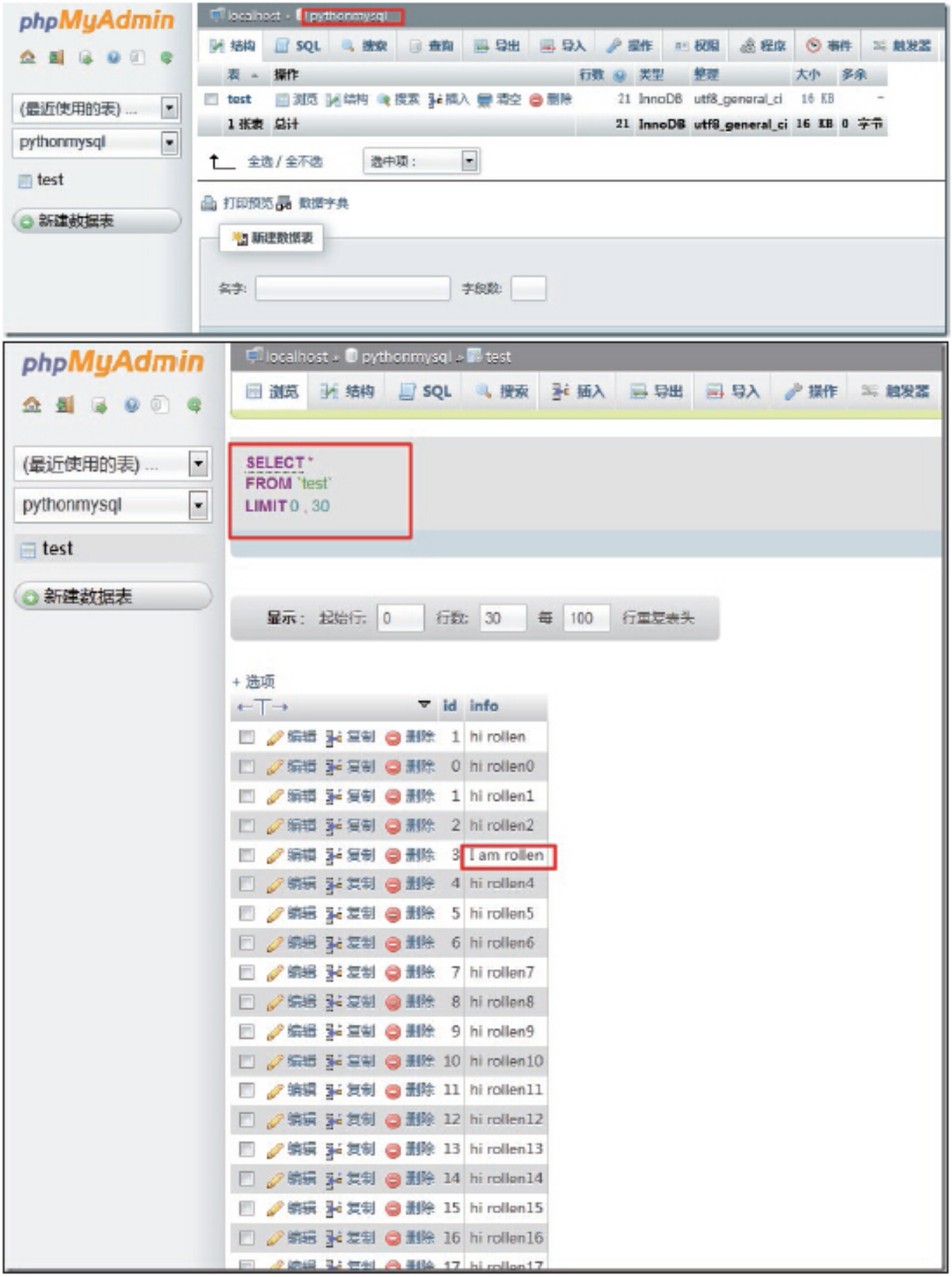


图6-6　pythonmysql数据库中新增了对应数据项

至此，就真正实现了通过Python的MySQLdb库去连接MySQL数据库，并且可以查询对应的数据项以及创建对应的数据表项。

第7章　Python与开源硬件

在介绍了Python的各种常见应用后，为了增加深入学习Python的动力和兴趣，下面再来介绍一下近期比较流行的开源硬件，以及如何用Python搭配开源硬件做一些更酷更有趣的东西。

7.1　Python和开源硬件之间的关系

在详细介绍开源硬件之前，为了增加读者的直观认识，从宏观上了解开源硬件和普通PC之间的关系，此处先通过对比来总结常见的开源硬件和Python之间的关系，如表7-1所示。

表7-1　Python和开源硬件之间的关系

不同硬件平台	运行的操作系统	Python 的运行环境
普通 PC	Windows	安装 Windows 版本的 Python 后，所带的 Python 运行环境
	Linux	各种 Linux 发行版（比如 Ubuntu）中都自带的 Python 运行环境
	Mac	Mac 中自带的 Python 运行环境
pcDuino	Lubuntu	Lubuntu 中也自带了 Python 运行环境
树莓派 (Raspberry Pi)	Arch Linux	自带的 Python 运行环境

至此读者应该很容易理解了：对于在硬件pcDuino中运行的软件操作系统Lubuntu，以及在Lubuntu中运行Python，和在普通的硬件即笔记本电脑或台式计算机，中运行操作系统Ubuntu，以及在Ubuntu中运行Python是没区别的。Python和开源硬件之间的关系可以总结为：

开源硬件类似于一个普通的计算机，就像在普通计算机的Ubuntu操作系统中安装Python一样，也可以在pcDuino的Lubuntu操作系统中安装Python。

7.2　pcDuino基础知识

7.2.1　什么是开源硬件

解释在开源硬件pcDuino上使用Python实现应用之前，先要对开源硬件进行一些基本的介绍。简要的解释可以总结为：开源软件是将软件的源代码公开，其出发点是本着开源和共享的精神去公开源码，使得他人可以借鉴和使用，且如果软件有问题或bug，他人也可以帮忙指出；并且任何人都可以将自己的研究成果，比如新加的功能加入其中，以此更加促进该软件的发展。而开源硬件类似于开源软件，不同之处在于，开源硬件是把硬件的“源代码”公开。此处硬件的源代码，对应的就是物理上的实体：硬件开发板。而硬件开发板这个硬件所对应的各种软件层面的资料则包括：

· 硬件原理图（schematics）

· 机械绘图（mechanical drawings）

· 物料清单（bills of material BOM）

· 板子布局（PCB layout data）

· 硬件模块的设计（HDL source code）

· 各个模块的电路图（integrated circuit layout）

如此多的相关资料汇总起来就可以理解为：硬件实体所对应的软件设计方面的资料。有了这些资料，就可以去找开发板代工厂生产出相应的开发板，换句话说，有了这些资料就可以去找代工厂生产了，最终可以获得物理上的实体：真正的开发板。而接下来就是软件人员的事情：在这个开发板上折腾软件方面的驱动和功能。

而上述的过程在此之前是另外一套，之前这些硬件的设计都是不同的硬件开发板设计厂家设计，各自有不同的设计且一般是私有的电路设计，是不会公开的。而这个开源硬件就是把这些都公开了，其核心目的在于：普通的开发人员可以通过公开途径去购买这些根据某开源硬件制造出来的开发板。其特点是价格相对不高，但功能相对丰富，在很大程度上降低了软件开发者获取一个可用的开发板和相关配置资料的成本，并且往往同时也提供了对应的各种不同的软件系统，方便了软件开发者基于该系统继续做自己的上层应用。

7.2 pcDuino基础知识

7.2.1 什么是开源硬件

解释在开源硬件pcDuino上使用Python实现应用之前，先要对开源硬件进行一些基本的介绍。简要的解释可以总结为：开源软件是将软件的源代码公开，其出发点是本着开源和共享的精神去公开源码，使得他人可以借鉴和使用，且如果软件有问题或bug，他人也可以帮忙指出；并且任何人都可以将自己的研究成果，比如新加的功能加入其中，以此更加促进该软件的发展。而开源硬件类似于开源软件，不同之处在于，开源硬件是把硬件的“源代码”公开。此处硬件的源代码，对应的就是物理上的实体：硬件开发板。而硬件开发板这个硬件所对应的各种软件层面的资料则包括：

· 硬件原理图（schematics）

· 机械绘图（mechanical drawings）

· 物料清单（bills of material BOM）

· 板子布局（PCB layout data）

· 硬件模块的设计（HDL source code）

· 各个模块的电路图（integrated circuit layout）

如此多的相关资料汇总起来就可以理解为：硬件实体所对应的软件设计方面的资料。有了这些资料，就可以去找开发板代工厂生产出相应的开发板，换句话说，有了这些资料就可以去找代工厂生产了，最终可以获得物理上的实体：真正的开发板。而接下来

就是软件人员的事情：在这个开发板上折腾软件方面的驱动和功能。

而上述的过程在此之前是另外一套，之前这些硬件的设计都是不同的硬件开发板设计厂家设计，各自有不同的设计且一般是私有的电路设计，是不会公开的。而这个开源硬件就是把这些都公开了，其核心目的在于：普通的开发人员可以通过公开途径去购买这些根据某开源硬件制造出来的开发板。其特点是价格相对不高，但功能相对丰富，在很大程度上降低了软件开发者获取一个可用的开发板和相关配置资料的成本，并且往往同时也提供了对应的各种不同的软件系统，方便了软件开发者基于该系统继续做自己的上层应用。

7.2.2 常见的开源硬件

解释了什么是开源硬件，接着再来简要介绍一下目前常见的开源硬件有哪些。近来国内比较流行的开源硬件有如下几种。

1.开源硬件之Arduino

Arduino是一个采用了Atmel AVR单片机的开放源代码的单芯片微电脑。

Arduino的Logo如图7-1所示。

图7-1 Arduino的Logo

典型的Arduino板子的实物图如图7-2所示。

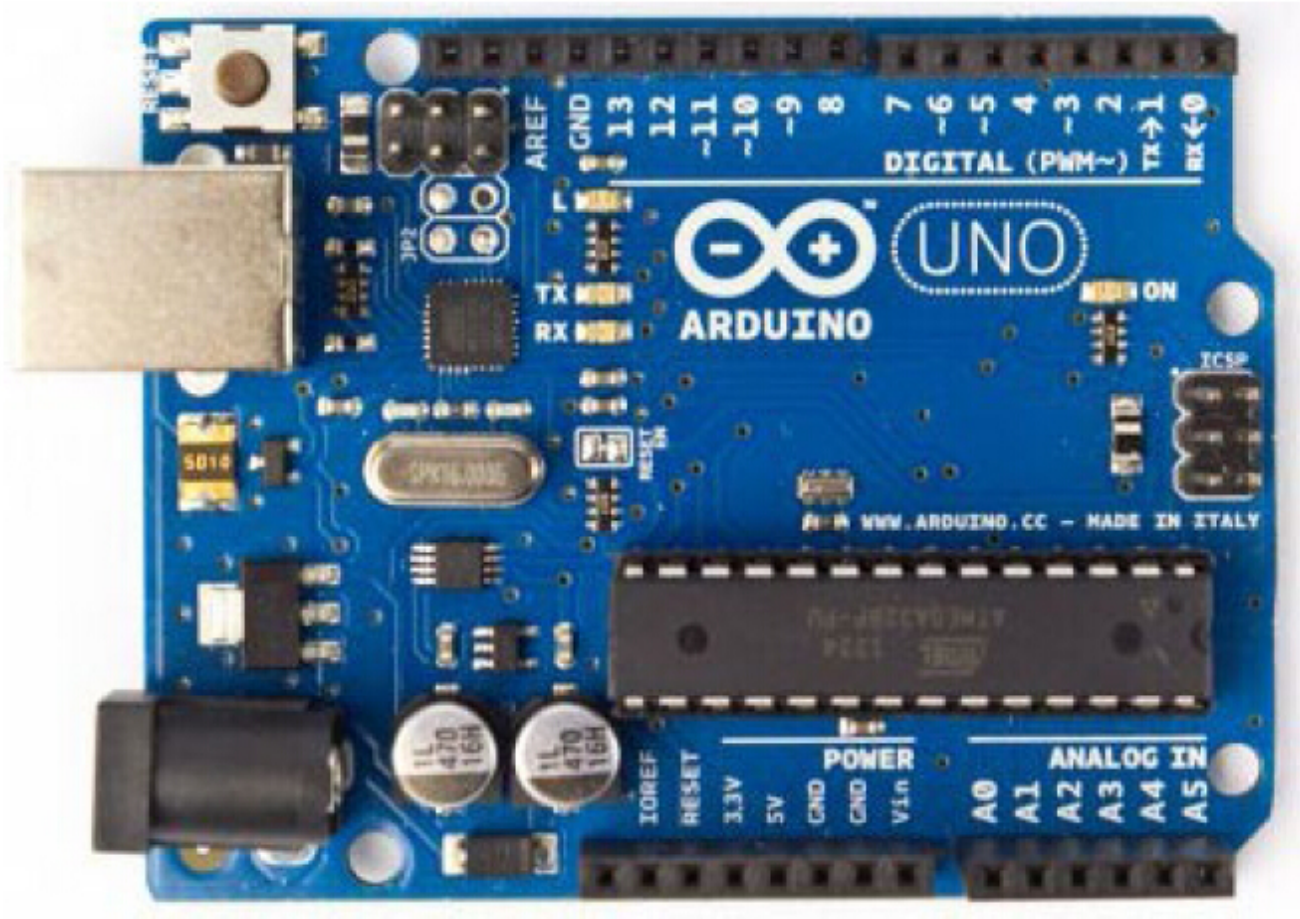


图7-2　Arduino板子实物图

Arduino的主要特点是在开源硬件的基础上设计了一个类似Java、C语言的Processing/Wiring开发环境。这就使得即使是对于硬件细节不熟悉的普通开发者，也可以直接通过此套开发环境编写基本的代码实现直接操控硬件的目的，实现各种很酷的功能。比如普通开发者可以通过简单的几行代码实现操作LED灯的开和关。所以Arduino的主要目的在于：以相对简单、统一、方便的方式去操控硬件，然后在此基础上实现很多相对复杂的各种应用，比如用Arduino去制作机器人小车。

关于更多的Arduino资料，可以参考其官网，地址为http://arduino.cc/。

2.开源硬件之树莓派

树莓派的英文是Raspberry Pi，它是一个运行着Linux系统的只有一张信用卡大小的单板机计算机。

树莓派的Logo如图7-3所示。

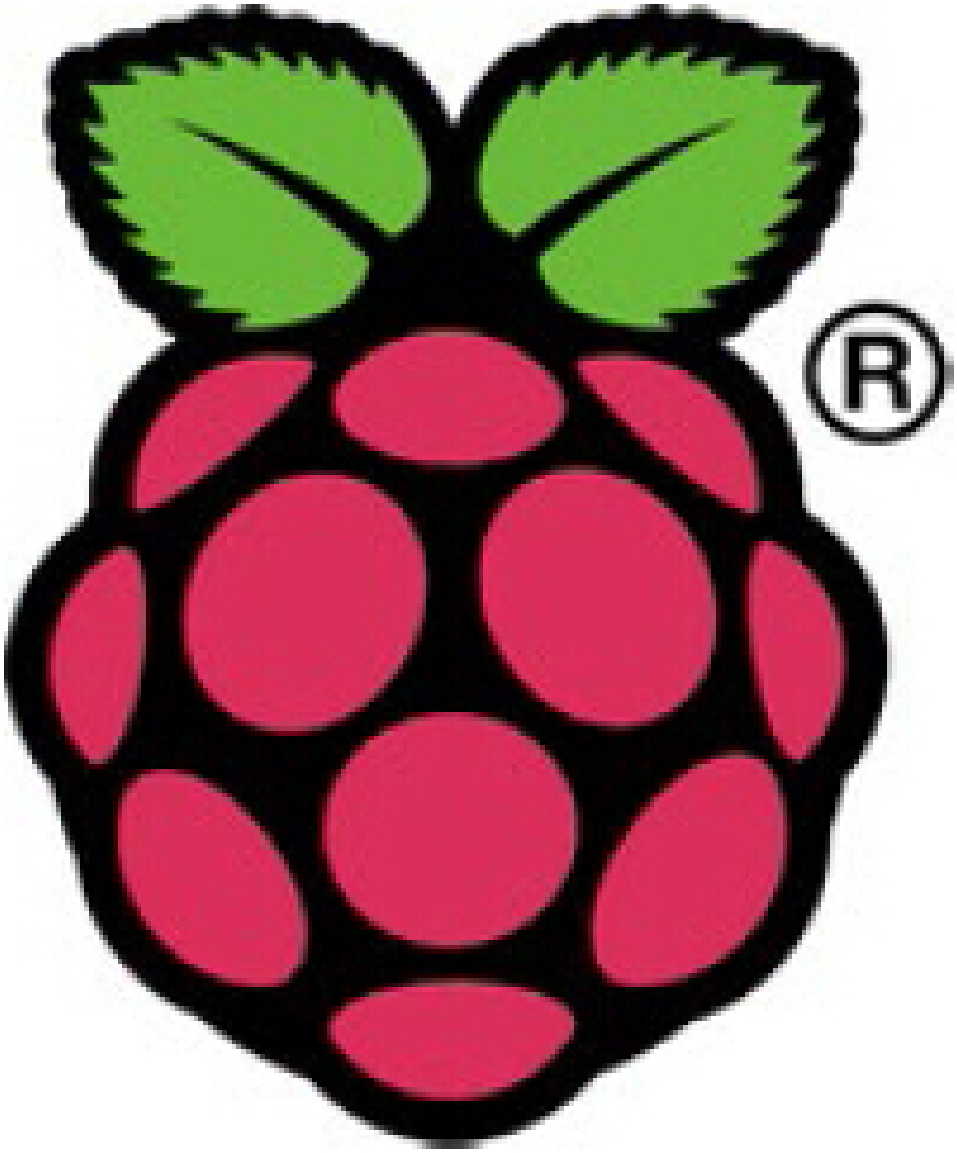


图7-3　树莓派的Logo

一个典型的树莓派板子的实物图如图7-4所示。

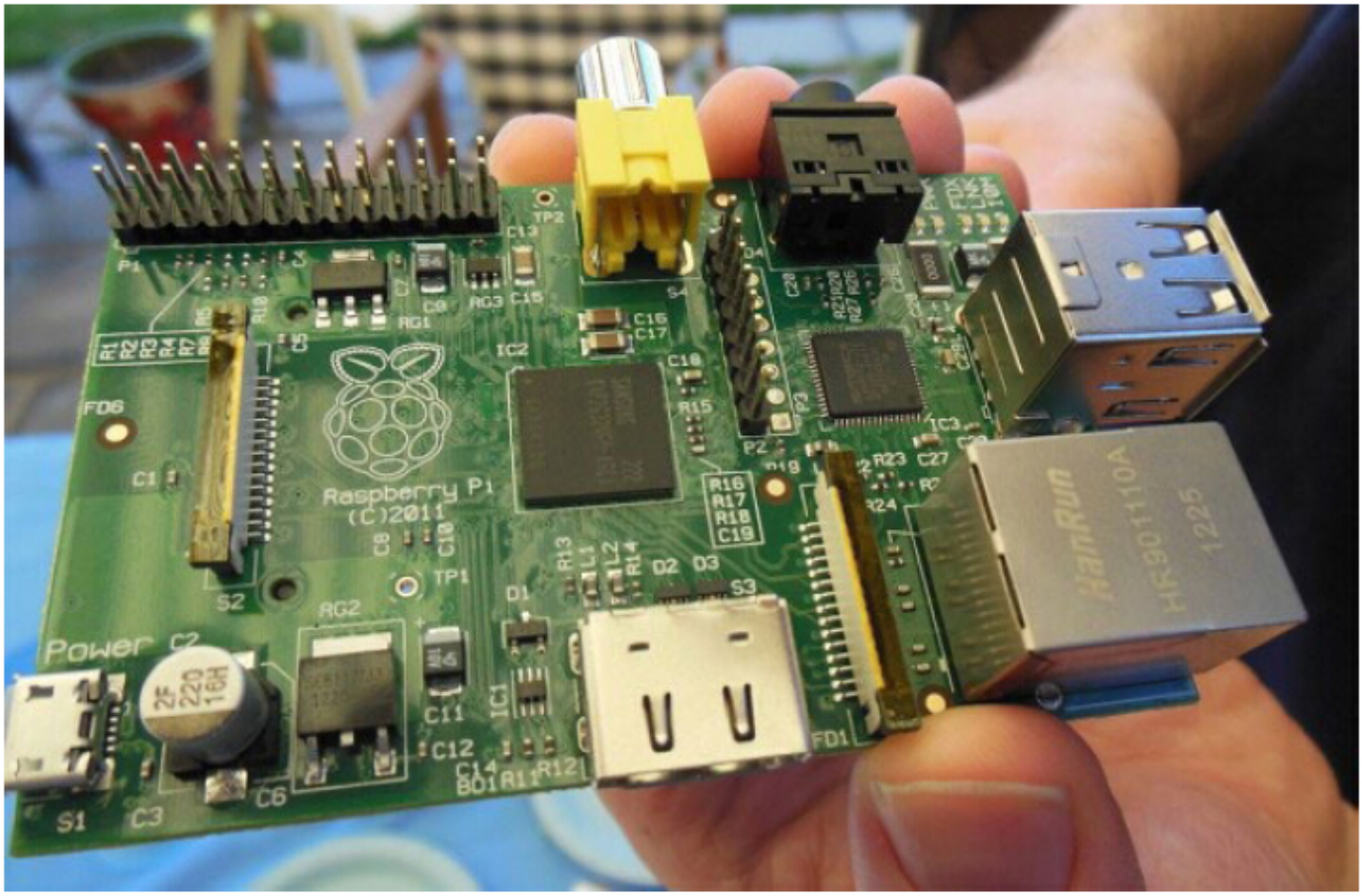


图7-4　树莓派板子实物图

树莓派是英国的树莓派基金会所开发的，是为了在学校内普及电脑科学教育而推出的，硬件价格相对较低。由于树莓派相当于一个普通PC，所以同样可以用来实现各种不同的功能，包括但不限于Web服务器、XMBC多媒体中心、有声书籍播放器、家用VPN服务器、电子相框等。甚至还有人对其进行更有趣的改造，比如像直升机一样的四旋翼飞行器、制作成一个酷坦克、被热气球送上太空等。

更多树莓派的资料可以参考其官网，地址为http://www.raspberrypi.org/。

3.开源硬件之pcDuino

pcDuino是一台可以运行Lubuntu、Android 4.0ICS等类似于PC版的操作系统的迷你电脑。

一个典型的pcDuino板子的实物图如图7-5所示。

图7-5　pcDuino板子实物图

（1）pcDuino的主要特点

1）pcDuino可以通过板子上的HDMI接口输出视频。这样就可以把带HDMI的超大的外置显示器如LCD显示屏当做pcDuino这个迷你电脑的显示器了。

2）pcDuino的硬件接口是和Arduino兼容的。这意味着对于Arduino的开发环境是完全可以在此pcDuino上面运行的。换句话说，之前别人或自己写的可以运行在Arduino中用于操作各种硬件的代码是可以拿到pcDuino上运行，去控制对应的各种硬件外设的。另外也兼容Arduino的外接设备的接口。

pcDuino和之前的树莓派等类似，就相当于一个普通的PC。正如我们知道的那样，在PC上可以实现你能够想象到的任何应用。因此关于可以把pcDuino打造为其他用途，就像在普通PC上实现应用，可以说是：想象力有多大，舞台就有多大。

（2）可在pcDuino上用Python实现的应用

下面列出一些已经可以在pcDuino上用Python实现的应用。

1）在pcDuino上通过网页的形式控制LED灯的打开和关闭。实现一个Web服务器，并且提供对应的按钮，可以实现LED灯的开关，由此实现多个LED灯的闪烁效果。由于LED灯用的是不同颜色，所以看到的就是各种彩灯闪烁的效果。

2）在pcDuino上运行simpleCV以便实现图形图像处理。通过使用Python的库函数simpleCV可以实现图像的处理，包括实时采集摄像头的数据以及可以对图片进一步处理，就像Photoshop软件那样做出很多复杂的图形处理。

3）用pcDuino实现远程无线视频监控。通过在pcDuino中使用Python，再加上其他语言，实现远程的视频控制功能。此应用还增加了额外复杂的功能：做了对应的安卓手机的应用程序，可以通过手机实现远程操控家里的pcDuino小车，然后通过pcDuino上面的摄像头，实时传输图像到手机端以达到远程无线视频监控的复杂应用。

如前所述，想要用pcDuino等开源硬件来实现何种功能，关键是要有好的创意，其次才是掌握和使用相关的技术去实现此创意。而在实现创意和功能时，很多都是选择相对比较方便好用的Python。

7.2.3 为何选择pcDuino

在得知了常见的开源硬件有Arduino、树莓派、pcDuino之后，下面再来对比介绍我们为何选择pcDuino，如表7-2所示。

表7-2 不同开源硬件对比

开源硬件 对比项	Arduino	树莓派	pcDuino
尺寸	各种尺寸均有：从指尖大小到大如一两张信用卡般大小	小巧：信用卡般大小	相对大一点：1.5 个信用卡大小
硬件配置	较低：性能相对较弱	中等：性能中等	相对较高：硬件性能更好
价格	从 130 元到 1600 元不等	150 元或 220 元	370 元左右
功能扩展	功能扩展性很好：各种功能的扩展板	扩展性一般：外接必需的设备	扩展性一般：外接部分设备
学习资源	很丰富：网上和官网提供很多上手资料	丰富：网上和官网提供了一些资料	很丰富：网上和官网提供很多上手资料和参考案例
上手易用程度	一般：需要使用者懂得对应的编程，外接一些接口板才能实现特定的功能	容易:需要自己准备 SD 卡，自己烧录镜像才能启动板子	很容易：板子买到后自带 Lubuntu 镜像，直接可以启动开发板
综合评价	价格便宜，可以外挂各种扩展板，实现各种有趣的功能，尤其是控制各种外设	适合代替普通 PC，实现各种丰富的应用	迷你型 PC，方便上手，可以运行不同系统和实现各种应用

由表7-2可知：如果是为了快速上手学习一个迷你型的PC，在该平台上实现各种创意和应用的话，相对来说pcDuino是个比较好的选择。因为虽然pcDuino价格相对高一些，但是配置更好、性能更好，能更流畅地运行各种系统和应用。并且，买到手的板子是已经烧好了操作系统的板子，可以直接上手实践。

7.2.4 如何配置开源硬件pcDuino

在选择了pcDuino之后，接下来介绍如何操作和控制pcDuino。对于pcDuino来说默认已经安装了Lubuntu系统，所以连接并操作pcDuino的方式有多种，此处介绍其中较常用的两种方式。

1.通过SSH2连接和控制pcDuino

在通过SSH2连接pcDuino中的Lubuntu之前，需要搞清楚Lubuntu的IP地址。此处获得当前Lubuntu的IP地址有如下两种方法。

（1）通过路由器查看当前已连接的IP

此处，通过当前的路由器管理软件：以网页形式登录路由器控制界面，找到对应的当前连接的电脑，再找到对应的IP地址，如图7-6所示。

图7-6　找到对应的IP地址

从图7-6中可见，此处的pcDuino中的Lubuntu对应着的用户名是ubuntu，其IP地址是192.168.1.107。

（2）通过网络扫描软件查看当前已连接的IP

另外也可以找个合适的网络扫描软件，去扫描当前网段中的电脑端口和IP地址。而端口扫描软件推荐使用nmap。

使用软件之前先明确我们的目标：已知Lubuntu开启了SSH服务，想要找到当前IP网段中开启了SSH服务的系统，其中就包含此处要找的那个Lubuntu。而目前本网段中没有其他的电脑开启了SSH，所以扫描出来开启了SSH的只有Lubuntu，就可以得知其IP了。

下面介绍如何具体操作。首先登录nmap的下载页面（地址：http://nmap.org/download.html）去下载最新版本的nmap-6.40-setup.exe，然后双击安装即可，如图7-7所示。

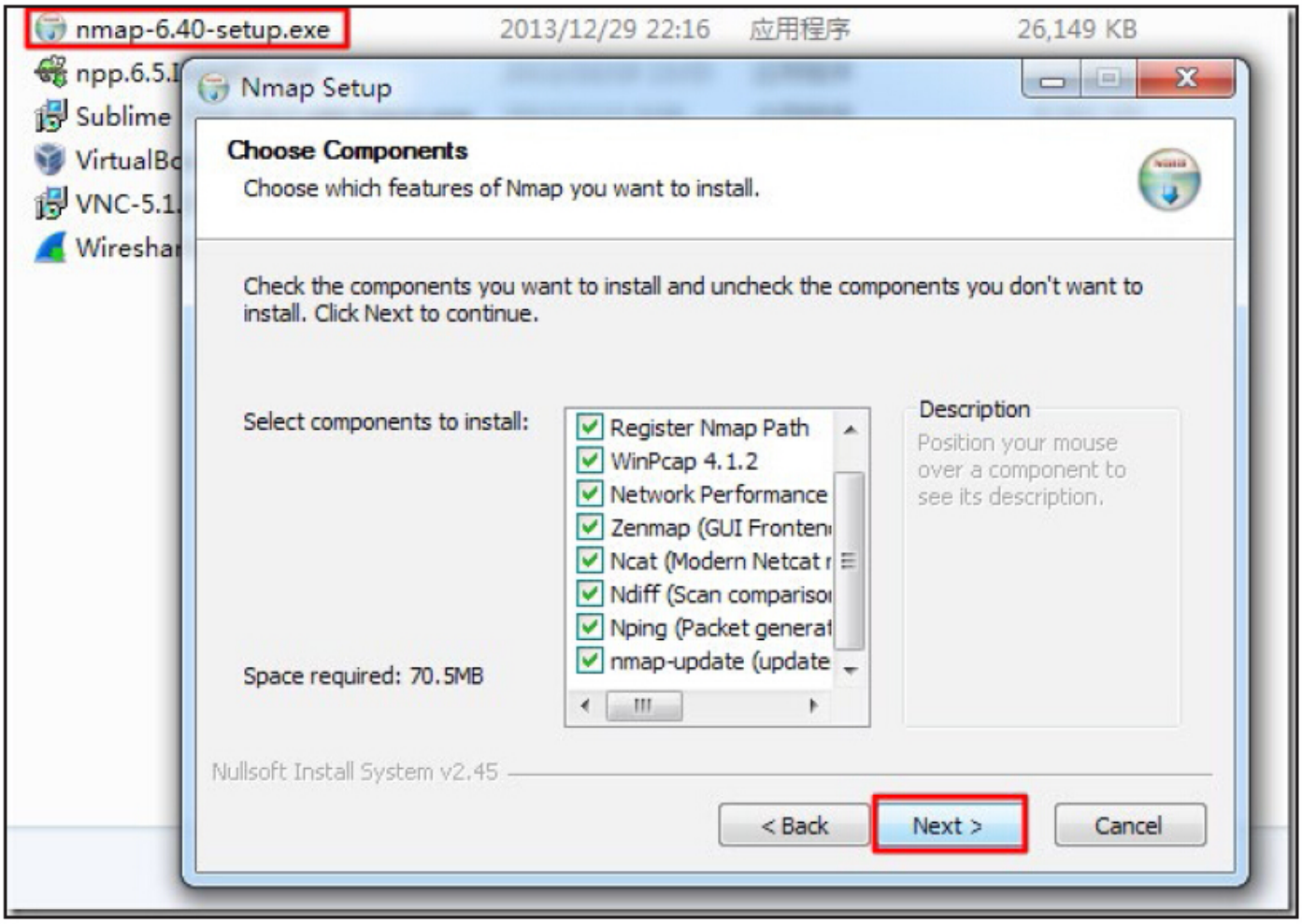


图7-7　在Windows 7中安装nmap 6.40

然后使用namp扫描，在Command中填写如下内容：

```
nmap --top-ports 10 192.168.1.*
```

单击“扫描”（Scan）按钮后，稍等片刻即可看到扫描结果，如图7-8所示。

可见开启了SSH的只有一个，所以其IP就是我们要找的Lubuntu的IP：192.168.1.107。

此处对于上述nmap参数的含义再详细解释一下。

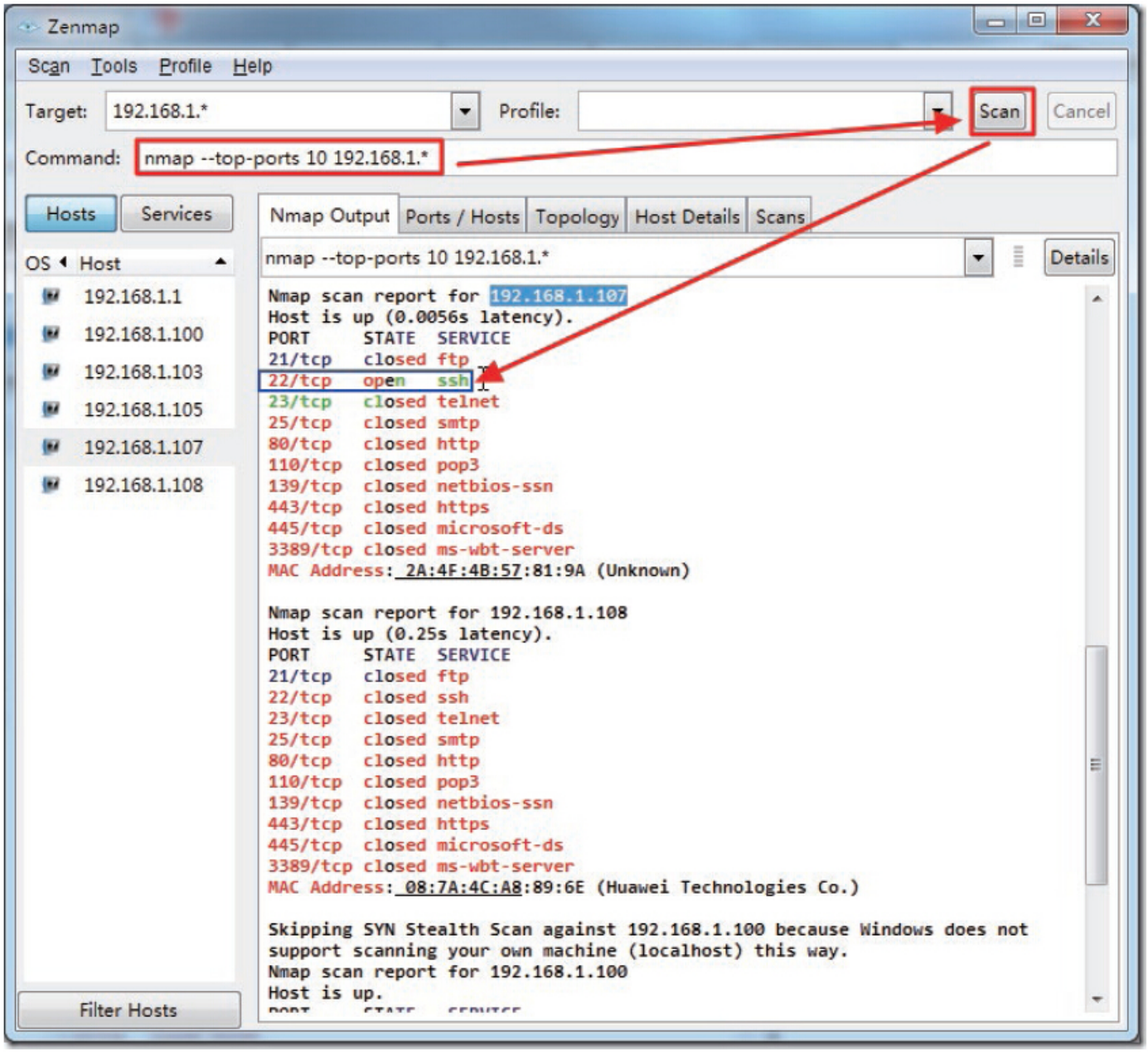


图7-8　使用namp扫描可以找到107的IP开启了SSH服务

参考nmap官网页面（地址：http://nmap.org/book/man.html）可知：参数“--top-ports 10”的含义是扫描最常用的10个端口。

到底常用的10个端口是哪几个呢？从图7-8中我们可以看到是21、22、23、25、80、110、139、443、445、3389。

在获得了当前Lubuntu的IP地址后，接着就可以用工具以SSH2的方式连接Lubuntu。支持SSH2且好用的工具很多，比如SecureCRT、putty等，此处推荐使用SecureCRT。

SecureCRT的操作步骤如下。

打开SecureCRT的连接界面，输入对应的参数：

· Protocol：SSH2

注意此处所用协议是SSH2，不是SSH。

· Hostname：192.168.1.107

此处所用地址就是我们上面查到的pcDuino中Lubuntu的地址。

· Port：22

SSH协议常用的也是默认的端口是22。

· Firewall：None

此处没有防火墙，所以设置为无。

· Username：ubuntu

pcDuino中的Lubuntu默认的用户名和密码都是ubuntu。

用Secure CRT通过SSH2连接pcDuino中Lubuntu的连接配置如图7-9所示。

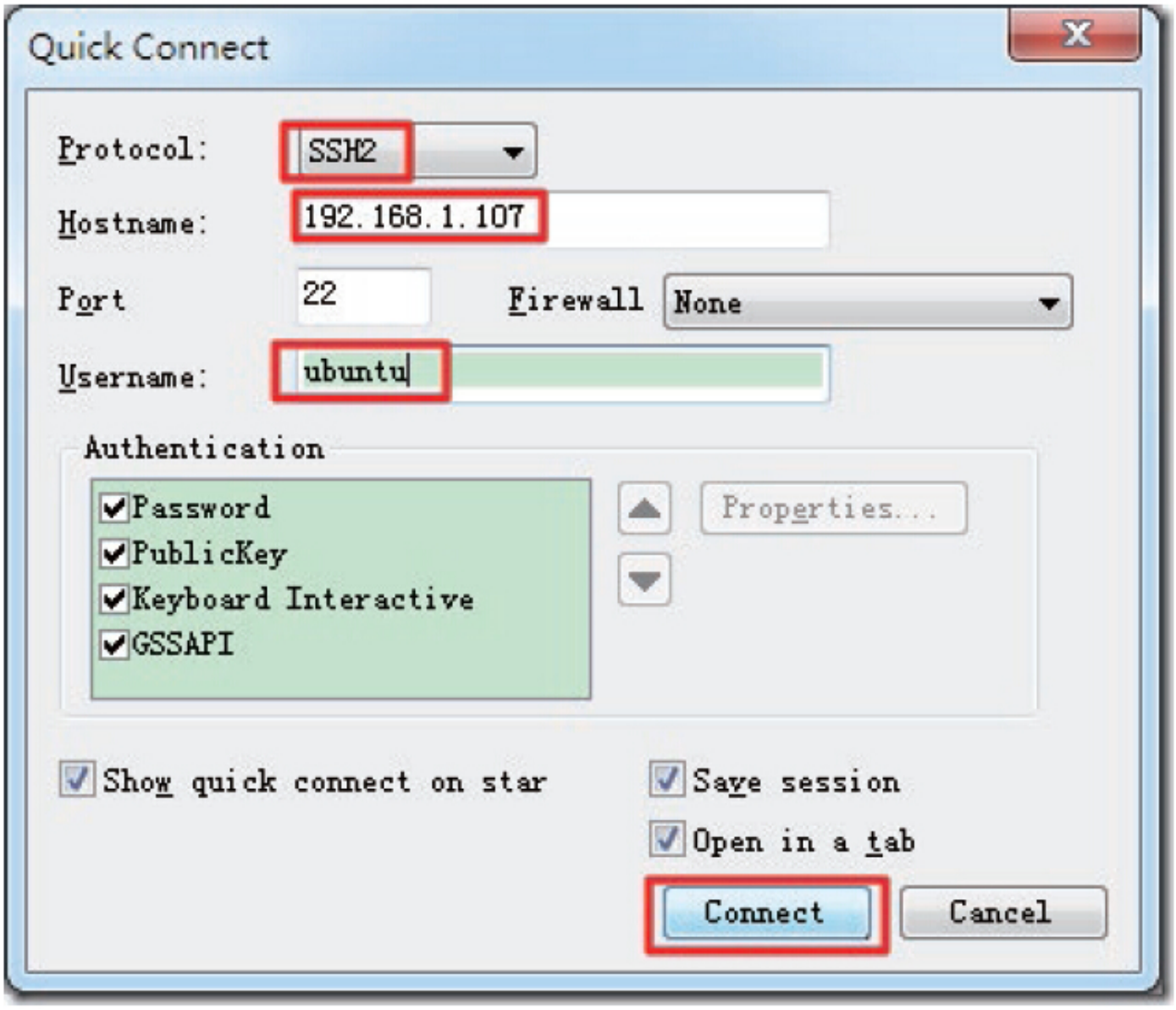


图7-9　用SecureCRT通过SSH2连接pcDuino中Lubuntu的连接配置

单击Connect按钮后SecureCRT会弹出对话框询问是否保存此连接的配置，单击Accept&Save按钮，如图7-10所示。

再输入密码，如图7-11所示。

前面提到了pcDuino中Lubuntu的默认用户名和密码都是ubuntu，所以此处密码填写ubuntu。接着通过SSH2连接pcDuino

的Lubuntu，如图7-12所示。

接下来就可以像普通的Linux类系统一样去操作了，比如用ls命令查看目录下有哪些文件，如图7-13所示。

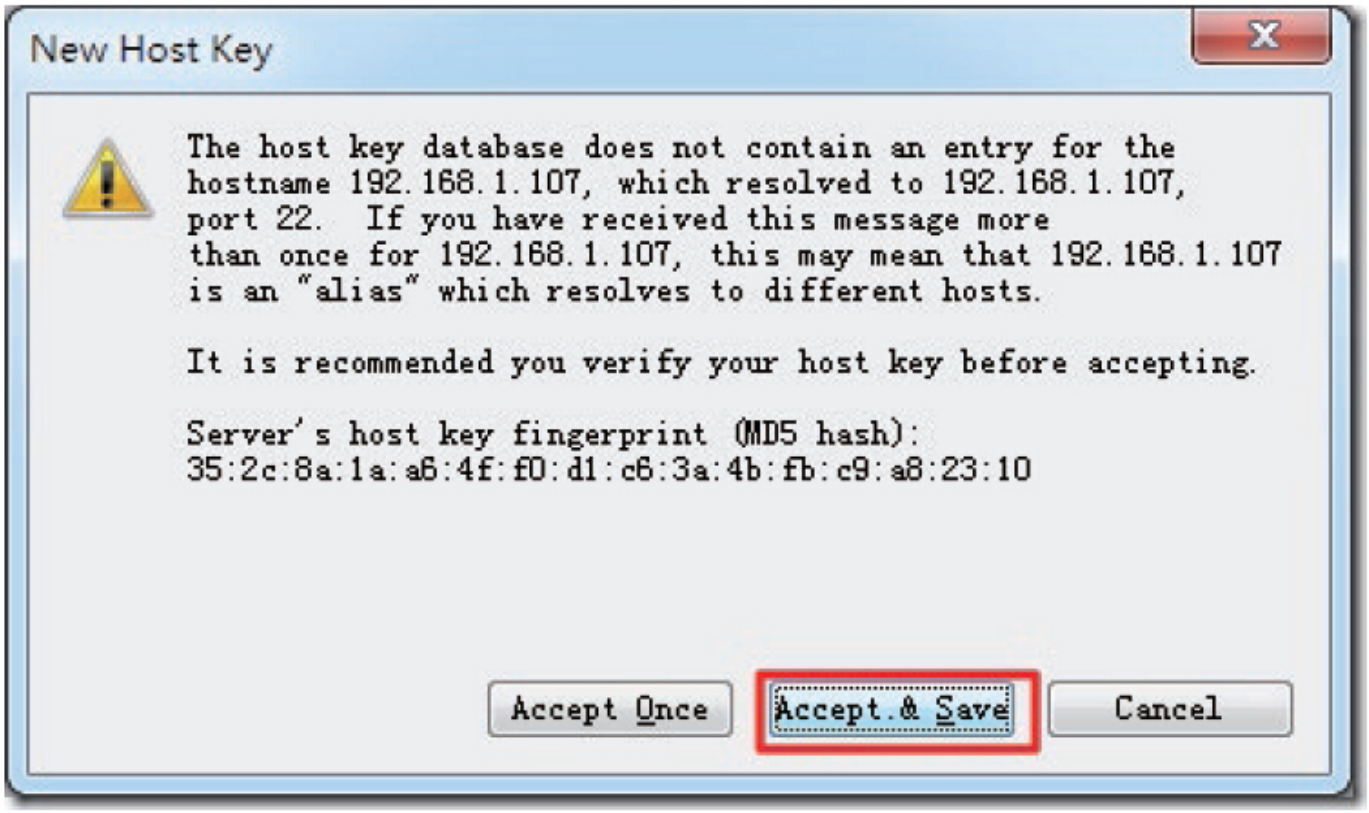


图7-10　保存SecureCRT的SSH2的连接配置

图7-11　输入pcDuino中Lubuntu的密码

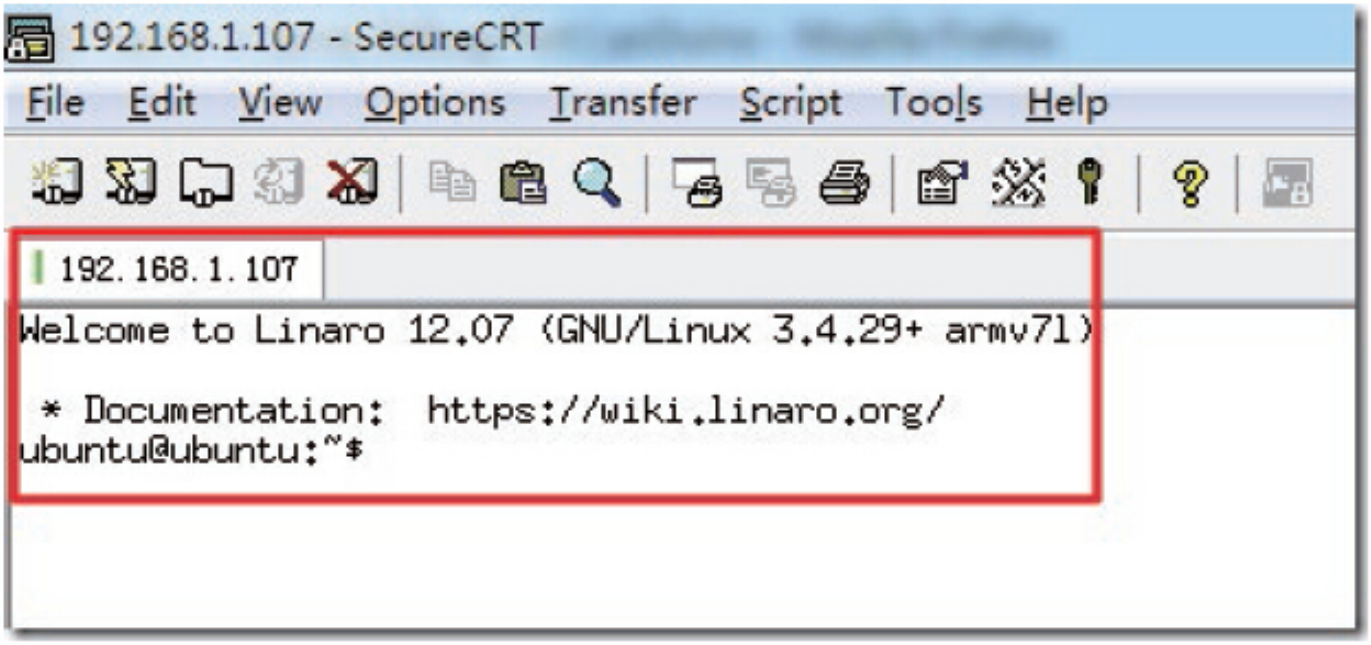


图7-12　用SecureCRT通过SSH2成功连接pcDuino的Lubuntu

至此已成功通过SSH2的方式连接pcDuino的Lubuntu系统，接下来读者根据自己的需求像操作普通的Ubuntu那样去操作即可。

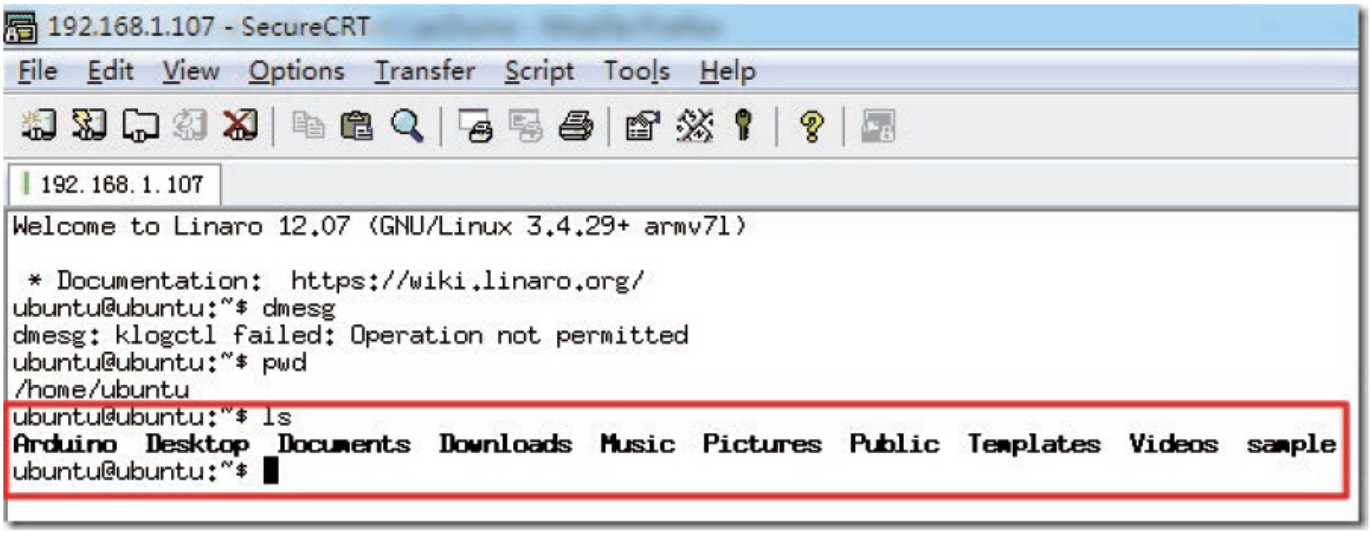


图7-13　通过SSH2在pcDuino的Lubuntu中执行ls

2.通过VNC操作pcDuino中的Lubuntu

除了使用SSH2连接pcDuino之外，还可以通过VNC连接pcDuino。下面介绍如何用VNC连接并操作pcDuino的Lubuntu系统。首先我们需要找个VNC的客户端软件，此处使用RealVNC的VNC Viewer，登录官网主页（地址：http://www.realvnc.com/）下载并安装VNC，如图7-14所示。

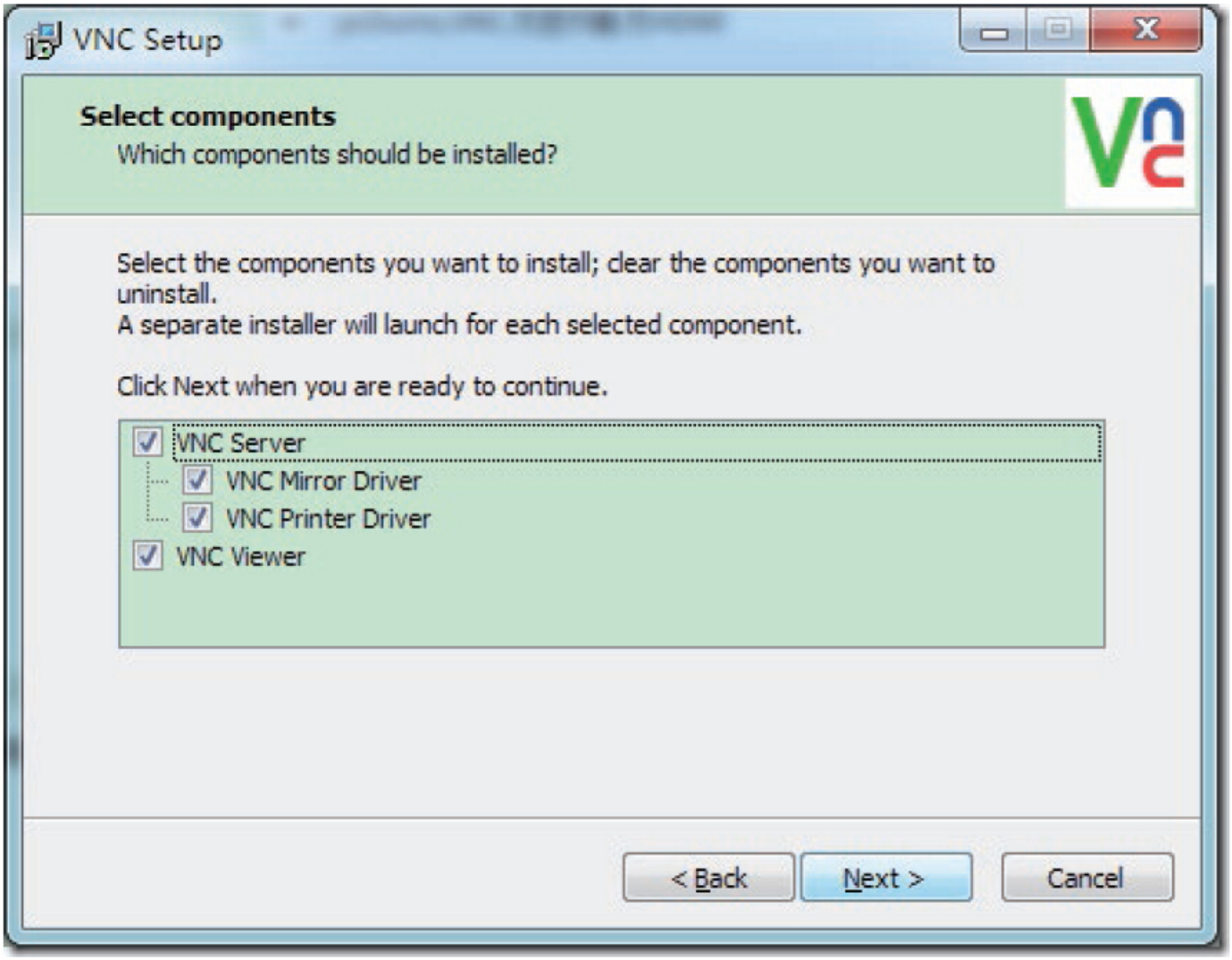


图7-14　安装VNC期间选择所要安装的功能

安装完毕后再启动VNC Viewer，然后在VNC Server中输入参数：192.168.1.107：5900，其中5900是VNC协议的默认端口地址。然后单击Connect按钮，如图7-15所示。

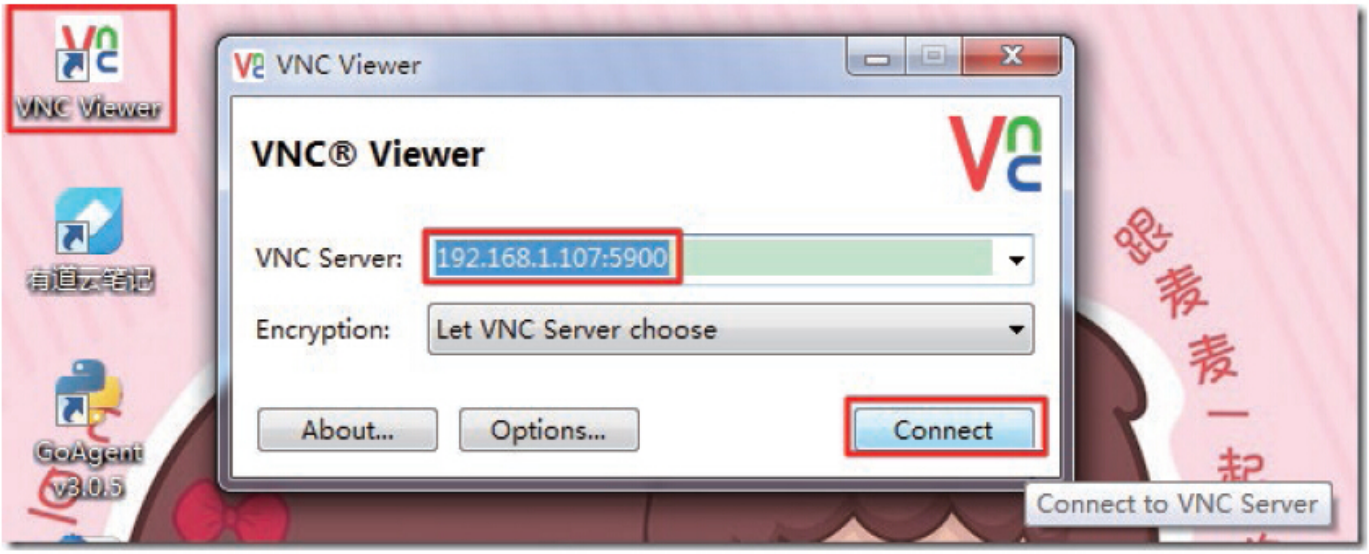


图7-15　在VNC Viewer中输入服务器地址并连接

这样就可以登录并连接到对应的Lubuntu系统中了，如图7-16所示。

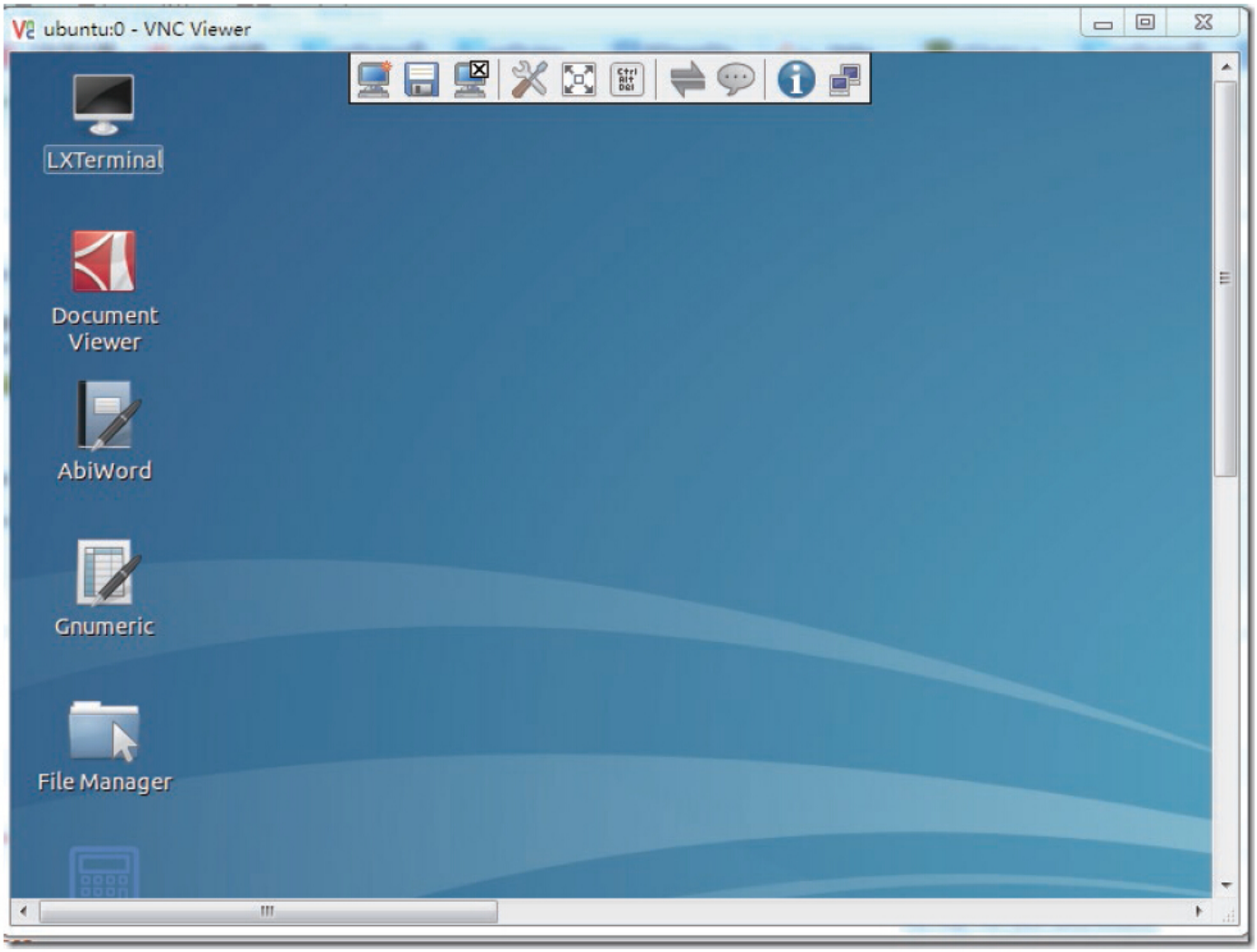


图7-16　通过VNC连接pcDuino的Lubuntu

确保pcDuino中的Lubuntu已经启动了图形桌面模式

在通过VNC方式连接Lubuntu期间需要注意的一点是，如果读者拿到pcDuino后使用VNC连接失败，如图7-17所示，那么需要读者确认一些配置是否正确。

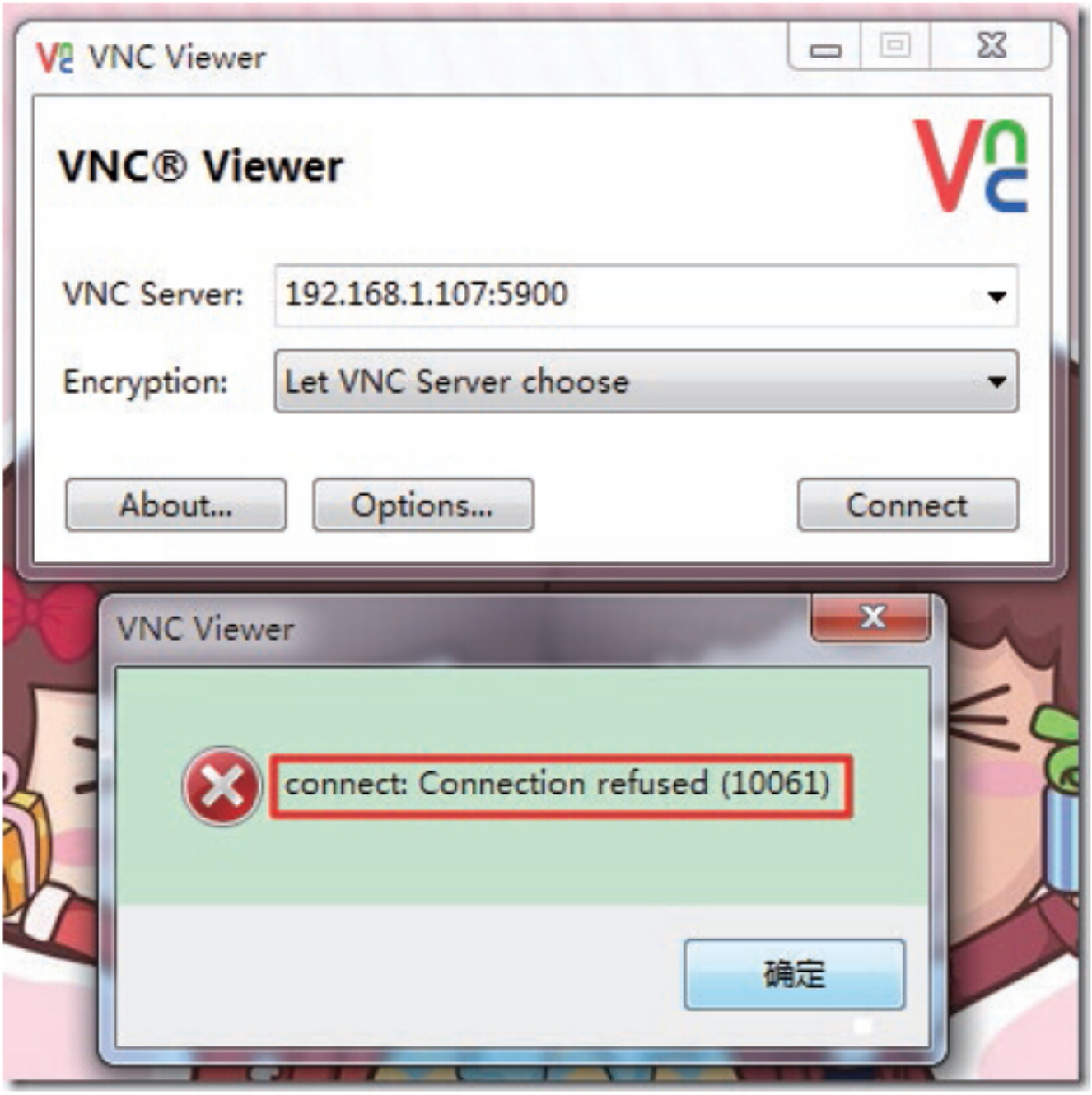


图7-17　VNC连接出错

（1）确保pcDuino启动后进入的是桌面模式

如果像笔者一样之前不小心修改了启动配置中的“Boot to cmdline only or desktop”为“command line”的话，则在连接VNC之前需要在SSH2中手动调用lightdm启动桌面，如图7-18所示，然后再用VNC连接才可以。

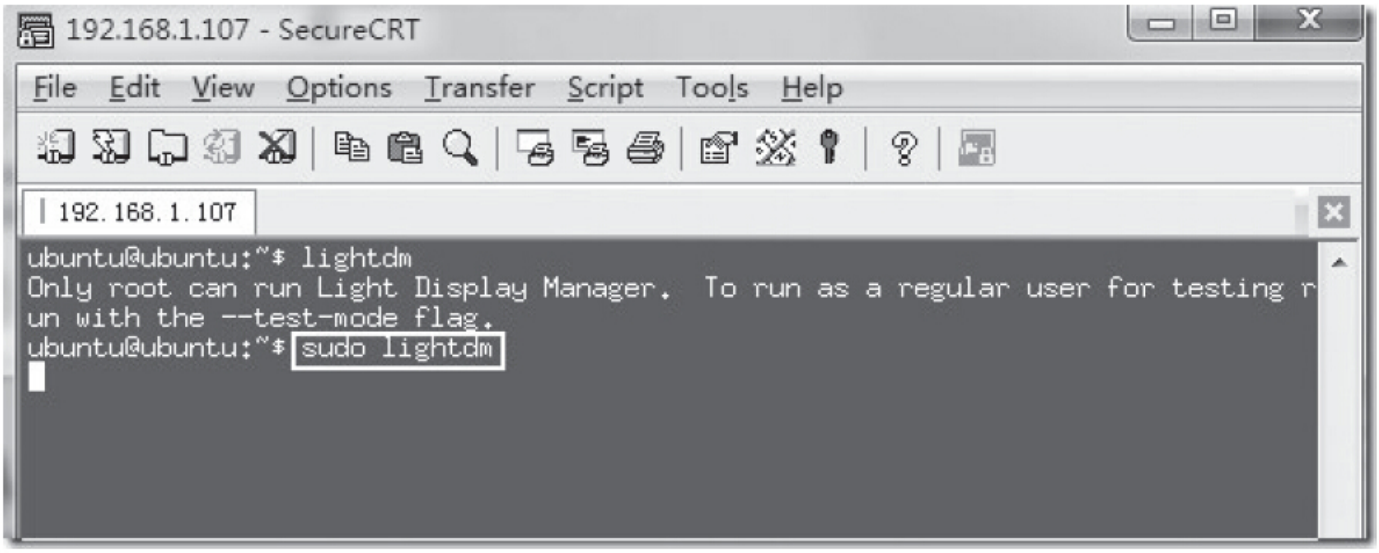


图7-18　通过SSH2运行pcDuino的Lubuntu中的lightdm

（2）确保VNC服务已正常运行

如果VNC服务没有运行，则需要启动VNC服务器，如图7-19所示。

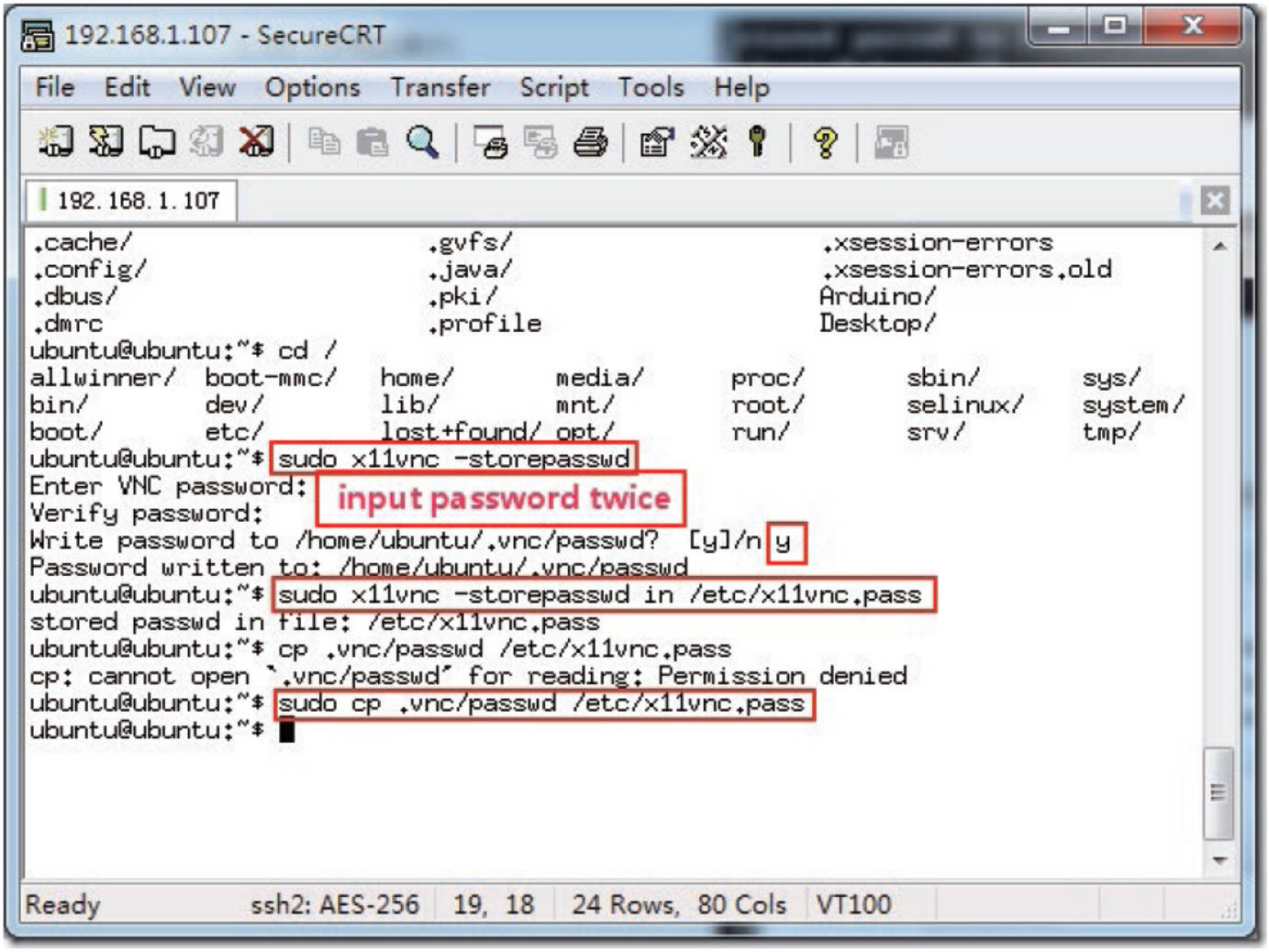


图7-19　通过SSH2启用pcDuino的Lubuntu中的x11vnc

然后再重启Lubuntu：

```
sudo reboot
```

最后用VNC连接就可以了。

（3）尝试手动启动VNC服务

如果仍是不行，可以试试手动启动VNC服务：

```
sudo x11vnc -forever -shared -rfbauth ~/.vnc/passwd
```

然后再运行VNC即可。

7.3 在开源硬件pcDuino上使用Python

下面通过各种不同实例来解释如何在pcDuino上使用Python实现各种功能。

7.3.1 Web服务器

1.功能背景与分析

接下来将通过例子来说明如何把pcDuino打造为一个Web服务器，如何在开源硬件pcDuino上使用Python。

2.物品清单

· pcDuino

用于实现对应的Web服务器的功能。

3.实现步骤与代码

通过前面的解释读者已经明白，其实此处想要在pcDuino中的Lubuntu中使用Python和普通的PC中使用Python是没什么区别的。不过可能对于从头到尾的每一步操作不是很清楚。下面就详细介绍如何从无到有的在pcDuino中的Lubuntu中用Python的第三方Web服务器框架Flask去搭建一个最简单的Web服务器。

先列出核心步骤：

· 在pcDuino的Lubuntu中安装flask

· 用Lubuntu中的文本编辑器写flask测试代码

· 运行测试flask的Python代码

· 在浏览器中确认Web服务器是否工作

接着再详细介绍每一步如何操作。在此先提醒一下flask的官网是http://flask.pocoo.org/，其中有很多值得参考的文档，有需要的读者可以去找官网资料学习和参考。

首先打开Lubuntu的终端去安装flask，如图7-20所示。

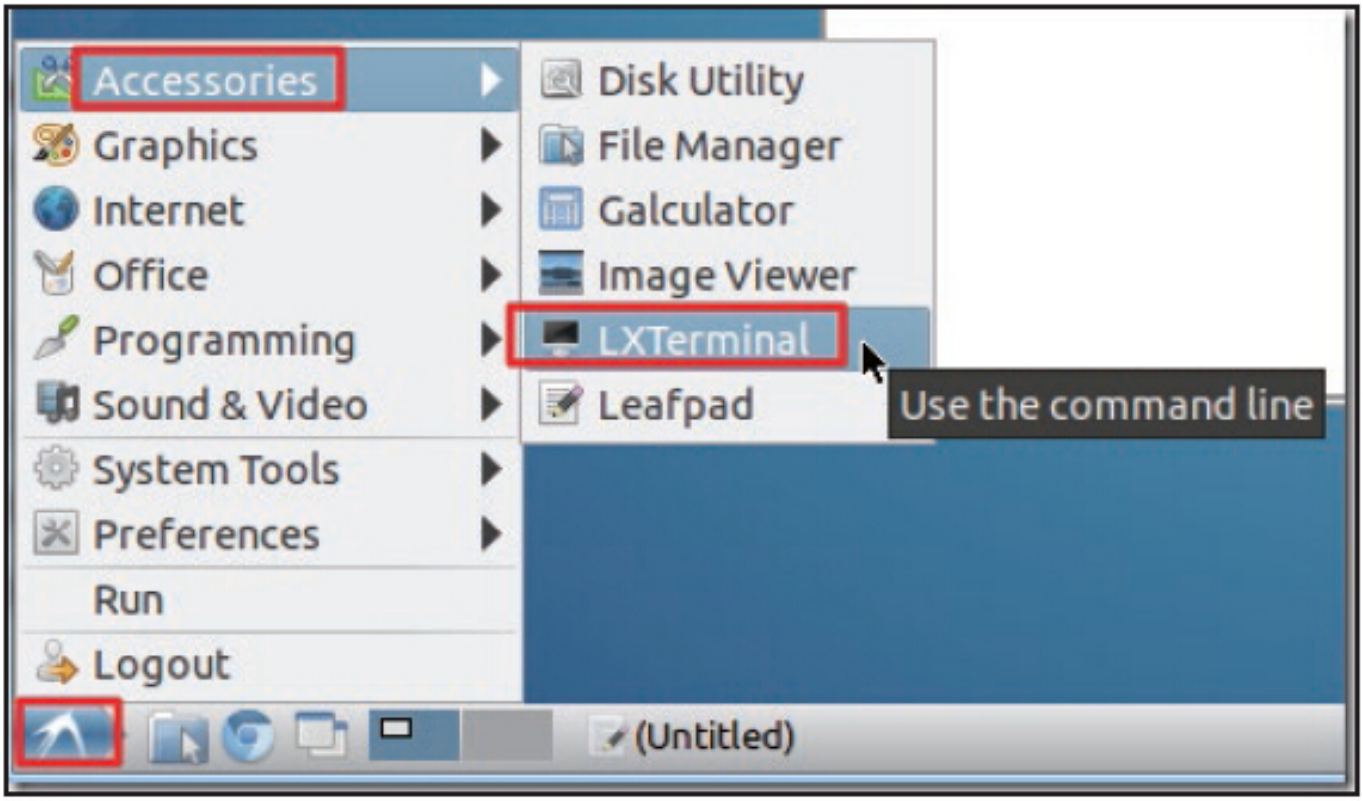


图7-20　Lubuntu中打开LXTerminal终端

再去终端中运行如下代码（如图7-21所示）。

```
sudo apt-get install python-pip
```

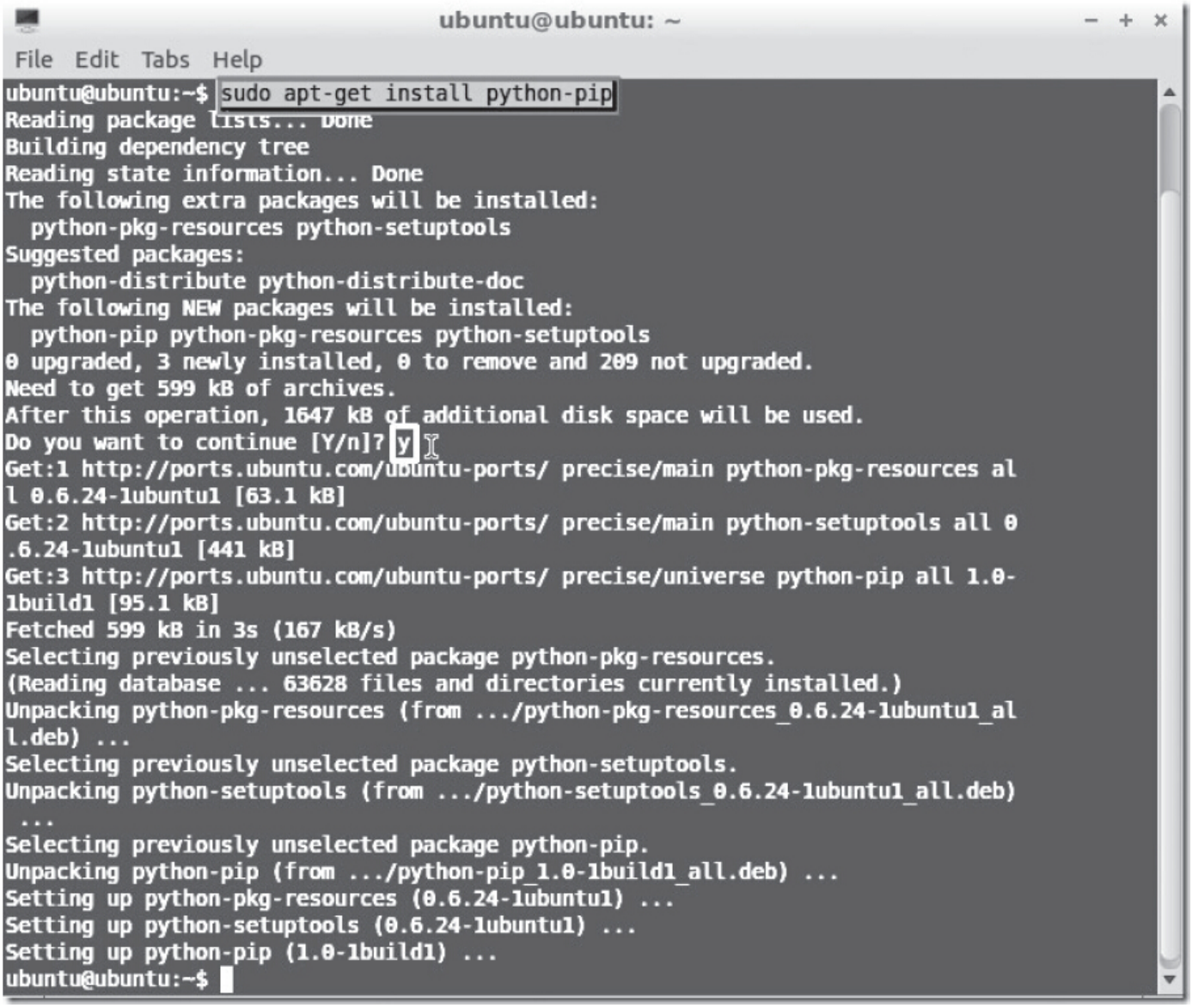


图7-21　Lubuntu中安装Python的pip

Python中的包管理工具：pip

pip是Python的一个包管理工具，用pip可以很方便地安装第三方的Python库。因为pip可以自动帮助下载并安装对应的第三方的库，且自动找到该库所依赖的其他的库，一并帮助用户一次性下载安装好，无需用户操作太多细节，极大地方便了Python的库的管理。

再通过pip安装flask，如图7-22所示。

```
sudo pip install flask
```

接着用Lubuntu中的文本编辑器Leafpad写flask测试代码。

先打开Lubuntu中的文本编辑器LeafPad，如图7-23所示。

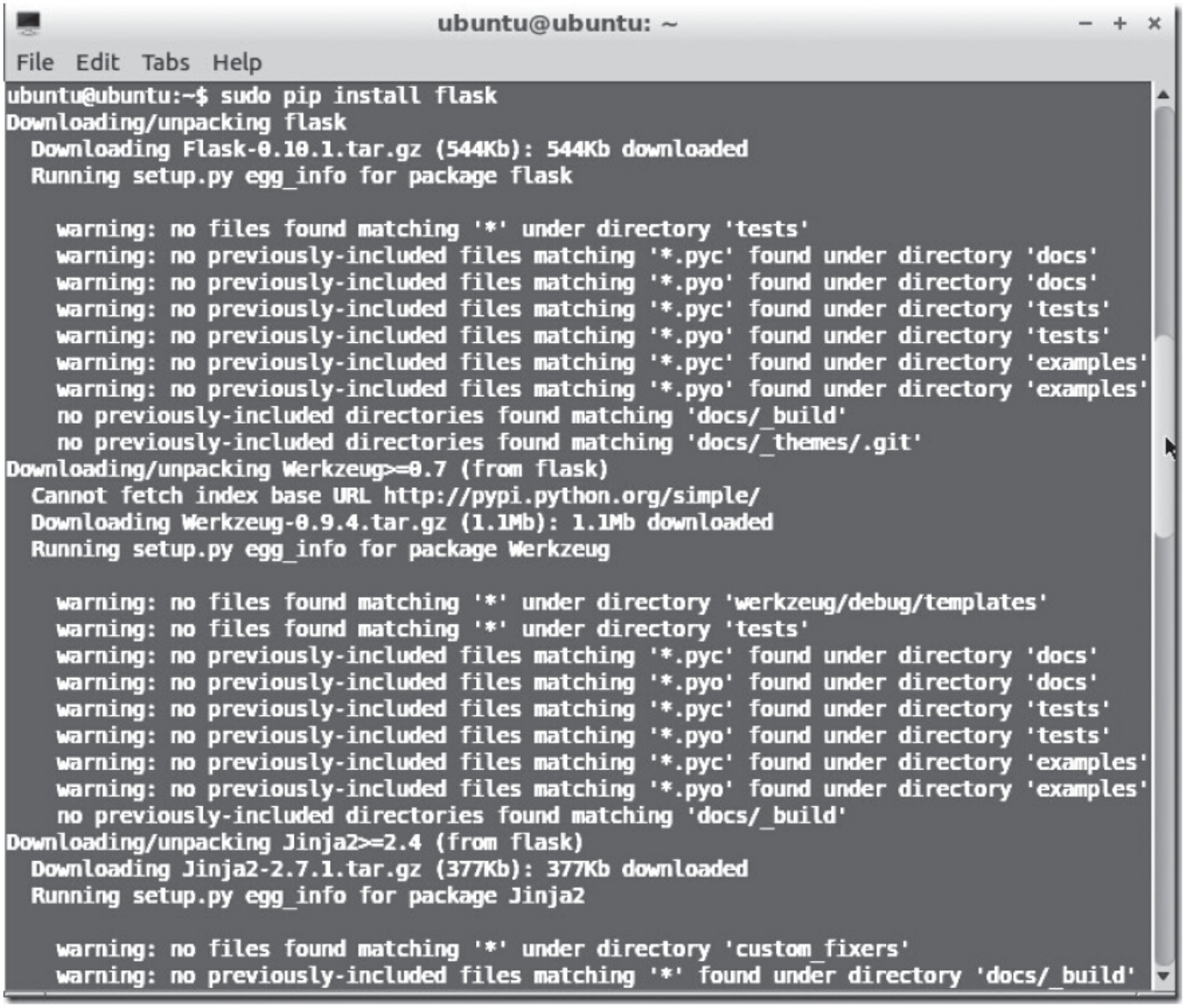


图7-22　Lubuntu中通过pip安装flask

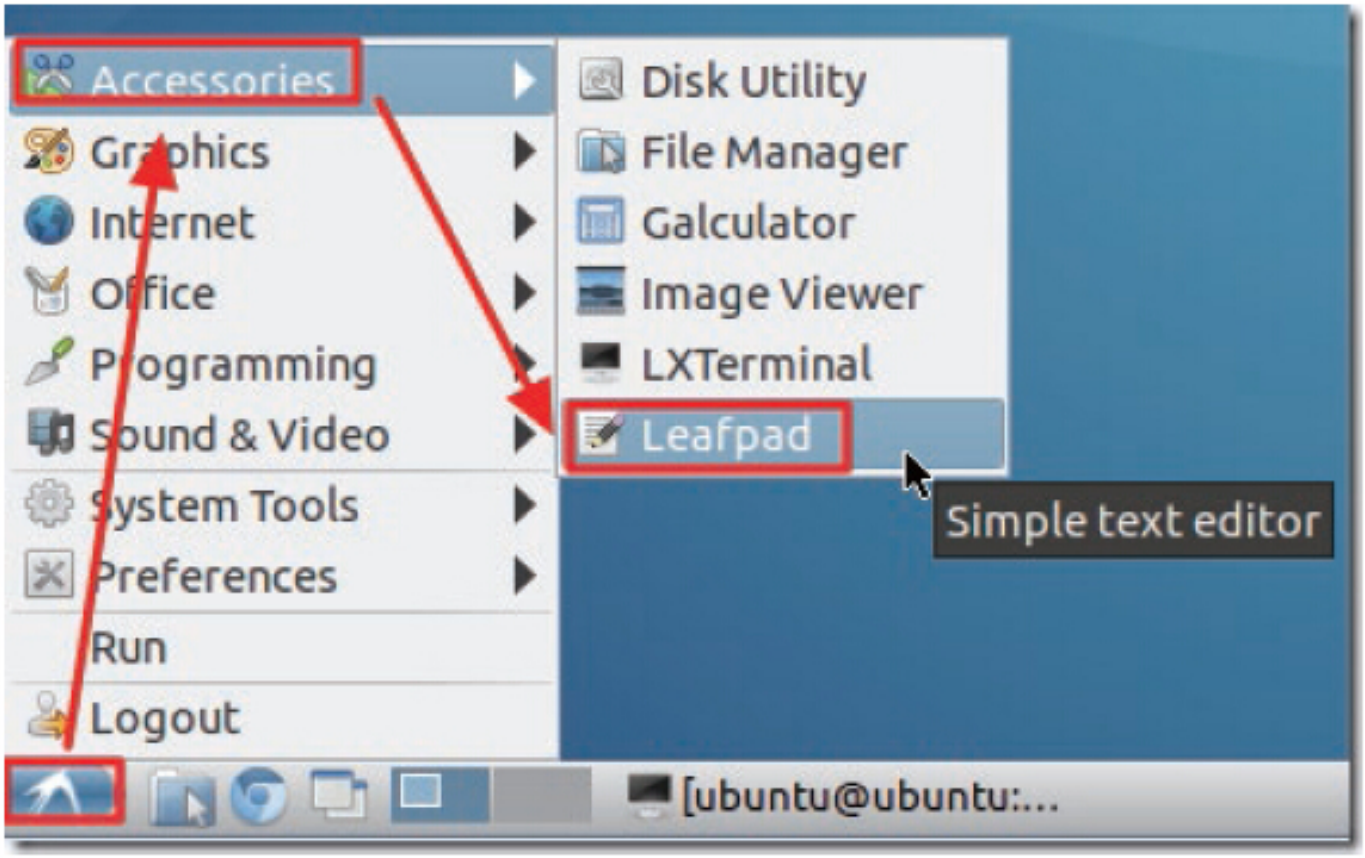


图7-23　Lubuntu中运行文本编辑器LeafPad

然后写入如下所示测试代码，输入后如图7-24所示。

```
from flask import Flask
app = Flask(__name__)
@app.route("/")
def hello():
    return "Demo webserver using python flask in Lubuntu pcDunino by Crifan Li"
if __name__ == "__main__":
    app.run(host='0.0.0.0', port=80, debug=True)
```

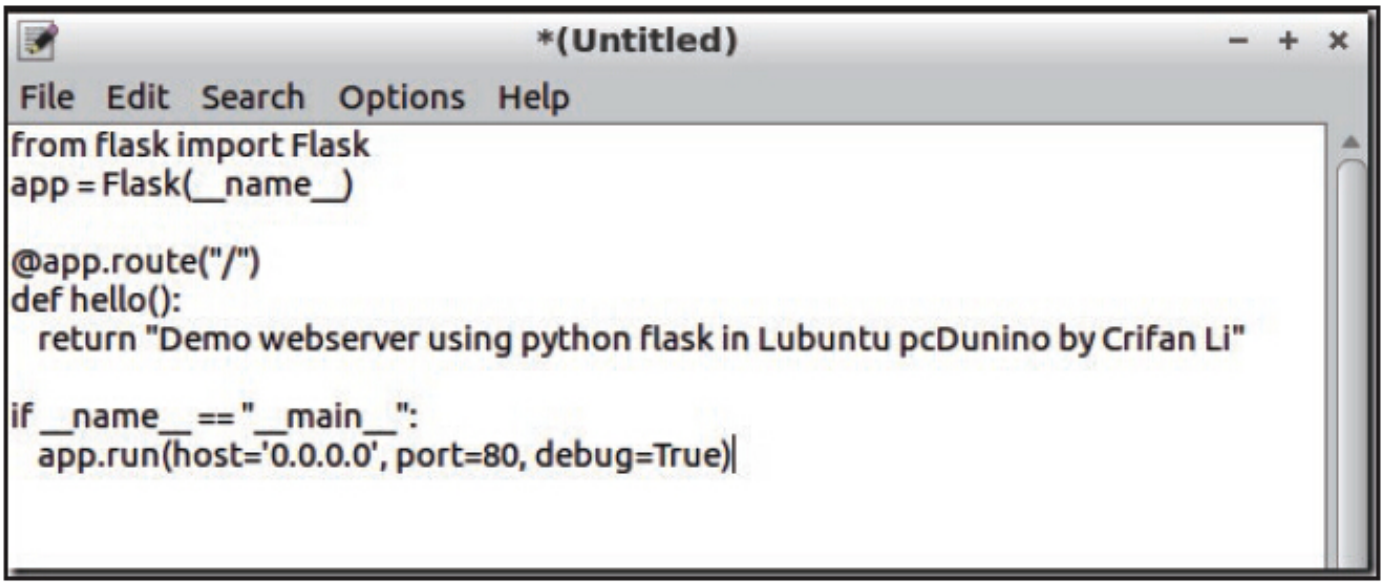


图7-24　LeafPad中的flask测试代码

再如上测试代码保存为demoWebserverViaFlask.py，如图7-25所示。

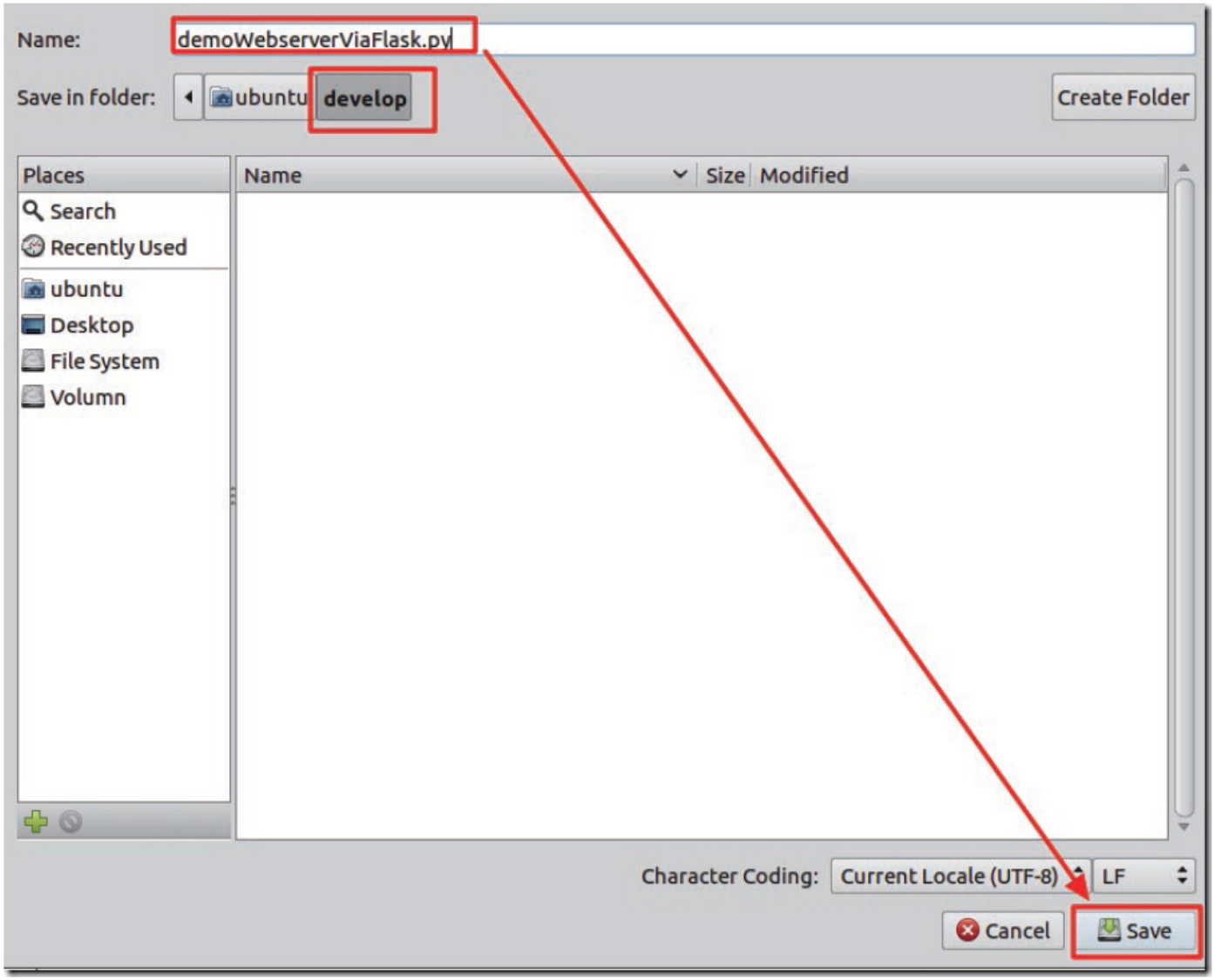


图7-25　保存LeafPad中的flask测试代码

到终端中切换到对应文件夹下以确认该文件存在，如图7-26所示。

```
ubuntu@ubuntu:~$ ls
2013-12-28-110454_1920x1080_scrot.png  Desktop    Music      Templates  sample
2013-12-28-110540_1920x1080_scrot.png  Documents  Pictures   Videos
Arduino                                Downloads  Public     develop
ubuntu@ubuntu:~$ cd develop/
ubuntu@ubuntu:~/develop$ ls
demoWebserverViaFlask.py
ubuntu@ubuntu:~/develop$ ls -l
total 4
-rw-rw-r-- 1 ubuntu ubuntu 207 Dec 29 07:17 demoWebserverViaFlask.py
ubuntu@ubuntu:~/develop$
```

图7-26　LXTerminal中切换到对应路径并确认文件存在

接着运行此测试flask的Python代码文件，如图7-27所示。

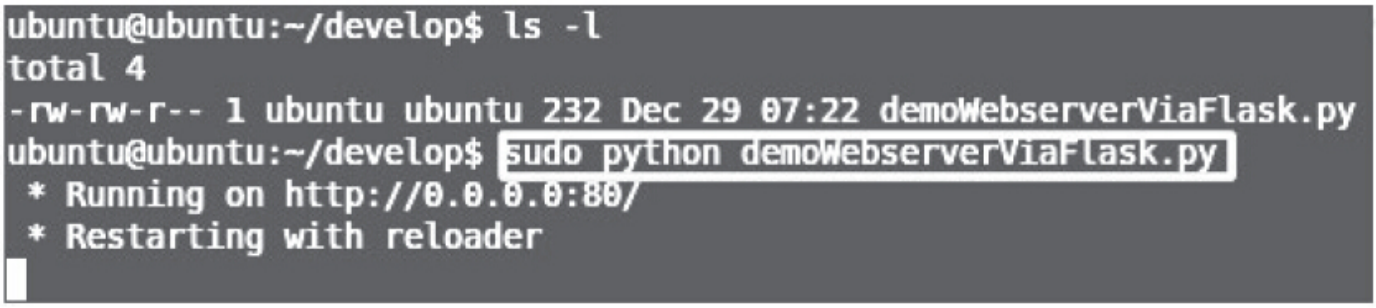


图7-27　Lubuntu中运行flask代码启动Web服务器

4.实验现象

接着需要去浏览器中确认此Web服务器是否正常运行了。不过在此之前需要先清楚作为Web服务器的pcDuino中的Lubuntu当前的IP地址是多少。虽然之前就已经知道了IP地址，但此处需要再通过ifconfig确认一下，如图7-28所示。

确认此处IP的确是192.168.1.107。然后再去主机端Windows7中使用浏览器打开对应的Web服务器的地址，查看该Web服务器是否已正常工作了，如图7-29所示。

由图7-29可见此处Web服务器的确已经正常工作，已经可以显示出对应的打印信息。在代码中用debug=True开启了flask的调试功能，同时经过多次访问页面，所以Lubuntu中的flask服务器端中可以看到页面访问所对应的调试信息，如图7-30所示。

至此，终于一步步实现了在pcDuino中的Lubuntu中使用Python的第三方库及Web服务器框架flask搭建一个简单的Web服务器。接下来读者就可以根据自己的需要在这类开源硬件上充分利用各种资源实现自己想要的功能了。

5.结果分析

实现此Web服务器的核心就是利用Python的服务器方面的库搭建一个Web服务器。

```
ubuntu@ubuntu: ~/develop
File  Edit  Tabs  Help
ubuntu@ub...  ×  ubuntu@ub...  ×
ubuntu@ubuntu:~/develop$ ifconfig
eth0      Link encap:Ethernet  HWaddr 2a:4f:4b:57:81:9a
          inet addr:192.168.1.107  Bcast:192.168.1.255  Mask:255.255.255.0
          inet6 addr: fe80::284f:4bff:fe57:819a/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:89126 errors:0 dropped:0 overruns:0 frame:0
          TX packets:109786 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:8716162 (8.7 MB)  TX bytes:113992884 (113.9 MB)
          Interrupt:55 Base address:0xb000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:164 errors:0 dropped:0 overruns:0 frame:0
          TX packets:164 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:25894 (25.8 KB)  TX bytes:25894 (25.8 KB)

wlan3     Link encap:Ethernet  HWaddr 00:79:03:01:17:77
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:949 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

ubuntu@ubuntu:~/develop$
```

图7-28　用ifconfig确认pcDuino的Lubuntu的IP

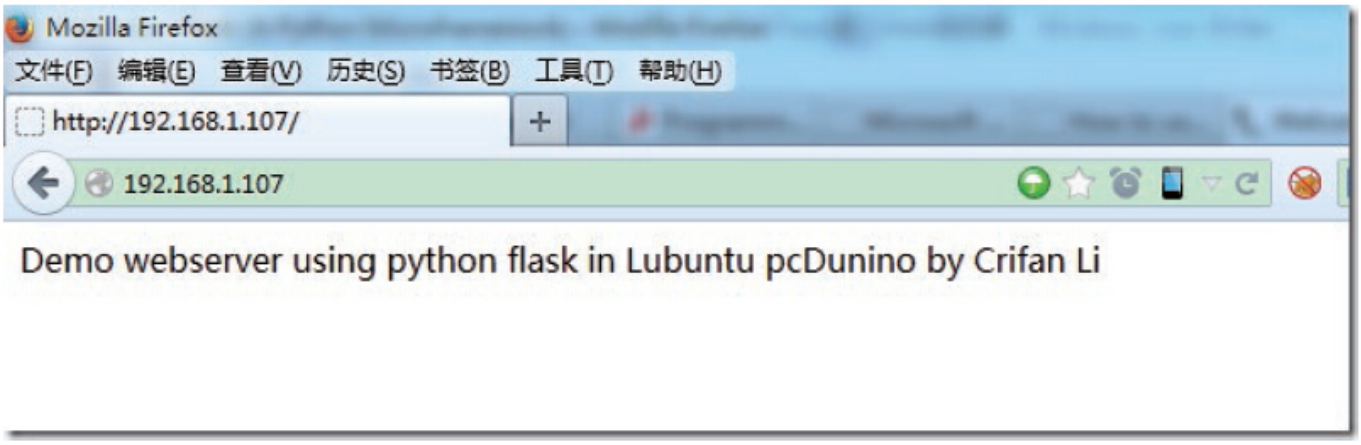


图7-29　用浏览器访问Web服务器地址

```
ubuntu@ubuntu:~/develop$ sudo python demoWebserverViaFlask.py
 * Running on http://0.0.0.0:80/
 * Restarting with reloader
192.168.1.100 - - [29/Dec/2013 07:28:34] "GET / HTTP/1.1" 200 -
192.168.1.100 - - [29/Dec/2013 07:28:34] "GET /favicon.ico HTTP/1.1" 404 -
192.168.1.100 - - [29/Dec/2013 07:28:34] "GET /favicon.ico HTTP/1.1" 404 -
192.168.1.100 - - [29/Dec/2013 07:28:34] "GET /favicon.ico HTTP/1.1" 404 -
192.168.1.100 - - [29/Dec/2013 07:30:15] "GET / HTTP/1.1" 200 -
```

图7-30　终端中可以看到flask的调试信息

7.3.2　漏水监测

1.功能背景与分析

接下来介绍一个Python搭配pcDuino一起使用例子，即实现通过湿度传感器检测洗衣机等物品漏水后，通过Python发送通知邮件的功能。

2.物品清单

· pcDuino

· 湿度传感器

此处用的是DFRobot的湿度传感器，它有三个引脚：GND、VCC和S。

· 洗衣机

用于测试漏水检测邮件通知是否工作正常。

3.实现步骤与代码

先进行物理上的硬件连接：分别将湿度传感器的S脚与pcDuino的A2脚、GND脚与pcDuino的GND脚、VCC脚与pcDuino的5V相连，如图7-31所示。

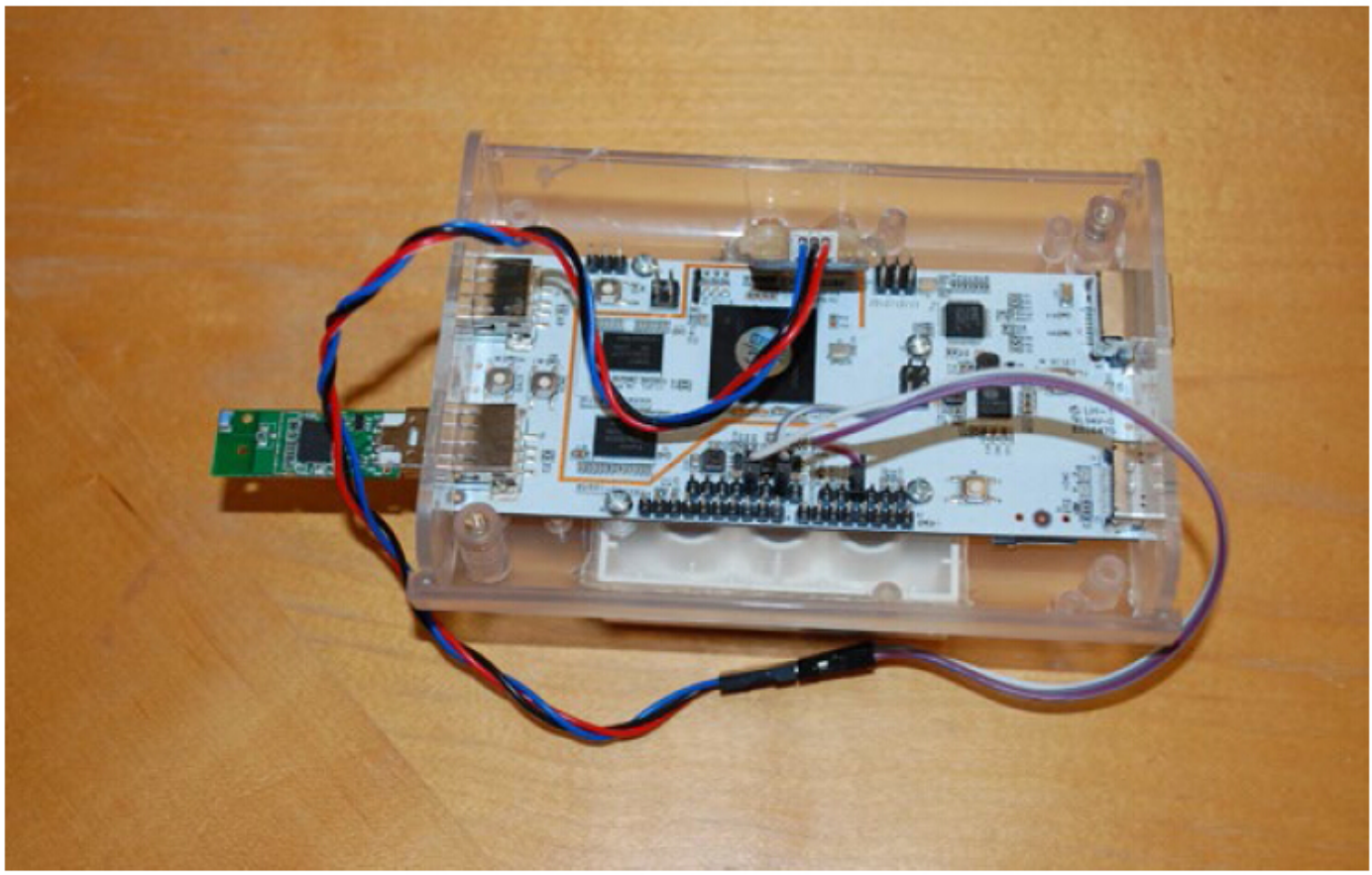


图7-31　pcDuino和湿度传感器物理连接图

然后再来进行软件上的代码操作和真正物理上的安装和使用。

（1）下载Python库

用下面的命令去下载pcDuino的Python库：

```
$git clone https://github.com/pcduino/python-pcduino.git
```

（2）写实现发邮件功能的Python代码

下载了Python库之后就可看到在pcDuino文件夹下面有对应的各种库文件，然后再在Samples下面创建一个名为waterdetect的文件夹，并添加如下的代码到waterdetect.py中。

```
#!/usr/bin/python
#-*- coding: utf-8 -*-
import smtplib
import time
from adc import analog_read
server= 'smtp.gmail.com'
port = 587
sender = 'xxxxx@gmail.com'
recipient = 'yyyy@linksprite.com'
password='xxxx'
subject = 'Water Leakage Detected'
body = 'Water detected at basement water heater 1.'
def delay(ms):
    time.sleep(1.0*ms/1000)
def setup():
        print "read channel ADC2 value ,the V-REF = 3.3V"
        delay(3000)
body = "" + body + ""
headers = ["From: " + sender,
          "Subject: " + subject,
          "To: " + recipient,
          "MIME-Version: 1.0",
          "Content-Type: text/html"]
headers = "\r\n".join(headers)
def loop():
    while(1):
        value = analog_read(2)
    voltage = (value * 3.3)/4096
        print ("value =  %4d"%value)
    print ("voltage =  %4.3f  V" %voltage)
        if voltage > 3.0 :
              session = smtplib.SMTP(server, port)
              session.ehlo()
              session.starttls()
              session.ehlo
              session.login(sender, password)
              session.sendmail(sender, recipient, headers + "\r\n\r\n" + body)
              session.quit()
    delay(1000)
def main():
    setup()
    loop()
main()
```

从如上代码中可见在系统启动时会先去配置相关内容，然后再运行对应的守护进程用于监测变化。

（3）创建对应的启动脚本

为了让程序在Linux系统启动时就运行，需要去创建对应的启动脚本，并放到etc/init.d/中，代码如下。

```
#!/bin/sh -e
DAEMON="/home/ubuntu/python-pcduino/Samples/waterdetect/waterdetect.py"
DAEMONUSER="ubuntu"
DEAMON_NAME="waterdetect.py"
PATH="/sbin:/bin:/usr/sbin:/usr/bin"
test -x $DAEMON || exit 0
```

```
. /lib/lsb/init-functions
d_start () {
        log_daemon_msg "Starting system $DEAMON_NAME Daemon"
        start-stop-daemon --background --name $DEAMON_NAME --start --user $DAEMONUSER --exec $DAEMON
        log_end_msg $
}
d_stop () {
        log_daemon_msg "Stopping system $DEAMON_NAME Daemon"
        start-stop-daemon --name $DEAMON_NAME --stop --retry 5 --name $DEAMON_NAME
          log_end_msg $
}
case "$1" in
        start|stop)
                d_${1}
                ;;
        restart|reload|force-reload)
                        d_stop
                        d_start
                ;;
        force-stop)
               d_stop
                killall -q $DEAMON_NAME || true
                sleep 2
                killall -q -9 $DEAMON_NAME || true
                ;;
        status)
               status_of_proc "$DEAMON_NAME" "$DAEMON" "system-wide $DEAMON_NAME" && exit 0 || exit $
                ;;
        *)
                echo "Usage: /etc/init.d/$DEAMON_NAME
{start|stop|force-stop|restart|reload|force-reload|status}"
                exit 1
                ;;
esac
exit 0
```

（4）添加脚本运行权限并运行或停止

同时不要忘记为启动文件添加对应的执行权限：

```
$ sudo chmod 755 /etc/init.d/waterdetect
$ sudo chmod 755
/home/ubuntu/python-pcduino/Samples/waterdetect/waterdetect.py
```

接着就可以使用如下命令去启动或停止对应服务了。

```
$ sudo service waterdetect start
 * Starting system waterdetect.py Daemon                          [ OK ]
$ sudo service waterdetect status
 * /home/ubuntu/python-pcduino/Samples/waterdetect/waterdetect.py is running
$ sudo service waterdetect stop
 * Stopping system waterdetect.py Daemon                          [ OK ]
```

其中，是使用如下代码将其设置为在后台运行的服务的。

```
$ sudo update-rc.d waterdetect defaults
```

4.实验现象

将“漏水检测器”安放到洗衣机边，如图7-32所示。

当洗衣机在洗衣服时溅出水时，此处的漏水检测器检测出有足够的湿度后用户就可以收到对应的邮件通知了，如图7-33所示。

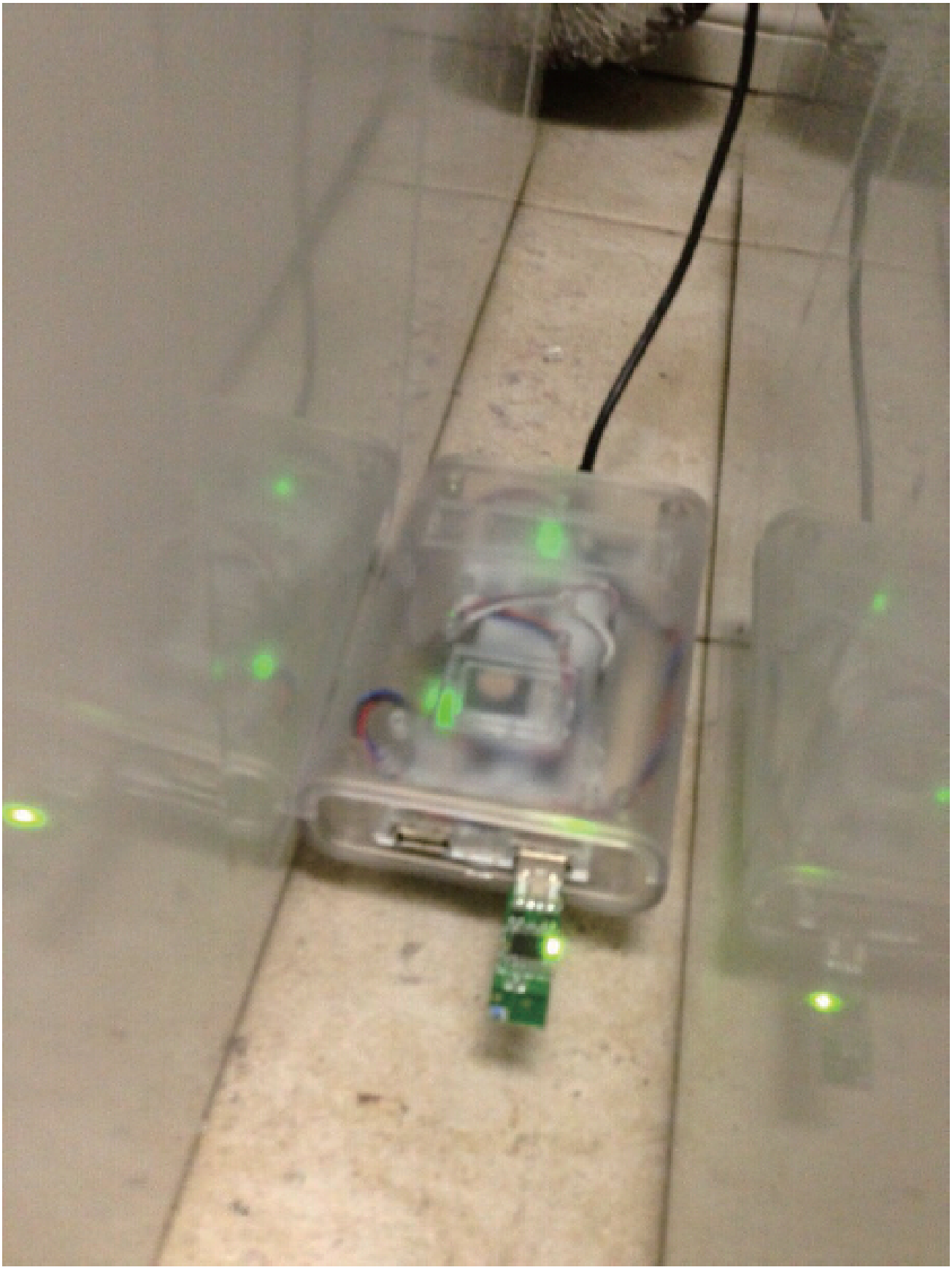


图7-32　洗衣机边的漏水检测器

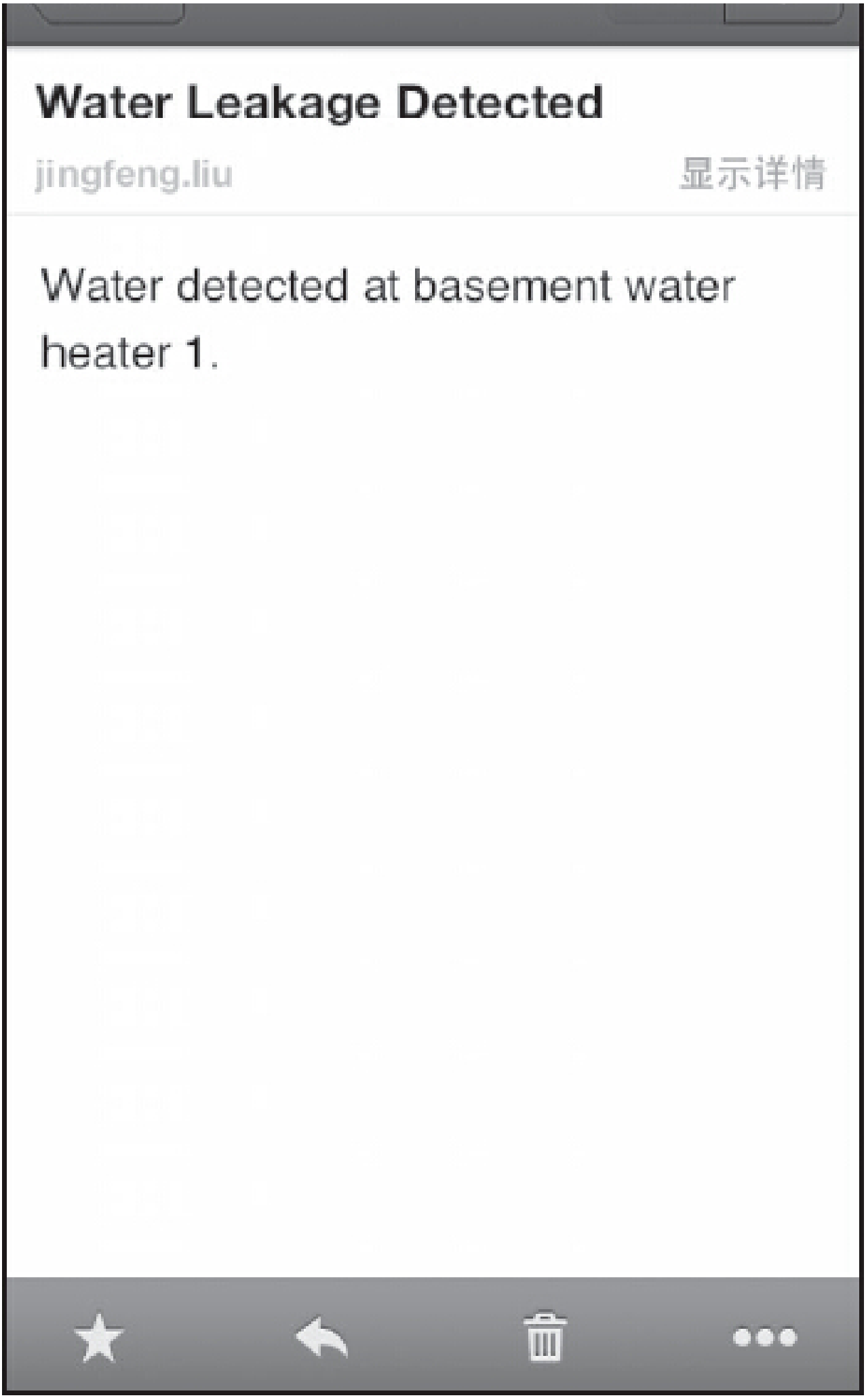

Water Leakage Detected
jingfeng.liu
显示详情
Water detected at basement water heater 1.

图7-33　收到漏水的邮件通知

如此就通过Python搭配pcDuino实现了当水泄漏后可以检测并发送邮件通知的功能了。

5.结果分析

实现此功能的核心是外接湿度传感器检测外部湿度，当pcDuino中运行的守护进程检测到湿度超过一定限度后，再调用Python去发送邮件实现通知的效果。

7.3.3　使用Z-Wave实现智能家居

1.功能背景与分析

经常开车的朋友可能会遇到下面这种情况：从车库开车出去后，但是忘记是否关了车库的门，希望可以做到，如果得知车库门没有关上，支持远程控制然后将车库的门关上。

下面我们就来通过pcDuino实现一个智能的车库，可以通过pcDuino+Python实现通过邮件得知当前车库门的状态，并且支持通过邮件发送指令去关闭车库的门。

想要实现此功能的话需要注意到一个细节，铰链离控制单元的远近可表明车库的门是何种状态：打开还是关闭，如图7-34所示。

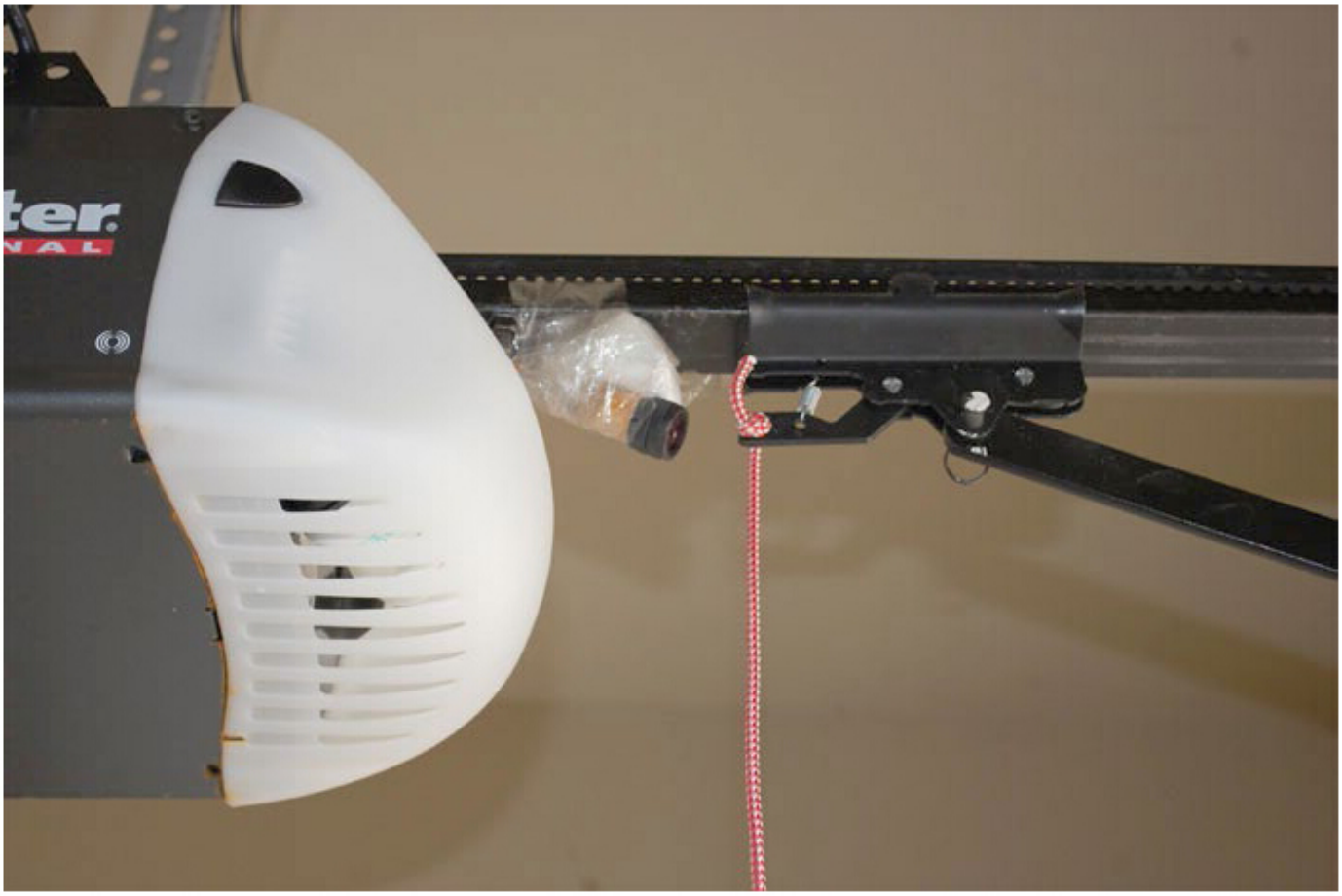


图7-34　铰链离的远近说明了车库门是否开关

2.物品清单

· pcDuino

· 车库

· LiftMaster车库门控制器

用于控制车库门的开和关。

· Linker中继模块

用于控制LiftMaster。

· Wi-Fi的USB电子狗

支持远程无线控制门铰链的开和关。

· 红外检测仪

红外检测仪在正常情况下输出高电平信号，当检测物理靠近时输出低电平信号，如图7-35所示。

3.实现步骤与代码

先将车库以及相关控制设备安装好，如图7-36所示。

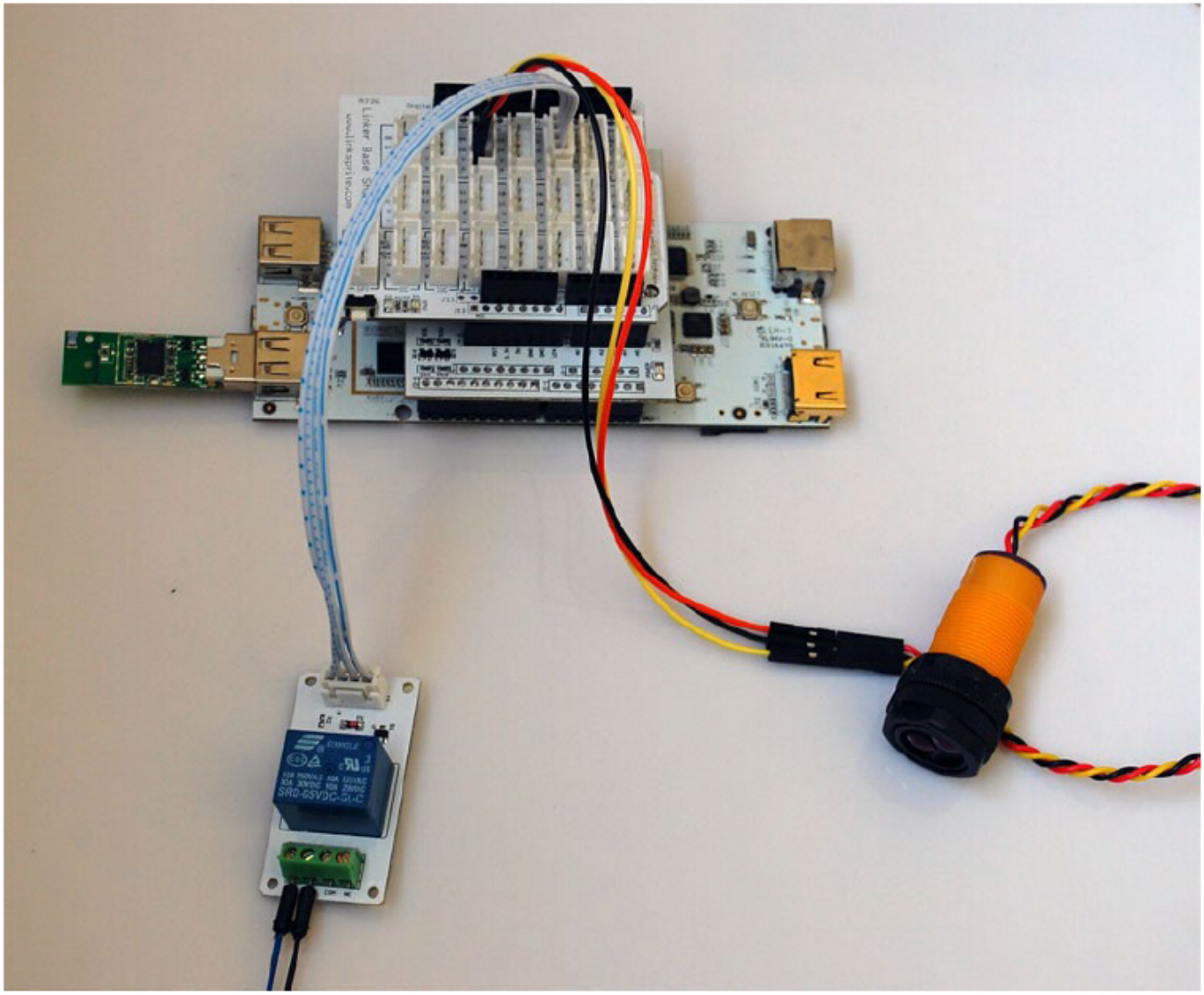


图7-35　CuteDigi的橘黄色的红外检测仪

图7-36　加装了控制设备的车库

然后将红外检测仪接到pcDuino的D2引脚上，将Linker中继模块接到pcDuino的D4引脚，再将pcDuino安装到控制设备上，如图7-37所示。

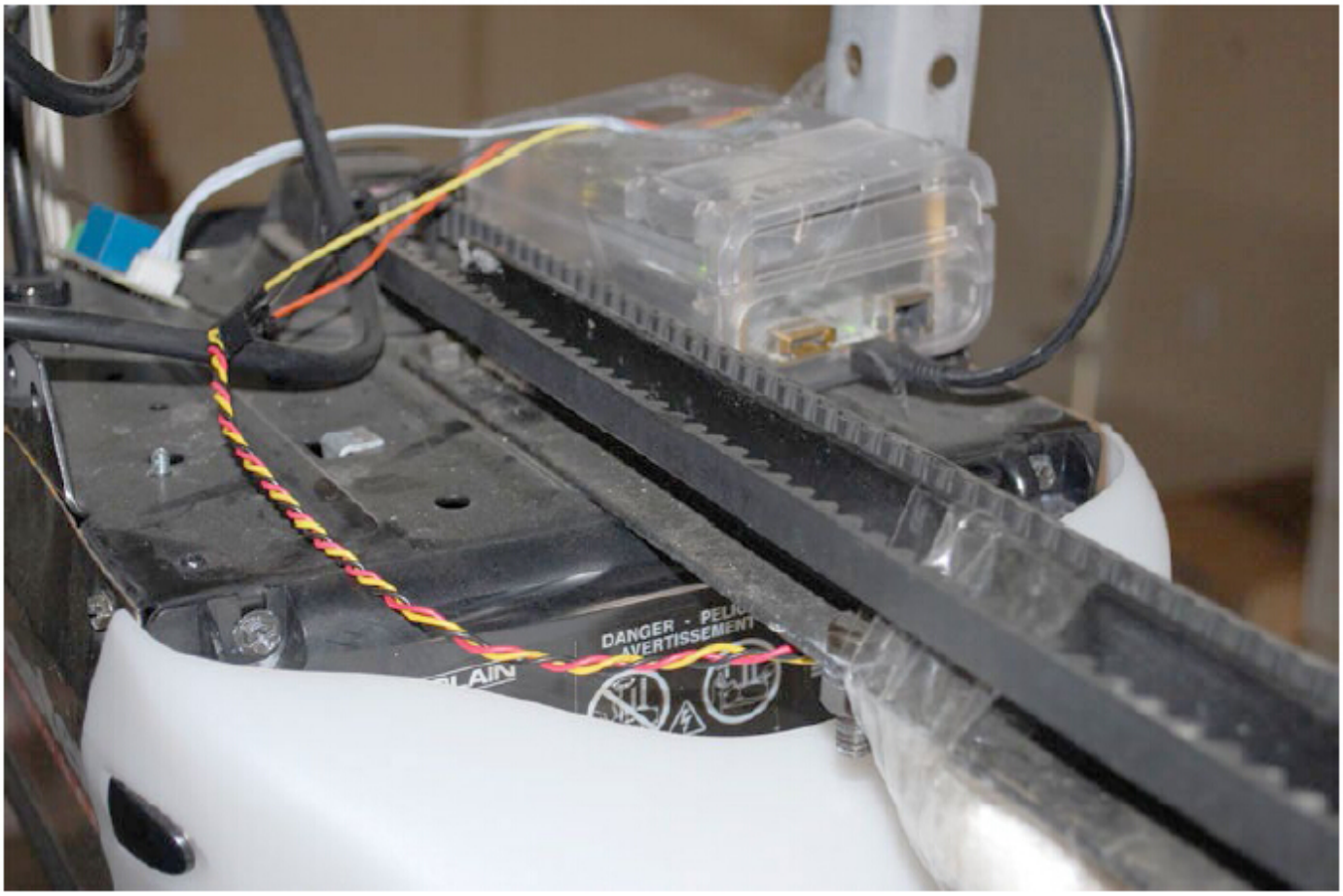


图7-37　pcDuino已安装到车库门控制设备上

接着写控制代码：

```
#!/usr/bin/python
#-*- coding: utf-8 -*-
import smtplib
import time
from adc import analog_read
import gpio
import time
import imaplib
import uuid
from email import email
import imaplib
import time
import uuid
from email import email
relay_pin = "gpio2"
sensor_pin="gpio4"
#the following is for sending email
server= 'smtp.gmail.com'
port = 587
sender = 'xxxxxx@gmail.com'
recipient = 'xxxxx@linksprite.com'
password='xxxxx'
subject = 'Garage Door Opened'
body = 'The garage door is open. Please check'
#the following is for receiving email
IMAP_SERVER = 'imap.gmail.com'
IMAP_PORT = '993'
IMAP_USE_SSL = True
imap_username = 'xxxxx@gmail.com'
imap_password = 'xxxxx'
num=0
message_content='1';
def delay(ms):
    time.sleep(1.0*ms/1000)
def setup():
    gpio.pinMode(relay_pin, gpio.OUTPUT)
    gpio.pinMode(sensor_pin, gpio.INPUT)
    gpio.digitalWrite(relay_pin, gpio.LOW)
body = "" + body + ""
headers = ["From: " + sender,
           "Subject: " + subject,
           "To: " + recipient,
           "MIME-Version: 1.0",
           "Content-Type: text/html"]
headers = "\r\n".join(headers)
replysubject = 'I closed Garage Door'
replybody = 'I closed Garage Door.'
replybody = "" + replybody + ""
replyheaders = ["From: " + sender,
           "Subject: " + replysubject,
           "To: " + recipient,
           "MIME-Version: 1.0",
           "Content-Type: text/html"]
replyheaders = "\r\n".join(replyheaders)
class MailBox(object):
    def __init__(self, user, password):
        self.user = user
        self.password = password
        if IMAP_USE_SSL:
            self.imap = imaplib.IMAP4_SSL(IMAP_SERVER, IMAP_PORT)
        else:
            self.imap = imaplib.IMAP4(IMAP_SERVER, IMAP_PORT)
    def __enter__(self):
        self.imap.login(self.user, self.password)
        return self
    def __exit__(self, type, value, traceback):
        self.imap.close()
        self.imap.logout()
    def get_count(self):
        self.imap.select('Inbox')
        status, data = self.imap.search(None, 'UnSeen')
        return sum(1 for num in data[0].split())
    def fetch_message(self, num):
        self.imap.select('Inbox')
        status, data = self.imap.fetch(str(num), '(RFC822)')
        email_msg = email.message_from_string(data[0][1])
        return email_msg
    def delete_message(self, num):
        self.imap.select('Inbox')
        self.imap.store(num, '+FLAGS', r'\Deleted')
        self.imap.expunge()
```

```
    def print_msgs(self):
        self.imap.select('Inbox')
        status, data = self.imap.search(None, 'UnSeen')
        #for num in reversed(data[0].split()):
        status, data = self.imap.fetch(num, '(RFC822)')
        self.imap.store(num, '+FLAGS','\\Deleted')
        msg=email.message_from_string(data[0][1])
        message_content=msg.get_all("Subject")
        if  'close garage door' in message_content :
              return 1
    def get_latest_email_sent_to(self, email_address, timeout=300, poll=1):
        start_time = time.time()
        while ((time.time() - start_time) < timeout):
            # It's no use continuing until we've successfully selected
            # the inbox. And if we don't select it on each iteration
            # before searching, we get intermittent failures.
            status, data = self.imap.select('Inbox')
            if status != 'OK':
                 time.sleep(poll)
                 continue
            status, data = self.imap.search(None, 'TO', email_address)
            data = [d for d in data if d is not None]
            if status == 'OK' and data:
                 for num in reversed(data[0].split()):
                    status, data = self.imap.fetch(num, '(RFC822)')
                    email_msg = email.message_from_string(data[0][1])
                    return email_msg
            time.sleep(poll)
        raise AssertionError("No email sent to '%s' found in inbox "
             "after polling for %s seconds." % (email_address, timeout))
    def delete_msgs_sent_to(self, email_address):
        self.imap.select('Inbox')
        status, data = self.imap.search(None, 'TO', email_address)
        if status == 'OK':
             for num in reversed(data[0].split()):
                status, data = self.imap.fetch(num, '(RFC822)')
                self.imap.store(num, '+FLAGS', r'\Deleted')
        self.imap.expunge()
def closedoor() :
        gpio.digitalWrite(relay_pin, gpio.HIGH)
        delay(200)
        gpio.digitalWrite(relay_pin, gpio.LOW)
if __name__ == '__main__':
    setup()
    while(1):
        if gpio.digitalRead(sensor_pin) == gpio.LOW  :
              print("Sensor pin is LOW")
              session = smtplib.SMTP(server, port)
              session.ehlo()
              session.starttls()
              session.ehlo
              session.login(sender, password)
              session.sendmail(sender, recipient, headers + "\r\n\r\n" + body)
              session.quit()
    delay(10)
        with MailBox(imap_username, imap_password) as mbox:
                 num=mbox.get_count()
                 if num !=0 :
                        if  mbox.print_msgs()  :
                              print 'turn on'
                              closedoor()
                              session = smtplib.SMTP(server, port)
                     session.ehlo()
                     session.starttls()
                              session.ehlo
                              session.login(sender, password)
                              session.sendmail(sender, recipient, replyheaders + "\r\n\r\n" + replybody)
                              session.quit()
```

参考7.3.2节的内容将上述代码加入守护进程，使得在pcDunino启动时就开始运行，从而起到监控的作用。

4.实验现象

如此就可以通过发送邮件得知当前的车库门是否关了，并且可以通过发送邮件由LiftMaster控制车库门的开和关。

5.结果分析

此功能的核心在于要有对应的物理实体和控制设备，并且搭建完整硬件环境，然后才可以通过写软件代码去实际操控这些硬件实现控制车库门的开关的功能。

附录A　如何利用Python的相关资源

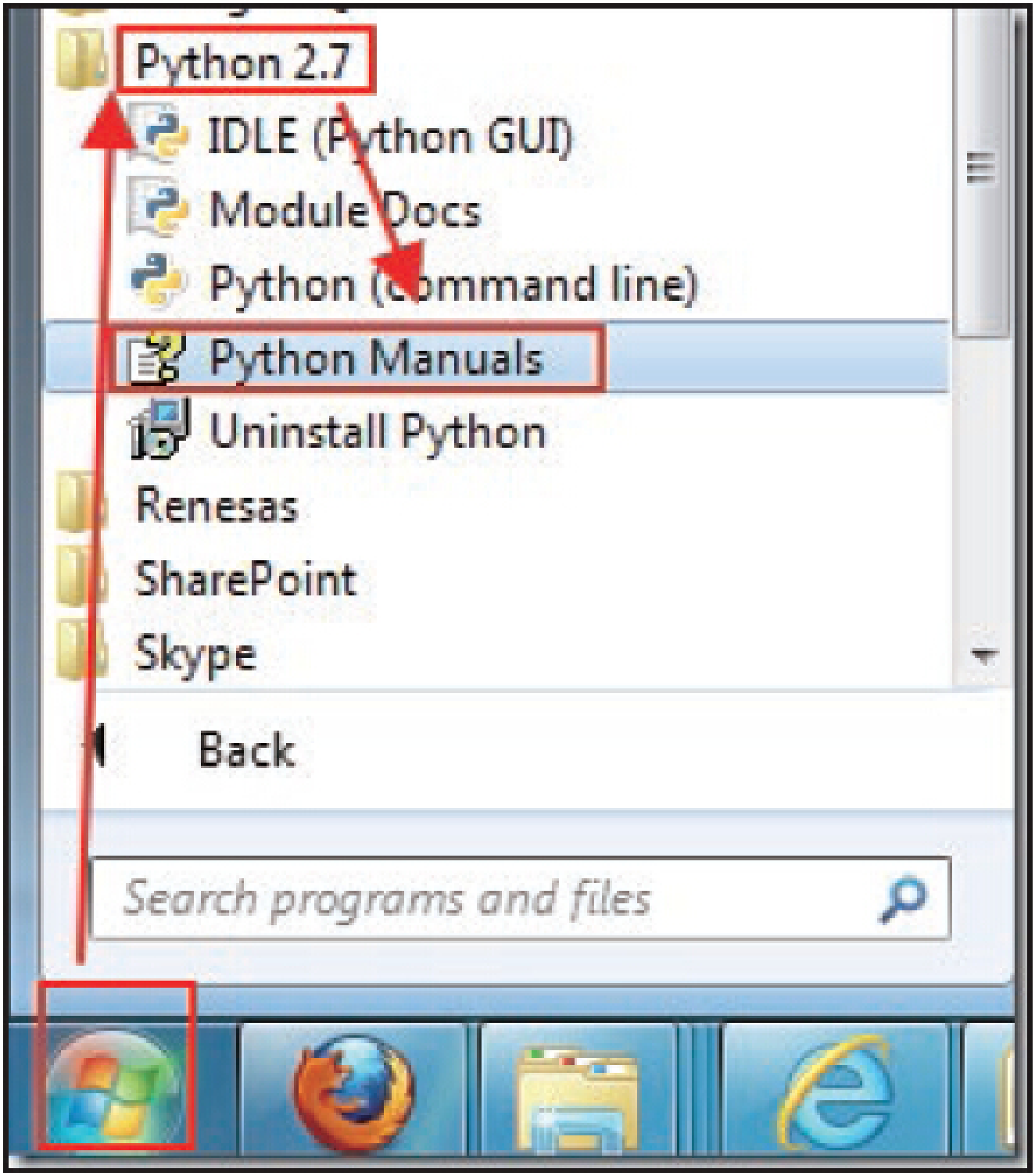
在学习Python的过程中，不论是查询某个库函数的使用还是遇到问题到网上查找解决办法，都会涉及一个问题：如何才能找到需要的参考资料呢？本部分就针对此问题来总结一些有价值的参考资料，并介绍如何利用它们。

A.1　如何利用Python的官方手册

在查询函数的接口和API定义方面，最好且最权威的资料是Python官网的手册。Python官方的手册主要有两种方式供用户使用。

1.自带chm格式手册：Python Manual

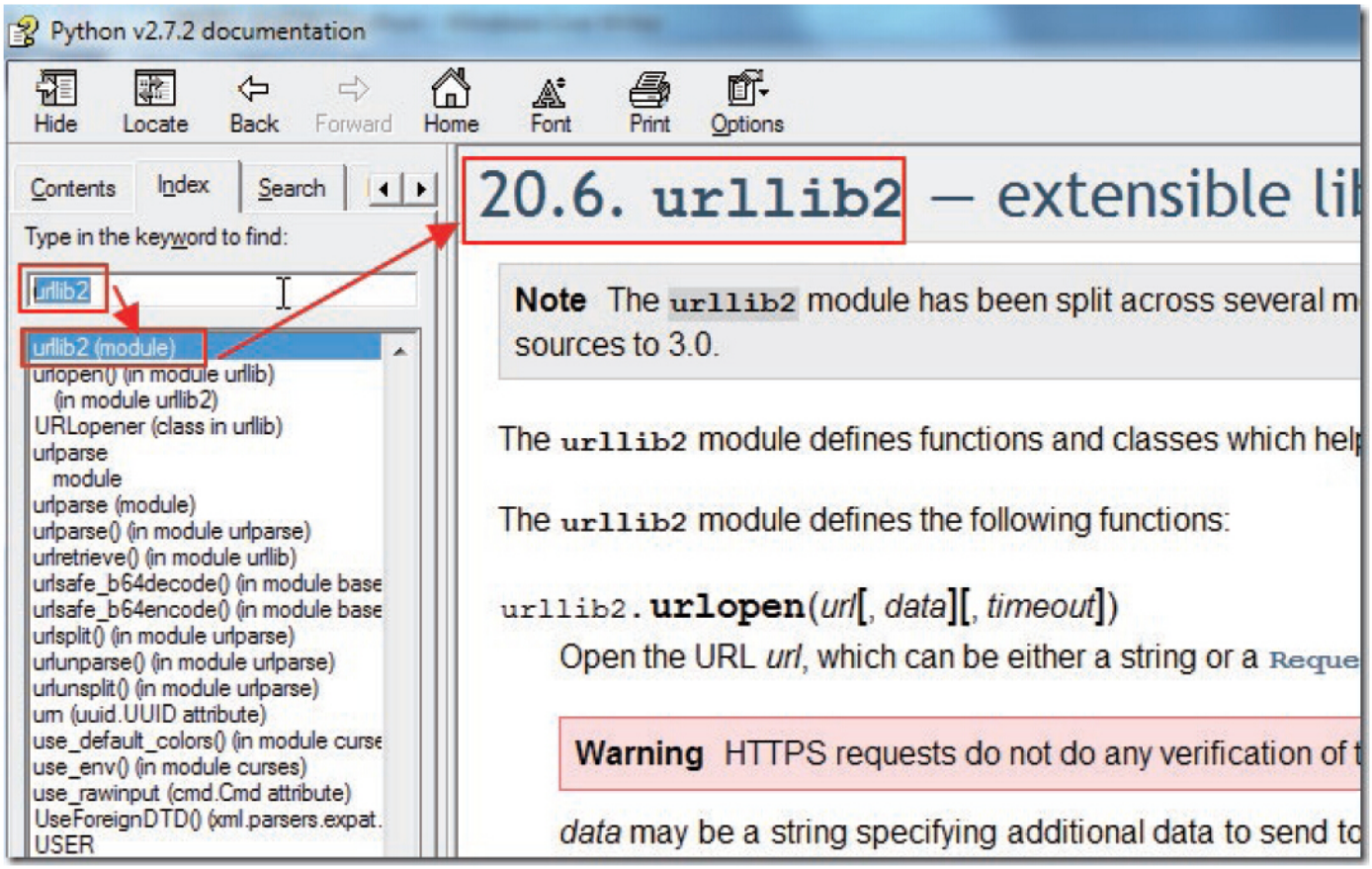
Windows平台在安装了Python之后就可以通过执行“开始”→Python 27→Python Manuals命令找到对应的Python手册了，如图A-1所示。



图A-1　Windows中的Python手册

打开手册后就可以查询自己想要找的各种资料了，包括库函数的各种定义、各种基本类型的解释等。接着再通过举例来解释如何使用Python手册查找所需资料。

比如学习Python期间涉及网络编程时需要用到urllib2模块，而此时你只知道有个叫作urllib2的模块但却不知道它有哪些函数以及函数的接口是什么，所以想要去查询对应的urllib2库的详细含义和用法，此时就可以去Python手册中查找urllib2，如图A-2所示。



图A-2　在Python手册中查找urllib2

然后就可以找到对应的解释了：

```
urllib2.urlopen(url[, data][, timeout])
```

Open the URL url，which can be either a string or a Request object.

Warning HTTPS requests do not do any verification of the server’s certificate.

data may be a string specifying additional data to send to the server，or None if no such data is needed.Currently HTTP requests are the only ones that use data；the HTTP request will be a POST instead of a GET when the data parameter is provided.data should be a buffer in the standard application/x-www-form-urlencoded format.The urllib.urlencode()function takes a mapping or sequence of 2-tuples and returns a string in this format.urllib2module sends HTTP/1.1requests with Connection：close header included.

The optional timeout parameter specifies a timeout in seconds for blocking operations like the connection attempt（if not specified，the global default timeout setting will be used）.This actually only works for HTTP，HTTPS and FTP connections.

This function returns a file-like object with two additional methods：

· geturl()-return the URL of the resource retrieved，commonly used to determine if a redirect was followed

· info()-return the meta-information of the page，such as headers，in the form of an mimetools.Message instance（see Quick Reference to HTTP Headers）

· getcode()-return the HTTP status code of the response

Raises URLError on errors.

Note that None may be returned if no handler handles the request（though the default installed global OpenerDirector uses UnknownHandler to ensure this never happens）.

In addition，if proxy settings are detected（for example，when a*_proxy environment variable like http_proxy is set），ProxyHandler is default installed and makes sure the requests are handled through the proxy.

Changed in version 2.6：timeout was added.

如此，通过认真仔细学习官网的解释，可以更好地理解对应的函数功能、参数等细节知识以及如何使用。比如：

1）解释中说了可以用geturl()，info()，getcode()去获得对应的信息。

于是在你写代码时，就通过已获得该类型的对象调用对应方式，即使用该函数去获得你想要的信息了。此处说一个笔者之前自己写代码时就遇到的实际例子：

```
#if not give user-agent, then will error: HTTP Error 403: Forbidden
request = urllib2.Request(fileUrl, headers={'User-Agent' : gConst['UserAgent']});
#print "request=",request;
resp = urllib2.urlopen(request, timeout=gConst['defaultTimeout']);
#print "resp=",resp;
realUrl = resp.geturl();
```

其中就是为了获得对应的urllib2所返回的响应resp后，再去调用geturl去获得当发生网页自动跳转后的真正的url地址。

2）注意到其中最后部分的解释，它提到了是2.6版本之后才加入timeout参数。

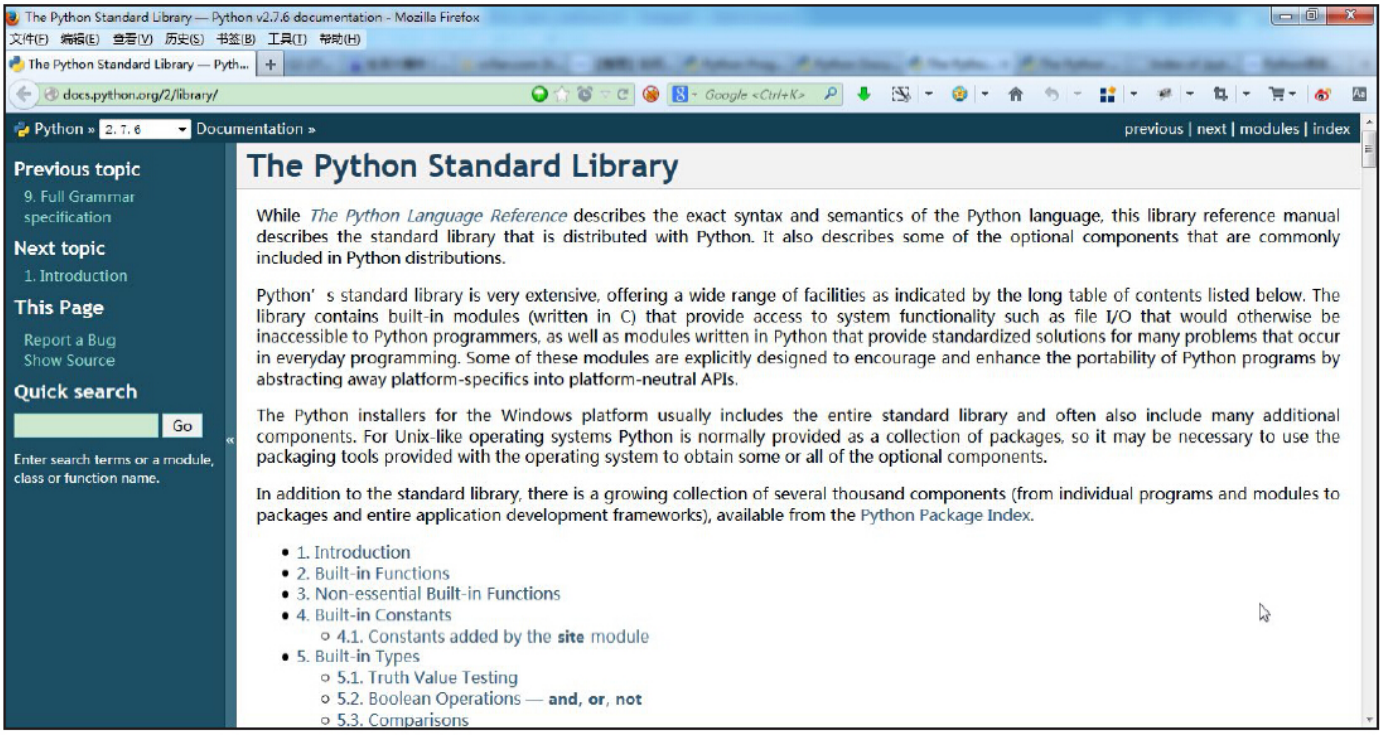
看到此处，细心的读者就应该想到，如果当前使用的Python版本是2.6或更早的版本，那么则是没有timeout参数可用的。

如此，读者在自己写代码期间当遇到不熟悉的、不懂的库或函数等内容时，第一时间想到的应该是尽量先去通过Python手册来看看来自Python官方的解释到底是什么样的、到底如何使用，然后再去参考其他第三方的资料，以此实现逐渐更加熟悉和深入对应的库和函数等的使用。

2.在线网页格式的在线手册

除了Windows中安装Python后就自带了手册之外，Python官网也提供了在线版本的Python手册，以方便Linux、Mac等平台中没有chm文件可供参阅时在线查看对应的手册内容。Python手册官网在线地址是如下。

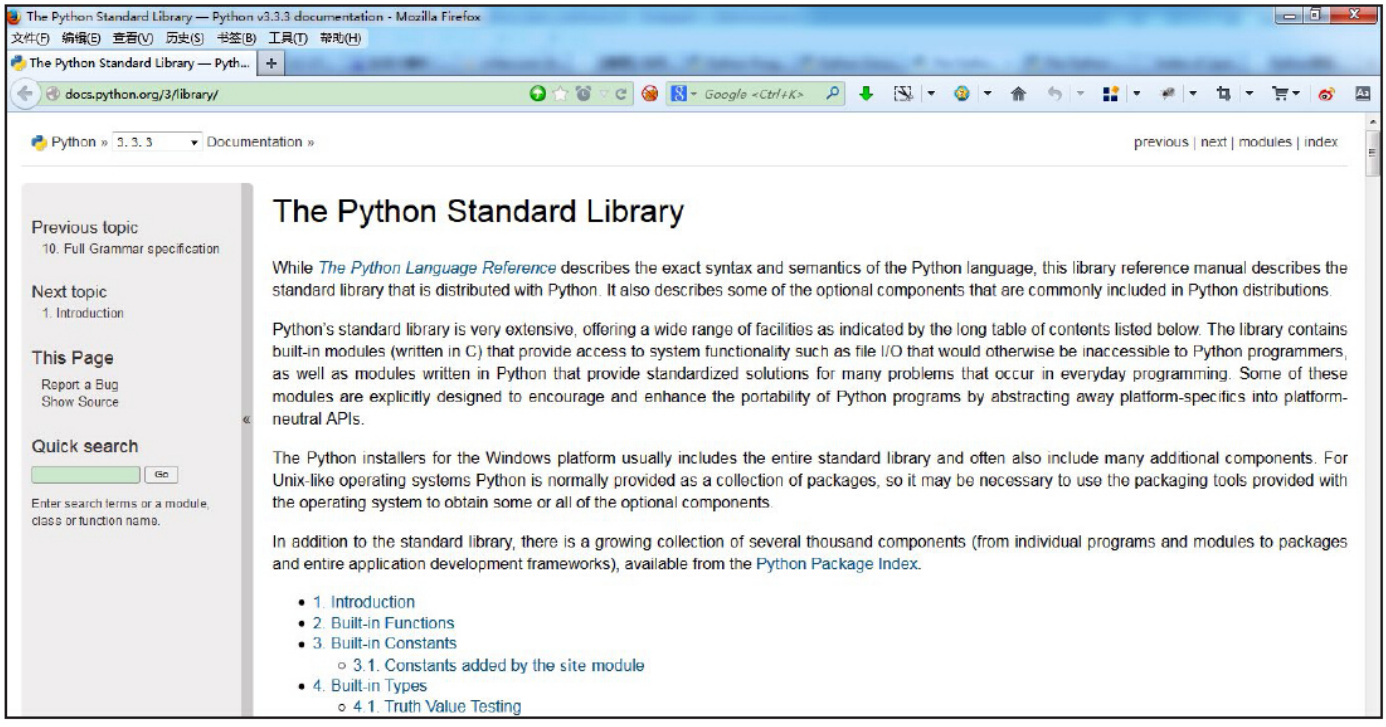
1）Python 2在线手册的地址为http://docs.python.org/2/library/。其网页如图A-3所示。



图A-3　Python 2网页版在线手册

2）Python 3在线手册的地址为http://docs.python.org/3/library/。其网页如图A-4所示。

这样的话，即使手上没有chm格式的Python手册，只要有网络也可以随时随地地查阅在线的Python手册了。



图A-4　Python 3网页版在线手册

A.2　如何利用在线的Python资源

前面已经介绍过了，在继续学习Python期间必然会涉及了解和使用第三方的库函数。可能很多读者都会觉得这些库函数的数量太庞大，不方便查阅。实际上，对于这个问题Python官网早已想到，早已有了专门的地方进行了分类整理，方便读者查阅所有的官网所收录的Python的第三方的库。

· 主入口地址：Index of Packages：Python Package Index

https://pypi.python.org/pypi

· 单一页面中查看所有的库函数

https://pypi.python.org/pypi%3Aaction=index

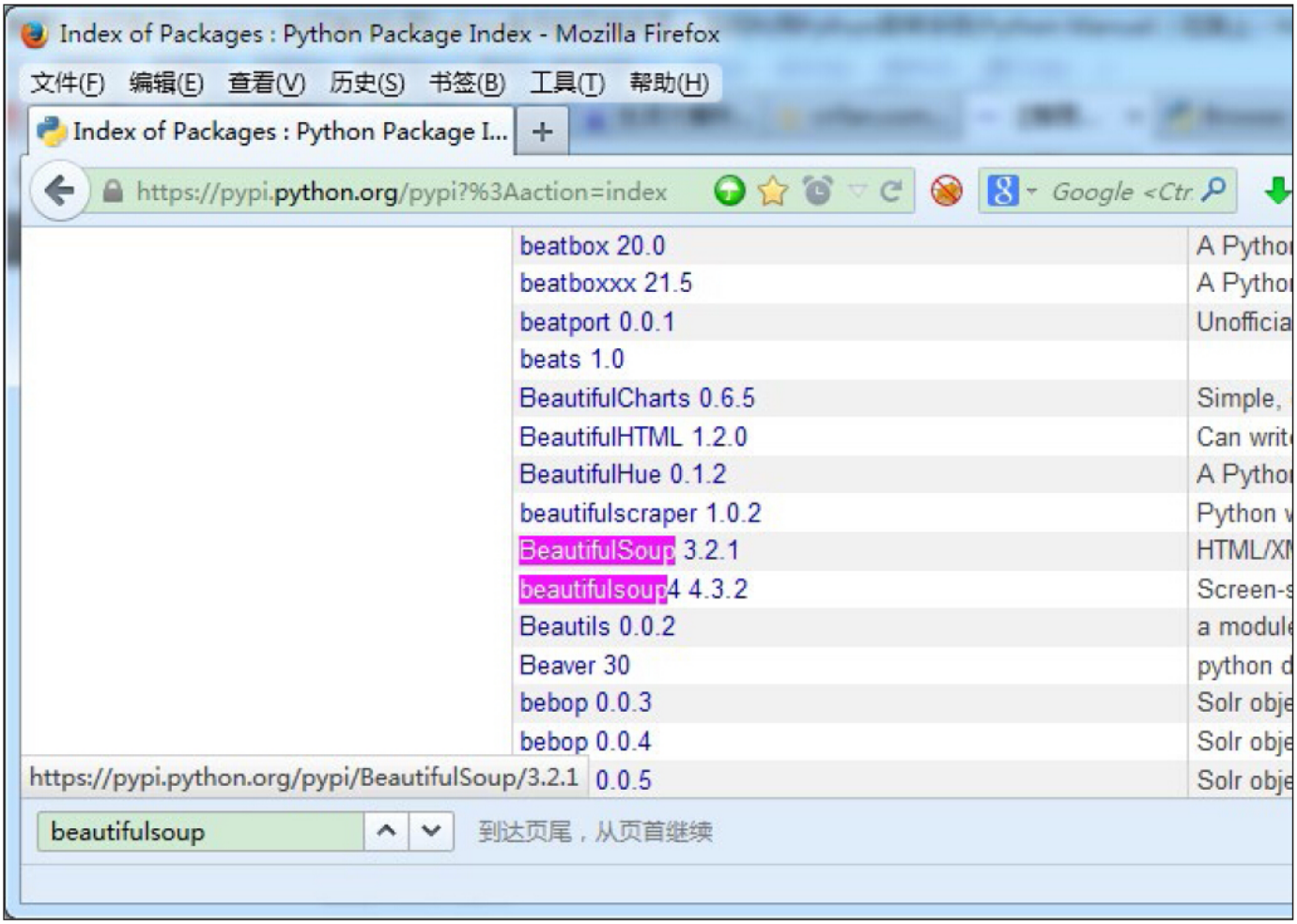
· 分类查看所有的库函数

http://pypi.python.org/pypi%3Aaction=browse

· 只查看Python 3的库函数

https://pypi.python.org/pypi：action=browse&c=533&show=all

如此，读者每次需要查看对应的第三方的库函数时，就可以根据需要去查询了。举个例子：在网络开发领域常会遇到处理已获得的网页，即解析网页，而通过网络搜索已经知道有个专门用来解析网页的叫作BeautifulSoup的库，但是想要进一步知道其具体的用法。此时就可以去上述“单一页面中查看所有的库函数”的地址https://pypi.python.org/pypi%3Aaction=index中找到对应的BeautifulSoup了，如图A-5所示。

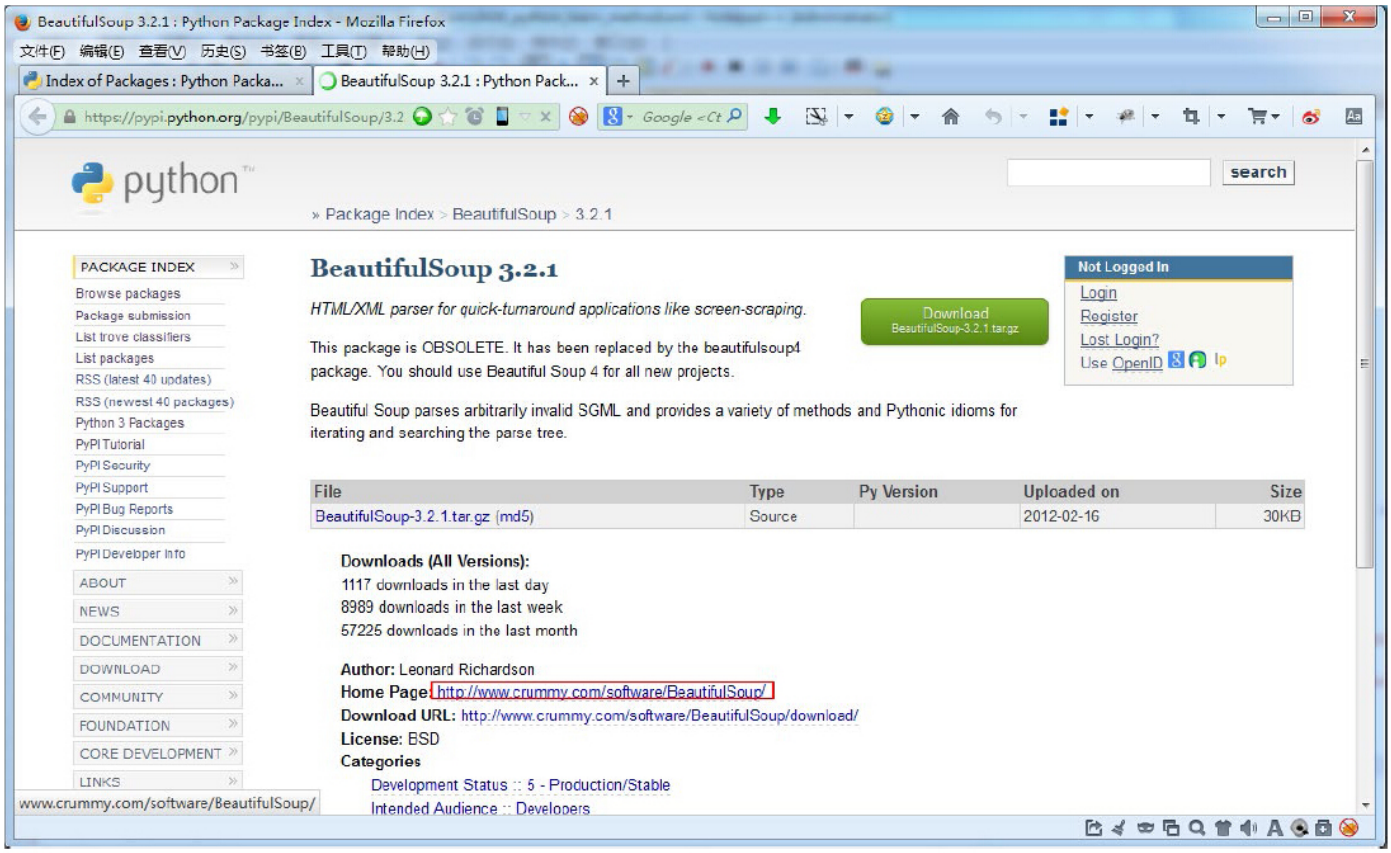


图A-5　Python库函数页面中找到BeautifulSoup

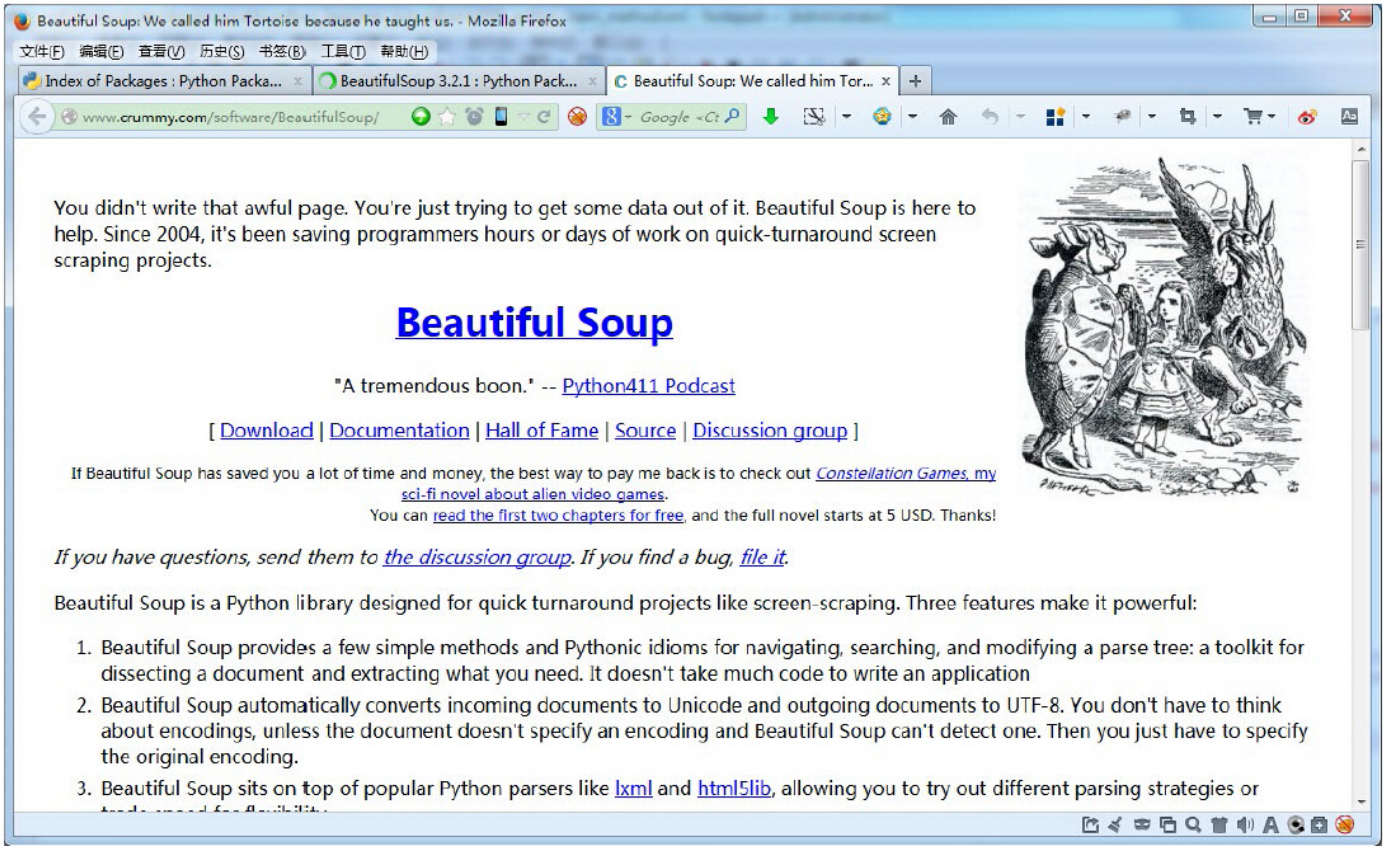
其中可以注意到BeautifulSoup分成两大版本：BeautifulSoup的3.2.1和beautifulsoup4的4.3.2。接着单击进入后就可以找到该模块的主页地址了，如图A-6所示。

然后就可以找到真正想要的模块的主页了，如图A-7所示。

然后在主页中可以看到各种资料，包括我们所需要的BeautifulSoup的文档（Documentation）。读者接下来就可以根据自己的需要查询不同的内容去获得最权威、最官方的解释了，学习每个不同模块的不同的功能和函数的具体用法。



图A-6　页面中可以找到BeautifulSoup主页地址



图A-7　已打开BeautifulSoup主页

附录B　如何继续深入学习Python

在已经掌握了Python的基本知识之后，如何继续深入学习Python呢？可以总结为一句话：兴趣是最好的导师。

对此句话更详细地阐述就是，基于一定的兴趣导向，寻找一个合适的目标，然后在实现目标的过程中一点点学习涉及的Python的相关知识。关于目标，可以是自己给自己找的任务，去实现一定的功能，也可以是为解决别人的某个问题或者帮别人实现一定的功能或目标。而后在实现目标的过程中，充分利用相关资源，边写Python代码边学习相关知识。即一边慢慢搞清楚需要实现的功能所涉及的思路和要做的事情有哪些，一边将搞懂后的思路用真正的Python代码实现出来。在实现目标的过程中如果遇到相关的Python的基本语法、库的使用、函数的接口等细节问题，再去参考相关的API手册说明，一点点学习并写出自己的Python代码。

举个例子，笔者之前就遇到一个自己的实际需求：需要将之前笔者在网易163博客、腾讯的QQ空间、百度的hi空间等地方的很多个博客的内容都迁移到自己新建的基于Wordpress的个人网站中，即实现多个国内博客搬家到Wordpress。如此以自己的实际需求为导向，一点点学习和参考他人的代码，然后自己继续调试代码和扩充功能。在此期间必然需要学习和了解相关技术，其中多数是和网络爬虫有关的背景知识和逻辑，然后再去参考他人写网络爬虫的相关代码。包括其中用到了经典的urllib等库函数，然后自己去参考Python自带手册，一点点学习其详细的语法，从而最终搞懂了网络爬虫的实现逻辑，以及如何用Python实现网络爬虫，Python中和网络处理相关的库函数的使用，从无到有地实现了支持多个博客搬家的Python程序：BlogsToWordpress（地址：http://code.google.com/p/blogs-to-wordpress/）。如此过程就是前面提到的在实现自己目标的过程中渐进式地学习Python。这样才能把学习Python的过程变成一个相对有意义、有趣的过程，达到学有所用的目的，才能使自己更加有动力、有目标，不会像一些人为了学习而学习，把学习和写Python代码变成简单抄写示例代码的枯燥过程。

附录C　Python学习资料

最后整理一些相对不错的Python的相关资料供读者在学习Python期间参考和学习。

· 简明Python教程（地址：http://woodpecker.org.cn/abyteofpython_cn/chinese/）Swaroop，C.H.著，沈洁元译。该资料的特点是内容很全。

· Python中文文档。文档中（地址：http://www.elias.cn/Python/HomePage）介绍了很多资源，其中就有Python教程的中文版（地址：http://wiki.woodpecker.org.cn/moin/March_Liu/PyTutorial），值得参考。